

# **DC Explorer**

# **User Guide**

## 中文学习译本

原版版本：S-2021.06，June 2021

Synopsys, Inc.

原文版权归 Synopsys, Inc. 所有（Version S-2021.06, June 2021）。  
本文件为非官方个人学习译本，仅供学习参考，不得用于商业传播。  
命令名、选项名、文件名等技术标识符保持英文原文不译。

# 目录

|                                        |     |
|----------------------------------------|-----|
| 关于本手册                                  | ii  |
| 本版本新增内容                                | ii  |
| 相关产品、文档与商标                             | ii  |
| 排版约定                                   | iii |
| 客户支持                                   | iii |
| 第一章 关于 <b>DC Explorer</b>              | 1   |
| 1.1 DC Explorer 设计流程                   | 1   |
| 1.2 运行 DC Explorer 设计流程                | 2   |
| 1.3 使用 DC Explorer 的收益                 | 3   |
| 1.4 DC Explorer 与 Design Compiler 的兼容性 | 3   |
| 1.4.1 与 Design Compiler 的脚本兼容性         | 4   |
| 1.5 IC Compiler 中的 Floorplan 探索        | 5   |
| 第二章 使用 <b>DC Explorer</b>              | 6   |
| 2.1 运行 DC Explorer                     | 6   |
| 2.1.1 启动 DC Explorer                   | 7   |
| 2.1.2 输入 de_shell 命令                   | 8   |
| 2.1.3 中断或终止命令处理                        | 8   |
| 2.1.4 在 de_shell 中打开或关闭 GUI            | 9   |
| 2.1.5 在命令行获取帮助                         | 9   |
| 2.1.6 Setup 文件                         | 10  |
| 2.1.7 在日志文件中查找会话信息                     | 12  |
| 2.1.8 运行 Tcl 脚本                        | 13  |
| 2.1.9 退出 DC Explorer                   | 14  |
| 2.2 许可证                                | 14  |
| 2.2.1 列出正在使用的许可证                       | 15  |
| 2.2.2 获取许可证                            | 15  |
| 2.2.3 释放许可证                            | 16  |
| 2.2.4 启用许可证排队                          | 16  |
| 2.3 使用多核技术                             | 17  |
| 2.3.1 启用多核处理                           | 17  |
| 2.3.2 报告运行时间                           | 18  |

|       |                                |    |
|-------|--------------------------------|----|
| 2.3.3 | 调整多核处理期间的核数                    | 18 |
| 2.4   | 并行运行命令                         | 19 |
| 2.4.1 | parallel_execute 命令            | 19 |
| 2.4.2 | 许可证要求                          | 20 |
| 2.4.3 | 多核处理                           | 20 |
| 2.4.4 | 支持的命令                          | 20 |
| 2.4.5 | 在设计流程中                         | 20 |
| 2.5   | 在后台运行命令以改善运行时间                 | 21 |
| 2.5.1 | 报告用 redirect 提交的作业             | 21 |
| 第三章   | 设置库                            | 23 |
| 3.1   | 库要求                            | 23 |
| 3.1.1 | 逻辑库                            | 24 |
| 3.1.2 | 符号库                            | 25 |
| 3.1.3 | DesignWare 库                   | 25 |
| 3.1.4 | 物理库                            | 26 |
| 3.2   | 指定逻辑库                          | 26 |
| 3.2.1 | 指定 DesignWare 库                | 27 |
| 3.2.2 | 指定库搜索路径                        | 27 |
| 3.2.3 | 指定最小时序库                        | 28 |
| 3.3   | 指定物理库                          | 28 |
| 3.4   | 在网表或 DEF 文件中映射物理变体单元           | 29 |
| 3.5   | 使用库                            | 29 |
| 3.5.1 | 加载库                            | 30 |
| 3.5.2 | 列出库                            | 30 |
| 3.5.3 | 报告库内容                          | 30 |
| 3.5.4 | 指定库对象                          | 30 |
| 3.5.5 | 从目标库中排除单元                      | 31 |
| 3.5.6 | 验证库一致性                         | 31 |
| 3.5.7 | 从内存中移除库                        | 31 |
| 3.5.8 | 保存库                            | 32 |
| 3.6   | 目标库子集                          | 32 |
| 3.6.1 | 设置目标库子集限制                      | 33 |
| 3.7   | 使用 TLUPlus 或 nxtgrd 文件进行 RC 估算 | 34 |
| 3.7.1 | 使用 TLUPlus 文件                  | 34 |
| 3.7.2 | 使用 nxtgrd 文件                   | 34 |
| 3.8   | 对黑盒的支持                         | 35 |
| 3.8.1 | 自动创建物理库单元                      | 36 |
| 3.8.2 | 定义物理尺寸                         | 36 |
| 3.8.3 | 为黑盒创建快速时序模型                    | 37 |
| 3.8.4 | 用于在黑盒中定义时序的命令                  | 39 |
| 3.8.5 | 识别黑盒                           | 39 |

|                              |    |
|------------------------------|----|
| 第四章 对不完整或不匹配数据的容错            | 41 |
| 4.1 容错类别                     | 41 |
| 4.1.1 限制                     | 42 |
| 4.2 默认容错设置                   | 43 |
| 4.2.1 总线引脚命名风格               | 43 |
| 4.2.2 引脚名大小写不匹配              | 44 |
| 4.2.3 逻辑库中缺失单元               | 44 |
| 4.2.4 物理库中缺失单元               | 45 |
| 4.2.5 逻辑库中缺失引脚               | 45 |
| 4.2.6 FRAM 视图中缺失引脚           | 46 |
| 4.2.7 逻辑库与物理库之间的引脚方向不匹配      | 46 |
| 4.2.8 端口宽度不匹配                | 47 |
| 4.3 调整总线引脚命名风格的容错设置          | 47 |
| 4.4 解析总线与 bit-blasted 命名风格   | 48 |
| 4.5 允许引脚名同义词                 | 49 |
| 4.5.1 示例 1                   | 49 |
| 4.5.2 示例 2                   | 49 |
| 4.6 允许未连接的接口引脚               | 50 |
| 4.7 在网表中包含锚点单元               | 50 |
| 4.8 创建报告与缺失约束                | 51 |
| 4.8.1 报告不完整或不匹配数据            | 51 |
| 4.8.2 报告缺失约束                 | 53 |
| 4.8.3 报告不可行路径                | 54 |
| 4.8.4 使用分类时序报告创建约束           | 55 |
| 4.8.5 为不可行路径创建例外             | 56 |
| 第五章 处理内存中的设计                 | 58 |
| 5.1 设计术语                     | 58 |
| 5.1.1 设计                     | 59 |
| 5.1.2 设计对象                   | 59 |
| 5.1.3 设计、实例与引用之间的关系          | 60 |
| 5.2 打开设计                     | 61 |
| 5.2.1 analyze 与 elaborate 命令 | 62 |
| 5.2.2 read_file 命令           | 62 |
| 5.2.3 两种读入方法的差异              | 63 |
| 5.2.4 打开 HDL 与 .ddc 文件       | 63 |
| 5.3 设置当前设计                   | 64 |
| 5.4 链接设计                     | 65 |
| 5.4.1 更改设计引用                 | 65 |
| 5.4.2 使用搜索路径定位设计             | 66 |
| 5.4.3 查询设计引用                 | 66 |
| 5.4.4 解析引用的过程                | 67 |
| 5.5 编辑设计                     | 68 |

|        |                                             |     |
|--------|---------------------------------------------|-----|
| 5.5.1  | 设计编辑任务与命令                                   | 69  |
| 5.5.2  | 查询设计与对象                                     | 69  |
| 5.5.3  | 指定设计对象                                      | 70  |
| 5.6    | 更改设计层次                                      | 72  |
| 5.6.1  | 添加层次级别                                      | 72  |
| 5.6.2  | 移除层次级别                                      | 73  |
| 5.6.3  | 合并来自不同子设计的单元                                | 76  |
| 5.7    | 使用设计对象属性                                    | 76  |
| 5.7.1  | 查询属性值                                       | 77  |
| 5.7.2  | 在对象上设置属性                                    | 78  |
| 5.7.3  | 更改对象属性                                      | 78  |
| 5.7.4  | 获取属性描述的命令                                   | 79  |
| 5.7.5  | 传播在单元实例上设置的用户属性                             | 80  |
| 5.7.6  | 在时序单元上保留用户属性                                | 81  |
| 5.7.7  | 对象搜索顺序                                      | 82  |
| 5.8    | 创建设计                                        | 82  |
| 5.9    | 复制设计                                        | 83  |
| 5.10   | 重命名设计                                       | 83  |
| 5.11   | 保存设计                                        | 84  |
| 5.12   | 移除设计                                        | 85  |
| 5.13   | 设计数据库与网表命名一致性                               | 85  |
| 5.13.1 | .synopsys_dc.setup 文件中的命名规则                 | 86  |
| 5.13.2 | 解决命名问题                                      | 86  |
| 5.13.3 | 避免 SystemVerilog 与 VHDL 结构中的 bit-blasted 端口 | 87  |
| 第六章    | 定义设计环境                                      | 89  |
| 6.1    | 工作条件                                        | 89  |
| 6.2    | 定义工作条件                                      | 90  |
| 6.2.1  | 多角多模设计                                      | 90  |
| 6.3    | 对系统接口建模                                     | 91  |
| 6.3.1  | 定义输入端口的驱动特性                                 | 91  |
| 6.3.2  | 定义输入与输出端口上的负载                               | 92  |
| 6.3.3  | 定义输出端口上的扇出负载                                | 92  |
| 6.4    | 在端口上设置逻辑约束                                  | 93  |
| 6.5    | 使用 UPF 的多电压设计                               | 94  |
| 第七章    | 定义设计约束                                      | 95  |
| 7.1    | DC Explorer 约束类型                            | 95  |
| 7.1.1  | 设计规则约束                                      | 96  |
| 7.1.2  | 优化约束                                        | 98  |
| 7.1.3  | 约束优先级                                       | 99  |
| 7.1.4  | 设计规则约束的优先顺序                                 | 100 |
| 7.1.5  | 生成约束报告                                      | 101 |

|       |                                    |            |
|-------|------------------------------------|------------|
| 7.2   | 定义设计规则约束                           | 102        |
| 7.2.1 | 定义最大转换时间                           | 102        |
| 7.2.2 | 定义最大扇出                             | 103        |
| 7.2.3 | 定义最大电容                             | 104        |
| 7.2.4 | 定义最小电容                             | 105        |
| 7.2.5 | 定义单元退化                             | 105        |
| 7.2.6 | 定义连接类                              | 106        |
| 7.3   | 对特殊网络禁用设计规则修复                      | 106        |
| 7.4   | 在层次化设计中传播约束                        | 106        |
| 7.4.1 | 表征子设计                              | 107        |
| 7.4.2 | 沿层次向上传播约束                          | 109        |
| 第八章   | <b>UPF 探索</b>                      | <b>111</b> |
| 8.1   | UPF 探索流程概览                         | 111        |
| 8.1.1 | 带 UPF 的设计探索                        | 112        |
| 8.1.2 | 运行带 UPF 的设计探索                      | 113        |
| 8.1.3 | 隔离与电平移位插入                          | 114        |
| 8.2   | 最小 UPF 文件                          | 116        |
| 8.2.1 | 必需命令                               | 116        |
| 8.2.2 | 可选命令                               | 117        |
| 8.2.3 | 被忽略的命令                             | 117        |
| 8.2.4 | 被忽略的供电网络域依赖性                       | 118        |
| 8.3   | 带 UPF 的探索综合                        | 118        |
| 8.4   | 报告功耗探索结果                           | 119        |
| 8.4.1 | Design Vision GUI                  | 119        |
| 第九章   | <b>使用 Floorplan 物理约束</b>           | <b>120</b> |
| 9.1   | 导入 Floorplan 信息                    | 120        |
| 9.1.1 | 在 IC Compiler 中使用 write_def 命令     | 121        |
| 9.1.2 | 在 DC Explorer 中读取 DEF 信息           | 121        |
| 9.1.3 | 导入的 DEF 信息                         | 122        |
| 9.1.4 | extract_physical_constraints 的名称匹配 | 128        |
| 9.1.5 | 解决 Site 名称不匹配                      | 129        |
| 9.1.6 | 在 IC Compiler 中保存 Floorplan 信息     | 129        |
| 9.1.7 | 在 DC Explorer 中读取 Floorplan 脚本     | 130        |
| 9.1.8 | 导入的物理约束                            | 130        |
| 9.1.9 | read_floorplan 的名称匹配               | 131        |
| 9.2   | 物理约束概览                             | 131        |
| 9.2.1 | 定义物理约束                             | 132        |
| 9.2.2 | Placement Bound 概览                 | 140        |
| 9.2.3 | 用于定义物理约束的命令                        | 141        |
| 9.3   | 保存物理约束                             | 142        |
| 9.3.1 | IC Compiler 格式                     | 142        |

|         |                                 |     |
|---------|---------------------------------|-----|
| 9.3.2   | IC Compiler II 格式               | 142 |
| 9.4     | 以 ASCII 格式保存供 IC Compiler II 使用 | 143 |
| 9.4.1   | 限制                              | 144 |
| 9.5     | 包含仅物理单元                         | 144 |
| 9.5.1   | 指定仅物理单元                         | 144 |
| 9.5.2   | 从 DEF 提取仅物理单元                   | 145 |
| 9.5.3   | 报告仅物理单元                         | 146 |
| 9.5.4   | 保存仅物理单元                         | 146 |
| 9.6     | 相对布局                            | 147 |
| 9.6.1   | 相对布局概览                          | 147 |
| 9.6.2   | 运行相对布局流程                        | 148 |
| 9.6.3   | 指定相对布局约束                        | 149 |
| 9.6.4   | 使用 HDL Compiler 指令创建相对布局        | 157 |
| 第十章     | 优化                              | 159 |
| 10.1    | Compile 策略                      | 159 |
| 10.2    | 执行自顶向下 Compile                  | 161 |
| 10.3    | 执行自底向上 Compile                  | 162 |
| 10.3.1  | 自底向上 Compile 概览                 | 162 |
| 10.3.2  | Compile 子块                      | 163 |
| 10.3.3  | 在顶层 Compile 设计                  | 164 |
| 10.4    | 面向高性能设计的优化                      | 166 |
| 10.4.1  | Datapath 优化                     | 166 |
| 10.4.2  | 面向最小面积的优化                       | 166 |
| 10.5    | 执行手动高扇出综合                       | 167 |
| 10.6    | 解析设计引用的多实例                      | 168 |
| 10.6.1  | Uniquify 方法                     | 168 |
| 10.6.2  | Compile-Once-Don't-Touch 方法     | 169 |
| 10.6.3  | Ungroup 方法                      | 170 |
| 10.6.4  | 保留子设计                           | 170 |
| 10.7    | 定义库子集限制                         | 171 |
| 10.8    | 优化多角多模式设计                       | 172 |
| 10.8.1  | 多角多模式概念                         | 172 |
| 10.8.2  | 多角多模式设计不支持的功能                   | 173 |
| 10.8.3  | 基本多角多模式流程                       | 173 |
| 10.8.4  | 在多角多模式流程中处理库                    | 174 |
| 10.8.5  | 定义 Scenario                     | 177 |
| 10.8.6  | 管理 Scenario                     | 177 |
| 10.8.7  | 并发多角多模式优化与时序分析                  | 178 |
| 10.8.8  | 多角多模式设计的报告命令                    | 179 |
| 10.8.9  | 多角多模式设计支持的 SDC 命令               | 179 |
| 10.8.10 | 在多角多模式设计中使用 Block Abstraction   | 179 |
| 10.9    | 带 Floorplan 物理约束的优化             | 180 |

|                                                |            |
|------------------------------------------------|------------|
| 10.10 执行功耗优化 . . . . .                         | 181        |
| 10.11 流水线逻辑 Retiming . . . . .                 | 181        |
| 10.11.1 流水线逻辑 Retiming 命令 . . . . .            | 182        |
| 10.11.2 寄存器 Retiming 示例 . . . . .              | 182        |
| 10.11.3 含 Path Group 约束的寄存器 Retiming . . . . . | 183        |
| 10.12 其他优化功能 . . . . .                         | 183        |
| 10.12.1 保留 Subdesign . . . . .                 | 183        |
| 10.12.2 修复多端口网络 . . . . .                      | 184        |
| 10.12.3 优化多比特寄存器 . . . . .                     | 186        |
| 10.12.4 物理流程中的拥塞报告 . . . . .                   | 187        |
| 10.12.5 控制 Path Group 创建 . . . . .             | 188        |
| <b>第十一章 使用层次化模型 . . . . .</b>                  | <b>190</b> |
| 11.1 层次化模型概览 . . . . .                         | 190        |
| 11.1.1 层次化模型中使用的信息 . . . . .                   | 191        |
| 11.1.2 多角多模式流程中的层次化模型 . . . . .                | 192        |
| 11.1.3 在 GUI 中查看层次化模型 . . . . .                | 193        |
| 11.2 关于 Block Abstraction . . . . .            | 193        |
| 11.2.1 Block Abstraction 层次化流程 . . . . .       | 194        |
| 11.2.2 创建并保存 Block Abstraction . . . . .       | 194        |
| 11.2.3 设置顶层设计选项 . . . . .                      | 195        |
| 11.2.4 控制为 Block Abstraction 加载的逻辑范围 . . . . . | 195        |
| 11.2.5 加载 Block Abstraction . . . . .          | 196        |
| 11.2.6 检查 Block Abstraction 就绪性 . . . . .      | 197        |
| 11.2.7 执行顶层综合 . . . . .                        | 198        |
| 11.2.8 忽略 Block Abstraction 内的时序路径 . . . . .   | 198        |
| 11.2.9 限制 . . . . .                            | 199        |
| <b>第十二章 使用 GUI . . . . .</b>                   | <b>200</b> |
| 12.1 功能与收益 . . . . .                           | 200        |
| 12.2 使用 GUI 窗口 . . . . .                       | 201        |
| 12.2.1 输入命令并查看结果 . . . . .                     | 202        |
| 12.2.2 查看 Man 页 . . . . .                      | 203        |
| 12.2.3 保存窗口或视图的图像 . . . . .                    | 203        |
| 12.2.4 在 GUI 中获取帮助 . . . . .                   | 204        |
| 12.3 执行 Floorplan 探索 . . . . .                 | 205        |
| 12.3.1 启用 Floorplan 探索 . . . . .               | 205        |
| 12.3.2 启用数据流分析 . . . . .                       | 207        |
| 12.3.3 使用 Floorplan 探索 GUI . . . . .           | 208        |
| 12.3.4 保存 Floorplan 或丢弃更新 . . . . .            | 209        |
| 12.3.5 退出会话 . . . . .                          | 209        |
| 12.3.6 在 Floorplan 变更后执行综合 . . . . .           | 210        |
| 12.3.7 在批处理模式下使用 Floorplan 探索 . . . . .        | 210        |

|                                        |            |
|----------------------------------------|------------|
| 12.3.8 黑盒、物理层次与块抽象                     | 211        |
| 12.4 使用 IC Compiler II 进行 Floorplan 探索 | 212        |
| 12.4.1 Floorplan 探索的先决条件               | 212        |
| 12.4.2 运行 Floorplan 探索流程               | 212        |
| 12.4.3 向 IC Compiler II 传输数据           | 214        |
| 12.5 分析 RTL                            | 214        |
| 12.6 RTL 交叉探测                          | 217        |
| 12.6.1 重复 RTL 交叉探测                     | 218        |
| <b>第十三章 分析并解决设计问题</b>                  | <b>219</b> |
| 13.1 修复由不受支持的 Technology 文件属性引起的错误     | 219        |
| 13.2 比较 DC Explorer 与 IC Compiler 环境   | 220        |
| 13.2.1 压缩格式的环境文件                       | 221        |
| 13.3 检查设计一致性                           | 221        |
| 13.4 检查未映射单元                           | 222        |
| 13.5 指定输出端口方向不正确时的严重级别                 | 223        |
| 13.6 分析设计问题                            | 223        |
| 13.7 分析面积                              | 223        |
| 13.7.1 将块抽象中单元面积报告为宏单元面积               | 225        |
| 13.8 分析时序                              | 225        |
| 13.9 生成结果质量                            | 226        |
| 13.9.1 度量结果质量                          | 226        |
| 13.9.2 分析结果质量                          | 226        |
| 13.9.3 报告结果质量                          | 228        |
| 13.9.4 通过控制路径组创建改善 QoR                 | 229        |
| 13.10 调试带有 dont_touch 属性的单元与网          | 230        |
| 13.10.1 报告 Don't Touch 单元与网            | 230        |
| 13.10.2 创建 Don't Touch 单元与网的集合         | 231        |
| 13.11 使用命令进行 RTL 交叉探测                  | 232        |
| 13.12 在批处理模式下报告逻辑级                     | 233        |
| <b>第十四章 使用 Milkyway 数据库</b>            | <b>235</b> |
| 14.1 关于 Milkyway 数据库                   | 235        |
| 14.1.1 所需许可证与文件                        | 236        |
| 14.1.2 调用 Milkyway Environment 工具      | 236        |
| 14.2 使用 Milkyway 数据库的指南                | 236        |
| 14.3 创建 Milkyway 设计库                   | 237        |
| 14.4 写出 Milkyway 数据库                   | 238        |
| 14.4.1 关于 write_milkyway 命令            | 238        |
| 14.4.2 使用 Milkyway 格式的限制               | 239        |

|       |                     |     |
|-------|---------------------|-----|
| 附录 A  | 面向综合的设计文件管理         | 240 |
| A.1   | 管理设计数据              | 240 |
| A.1.1 | 设计数据控制              | 240 |
| A.1.2 | 设计数据组织              | 241 |
| A.2   | 面向综合的划分             | 242 |
| A.2.1 | 为设计重用而划分            | 242 |
| A.2.2 | 将相关组合逻辑放在一起         | 242 |
| A.2.3 | 对块输出寄存器化            | 243 |
| A.2.4 | 按设计目标划分             | 244 |
| A.2.5 | 按 Compile 技术划分      | 244 |
| A.2.6 | 将可共享资源放在一起          | 244 |
| A.2.7 | 将用户定义资源与其所驱动的逻辑放在一起 | 245 |
| A.2.8 | 隔离特殊功能              | 245 |
| A.3   | 面向综合的 HDL 编码        | 246 |
| A.3.1 | 编写与工艺无关的 HDL        | 246 |
| A.3.2 | 使用 HDL 构造           | 248 |
| A.3.3 | 编写有效代码              | 252 |
| 附录 B  | 设计示例                | 255 |
| B.1   | 设计描述                | 255 |
| B.2   | Setup 文件            | 262 |
| B.3   | 默认约束文件              | 262 |
| B.4   | Read 脚本             | 263 |
| B.5   | Compile 脚本          | 264 |
| 附录 C  | 基本命令                | 271 |
| C.1   | 定义设计规则的命令           | 271 |
| C.2   | 定义设计环境的命令           | 271 |
| C.3   | 设置设计约束的命令           | 272 |
| C.4   | 分析并解决设计问题的命令        | 272 |
| 术语表   |                     | 274 |

# 关于本手册

## *About This Manual*

DC Explorer 工具支持早期 RTL 探索 (early RTL exploration)，从而为 RTL 综合提供更好的起点，并加速设计实现。*DC Explorer User Guide* 介绍综合相关概念与命令，说明对不完整或不匹配数据的容错 (tolerance)，并给出基本综合策略示例。

本手册不涵盖异步设计、I/O pad 综合、测试综合、仿真、物理设计技术 (如 floorplanning 或 place and route)，以及物理设计信息的反向标注 (back-annotation)。

此处内容是对 Synopsys 综合参考手册的补充，而非替代。未涵盖主题请参阅其他 Synopsys 文档。

本手册面向使用 Synopsys 综合工具、并以 VHDL 或 Verilog 硬件描述语言 (HDL) 工作的逻辑设计人员与工程师。使用本手册前，应熟悉：

- 高层次设计技术 (high-level design techniques)
- ASIC 设计原理
- 时序分析原理
- 功能分区技术 (functional partitioning)

本前言包括以下节：

- 本版本新增内容
- 相关产品、文档与商标
- 排版约定
- 客户支持

## 本版本新增内容

关于新功能、增强与变更、已知限制，以及已解决的 Synopsys Technical Action Requests (STAR) 信息，请参阅 SolvNetPlus 站点上的 *DC Explorer Release Notes*。

## 相关产品、文档与商标

有关 DC Explorer 工具的更多信息，请参阅 Synopsys SolvNetPlus 支持站点上的文档：<https://solvnetplus.synopsys.com>

您可能还希望参阅以下相关 Synopsys 产品文档：

- Design Compiler®
- Design Vision
- DesignWare components
- TestMAX DFT 与 DFTMAX™
- HDL Compiler™
- IC Compiler™
- Power Compiler™
- PrimeTime®

## 排版约定

Synopsys 文档使用下列约定。

表 1: 文档排版约定

| 约定                    | 说明                                                                                                                                         |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Courier</b>        | 表示语法，例如 <code>write_file</code>                                                                                                            |
| <b>Courier italic</b> | 表示语法中的用户自定义值，例如 <code>write_file design_list</code>                                                                                        |
| <b>Courier bold</b>   | 表示示例中用户原样输入的内容，例如 <code>prompt&gt; write_file top</code>                                                                                   |
| 紫色高亮                  | <ul style="list-style-type: none"> <li>• 在示例中表示特别关注的信息；</li> <li>• 在命令语法中表示默认值，例如 <code>include_enclosing = true   false</code></li> </ul> |
| [ ]                   | 表示语法中的可选参数，例如 <code>write_file [-format fmt]</code>                                                                                        |
| ...                   | 表示参数可按需要重复任意次数，例如 <code>pin1 pin2 ... pinN</code>                                                                                          |
|                       | 表示在多个备选中选择其一，例如 <code>low   medium   high</code>                                                                                           |
| \                     | 表示命令行续行                                                                                                                                    |
| /                     | 表示目录层次                                                                                                                                     |
| 粗体                    | 表示带有关联操作的图形用户界面（GUI）元素                                                                                                                     |
| <b>Edit &gt; Copy</b> | 表示菜单命令路径，例如打开 Edit 菜单并选择 Copy                                                                                                              |
| <b>Ctrl+C</b>         | 表示组合键，例如按住 Ctrl 再按 C                                                                                                                       |

## 客户支持

客户支持通过 SolvNetPlus 提供。

### 访问 SolvNetPlus

SolvNetPlus 站点包含技术文章知识库以及对 Synopsys 工具常见问题的解答，并可访问软件下载、文档与技术支持等在线服务。

访问地址: <https://solvnetplus.synopsys.com>

如提示, 请输入用户名与密码。若尚无 Synopsys 账号, 请按页面说明注册。若使用 SolvNetPlus 时需要帮助, 请点击右上角菜单栏中的 **REGISTRATION HELP**。

## 联系客户支持

请访问 <https://solvnetplus.synopsys.com>。

# Chapter 1

## 关于 DC Explorer

### *About DC Explorer*

如果设计人员缺少快速、高效的方法来探索和改进数据、修复设计问题，并为 RTL 综合建立更好的起点，则开发新 RTL 并将其与第三方 IP 和大量遗留 RTL block 集成会非常耗时。DC Explorer 通过支持早期 RTL 探索来解决这些问题，从而为 RTL 综合提供更好的起点，并加速设计实现。

凭借对不完整设计数据的容错能力以及显著短于完整综合的 runtime，DC Explorer 可尽早提供实现结果；这些结果与 Design Compiler topographical mode 所得结果的偏差通常在 10% 以内。该工具支持在设计周期早期高效地对不同设计配置执行 what-if analysis，从而加快高质量 RTL 与约束的开发，并推动更快、更易收敛的 design flow。它还可生成早期 netlist，用于在 IC Compiler 中启动物理探索。通过一键访问 IC Compiler design planning，DC Explorer 支持在设计周期的极早阶段创建和修改 floorplan。

本章包括以下主题：

- DC Explorer 设计流程
- 运行 DC Explorer 设计流程
- 使用 DC Explorer 的收益
- DC Explorer 与 Design Compiler 的兼容性
- IC Compiler 中的 Floorplan 探索

## 1.1 DC Explorer 设计流程

### DC Explorer Design Flow

DC Explorer design flow 提供快速综合，并帮助检测不完整或不匹配的数据。DC Explorer 接受以下输入文件：RTL netlist、SDC 文件、logic library、physical library（可选）以及 floorplan（可选）。工具生成报告、消息，以及可作为 IC Compiler 早期 floorplanning 输入的 netlist。

图 1.1 给出 DC Explorer 中的典型设计流程。

(原书 *Figure 1: Design Flow in DC Explorer*)  
请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 1.1: DC Explorer 中的设计流程

### 参见

运行 DC Explorer 设计流程，见下一节。

## 1.2 运行 DC Explorer 设计流程

### Running the DC Explorer Design Flow

按下列步骤运行 DC Explorer 设计流程：

1. 创建 **RTL** 和约束。  
DC Explorer 的输入设计文件为 RTL 描述、Synopsys Design Constraints (SDC) 文件、逻辑库、物理库（可选）以及 floorplan（可选）。
2. 打开设计。  
可用以下任一方法打开设计：
  - `analyze` 与 `elaborate` 命令
  - `read` 类命令，例如 `read_file`
3. 报告不完整或不匹配的数据，使用 `current_design`、`link`、`check_design` 与 `report_design_mismatch` 命令。  
继续执行前，不一定需要修复这些不完整或不匹配的数据。
4. 定义设计环境并施加设计约束。
  - 使用下列命令设置设计环境：`set_operating_conditions`、`set_driving_cell`、`set_load` 与 `set_fanout_load`。
  - 施加设计约束时使用 `read_sdc` 命令。
5. 运行快速 **compile** 并优化设计，使用 `compile_exploration` 命令。
6. 创建 **HTML** 格式的分类时序报告，使用 `create_qor_snapshot` 与 `query_qor_snapshot` 命令。  
检查报告中的 `missing constraints` 与 `infeasible paths`；继续执行前，可能需要修复这些不匹配。
7. 写出网表，对 `write_file` 使用 `-format` 选项。

## 参见

- DC Explorer 设计流程（上文）
- 第 四 章「对不完整或不匹配数据的容错」
- 第 三 章「设置库」
- 第 五 章「打开设计」
- 第 六 章「定义设计环境」
- 第 七 章「定义设计约束」
- 第 十 章「优化」
- 第 十三 章「报告结果质量」
- 第 五 章「保存设计」
- 附录 A「面向综合的设计文件管理」

## 1.3 使用 DC Explorer 的收益

### Benefits of Using DC Explorer

在 RTL 与约束开发阶段，DC Explorer 可通过早期快速综合提升效率。使用 DC Explorer 具有以下收益：

- 产出快速综合结果
- 容忍不完整或不匹配的设计数据
- 在使用同一组约束时提供准确的初步综合结果，其时序、面积和功耗数值与最终实现的偏差通常在 10% 以内
- 自动协调设计不匹配，并以简洁格式报告
- 在优化其他关键路径的同时检测不可行时序路径
- 生成易于浏览的 HTML timing report，以加快调试
- 创建可用于 IC Compiler floorplanning 的 netlist

## 1.4 DC Explorer 与 Design Compiler 的兼容性

### DC Explorer and Design Compiler Compatibility

DC Explorer 在下列方面与 Design Compiler 兼容。默认情况下，DC Explorer：

- 支持下列类别中的大多数 Design Compiler 命令：constraints、scripting、reports、design handling、design inputs and outputs、modeling 和 library setup
- 可运行既有 Design Compiler 脚本

- 跳过所有不受支持的 Design Compiler 命令，并发出 warning message  
要查看不支持的 Design Compiler 命令列表，请在 DC Explorer shell 提示符下输入 `help` 命令。  
DC Explorer 与 Design Compiler 的差异如下：
- DC Explorer 命令行界面(`de_shell`)默认运行于 `topographical` 模式，因此无需指定 `-topographical_mode` 选项。
- 若尝试运行 `compile_ultra`，DC Explorer 会改为调用 `compile_exploration`。  
若对 `compile_ultra` 或 `compile` 使用不受支持的选项，DC Explorer 会发出 warning 并跳过该选项。若使用 `-incremental` 选项，DC Explorer 会发出 error message，并完全跳过该命令。

– DC Explorer 会忽略或转换不支持的 Design Compiler 命令，并继续执行脚本。

- \* 对设计或 QoR 影响很小的不受支持命令会被忽略，并给出 DESH-009 warning。例如：

```
Warning: Command 'set_fix_hold' is not supported in DC Explorer.
The command is ignored.(DESH-009)
```

- \* 影响显著的不受支持命令会被忽略，并给出 DESH-008 error。例如：

```
Error: Command 'insert_dft' is not supported in DC Explorer.
(DESH-008)
```

- \* Automated chip synthesis 命令无法识别，DC Explorer 会发出 CMD-005 error message。

– DC Explorer 支持针对 pre-DFT 规则违例的测试设计规则检查 (test design rule checking)。注意下列限制：

- \* 多数 DFT 规范命令可在已映射 (mapped) 设计上运行，但不能在未映射 (unmapped) 设计上运行。
- \* 不支持的命令会按影响程度以 DESH-009 警告或 DESH-008 错误忽略。
- \* 工具不支持 post-DFT DRC，即在已 stitch 的设计上运行 `dft_drc` 命令。

有关 DFT 流程的更多信息，请参阅 TestMAX DFT 文档。

### 1.4.1 与 Design Compiler 的脚本兼容性

#### Script Compatibility With Design Compiler

在 DC Explorer 中运行 Design Compiler 脚本时，应遵循下列指南：

- 若脚本检查程序名为 `dc_shell` 及其提示符，需要将提示符从 `de_shell` 改为 `dc_shell`。为此，将变量 `de_rename_shell_name_to_dc_shell` 设为 `true`。此设置会将只读环境变量 `synopsys_program_name` 设为 `dc_shell`。
- 若要编写仅在 DC Explorer（而非 Design Compiler）中执行某些命令的脚本，可使用如下结构：

```
if {[shell_is_in_exploration_mode]} {do_DC_Explorer_setup}
```

- 要优化漏电功耗，请用 `set_multi_vth_constraint` 替换 `set_max_leakage_power`。
- 用一次 `compile_exploration` 替换多次 `compile_ultra`。

## 1.5 IC Compiler 中的 Floorplan 探索

### Floorplan Exploration in IC Compiler

默认情况下，DC Explorer 使用 topographical technology，使您能在 RTL 综合阶段准确预测 post-layout timing、面积和功耗，而无需采用基于 wire load model 的时序近似。它利用 Synopsys 的 placement 和 optimization technology 在综合过程中实现准确的时序预测，从而提高与最终物理设计的相关性，并为物理实现提供更好的起点。

您可使用 DC Explorer 为 IC Compiler 的 floorplan exploration flow 创建数据库。支持的数据库是 .ddc 或 ASCII Verilog 格式的 mapped netlist。netlist 中大多数不完整或不匹配的数据都会得到解决，但 logic library 与 physical library 之间的不一致除外；在继续执行 IC Compiler floorplan exploration 之前，必须解决这些不一致。

从 DC Explorer 到 IC Compiler 设计规划的流程概览见原书插图；有关 GUI 信息，请参阅 *Design Vision User Guide*。

#### 参见

- 第十二章「执行 Floorplan 探索」
- 第九章「使用 Floorplan 物理约束」

# Chapter 2

## 使用 DC Explorer

### *Working With DC Explorer*

DC Explorer 为综合与时序分析提供两种界面：`de_shell` command-line interface (shell) 和 Design Vision graphical user interface (GUI)。`de_shell` 是纯文本环境，您在命令行提示符下输入命令。Design Vision 是 Synopsys 综合环境的 GUI，用于可视化设计数据并分析结果。

要了解在 DC Explorer 环境中的标准操作任务，请参阅：

- 运行 DC Explorer
- 许可证 (Licensing)
- 使用多核技术
- 并行运行命令
- 在后台运行命令以缩短 runtime

## 2.1 运行 DC Explorer

### Running DC Explorer

下列各节介绍如何使用 `de_shell` 与 Design Vision GUI 运行 DC Explorer：

- 启动 DC Explorer
- 输入 `de_shell` 命令
- 中断或终止命令处理
- 在 `de_shell` 中打开或关闭 GUI
- 在命令行获取帮助
- Setup 文件
- 在日志文件中查找会话信息
- 运行 Tcl 脚本

- 退出 DC Explorer

## 2.1.1 启动 DC Explorer

### Starting DC Explorer

DC Explorer 在 Linux 的 X Window 环境下运行。启动前请确保 `$PATH` 变量包含指向 `bin` 目录的路径。在 Linux shell 中输入 `de_shell` 即可启动 DC Explorer。例如：

```
% de_shell
```

默认情况下，该命令以命令行界面（`de_shell`）启动工具。

启动时还可在命令行加入其他选项。例如，可以使用：

- `-checkout`: 访问程序默认检出功能以外的 licensed feature
- `-wait`: 为检出任何额外 license 设置等待时间上限
- `-f`: 在显示初始 `de_shell` 提示符之前执行脚本文件
- `-x`: 在启动时执行一条 `de_shell` 语句
- `-no_init`: 指定 `de_shell` 不执行任何 `.synopsys_dc.setup` startup file  
仅当要包含 command log 或其他 script file，以复现先前的 `de_shell` session 时，才使用此选项。
- `-no_home_init`: 指定不执行 home directory 中的 `.synopsys_dc.setup` startup file
- `-no_local_init`: 指定不执行 local `.synopsys_dc.setup` startup file
- `-gui`: 以 GUI 启动 `de_shell`
- `-no_gui`: 不以 GUI 启动 `de_shell`

详细选项列表见 `de_shell man` 页。

启动时，`de_shell` 执行下列任务：

1. 创建命令日志文件。
2. 读取并执行 `.synopsys_dc.setup` 文件。
3. 分别执行命令行上由 `-f` 或 `-x` 指定的脚本文件或命令。
4. 在调用 `de_shell` 的窗口中显示 program header 与 `de_shell` prompt。program header 列出站点已获得 license 的所有 feature。

#### 注意

若以路径形式输入 `de_shell` 命令，必须指定完整的 absolute path，而不能使用 relative path。若使用 relative path（如 `../`），DC Explorer 将无法访问位于 root directory 中的 library。

下列示例给出正确指示包含 DC Explorer 安装的 Synopsys 根目录的方式：

```
/tools/synopsys/2012.06/bin/de_shell
```

## 2.1.2 输入 de\_shell 命令

### Entering de\_shell Commands

您通过 `de_shell` 命令与 DC Explorer 交互；这些命令基于工具命令语言 (Tcl)，并包含实现特定 DC Explorer 功能所需的命令扩展。DC Explorer 命令语言提供与 Linux 命令 shell 类似的能力，包括变量、条件执行与控制流命令。您可以：

- 在 `de_shell>` 提示符下交互输入单条命令
- 在 Design Vision GUI 的控制台命令行上交互输入单条命令
- 运行一个或多个 Tcl 命令脚本（包含 `de_shell` 命令的文本文件）

在 `de_shell` 中输入命令的方式与标准 Linux shell 相同。GUI 打开时，可在控制台命令行输入命令，也可通过菜单界面使用 DC Explorer 命令。

在 `de_shell` 中输入命令：

1. 在命令行上键入命令。
2. 按 `Return`。

在控制台命令行上输入命令：

1. 单击控制台任意处以激活命令行。
2. 键入命令。
3. 单击 `de_shell>` 提示按钮或按 `Return`。

DC Explorer 会在 `de_shell` 与控制台日志视图中回显命令输出，包括处理消息以及警告或错误消息。

要显示某 `de_shell` 命令可用的选项，在命令行上输入命令名与 `-help`。例如，查看 `compile_exploration` 的选项：

```
de_shell> compile_exploration -help
```

输入命令、选项或文件名时，键入足以唯一标识名称的字符后可按 `Tab` 键，由 DC Explorer 补全其余字符。若已键入的字符可对应多个名称，工具会列出符合条件的名称，可用方向键和 `Enter` 键进行选择。

要从命令行界面的输出中复用命令，可选中相应文本，将指针移到 `de_shell` 命令行，再单击鼠标中键粘贴。

## 2.1.3 中断或终止命令处理

### Interrupting or Terminating Command Processing

若为命令输入了错误选项或输入了错误命令，可中断命令处理并仍留在 `de_shell` 中。要中断或终止命令，按 `Ctrl-C`。

某些命令与进程（例如 `update_timing`）无法中断。要停止这些命令或进程，必须在系统级终止 `de_shell`。终止进程或 shell 时不会保存任何数据。

按 `Ctrl-C` 时请注意：

- 若正在处理脚本文件且您中断了其中某条命令，则脚本处理被中断，后续脚本命令不再执行。

- 若在命令响应中断之前按了三次 Ctrl-C，`de_shell` 会被中断并以如下消息退出：

```
Information: Process terminated by interrupt.
```

少数例外行为记录在相应命令的 `man` 页中。

## 2.1.4 在 `de_shell` 中打开或关闭 GUI

### Opening or Closing the GUI in `de_shell`

启动命令行界面时，Linux shell 中会出现 `de_shell>` 提示符。启动命令行界面后，可使用 Design Vision GUI。

Design Vision 在 Linux 的 X Window 环境下运行。打开 GUI 前，请确保 `$DISPLAY` 环境变量设置为您 Linux 系统显示的名称。

若以无 GUI 方式启动 `de_shell`，可在 `de_shell>` 提示符下输入 `gui_start` 打开 GUI。例如：

```
de_shell> gui_start
```

也可在启动 DC Explorer 时对 `de_shell` 指定 `-gui` 选项以打开 GUI。要从 `de_shell` 会话使用 GUI，必须持有 Design Vision 许可证。

在会话中的任意时刻，都可打开或关闭 GUI 而不退出 DC Explorer。例如，若需节省系统资源，可关闭 GUI 而让 DC Explorer 以命令行界面继续运行。

要关闭 GUI 而不退出 DC Explorer，可任选其一：

- 选择 File > Close GUI
- `de_shell> gui_stop`

打开 GUI 时，它会读取一组名为 `.synopsys_dv_gui.tcl` 的 `setup` 文件。可用这些文件执行 GUI 专用设置。`.synopsys_dv_gui.tcl` 中的设置会覆盖 `.synopsys_dc.setup` 中的设置。有关 `.synopsys_dv_gui.tcl` 的更多信息，请参阅 *Design Vision User Guide*。

除读取 `setup` 文件外，GUI 还会从主目录中的 `~/synopsys_dc_gui/preferences.tcl` 加载首选项与视图设置。您不应编辑该文件。默认系统首选项面向最优工具运行，对多数设计适用。但必要时可在会话中通过 Application Preferences 对话框更改 GUI 首选项。

可设置控制 GUI 窗口文本外观的首选项，以及选择或交互操作相关命令是否出现在会话日志中。还可设置各类全局、原理图视图与版图视图的默认控件。

更多信息见 *Design Vision User Guide* 中的“Setting GUI Preferences”主题。

## 2.1.5 在命令行获取帮助

### Getting Help on the Command Line

使用 DC Explorer 时可利用下列联机信息资源：

- 命令帮助：指定 `de_shell` 命令所用选项与参数的列表，显示在 shell 中；GUI 打开时也显示在控制台日志视图中
- Man 页：显示在 shell 中；GUI 打开时也显示在控制台日志视图中
- Man 页查看器：使用 GUI 时显示命令、变量与错误消息的 man 页

要显示 `de_shell` 命令的简要说明：

```
de_shell> help command_name
```

要显示命令选项与参数列表:

```
de_shell> command_name -help
```

要在 de\_shell (以及 GUI 打开时的控制台日志视图) 中显示 man 页:

```
de_shell> man command_or_variable_name
```

要在 GUI man 页查看器中显示 man 页, 可任选其一:

- 在 GUI 控制台命令行上输入 man 后跟命令或变量名:

```
de_shell> man command_or_variable_name
```

- 在 de\_shell 或 GUI 控制台命令行上输入 gui\_show\_man\_page 后跟命令或变量名:

```
de_shell> gui_show_man_page command_or_variable_name
```

要在 GUI man 页查看器中交互浏览 man 页:

1. 选择 Help > Man Pages。  
Man 页查看器打开。
2. 选择要查看的 man 页类型: Commands、Variables 或 Messages。  
随即出现 man 页列表。
3. 选择要查看的 man 页。

## 2.1.6 Setup 文件

### Setup Files

调用 DC Explorer 时, 它会自动执行三个 setup 文件中的命令。这些文件同名, 均为 .synopsys\_dc.setup, 但位于不同目录。这些文件包含用于初始化参数和变量、声明 design library 等操作的命令。

DC Explorer 按下列顺序从三个目录读取三个 .synopsys\_dc.setup 文件:

1. Synopsys 根目录
2. 您的主目录 (home directory)
3. 当前工作目录 (即您调用 DC Explorer 的目录)

表 2.1 说明这三个 setup 文件的作用。

示例 2.1 给出 .synopsys\_dc.setup 文件示例。

```
# Define the target library, symbol library,
# and link libraries
set_app_var target_library lsi_10k.db
set_app_var synthetic_library dw_foundation.sldb
set_app_var link_library "*" $target_library $synthetic_library"
set_app_var search_path [concat $search_path ./src]
set_app_var designer "Your Name"
# Define aliases
```

表 2.1: Setup 文件

| 文件                      | 位置                                   | 作用                                                                                                                                       |
|-------------------------|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| 系统级 .synopsys_dc.setup  | Synopsys 根目录（\$SYNOPSYS/admin/setup） | 包含由 Synopsys 定义的系统变量以及面向站点全部用户的一般 DC Explorer 设置。仅系统管理员可修改此文件。<br>注 意：\$SYNOPSYS 为 DC Explorer 安装目录路径。                                   |
| 用户定义 .synopsys_dc.setup | 您的主目录                                | 包含用于定义 DC Explorer 工作环境 preference 的变量。此文件中的变量覆盖 system-wide setup 文件中的对应变量。                                                             |
| 设计专用 .synopsys_dc.setup | 启动 DC Explorer 的工作目录                 | 包含影响该目录中所有设计优化的 project-specific 或 design-specific 变量。要使用此文件，必须从该目录调用 DC Explorer。此文件中定义的变量覆盖 user-defined 和 system-wide setup 文件中的对应变量。 |

```
alias h history
alias rc "report_constraint -all_violators"
```

代码 2.1: .synopsys\_dc.setup 文件

## 2.1.7 在日志文件中查找会话信息

### Finding Session Information in the Log Files

可在下列日志文件中查找会话信息（例如已处理的 `de_shell` 命令与已访问的文件）：

- 命令日志文件（Command Log Files）
- 编译日志文件（Compile Log Files）
- 文件名日志文件（Filename Log Files）

#### 命令日志文件

##### Command Log Files

命令日志文件记录 DC Explorer 处理的 `de_shell` 命令，包括 `setup` 文件命令与变量赋值。默认情况下，DC Explorer 将命令日志写入调用 `de_shell` 的目录中名为 `command.log` 的文件。

可在 `.synopsys_dc.setup` 文件中用 `sh_command_log_file` 变量更改 `command.log` 的文件名。应在启动 DC Explorer 之前更改此变量。若用户定义或项目专用的 `.synopsys_dc.setup` 不含该变量，DC Explorer 会自动创建 `command.log` 文件。

每次 DC Explorer 会话都会覆盖命令日志文件。要保存命令日志，请移动或重命名它。可用命令日志文件来：

- 为特定综合策略生成脚本
- 记录设计探索过程
- 记录所遇到的问题

#### 编译日志文件

##### Compile Log Files

每次编译设计时，DC Explorer 会生成 ASCII 日志、HTML 日志文件以及 `.DE_log_snapshot_date_process_id` 目录：

- **ASCII log**

该 log 在屏幕上显示每次 DC Explorer 运行的输出（例如已处理的命令与 error message），便于快速查看和调试。日志末尾的摘要显示有多少消息被重定向到 `.DE_log_snapshot_date_process_id` 目录下各具体日志文件。

- **HTML log file**

该文件位于当前工作目录下，以 HTML 格式包含 ASCII 日志的完整内容以及全部被重定向的消息。文件末尾的摘要表按消息 ID 分组给出所有消息出现情况的概览；单击消息 ID 链接即可显示该消息。默认文件名为 `default.html`。要更改文件名，请设置 `de_log_html_filename` 变量。例如：

```
de_shell> set_app_var de_log_html_filename test1.log.html
```

### 注意

必须安装 Python 才能生成 HTML log file。下载信息见: <https://www.python.org/download>  
默认情况下, 工具将非关键 warning 和 information message 从 ASCII log 重定向到 HTML log file。  
要禁用重定向, 将 `de_log_redirect_enable` 变量设为 `false`。

#### • `.DE_log_snapshot_date_process_id` 目录

该目录存储从 ASCII 日志重定向出的信息与警告消息。错误消息显示在 ASCII 日志中, 并保存在 HTML 日志文件中。该目录包含下列日志文件:

- read\_design\_log
- read\_library\_log
- timing\_message\_log
- power\_message\_log
- ungroup\_hierarchies\_log
- register\_removal\_log
- optimization\_message\_log

文件名日志文件

### Filename Log Files

DC Explorer 将已读取文件的名称写入调用 `de_shell` 的目录中的文件名日志文件。若 DC Explorer 异常终止, 可用该文件识别复现错误所需的数据文件。要指定文件名日志文件的名称, 在 `.synopsys_dc.setup` 中设置 `filename_log_file` 变量。

## 2.1.8 运行 Tcl 脚本

### Running Tcl Scripts

可用 Tcl 脚本来完成例行、重复或复杂任务。将一系列 `de_shell` 命令放入文本文件即可创建命令脚本文件。脚本文件中可执行任意 `de_shell` 命令。要从 `de_shell` 命令行运行脚本文件, 输入:

```
de_shell> source file_name
```

在 Tcl 中, 行首的井号 (#) 表示注释。例如:

```
# This is a comment
```

要在 GUI 中运行脚本, 使用 File > Execute Scripts 对话框。

更多信息另见 *Using Tcl With Synopsys Tools* 文档。

## 2.1.9 退出 DC Explorer

### Exiting DC Explorer

可随时退出 DC Explorer 并返回操作系统。默认情况下, `de_shell` 将会话信息保存在 `command.log` 文件中。但若在启动 DC Explorer 之后更改 `sh_command_log_file` 的文件名, 会话信息可能会丢失。此外, `de_shell` 不会自动保存已加载到内存中的设计。若要在退出前保存这些设计, 请使用 `write_file` 命令。例如:

```
de_shell> write_file -format ddc -hierarchy -output my_design.ddc
```

要退出 `de_shell`, 可任选其一:

- 输入 `quit`。
- 输入 `exit`。
- 若以交互模式运行且工具正忙, 按 `Ctrl-d`。

要在 Design Vision GUI 中退出 DC Explorer, 选择 `File > Exit`。

## 2.2 许可证

### Licensing

下列各设计流程适用特定许可证要求:

- DC Explorer
  - RTL-Exploration 许可证
  - DC-Explorer-Shell 许可证
- DC Explorer floorplan 探索
  - 一套 DC Explorer 许可证、一份 DC-Extension 许可证以及一份 ICC-DP 许可证;  
或者
  - 两套 DC Explorer 许可证与两份 DC-Extension 许可证
- Pre-DFT 设计规则检查
  - TestMAX DFT 许可证

Design Vision (GUI) 和 DC-Extension license 会按需自动检出。

#### 注意

使用 `multicore processing` 时, 每两个 `core` 需要一份 `license`。要移除 `multicore processing` 期间额外检出的 `license`, 请使用 `remove_license` 命令。

本节包括以下主题:

- 列出正在使用的许可证
- 获取许可证

- 释放许可证
- 启用许可证排队

## 2.2.1 列出正在使用的许可证

### Listing the Licenses in Use

要查看当前已检出的许可证，使用 `list_licenses` 命令。例如：

```
de_shell> list_licenses
Licenses in use:
    DC-Explorer-Shell
    Design-Vision
    RTL-Exploration
1
```

要显示哪些 license 已被检出，使用 `license_users` 命令。例如：

```
de_shell> license_users
jack@eng1 RTL-Exploration
john@eng2 RTL-Exploration, Design-Vision
2 users listed.
```

## 2.2.2 获取许可证

### Obtaining Licenses

调用 DC Explorer 时，Synopsys Common Licensing 软件会自动检出相应许可证。例如，若读入 HDL 设计描述，Synopsys Common Licensing 会为相应的 HDL Compiler 检出许可证。

可在 `de_shell` 或 GUI 中获取许可证。要从 `de_shell` 会话启动 GUI，必须持有 Design Vision 许可证。

- 使用 `de_shell`

若已知所需工具与接口，可用 `get_license` 命令检出这些许可证，以确保准备使用时各许可证可用。默认情况下，每个 feature 仅检出一份 license。license 检出后会一直保留，直到您释放它或退出 `de_shell`。

例如，下列命令为 HDL Compiler 功能检出一份许可证：

```
de_shell> get_license HDL-Compiler
```

对于可能需要多份许可证的多核处理，可对 `get_license` 使用 `-quantity` 选项指定所需许可证总数。若已检出部分许可证，该命令仅再获取使总数达到指定数量所需的额外许可证。

- 使用 GUI

- 要显示当前许可证信息，选择 File > Licenses。

出现 Application Licenses 对话框。Allocated Licenses 列表显示当前正在使用的许可证；Available Licenses 列表显示其他可用许可证。

- 要检出额外许可证，选择 **File > Licenses**，在 **Available Licenses** 列表中单击许可证名，再单击 **Allocate**。
- 若有可用许可证，工具会检出一份副本；若许可证均已被占用，则显示错误消息。

#### 参见

- 列出正在使用的许可证
- 释放许可证

### 2.2.3 释放许可证

#### Releasing Licenses

可在 `de_shell` 或 GUI 中释放许可证。要从 `de_shell` 会话启动 GUI，必须持有 Design Vision 许可证。

要释放已检出到您名下的许可证，使用 `remove_license` 命令。例如：

```
de_shell> remove_license HDL-Compiler
```

可用 `-keep` 选项指定命令完成后为每个 feature 保留的 license 数量。若不指定该选项，运行 `remove_license` 时会释放每个 feature 的所有 license。

要在 GUI 中释放许可证：

1. 选择 **File > Licenses**。
2. 在 **Allocated Licenses** 列表中选择一份许可证。
3. 单击 **Release**。

#### 参见

- 列出正在使用的许可证
- 获取许可证

### 2.2.4 启用许可证排队

#### Enabling License Queuing

DC Explorer 提供 license queuing 功能：当所有 license 均在使用时，应用程序可等待 license 变为可用。要启用该功能，将环境变量 `SNPSLMD_QUEUE` 设为 `true`。将显示如下消息：

```
Information: License queuing is enabled. (DCSH-18)
```

启用许可证排队后，可能出现您持有许可证 L1 却在等待 L2，而另一用户持有 L2 却在等待 L1 的情况；此时双方将一直等待。

为防止此类情况，请使用 `SNPS_MAX_WAITTIME` 与 `SNPS_MAX_QUEUEUETIME` 环境变量。在使用这两个变量之前，必须先将 `SNPSLMD_QUEUE` 设为 `true`。

- `SNPS_MAX_WAITTIME` 指定您所需第一份关键许可证的最大等待时间。

- `SNPS_MAX_QUEUE_TIME` 指定在同一 `de_shell` 进程中检出后续许可证的最大等待时间。在成功检出第一份许可证以启动 `de_shell` 之后使用该变量。

当您将设计跑过综合流程时，排队功能可能显示其他状态消息，例如：

```
Information: Started queuing for feature 'HDL-Compiler'. (DCSH-15)
Information: Still waiting for feature 'HDL-Compiler'. (DCSH-16)
Information: Successfully checked out feature 'HDL-Compiler'. (DCSH-14)
```

## 2.3 使用多核技术

### Using Multicore Technology

DC Explorer 的 multicore technology 支持使用多个 core 来缩短工具 runtime。在综合期间，该技术可将大型优化任务划分为较小任务，以便在多个 core 上处理。

#### 2.3.1 启用多核处理

##### Enabling Multicore Processing

要在 DC Explorer 中启用多核处理，使用 `set_host_options` 命令。例如，要使工具使用六个核运行进程，输入：

```
de_shell> set_host_options -max_cores 6
```

所有 `compile_exploration` 命令选项均支持使用多个 core 进行优化。

启用多核处理后，日志文件会包含类似如下的信息消息：

```
Information: Running optimization using a maximum of 6 cores. (OPT-1500)
```

为获得最高效率，`-max_cores` 设置不应大于实际可用 core 数。

设置 `compile_adjust_max_processes_used` 变量时，工具按该变量的下列取值运行多核进程：

- 1: 默认情况下，若 `overcommit_memory` 参数设为 2，工具会在 multiprocessing 期间调整 core 数。若资源不足以并发运行新进程：
  - `compile` 或 `compile_ultra` 会调整 core 数并继续执行，而不是终止进程。但是，当且仅当至少有一个新进程成功启动时，命令才会继续执行。
  - `parallel_execute` 会减少要用的进程数；若第一个新进程失败，则在主进程中继续执行。
  - `redirect -bg` 即使新进程失败，也会在主进程中继续执行。
- 2: 无论 `overcommit_memory` 参数设置如何，都调整核数。
- 0: 不调整核数。

#### 参见

调整多核处理期间的核数（下文）。

## 2.3.2 报告运行时间

### Reporting Runtime

要报告总体 compile wall clock time, 请运行 `report_qor` 命令, 如下例所示。该命令报告 `compile_exploration` 的 wall clock time。

```
de_shell > report_qor
*****
Report : qor
Design : TEST_TOP
...
*****
...
Hostname: machine
Compile CPU Statistics
-----
...
-----
Overall Compile Time:          631.32
Overall Compile Wall Clock Time: 288.11
```

所报告的挂钟时间与下列脚本报告的时间类似:

```
de_shell> set_host_options -max_cores 2
de_shell> set pre_compile_clock [clock seconds]
de_shell> compile_exploration
de_shell> set post_compile_clock [clock seconds]
de_shell> set diff_clock [expr $post_compile_clock - $pre_compile_clock]
```

衡量 multicore optimization 的 runtime speedup 时, 应使用进程的 wall clock time。CPU time 不是衡量 multicore runtime speedup 的正确指标。

#### 参见

报告结果质量 (QoR)。

## 2.3.3 调整多核处理期间的核数

### Adjusting the Number of Cores During Multicore Processing

要在 `overcommit_memory` 参数设为 2 时, 默认允许 Design Compiler 工具在多核处理期间调整核数, 请使用 `compile_adjust_max_processes_used` 变量。该变量允许工具通过发出下列消息继续执行:

```
UIO-306 (Warning) There is insufficient memory to launch all jobs,
reducing the number of
parallel processes to %d for this session. Tool performance may be
impacted. Please set
overcommit_memory to 0.
```

若未使用 `compile_adjust_max_processes_used` 变量且将 `overcommit_memory` 参数设为 2，操作系统可能对多进程程序错误地使用 `copy-on-write` 优化策略，为所有进程增加虚拟内存，即使有空闲内存也不分配。这会导致正在运行的进程或创建新进程时被阻塞，并使工具崩溃，发出如下消息：

```
OPT-1603 (error) Insufficient virtual memory. Please increase swap space,
or reduce other processes on this host.
```

### 参见

启用多核处理（上文）。

## 2.4 并行运行命令

### Running Commands in Parallel

在脚本中串行执行检查或报告命令可能占用总体运行时间的很大一部分。为缩短 `runtime` 并生成相同的报告，可使用 `parallel command execution` 并行运行这些命令。

- `parallel_execute` 命令
- 许可证要求
- 多核处理
- 支持的命令

#### 2.4.1 `parallel_execute` 命令

##### The `parallel_execute` Command

要启用并行命令执行，在脚本中用 `parallel_execute` 命令列出要并行执行的检查或报告命令。若列表中的某命令需要此前未执行过的时序更新，该命令会在并行命令执行期间自动调用 `update_timing`。

例如，下列脚本先更新时序信息，然后使用八个核并行执行 `report_timing`、`report_qor`、`report_cell` 与 `report_area`：

```
set_host_options -max_cores 8
update_timing
parallel_execute [list \
  "report_timing > $mylogfile" \
  "report_qor >> $mylogfile" \
  "report_cell" \
  "report_area"]
```

下列脚本在命令字符串中对选项参数与日志文件使用变量（包括 `$MAX`、`$NWORST`、`$cstr_log` 与 `$qor_log`）：

```
set MAX 10000; set_app_var NWORST 10
set rpt_const_options "-all_violators"
set cstr_log "rpt_cstr.log"
set qor_log "rpt_qor.log"
parallel_execute [list \
```

```
"report_timing -max $MAX -nworst $NWORST > rpt_tim.log" \
"report_constraints $rpt_const_options > $cstr_log" \
"report_qor > $qor_log"]
```

## 2.4.2 许可证要求

### License Requirements

对使用多核处理的并行命令执行，适用与串行运行命令相同的许可证要求。默认情况下，每八个 core 需要一套 DC Explorer license。

## 2.4.3 多核处理

### Multicore Processing

parallel command execution 最多可使用八个 core。一次可并行执行的最大命令数由 set\_host\_options 的 -max\_cores 选项决定。若指定大于八的数字，工具会发出 warning，并以可用 core 数覆盖该设置。工具会阻塞 de\_shell，直到并行执行列表中运行时间最长的命令完成。

下列示例使用三个核运行并行命令执行：

```
de_shell> parallel_execute [list \
    "report_cell" "report_timing" "report_area"]
Information: Running parallel report using a maximum of 3 cores.
(RPT-100)
```

下列示例为并行命令执行指定八个核：

```
de_shell> set_host_options -max_cores 8
```

## 2.4.4 支持的命令

### Supported Commands

仅对报告与检查命令使用并行命令执行。要查明支持哪些命令，对 parallel\_execute 指定 -list\_all 选项。若指定不支持的命令，工具会跳过该命令并发出类似如下的警告：

```
de_shell> parallel_execute [list report_libcell_subset]
Warning: 'report_libcell_subset' report command can't run in
parallel_execute mode.
(RPT-106)
```

## 2.4.5 在设计流程中

### In the Design Flow

下图说明如何在 DC Explorer 设计流程中并行运行检查与报告命令以改善运行时间。

参见

并行运行命令（本节上文）。

(原书 *Figure 2: Parallel Command Execution in the DC Explorer Design Flow*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 2.1: DC Explorer 设计流程中的并行命令执行

## 2.5 在后台运行命令以改善运行时间

### Running Commands in the Background to Improve Runtime

要缩短 DC Explorer runtime, 可对 `redirect` 命令使用 `-bg` 选项, 在前台运行其他命令的同时, 在后台运行命令。可运行单条命令、`source` 一个 Tcl 脚本以并行运行若干命令, 或运行一个 Tcl 过程。

要用 `redirect -bg` 指定是在后台运行只读命令还是非只读命令, 对 `enable_redirect_bg_commands` 使用下列选项:

- `-all` (默认): 允许工具用 `redirect -bg` 在后台运行只读与非只读命令。
- `-read_only`: 仅用 `redirect -bg` 在后台运行只读命令; 所有其他 (非只读) 命令在前台运行。若对 `-read_only` 指定了非只读命令, 工具发出 BGE-011 信息消息。

运行后台作业时必须指定要用的核数。`redirect -bg` 可用的核数从 `set_host_options` 设置的核数中扣除。使用 `redirect` 运行后台作业时, `de_shell` 立即返回以执行下一条命令。

下列示例将 Tcl 脚本重定向到后台运行, 并使用 GUI 进行分析:

```
set_host_options -max_cores 8
read_verilog
read_sdc
...
compile_exploration
redirect -bg -max_cores 3 -file y.out {source y.tcl}
remote_execute_source1.log {source bg_script.tcl}
Information: redirect -bg with max_cores 3 started. The maximum number
of cores available in parent is reduced to 5. (BGE-004)
start_gui
```

同一时间最多只能有两个活动的后台作业。若再重定向第三条命令或脚本, 工具会等待先前命令完成后再启动新作业。

支持的命令仅包括报告、分析与设计导出命令。会更改网表或其他数据的命令不受支持。

也可在 `redirect -bg` 命令内部运行 `parallel_execute`, 以并行运行一组检查与报告命令。

### 2.5.1 报告用 `redirect` 提交的作业

#### Reporting Jobs Submitted With the `redirect` Command

要报告您已用 `redirect -bg` 提交到后台运行的全部作业, 使用 `report_background_jobs` 命令。该命令报告已完成作业以及当前正在后台运行的作业, 如下例所示:

```
de_shell> report_background_jobs
```

```
JOB 'redirect -bg -file {background.log} source bg_script.tcl  
-max_cores 4 ' completed  
JOB(pid:13010) 'redirect -bg -file {background_1.log} source  
bg_script_1.tcl -max_cores 3 ' is running
```

要省略已完成作业，对 `report_background_jobs` 使用 `-reset` 选项。

# Chapter 3

## 设置库

### *Setting Up the Libraries*

DC Explorer 使用 technology library、symbol library 以及 synthetic library 或 DesignWare library 来实现综合，并以图形方式显示综合结果。必须正确设置这些 library，DC Explorer 才能正确使用其中的数据。

下列各节说明如何设置这些库并执行简单的库命令，以便 DC Explorer 正确使用库数据：

- 库要求
- 指定逻辑库
- 指定物理库
- 在网表或 DEF 文件中映射物理变体单元
- 使用库
- 目标库子集（Target Library Subsets）
- 使用 TLUPlus 或 nxtgrd 文件进行 RC 估算
- 对黑盒（black boxes）的支持

### 3.1 库要求

#### **Library Requirements**

DC Explorer 使用下列库：

- 逻辑库（Logic Libraries）
- 符号库（Symbol Libraries）
- DesignWare 库
- 物理库（Physical Libraries）

### 3.1.1 逻辑库

#### Logic Libraries

Logic library 由半导体厂商维护和发布，包含每个 cell 的特性与功能信息，例如 cell name、pin name、面积、delay arc 和 pin loading。Logic library 还定义必须满足的条件，例如 net 的 maximum transition time；这些条件称为 design rule constraints。此外，logic library 还为特定工艺指定 operating conditions 和 wire load models。

DC Explorer 支持使用 nonlinear delay model (NLDM)、Composite Current Source (CCS) model (compact 或 noncompact)，或同时包含 NLDM 和 CCS model 的 logic library。工具根据 logic library 的内容自动选择 timing model 类型。若 library 同时包含 NLDM 和 CCS model，DC Explorer 使用 CCS model。在 logic synthesis 和 preroute optimization 期间，工具可能不会使用所有可用的 CCS 数据，以缩短 runtime。

DC Explorer 要求逻辑库为 .db 格式。多数情况下，半导体厂商会提供 .db 格式库。若仅提供库源码，请参阅 Library Compiler 文档以了解如何生成 .db 格式逻辑库。设置逻辑库见「指定逻辑库」。

DC Explorer 将逻辑库用于下列目的：

- 实现设计功能

优化期间 DC Explorer 映射到的 logic library 称为 target library。Target library 包含用于生成 netlist 的 cell，以及设计 operating conditions 的定义。用于 compile 或 translate 设计的 target library 会成为该设计的 local link library。DC Explorer 将此信息保存在设计的 local\_link\_library 属性中。

- 解析单元引用

DC Explorer 用于解析 cell reference 的 logic library 称为 link library。Link library 包含 mapped netlist 中 library cell 和 subdesign 的描述，也可包含 design file。Link library 包括 local\_link\_library 属性中定义的 local link library，以及 link\_library 变量中指定的 system link library。

- 计算时序值与路径延时

链接库定义用于计算时序值与路径延时的延时模型。有关各类延时模型，见 Library Compiler 文档。

- 计算功耗

有关功耗计算，见 *Power Compiler User Guide*。

DC Explorer 将 link\_library 变量中找到的第一个 logic library 作为 main library。若其他 library 的度量单位与 main library 不同，DC Explorer 会将所有单位转换为 main library 中指定的单位。DC Explorer 从主库获取下列默认值与设置：

- 单位定义
- 工作条件
- K 因子 (K-factors)
- 输入与输出电压
- 时序范围 (timing ranges)
- RC slew trip points
- 网络 transition 时间退化表 (net transition time degradation tables)

逻辑库设置包含目标库与链接库：

- 目标库

映射期间，DC Explorer 从目标库中选择功能正确的门以构建电路，并使用厂商提供的这些门的时序数据计算电路时序。

要用 `target_library` 变量指定目标库。应仅指定希望 DC Explorer 用于映射设计中 standard cell（如组合逻辑和寄存器）的 standard cell library。不应指定任何 DesignWare library 或 macro library（如 pad 或 memory）。

- 链接库

要使设计完整，设计中的全部单元实例必须链接到所引用的库组件与设计。该过程称为链接设计（linking the design）或解析引用（resolving references）。为解析引用，DC Explorer 使用由下列变量与属性设置的链接库：

- 系统变量 `link_library` 列出 DC Explorer 用于解析引用的库与设计文件。

DC Explorer 从左到右搜索 `link_library` 中列出的文件，找到引用后即停止搜索。在 `link_library` 中指定星号，表示 DC Explorer 在内存中已加载的 library 中搜索 reference。例如，若将 `link_library` 设为 `{"*" lsi_10k.db}`，工具先在内存中搜索，再在 `lsi_10k` 库中搜索。

- `local_link_library` 属性列出在链接过程中添加到 `link_library` 变量开头的设计文件与库。解析引用时，DC Explorer 首先搜索该属性中的文件。可用 `set_local_link_library` 命令设置此属性。

- `search_path` 变量指定目录路径列表：当您仅指定不含路径的普通文件名时，工具用其查找逻辑库及其他文件。该变量还指定 DC Explorer 搜索完 link library 后，可继续搜索 unresolved reference 的路径。

### 3.1.2 符号库

#### Symbol Libraries

Symbol library 由半导体厂商维护和发布，包含 design schematic 中表示 library cell 的图形符号定义。生成 design schematic 时，DC Explorer 将 netlist 中的 cell 一对一映射到 symbol library 中的 cell。要查看 design schematic，请使用 Design Vision。

### 3.1.3 DesignWare 库

#### DesignWare Libraries

DesignWare library 是可重用 intellectual property (IP) building block 或 component 的集合，并紧密集成到 Synopsys 综合环境中。DesignWare 组件可用于实现许多内置 HDL 运算符（如 +、-、\*、<、>、<=、>=）以及由 if 与 case 语句定义的运算。

可用 DesignWare Developer 开发额外 DesignWare 库，也可从 Synopsys 或第三方许可 DesignWare 库。要使用已许可的 DesignWare 组件，需要相应组件的许可证密钥。

### 3.1.4 物理库

#### Physical Libraries

若要用 DC Explorer 生成可在 IC Compiler design planning 中启动物理探索的早期 netlist, 除 logic library 外, 还需指定 physical library。使用 Milkyway design library 指定 physical library, 并以 Milkyway 格式保存设计。

创建 Milkyway 设计库所需的输入包含 Milkyway 参考库与 Milkyway 工艺文件:

- **Milkyway 参考库**

Milkyway 参考库包含标准单元与宏的物理表示。在 topographical 模式下, Milkyway 参考库使用 FRAM 抽象视图存储信息。Reference library 还定义 placement unit tile, 即最小可布局 instance 的宽度和高度, 以及 routing direction。

- **Milkyway 工艺文件**

Milkyway 工艺文件 (.tf) 包含布线设计所需的工艺专用信息。若您的 Milkyway 库文件缺少布线层方向信息, DC Explorer 会自动推导。推导出的布线层方向保存在 .ddc 文件中。可用 set\_preferred\_routing\_direction 命令覆盖推导的布线层方向。要用 report\_preferred\_routing\_direction 报告全部布线方向。

更多信息见 *Power Compiler User Guide*。

#### 参见

- 指定逻辑库
- 指定物理库
- 指定库搜索路径
- 链接设计

## 3.2 指定逻辑库

#### Specifying Logic Libraries

表 3.1 列出控制各类库读取的变量及典型文件名。用这些变量指定逻辑库与 DesignWare 库。

表 3.1: 库变量

| 库类型          | 变量                | 默认                  | 扩展名   |
|--------------|-------------------|---------------------|-------|
| 目标库          | target_library    | "your_library.db"   | .db   |
| 链接库          | link_library      | "* your_library.db" | .db   |
| DesignWare 库 | synthetic_library | (无)                 | .sldb |

要设置对逻辑库的访问, 必须指定目标库与链接库。

在下列示例中, 目标库是第一个链接库。为简化链接库定义, 示例包含用户定义变量 additional\_link\_lib\_files 用于 pad 与宏等库。

```
de_shell> set_app_var target_library [list_of_standard_cell_libraries]
de_shell> set_app_var synthetic_library [sldb_files_for_designware]
de_shell> set additional_link_lib_files [additional_libraries]
de_shell> set_app_var link_library [list * $target_library \
    $additional_link_lib_files $synthetic_library]
```

若执行 technology translation, 请将现有 mapped gate 所使用的 standard cell library 加入 link library, 并将转换目标 standard cell library 加入 target library。

### 3.2.1 指定 DesignWare 库

#### Specifying the DesignWare Libraries

无需指定实现内置 HDL 运算符的标准综合库 `standard.sldb`; DC Explorer 会自动使用该库。若使用额外 DesignWare 库, 必须用 `synthetic_library` 变量 (用于优化) 与 `link_library` 变量 (用于单元引用) 指定这些库。例如, 要使用 DesignWare minPower component, 请在 `synthetic_library` 的设置中包含 `dw_minpower.sldb` library。此操作不会执行功耗优化。

有关使用 DesignWare 库的更多信息, 见 DesignWare 文档。

#### 参见

- 库要求
- 指定最小时序库
- 指定库搜索路径

### 3.2.2 指定库搜索路径

#### Specifying a Library Search Path

可用完整路径或仅用文件名指定库位置。若仅指定文件名, DC Explorer 使用 `search_path` 变量中定义的搜索路径定位库文件。默认情况下, 搜索路径包括当前工作目录与 `$SYNOPSIS/libraries/syn`, 其中 `$SYNOPSIS` 为安装目录路径。DC Explorer 从 `search_path` 中最左侧目录开始查找库文件, 并使用找到的第一个匹配库文件。

例如, 假设 `lib` 与 `vhdl` 目录中都有名为 `my_lib.db` 的逻辑库。DC Explorer 使用 `lib` 目录中的 `my_lib.db`, 因为它先找到 `lib` 目录。

```
de_shell> set_app_var search_path "lib vhdl default"
```

可用 `which` 命令按 DC Explorer 找到的顺序列出库文件。

```
de_shell> which my_lib.db
/usr/lib/my_lib.db, /usr/vhdl/my_lib.db
```

#### 参见

- 库要求
- 指定逻辑库

### 3.2.3 指定最小时序库

#### Specifying Minimum Timing Libraries

若同时进行最小与最大时序分析，`link_library` 变量指定的逻辑库会同时用于最大与最小时序信息。要指定单独的最小时序库，使用 `set_min_library` 命令。该命令将最小时序库与 `link_library` 中指定的最大时序库关联。例如：

```
de_shell> set_app_var link_library "* maxlib.db"
de_shell> set_min_library maxlib.db -min_version minlib.db
```

要查明哪些库被设为最小与最大库，使用 `list_libs` 命令。在生成的报告中，小写字母 `m` 出现在最小库旁，大写字母 `M` 出现在最大库旁。

参见

库要求。

## 3.3 指定物理库

#### Specifying Physical Libraries

创建 Milkyway 设计库所需的输入为 Milkyway 参考库与 Milkyway 工艺文件。

要创建 Milkyway 设计库：

1. 定义电源与地网络。例如，设置下列变量：

```
de_shell> set_app_var mw_power_net VDD
de_shell> set_app_var mw_ground_net VSS
de_shell> set_app_var mw_logic1_net VDD
de_shell> set_app_var mw_logic0_net VSS
de_shell> set_app_var mw_power_port VDD
de_shell> set_app_var mw_ground_port VSS
```

若不设置这些变量，执行 `write_milkyway` 时不会建立 power 和 ground connection；power net 和 ground net 可能会被转换为 signal net。

2. 用 `create_mw_lib` 命令创建 Milkyway 设计库。例如：

```
de_shell> create_mw_lib -technology $mw_tech_file \
    -mw_reference_library $mw_reference_library $mw_design_library_name
```

3. 用 `open_mw_lib` 命令打开刚创建的 Milkyway 库。例如：

```
de_shell> open_mw_lib $mw_design_library_name
```

4. (可选) 用 `set_tlu_plus_files` 命令附加 TLUPlus 文件。例如：

```
de_shell> set_tlu_plus_files -max_tluplus $max_tlu_file \
    -min_tluplus $min_tlu_file -tech2itf_map $prs_map_file
```

5. 在后续会话中，使用 `open_mw_lib` 打开 Milkyway 库。若使用 TLUPlus 文件进行 RC 估算，用 `set_tlu_plus_files` 附加这些文件。例如：

```
de_shell> open_mw_lib $mw_design_library_name
de_shell> set_tlu_plus_files -max_tluplus $max_tlu_file \
    -min_tluplus $min_tlu_file -tech2itf_map $prs_map_file
```

还支持下列 Milkyway 库命令：`copy_mw_lib`、`close_mw_lib`、`report_mw_lib`、`current_mw_lib` 与 `check_tlu_plus_files`。

#### 参见

- 库要求
- 使用 Milkyway 数据库
- 使用 TLUPlus 或 `nxtgrd` 文件进行 RC 估算

## 3.4 在网表或 DEF 文件中映射物理变体单元

### Mapping of Physical Variant Cells in Netlists or DEF Files

DC Explorer 可将网表或 DEF 文件中的物理变体单元（physical variant cells）映射到库中的主单元（master cell）。Physical variant cell 与 master cell 的差异在于 mask color 或细微的物理几何形状。对于先进工艺节点，这些 cell 的 timing characteristic 不会发生变化，因此 master cell 及其 variant 可使用同一个 Liberty model。

为高效管理库，需用 Liberty 语法中的 `physical_variant_cells` 属性定义主库单元的物理变体单元列表。读取包含物理变体单元的网表或 DEF 文件时，DC Explorer：

- 通过搜索 `physical_variant_cells` 属性指定的列表来解析物理变体单元引用
- 将物理变体单元映射到其主单元，并在整个设计流程中保留这些单元

要启用此功能，将 `link_allow_physical_variant_cells` 变量设为 `true`。默认值为 `false`。

## 3.5 使用库

### Working With the Libraries

可用简单库命令执行下列任务：

- 加载库
- 列出库
- 报告库内容
- 指定库对象
- 从目标库中排除单元
- 验证库一致性

- 从内存中移除库
- 保存库

### 3.5.1 加载库

#### Loading Libraries

DC Explorer 使用二进制库（逻辑库为 `.db` 格式，符号库为 `.sdb` 格式），并在需要时自动加载。要手动加载二进制库，使用 `read_file` 命令。例如：

```
de_shell> read_file my_lib.db
de_shell> read_file my_lib.sdb
```

若 library 不是相应的 binary format，请使用 `read_lib` 命令编译 library source。`read_lib` 命令需要 Library-Compiler license。

### 3.5.2 列出库

#### Listing Libraries

DC Explorer 按名称引用已加载到内存中的库。库源中的 `library` 语句定义库名。要用 `list_libs` 命令列出内存中已加载库的名称。例如：

```
de_shell> list_libs
Logical Libraries:
Library      File      Path
-----
my_lib       my_lib.db  /synopsys/libraries
my_symbol_lib my_lib.sdb /synopsys/libraries
```

### 3.5.3 报告库内容

#### Reporting Library Contents

要用 `report_lib` 命令报告库内容。该命令报告下列信息：

- 库单位
- 工作条件
- 单元（包括单元排除、偏好及其他属性）

### 3.5.4 指定库对象

#### Specifying Library Objects

Library object 是 vendor-specific cell 及其 pin。指定 library object 时，使用下列命名约定：

```
[file:]library/cell[/pin]
```

其中 **file** 为逻辑库文件名，**library** 为内存中已加载库的名称，**cell** 为库单元，**pin** 为单元引脚。若内存中加载了多个同名库，必须指定文件名。

例如，要在 **my\_lib** 库的 **AND4** 单元上设置 **dont\_use** 属性，输入：

```
de_shell> set_dont_use my_lib/AND4
```

要在 **my\_lib** 库 **AND4** 单元的 **Z** 引脚上设置 **disable\_timing** 属性，输入：

```
de_shell> set_disable_timing [get_pins my_lib/AND4/Z]
```

### 3.5.5 从目标库中排除单元

#### Excluding Cells From the Target Libraries

当 DC Explorer 将设计映射到逻辑库时，它从该库中选择库单元。要指定优化期间要从目标库排除的单元，使用 **set\_dont\_use** 命令。

例如，要阻止 DC Explorer 使用 **INV\_HD** 高驱 inverter，输入：

```
de_shell> set_dont_use MY_LIB/INV_HD
```

该命令仅影响当前加载到内存中的 **library** 副本，对磁盘上的版本没有影响。但是，如果保存该 **library**，排除设置也会被保存，相关 **cell** 将被永久排除。

要移除由 **set\_dont\_use** 设置的 **dont\_use** 属性，使用 **remove\_attribute** 命令。例如：

```
de_shell> remove_attribute MY_LIB/INV_HD dont_use  
MY_LIB/INV_HD
```

### 3.5.6 验证库一致性

#### Verifying Library Consistency

Logic library 与 physical library 之间的一致性对获得良好结果至关重要。处理设计之前，请运行 **check\_library** 命令以确保 **library** 一致。

```
de_shell> check_library
```

默认情况下，**check\_library** 在 **link\_library** 变量指定的逻辑库与当前 Milkyway 设计库中的物理库之间执行一致性检查。也可分别用 **-logic\_library\_name** 或 **-mw\_library\_name** 显式指定逻辑库或 Milkyway 参考库。若显式指定库，它们会覆盖默认库。

可用 **set\_check\_library\_options** 命令为 **check\_library** 设置选项，以执行各类逻辑库与物理库检查，例如总线命名风格、各单元面积等。

要查看已启用的库一致性检查，对 **report\_check\_library\_options** 指定 **-logic\_vs\_physical** 选项。

### 3.5.7 从内存中移除库

#### Removing Libraries From Memory

要从 **de\_shell** 内存中移除库，使用 **remove\_design** 命令。若内存中加载了多个同名库，必须同时指定路径与库名。用 **list\_libs** 命令查看内存中每个库的路径。

### 3.5.8 保存库

#### Saving Libraries

`write_lib` 命令以 Synopsys 数据库或 VHDL 格式保存（写入磁盘）已编译库。

参见

- 库要求
- 指定逻辑库
- 指定库搜索路径

### 3.6 目标库子集

#### Target Library Subsets

默认情况下,优化期间工具可从 `target library` 中选择任意 `library cell`。要将 `library cell` 限制为 `target library` 的一个 `subset`,或针对特定 `design block` 过滤 `target library cell`,请使用 `set_target_library_subset` 命令。例如,可在优化期间对某些设计块从目标库中排除特定双高度单元。

使用 `set_target_library_subset` 时请注意:

- Subset restriction 仅适用于优化期间新建或映射的 `cell`。除非工具在优化期间修改已 `mapped cell`, 否则该限制不影响这些 `cell`。
- 该命令将 `subset restriction` 应用于指定 `design` 及其 `hierarchy` 中的 `subdesign`, 但已设置其他 `subset restriction` 的 `subdesign` 除外。
- 较低 `hierarchy level` 指定的 `subset` 会覆盖较高 `level` 指定的任何 `subset`。
- 由该命令设置的子设计不能被 `ungroup` 命令或工具的自动 `ungroup` 能力解除分组。
- Subset restriction 仅适用于 `current design`。

表 3.2 列出 `set_target_library_subset` 的选项。

表 3.2: `set_target_library_subset` 选项

| 目的                                          | 使用此选项                                                |
|---------------------------------------------|------------------------------------------------------|
| 指定作为目标库子集、用于优化指定设计实例的库                      | <code>library_list</code>                            |
| 指定顶层设计实例、层次单元实例或两者作为设计。                     | <code>-top</code>   <code>-object_list cells</code>  |
| 若两者都不指定,默认为 <code>-top</code>               |                                                      |
| 在优化期间从目标库排除库单元                              | <code>-dont_use lib_cells</code>                     |
| 除 <code>library_list</code> 中列出的单元外,再为指定块指定 | <code>-use lib_cells</code>                          |
| 用于优化的库单元                                    |                                                      |
| 将命令作用域限制为仅时钟路径中的单元                          | <code>-clock_path</code>                             |
| 指定库子集仅用于指定设计                                | <code>-only_here lib_cells</code>                    |
| 指定要与所识别实例关联的 Milkyway 参考库路径                 | <code>-milkyway_reflibs milkyway_reflib_paths</code> |

有关如何设置目标库子集，见「设置目标库子集限制」。

### 3.6.1 设置目标库子集限制

#### Setting Target Library Subset Restrictions

要用目标库子集限制设计块的优化，执行下列任务：

- 指定目标库子集
- 检查目标库子集
- 报告目标库子集
- 移除目标库子集

#### 指定目标库子集

#### Specifying Target Library Subsets

在设置目标库子集限制之前，必须先设置 `target_library` 变量，因为要用该变量中列出的库单元来设置限制。要在优化期间为特定设计块指定目标库子集或过滤目标库单元，使用 `set_target_library_subset` 命令。

例如，下列命令将目标库列表设为包含 `lib1.db` 与 `lib2.db`，并将块 `u1` 中使用的库单元限制为 `lib2.db` 中的单元：

```
de_shell> set_app_var target_library "lib1.db lib2.db"
de_shell> set_target_library_subset "lib2.db" \
-object_list [get_cells u1]
```

下列示例限制 `INVX1`、`MUX1` 与 `NOR2` 库单元，使其在优化期间用于时钟路径：

```
de_shell> set_target_library_subset clocklib.db \
-clock_path -use [INVX1 MUX1 NOR2]
```

#### 检查目标库子集

#### Checking Target Library Subsets

要检查并显示由目标库子集引入的不一致设置，使用 `check_target_library_subset` 或 `check_mv_design -target_library_subset` 命令。这些命令检查目标库、目标库子集与工作条件之间的不一致设置。

`check_mv_design -target_library_subset` 在设计中搜索任何不遵循子集规则的单元。工具将找到的单元报告为警告，因为它们可能在子集设置之前就已存在于设计中。可在优化前后运行该命令以识别问题并检查是否得到相同结果。运行该命令需要 Power-Optimization 许可证。

#### 报告目标库子集

#### Reporting Target Library Subsets

要查明已为指定设计定义了哪些目标库子集，使用 `report_target_library_subset` 命令。

由 `report_cell`、`report_timing` 等报告命令生成的报告中，对由 `-dont_use` 或 `-only_here` 选项指定的单元会附加 `td` 属性。

移除目标库子集

### Removing Target Library Subsets

要从设计或设计实例移除目标库子集限制,使用 `remove_target_library_subset` 或 `reset_design` 命令。

#### 参见

目标库子集（上文）。

## 3.7 使用 TLUPlus 或 nextgrd 文件进行 RC 估算

### Using TLUPlus or nextgrd Files for RC Estimation

TLUPlus 文件包含电阻与电容查找表,并对超深亚微米 (UDSM) 工艺效应建模。若 TLUPlus 文件可用或已用于后端流程,应将其用于 RC 估算。作为 TLUPlus 文件的替代,可使用 nextgrd 文件,以使 DC Explorer 与签核 (signoff) 工具的 RC 估算对齐。

#### 3.7.1 使用 TLUPlus 文件

##### Using TLUPlus Files

TLUPlus 文件提供更准确的电容与电阻数据,从而改善与后端结果的相关性。

用 `set_tlu_plus_files` 命令指定 TLUPlus 文件。用 `-tech2itf_map` 选项指定映射文件,该文件在 Milkyway 工艺文件与工艺互连技术格式 (ITF) 文件之间映射层名。

例如:

```
de_shell> set_tlu_plus_files -max_tluplus $max_tlu_file \
-min_tluplus $min_tlu_file -tech2itf_map $prs_map_file
```

可用 `check_tlu_plus_files` 命令检查 TLUPlus 设置。

有关映射文件的更多信息,见 Milkyway 文档。要确认正在使用 TLUPlus 文件,请在编译日志中检查下列消息:

```
Information: TLU Plus based RC computation is enabled. (RCEX-141)
```

可用 `extract_rc` 命令执行 2.5D 抽取。该命令基于 Elmore 延时模型计算延时,并可更新网络上的回标 (back-annotated) 延时与电容数值。在网表被编辑后使用此命令。若已用 `set_tlu_plus_files` 指定 TLUPlus 工艺文件,工具基于 TLUPlus 工艺执行抽取;否则使用物理库中的抽取参数执行抽取。用 `set_extraction_options` 指定影响抽取的参数,用 `report_extraction_options` 报告影响布线后抽取的参数。

#### 3.7.2 使用 nextgrd 文件

##### Using nextgrd Files

nextgrd 文件包含芯片制造工艺的参考寄生参数。使用从晶圆厂获得的 nextgrd 文件是分析特定工艺技术寄生参数最准确的方式。也可用 StarRC 实用程序 `grdgenxo` 创建 nextgrd 文件; `grdgenxo` 从以互连技术格式 (ITF) 编写的工艺描述文件生成 nextgrd 文件。

**注意**

使用 `nxtgrd` 文件可使综合与签核工具对齐，从而不再需要维护单独的 TLUPlus 文件。但必须使用 N-2017.12 或更高版本的 `grdgenxo` 工具生成 `nxtgrd` 文件。

要用 `set_tlu_plus_files` 命令为 RC 估算指定 `nxtgrd` 文件。如下例所示，该命令将 `NXTGRD_file_max.nxtgrd` 设为电阻与电容最大条件计算所用文件：

```
de_shell> set_tlu_plus_files -max_tluplus NXTGRD_file_max.nxtgrd \
    -tech2itf_map design.map
```

当用 `set_tlu_plus_files` 为 5 nm 及更小的 technology node 读取 `nxtgrd` 文件时，必须持有 DC-Synth-AdvGeo license。若许可证不可用，工具发出错误消息。

## 3.8 对黑盒的支持

### Support for Black Boxes

DC Explorer 支持带黑盒的综合。支持下列类型的黑盒：

- **Functionally unknown black box**

逻辑功能未知的 cell。示例包括：

- 宏单元
- 空层次单元或已黑盒化的模块
- 未链接或未解析的单元

- **Logical black box**

未链接到 logic library 中某个 cell 的 cell。此类 black box 归入 functionally unknown black box。示例包括：

- 空层次单元或已黑盒化的模块
- 未链接或未解析的单元

可为逻辑黑盒单元定义时序，见「为黑盒创建快速时序模型」。

- **Physical black box**

没有 physical representation 的 cell。示例包括：

- 空层次单元或已黑盒化的模块
- 未链接或未解析的单元

可为物理黑盒单元定义物理尺寸。若物理库中没有某单元的物理表示，DC Explorer 会自动定义，见「定义物理尺寸」。

图 3.1 给出为黑盒单元定义时序与物理尺寸并综合设计的流程。

可在 Design Vision 版图窗口中查看 floorplan 中的黑盒，以进行可视化检查。可通过 View Settings 面板上的选项控制活动版图视图中黑盒单元的可见性、选择与显示风格属性。默认情况下，黑盒可见且可被选择。

(原书 *Figure 3: Supported Black Box Flow*)  
 请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 3.1: 支持的黑盒流程

更多信息另见 *Design Vision User Guide*。

#### 参见

- 用于在黑盒中定义时序的命令
- 自动创建物理库单元
- 定义物理尺寸
- 为黑盒创建快速时序模型

### 3.8.1 自动创建物理库单元

#### Automatic Creation of Physical Library Cells

若物理库中没有某单元的物理表示，DC Explorer 会自动定义它。工具可为下列单元创建物理库单元：

- 逻辑库单元（叶单元与宏）
- 空层次单元或已黑盒化的模块
- 未链接或未解析的单元
- 未映射单元

创建物理库单元时，工具发出下列警告消息：

Warning: Created physical **library** cell **for** logical **library** %s. (OPT-1413)

### 3.8.2 定义物理尺寸

#### Defining Physical Dimensions

DC Explorer 支持根据 physical black box 被真实逻辑替换后所含 object 的估算结果，设置该 black box cell 的尺寸。还可定义 gate equivalence calculation 的 base unit area，以估算 black box 的尺寸。下列各节讨论定义黑盒单元物理尺寸的这些策略：

- 估算黑盒尺寸
- 确定门等效面积

## 估算黑盒尺寸

### Estimating the Size of Black Boxes

要根据所含对象的估算为物理黑盒单元设置尺寸与形状，使用 `estimate_fp_black_boxes` 命令。

- 要用 `-sm_size` 选项设置宽度与高度：

下列示例估算名为 `alu1` 的黑盒，并将其指定为尺寸  $100 \times 100$ 、利用率 0.7 的软宏：

```
de_shell> estimate_fp_black_boxes \
  -sm_size {100 100} -sm_util 0.7 \
  [get_cells "is_physical_black_box==true" alu1]
```

- 要用 `-polygon` 选项创建直角多边形（rectilinear）黑盒：

下列示例估算名为 `alu1` 的黑盒，并将其指定为直角多边形软宏：

```
de_shell> estimate_fp_black_boxes \
  -polygon {{1723.645 1925.365} {1723.645 1722.415} \
    {1803.595 1722.415} {1803.595 1530.535} \
    {799.915 1530.535} {799.915 1925.365} \
    {1723.645 1925.365}} \
  [get_cells -filter "is_physical_black_box==true" alu1]
```

- 要用 `-hard_macros` 选项创建硬宏黑盒：

在下列示例中，名为 `U1` 的黑盒尺寸由硬宏 `ram16x128` 的尺寸估算：

```
de_shell> estimate_fp_black_boxes -hard_macros ram16x128 U1
```

## 确定门等效面积

### Determining the Gate Equivalent Area

DC Explorer 还允许您定义用于估算黑盒尺寸的门等效面积计算的基本单位面积。用 `set_fp_base_gate` 命令指定库单元面积或用户指定单元面积作为门等效计算的基本单位面积。

- 要用 `-cell` 选项指定库中的门作为计算所用基本单位面积：

```
de_shell> set_fp_base_gate -cell UNIT
```

此例中，`UNIT` 指定用作门等效计算基本单位面积的库单元参考名。

- 要用 `-area` 选项以平方微米指定用作计算基本单位面积的单元面积：

```
de_shell> set_fp_base_gate -area 10
```

此例中，基本门面积设为 10 平方微米。

## 3.8.3 为黑盒创建快速时序模型

### Creating Quick Timing Models for Black Boxes

DC Explorer 允许您为逻辑黑盒单元定义时序。可创建逻辑库单元，并以快速时序模型（quick timing model）格式为新单元提供时序模型。Quick timing model 是一种近似 timing model，适合在设

计周期早期描述 black box 的粗略初始时序。通过指定模型端口、输入上的 setup/hold 约束、clock-to-output 路径延时以及 input-to-output 路径延时来创建黑盒的快速时序模型。也可指定输入端口上的负载与输出端口的驱动强度。DC Explorer 将时序模型保存为 .db 格式，随后可在综合期间使用。

要为简单黑盒创建快速时序模型，按下列步骤操作：

1. 用 create\_qtm\_model 命令创建新模型。

```
de_shell> create_qtm_model BB
```

BB 为模型名。

2. 指定逻辑信息，例如逻辑库名称、最大 transition 时间、最大电容以及线负载信息。

```
de_shell> set_qtm_technology -library library_name
de_shell> set_qtm_technology -max_transition trans_value
de_shell> set_qtm_technology -max_capacitance cap_value
```

3. 指定全局参数，例如 setup 与 hold 特性。

```
de_shell> set_qtm_global_parameter -param setup -value setup_value
de_shell> set_qtm_global_parameter -param hold -value hold_value
de_shell> set_qtm_global_parameter -param clk_to_output \
    -value cto_value
```

4. 用 create\_qtm\_port 命令指定端口。

```
de_shell> create_qtm_port -type input A
de_shell> create_qtm_port -type input B
de_shell> create_qtm_port -type output OP
```

5. 用 create\_qtm\_delay\_arc 命令指定延时弧。

```
de_shell> create_qtm_delay_arc -name A_OP_R -from A \
    -from_edge rise -to OP -value 0.10 -to_edge rise
de_shell> create_qtm_delay_arc -name A_OP_F -from A \
    -from_edge fall -to OP -value 0.11 -to_edge fall
de_shell> create_qtm_delay_arc -name B_OP_R -from B \
    -from_edge rise -to OP -value 0.15 -to_edge rise
de_shell> create_qtm_delay_arc -name B_OP_F -from B \
    -from_edge fall -to OP -value 0.16 -to_edge fall
```

6. (可选) 生成报告，显示快速时序模型中已定义的参数、端口与时序弧。

```
de_shell> report_qtm_model
```

7. 用 save\_qtm\_model 命令保存快速时序模型：

```
de_shell> save_qtm_model
```

8. 用 write\_qtm\_model 命令写出快速时序模型的 .db 文件：

```
de_shell> write_qtm_model -out_dir QTM
```

9. 将 .db 文件加入 link\_library 变量以加载快速时序模型:

```
de_shell> lappend link_library "QTM/BB.db"
```

参见

用于在黑盒中定义时序的命令（下文）。

3.8.4 用于在黑盒中定义时序的命令

Commands for Defining Timing in Black Boxes

表 3.3 列出可用于为黑盒单元定义时序的快速时序模型命令。

表 3.3: 用于在黑盒单元中定义时序的命令

| 目的                    | 使用此命令                      |
|-----------------------|----------------------------|
| 创建快速时序模型时钟            | create_qtm_clock           |
| 定义 setup 与 hold 弧     | create_qtm_constraint_arc  |
| 为快速时序模型创建延时弧          | create_qtm_delay_arc       |
| 在快速时序模型描述中创建驱动类型      | create_qtm_drive_type      |
| 在快速时序模型中创建生成时钟        | create_qtm_generated_clock |
| 为快速时序模型在时钟端口上创建插入延时   | create_qtm_insertion_delay |
| 为快速时序模型描述创建负载类型       | create_qtm_load_type       |
| 开始定义快速时序模型            | create_qtm_model           |
| 在快速时序模型中创建路径类型        | create_qtm_path_type       |
| 创建快速时序模型端口            | create_qtm_port            |
| 报告当前快速时序模型的详细信息       | report_qtm_model           |
| 保存快速时序模型              | save_qtm_model             |
| 设置全局 setup、hold 或时钟参数 | set_qtm_global_parameter   |
| 在端口上设置驱动              | set_qtm_port_drive         |
| 在端口上设置负载              | set_qtm_port_load          |
| 设置各类工艺参数              | set_qtm_technology         |
| 写出快速时序模型 .db 文件       | write_qtm_model            |

参见

为黑盒创建快速时序模型（上文）。

3.8.5 识别黑盒

Identifying Black Boxes

DC Explorer 在黑盒上设置下列属性:

- is\_black\_box

对 get\_cells 指定 is\_black\_box 属性时, DC Explorer 识别功能未知黑盒:

```
de_shell> get_cells -hierarchical -filter "is_black_box==true"
```

- **is\_logical\_black\_box**

对 `get_cells` 指定 `is_logical_black_box` 属性时，DC Explorer 识别设计中的逻辑黑盒：

```
de_shell> get_cells -hierarchical -filter "is_logical_black_box==true"
```

- **is\_physical\_black\_box**

对 `get_cells` 指定 `is_physical_black_box` 属性时，DC Explorer 识别设计中的物理黑盒：

```
de_shell> get_cells -hierarchical \  
-filter "is_physical_black_box==true"
```

# Chapter 4

## 对不完整或不匹配数据的容错

### *Tolerance for Incomplete or Mismatched Data*

即使 RTL netlist 与 library 之间、logic library 与 physical library 之间，或 top-level RTL instantiation 与 RTL subblock 的 interface pin 定义之间存在不完整或不匹配的数据，DC Explorer 仍可继续执行 link process。对于这些数据不一致，工具会汇总 link error 和 warning message，并提供详细报告。当工具遇到 infeasible path 时，会推断 path assumption，并继续优化其他 critical path。

对不完整或不匹配数据的容错默认启用。在 DC Explorer 读入并链接设计之前，可以更改控制特定容错行为的部分设置，例如 bus 与 bit-blasted 命名风格。

要进一步了解对不完整或不匹配数据的容错，请参阅：

- 容错类别
- 默认容错设置
- 调整总线引脚命名风格的容错设置
- 解析总线与 bit-blasted 命名风格
- 允许引脚名同义词
- 允许未连接的接口引脚
- 在 netlist 中包含 anchor cell
- 创建报告与缺失约束

### 4.1 容错类别

#### **Tolerance Categories**

表 4.1 列出 DC Explorer 支持的对不完整或不匹配数据的容错类别。勾选 (X) 表示该容错类别适用且受支持。

表 4.1: 对不完整或不匹配数据的容错类别

| 容错类别              | Library 定义 vs instantiation | Top-level RTL vs RTL block | Logic library vs physical library |
|-------------------|-----------------------------|----------------------------|-----------------------------------|
| 不匹配的总线引脚命名风格      | X                           | X                          | X                                 |
| 总线 vs bit-blasted | X                           | X                          | X                                 |
| 引脚名大小写不匹配         | X                           | X                          | X                                 |
| 逻辑库中缺失单元          | X                           |                            |                                   |
| 物理库中缺失单元          |                             |                            | X                                 |
| 逻辑库中缺失引脚          | X                           |                            | X                                 |
| 物理库中缺失引脚          |                             |                            | X                                 |
| RTL 模块中缺失引脚       |                             | X                          |                                   |
| 缺失 RTL 模块         |                             | X                          |                                   |
| 引脚方向不匹配           |                             |                            | X                                 |
| 引脚名同义词（引脚映射）      | X                           | X                          | X                                 |
| 端口宽度不匹配           | X                           | X                          |                                   |
| 未连接的接口引脚          | X                           | X                          |                                   |

## 参见

- 默认容错设置
- 调整总线引脚命名风格的容错设置
- 解析总线与 bit-blasted 命名风格
- 允许引脚名同义词
- 允许未连接的接口引脚
- 报告不完整或不匹配数据

## 4.1.1 限制

## Limitations

DC Explorer 支持 Verilog 文件中所有容错类别的不匹配，但不支持 VHDL 文件中 subblock instantiation 与 top-level component instantiation 之间的不匹配。以下 VHDL 代码属于「RTL module 中缺失 pin」这一容错类别。代码显示 SUB1 与 TOP 两个 instantiation 之间存在不匹配，其中 SUB1 instantiation 缺少 MSGPIN pin。分析设计时，工具会对此类不匹配发出 error message。

```
entity SUB1 is
    port (AA, BB: in bit; CC: out bit);
end entity SUB1;

architecture RTL of SUB1 is
begin
```

```
    CC <= AA and BB;
end architecture RTL;

library WORK;
use WORK.all;

entity TOP is
    port (a, b, e: in bit; c: out bit);
end entity TOP;

architecture RTL of TOP is
begin
    U1 : entity WORK.SUB1(RTL)
        port map (AA=>a, BB =>b, MSGPIN=>e, CC=>c);
end architecture RTL;
```

### 参见

容错类别（上文）。

## 4.2 默认容错设置

### Default Tolerance Setup

默认情况下，DC Explorer 已针对下列容错类别完成设置：除 bus pin naming style 的容错外，其他设置均不可配置。有关 DC Explorer 的默认行为，请参阅各容错类别的说明：

- 总线引脚命名风格
- 引脚名大小写不匹配
- 逻辑库中缺失单元
- 物理库中缺失单元
- 逻辑库中缺失引脚
- FRAM 视图中缺失引脚
- 逻辑库与物理库之间的引脚方向不匹配
- 端口宽度不匹配

### 4.2.1 总线引脚命名风格

#### Bus Pin Naming Styles

默认情况下，DC Explorer 可根据 `bit_blasted_bus_linking_naming_styles` 变量定义的命名约定，将 RTL block 与 logic library 中的对应 block 进行匹配。要为不同总线命名风格调整该变量的默认设置，见「调整总线引脚命名风格的容错设置」。表 4.2 给出基于该变量默认设置的匹配示例。

表 4.2: 默认总线名匹配示例

| RTL       | 逻辑库文件     |
|-----------|-----------|
| OPRNDA_0_ | OPRNDA[0] |
| OPRNDA_1_ | OPRNDA[1] |
| OPRNDB(0) | OPRNDB_0_ |
| OPRNDB(1) | OPRNDB_1_ |
| OPRNDC[0] | OPRNDC(0) |
| OPRNDC[1] | OPRNDC(1) |

参见

调整总线引脚命名风格的容错设置。

4.2.2 引脚名大小写不匹配

Case Mismatches Between Pin Names

默认情况下，DC Explorer 允许下列对象之间存在 pin name 大小写不匹配：

- 顶层例化与 RTL 块
- 逻辑库与 RTL 例化
- 逻辑库与物理库（FRAM 视图）

图 4.1 给出顶层例化与 RTL 块之间引脚名大小写不匹配的示例。加载到内存中的设计显示这些不匹配已被解析。

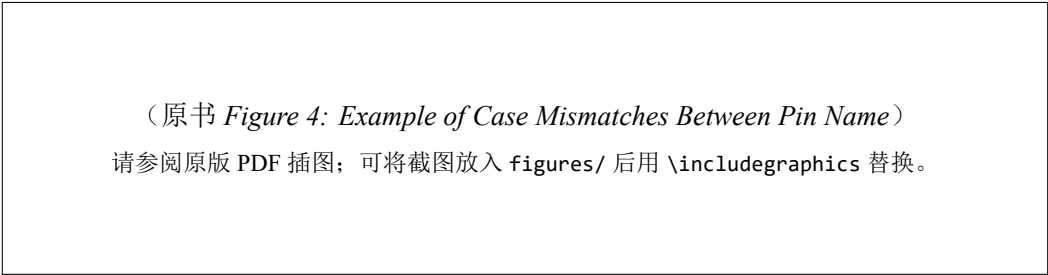


图 4.1: 引脚名大小写不匹配示例

4.2.3 逻辑库中缺失单元

Missing Cells in Logic Libraries

在 link process 中，即使 logic library 中缺少 cell reference，DC Explorer 仍可完成 cell linking，如图 4.2 所示。在此类情况下，DC Explorer 不发出 error message，但解析后得到的 library cell、macro 或 module 不包含 timing arc。

(原书 *Figure 5: Tolerance for Missing Cells in Logic Libraries*)  
请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 4.2: 对逻辑库中缺失单元的容错

#### 4.2.4 物理库中缺失单元

##### Missing Cells in Physical Libraries

当 physical library 中缺少某个 cell 时，DC Explorer 执行下列操作以解决该不匹配：

- 在 local Milkyway design library 中为缺失 cell 创建 FRAM view，并使用 one-unit-tile bounding box
- 从 logic library 推导缺失 cell 的 pin direction
- 在 FRAM view 的原点添加 physical pin

对物理库中较大的缺失宏，应按黑盒流程中所述步骤解析不匹配。有关黑盒流程的更多信息，见 *IC Compiler Design Planning User Guide* 与 *IC Compiler User Guide*。图 4.3 给出为子逻辑单元创建的 FRAM 视图。

(原书 *Figure 6: Examples of Missing Cells in Physical Libraries*)  
请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 4.3: 物理库中缺失单元的示例

#### 4.2.5 逻辑库中缺失引脚

##### Missing Pins in Logic Libraries

当 logic library cell 或 RTL reference 存在缺失 pin 时，DC Explorer 通过下列方式容许该不匹配：

- 为每个缺失 pin 创建 anchor cell，以保留连接并防止该 pin 被移除
- 忽略缺失 pin 上的 timing constraint
- 写出设计时，自动从 ASCII netlist 中移除 anchor cell，并恢复缺失 pin 的连接

如图 4.4 所示，为缺失的 `wr` 与 `ovf` 引脚创建了两个锚点单元。

参见

在网表中包含锚点单元。

(原书 *Figure 7: Example of Missing Pins in Logic Libraries*)  
请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 4.4: 逻辑库中缺失引脚的示例

#### 4.2.6 FRAM 视图中缺失引脚

##### Missing Pins in FRAM Views

当 logic library cell 的 pin 数量多于 FRAM view 时，DC Explorer 会为每个额外 pin 创建一个新的 physical pin。新的 physical pin 添加在 cell bounding box 的左下角，并放置在最低的允许 routing layer 上。

如图 4.5 所示，为 FRAM 视图中缺失的 `wr` 引脚创建了虚设物理引脚，并放置在子块的左下角。

(原书 *Figure 8: Example of a Missing Pin in the FRAM View*)  
请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 4.5: FRAM 视图中缺失引脚的示例

#### 4.2.7 逻辑库与物理库之间的引脚方向不匹配

##### Pin Direction Mismatches Between Logic and Physical Libraries

DC Explorer 允许 port direction mismatch。例如，当 logic library 中指定的 pin direction 与 FRAM view 中的方向不同时，DC Explorer 使用 logic library 中指定的方向，为 logic library cell 创建 physical link。图 4.6 说明 DC Explorer 如何自动解析 `di` 与 `ovf` 引脚方向不匹配。

(原书 *Figure 9: Example of Pin Direction Mismatches*)  
请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 4.6: 引脚方向不匹配示例

### 4.2.8 端口宽度不匹配

#### Port Width Mismatches

DC Explorer 允许 port width mismatch。例如，当 top-level design 所实例化的 RTL block 存在 port width mismatch 时，DC Explorer 会截断或 zero-extend bus，并添加 anchor cell，以保留 top level 上的额外 signal connection，如图 4.7 所示。

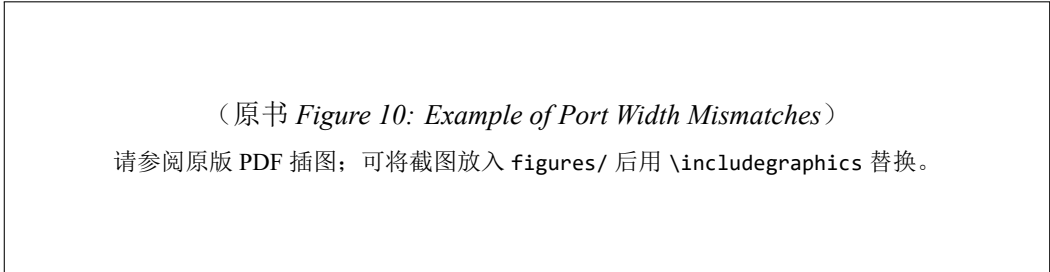


图 4.7: 端口宽度不匹配示例

### 4.3 调整总线引脚命名风格的容错设置

#### Adjusting the Tolerance Setting for Bus Pin Naming Styles

DC Explorer 默认可匹配某些总线引脚命名风格。通过设置 bit\_blasted\_bus\_linking\_naming\_styles 变量，可使 DC Explorer 匹配其他总线引脚命名风格。该变量默认为：

```
"%s\[%d\] %s(%d) %s_%d_"
```

要允许 DC Explorer 匹配表 4.3 所示的总线引脚命名风格，输入：

```
de_shell> set_app_var enable_bit_blasted_bus_linking true
de_shell> set_app_var bit_blasted_bus_linking_naming_styles \
"%s_%d %s\[%d\] %s<%d>"
```

在此示例中，%s\_%d 表示以一个下划线和一个数字结尾的字符串。该形式的名称可匹配由同一字符串后跟方括号内同一数字（%s\[%d\]），或同一字符串后跟尖括号内同一数字（%s<%d>）组成的任意名称。在匹配时，采用这三种指定形式的任意一对名称均视为等价。表 4.3 给出此示例的若干匹配名称。

表 4.3: RTL 与逻辑库文件之间不匹配的总线引脚命名风格

| RTL       | 逻辑库文件     |
|-----------|-----------|
| OPRNDA_0  | OPRNDA[0] |
| OPRNDA_1  | OPRNDA[1] |
| OPRNDB<0> | OPRNDB_0  |
| OPRNDB<1> | OPRNDB_1  |
| OPRNDC<0> | OPRNDC[0] |
| OPRNDA_0  | OPRNDA[0] |

## 参见

总线引脚命名风格。

## 4.4 解析总线与 bit-blasted 命名风格

### Resolving Bus and Bit-Blasted Naming Styles

要使 DC Explorer 匹配不同的总线与 bit-blasted 命名风格：

1. 将 `enable_bit_blasted_bus_linking` 变量设为 `true`（默认值）。
2. 将 `bit_blasted_bus_linking_naming_styles` 变量设为具体命名风格。

如图 4.8 所示，子 RTL 块的 `\sel[1]` 与 `\sel[0]` bit-blasted 信号不匹配。可设置下列变量以解析不匹配：

```
de_shell> set_app_var enable_bit_blasted_bus_linking true
de_shell> set_app_var bit_blasted_bus_linking_naming_styles \
    "%s\[%d\] %s(%d) %s_%d_"
```

（原书 *Figure 11: Example of Bus and Bus-Blasted Naming Styles*）  
请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 4.8: 总线与 bit-blasted 命名风格示例

使用与上一示例相同的两个变量设置，DC Explorer 可匹配多维总线，例如下列 RTL 示例中 `sub1` RTL 块的 `a` 总线与 `b` 总线：

```
module sub1 (input \a[1][1], \a[1][0], \a[0][1], \a[0][0],
             input [1:0][1:0] b,
             output [1:0][1:0] c);

module top (input [1:0][1:0] ain,
            input [1:0][1:0] bin,
            output [1:0][1:0] cout);

sub1 U1 (.a(ain),
        .\b[1][1](bin[1][1]), .\b[1][0]),
        .\b[0][1](bin[0][1]), .\b[0][0]),
        .c(cout));

endmodule
```

## 4.5 允许引脚名同义词

### Allowing Pin Name Synonyms

在 link process 中，可以根据指定的 synonym 启用 pin mapping 功能。该功能可解决下列各类不匹配：

- 顶层例化与 RTL 块
- 逻辑库与 RTL 例化
- 逻辑库与物理库（FRAM 视图）

要启用引脚名同义词匹配：

1. 确保 link\_allow\_pin\_name\_synonym 变量设为 true（默认值）。
2. 用 set\_pin\_name\_synonym 命令为 RTL、逻辑库与物理库引脚名指定同义词。

要移除引脚名同义词，使用 remove\_pin\_name\_synonym 命令。应在 link process 之后移除所有 pin name synonym，然后在加载 SDC 文件之前重新应用这些 synonym。要报告引脚名同义词，使用 report\_pin\_name\_synonym 命令。

#### 4.5.1 示例 1

##### Example 1

下列示例在顶层例化 top 与 RTL 块 sub 之间启用引脚映射。set\_pin\_name\_synonym 命令允许用 s 作为 sel 引脚的同义词，用 do 作为 DOUT 引脚的同义词。

```
de_shell> set_app_var link_allow_pin_name_synonym true
de_shell> set_pin_name_synonym s sel
de_shell> set_pin_name_synonym do DOUT
```

（原书 *Figure 12: Pin Mapping Between Top-Level Instantiations and RTL Blocks*）

请参阅原版 PDF 插图；可将截图放入 figures/ 后用 \includegraphics 替换。

图 4.9: 顶层例化与 RTL 块之间的引脚映射

#### 4.5.2 示例 2

##### Example 2

要在逻辑库与物理库之间启用引脚映射（如下例所示），set\_pin\_name\_synonym 命令：

1. 创建 FRAM 视图，并保存在 Milkyway 设计库中。
2. 根据指定的引脚名同义词，自动更正 FRAM 视图的引脚名以匹配逻辑库引脚名。

该示例对 do 引脚使用同义词 Dout，对 di 引脚使用同义词 Din。

```
de_shell> set link_allow_pin_name_synonym true
de_shell> set_pin_name_synonym Dout do
de_shell> set_pin_name_synonym Din di
```

(原书 *Figure 13: Example of Pin Mapping Between Logic Library and Physical Library*)

请参阅原版 PDF 插图；可将截图放入 figures/ 后用 \includegraphics 替换。

图 4.10: 逻辑库与物理库之间引脚映射示例

## 4.6 允许未连接的接口引脚

### Allowing Unconnected Interface Pins

为防止 unconnected interface pin:

- 连接到 constant high 或 constant low
- 跨 unconnected interface pin 传播 constant value

可将 link\_preserve\_dangling\_pins 变量从默认值 false 改为 true。将该变量设为 true 后，DC Explorer 可为每个 dangling interface pin 添加 anchor cell。如图 4.11 所示，DC Explorer 为未连接的 wr 信号添加了 SNPS\_ANCHOR\_0 锚点单元。

(原书 *Figure 14: Example of Unconnected Interface Pins*)

请参阅原版 PDF 插图；可将截图放入 figures/ 后用 \includegraphics 替换。

图 4.11: 未连接接口引脚示例

## 4.7 在网表中包含锚点单元

### Including Anchor Cells in Netlists

默认情况下，DC Explorer 会为 logic library 或 RTL 中的每个缺失 pin 创建 anchor cell，以防止该 pin 被移除并保留连接。保存设计时，anchor cell 会自动从 netlist 中移除。要在 Verilog 格式的 netlist 中包含 anchor cell，请对 write\_file 命令使用 -include\_anchor\_cells 选项。

例如，下列命令以 Verilog 格式保存带锚点单元的层次设计：

```
de_shell> write_file -hierarchy -format verilog \
-output mapped.vg -include_anchor_cells
```

图 4.12 说明网表如何包含锚点单元。

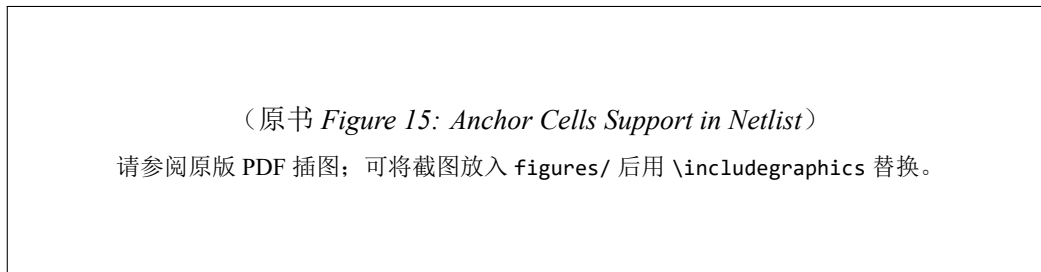


图 4.12: 网表中的锚点单元支持

## 4.8 创建报告与缺失约束

### Creating Reports and Missing Constraints

设计中可能包含不完整或不匹配的数据，例如 pin name 大小写不匹配、logic library 中缺失 cell、缺失 RTL module 以及 missing constraint。不完整或缺失的 timing constraint 可能导致 infeasible path。要报告不完整数据和 infeasible path，并创建 timing exception 和 missing constraint，请参阅：

- 报告不完整或不匹配数据
- 报告缺失约束
- 报告不可行路径
- 使用分类时序报告创建约束
- 为不可行路径创建例外

参见

容错类别。

#### 4.8.1 报告不完整或不匹配数据

##### Reporting Incomplete or Mismatched Data

要报告不完整或不匹配数据，可任选下列方法之一：

- 使用 link 命令显示所发现并已解析的不匹配警告列表。例如：

```
Warning: Automatically reconstructed 'sel' bus with width of 2 from
{sel[0] ... sel[1]} bit-blasted bus pins in 't1' reference.
(LINK-912)

Warning: Automatically reconstructed 'd2' bus with width of 4 from
{ d2[0] d2[1] ... d2[3] } bit-blasted bus pins in 'U1' cell.
```

```
(LINK-912)

Warning: 'wr' pin on 'inst1' cell in 'top' design is missing from the
'SliceInit' reference. (LINK-902)

Warning: Unable to connect 'wr' net to the 'wr' pin of 'inst1' cell
in 'top' design. (LINK-903) . . . .

Warning: Design mismatches were detected by the linker and resolved
to link the current design. You can use report_design_mismatch
and check_design commands explicitly to see warning messages about
the design mismatches. (DESH-017)
```

- 使用 `check_design` 命令显示工具遇到并解析的全部设计不匹配的摘要。例如：

| Design Mismatches                                     | Logical | Physical |
|-------------------------------------------------------|---------|----------|
| Missing pin on logical <b>library</b> part (LINK-902) | 1       |          |
| Can't connect net to missing pin (LINK-903)           | 2       |          |
| Missing physical pin (LINK-905)                       |         | 1        |
| Presence of linker anchor cells (LINK-911)            | 2       |          |
| Reconstructed bus (LINK-912)                          | 1       |          |
| Total netlist elements                                | 6       | 1        |

- 使用 `report_design_mismatch` 命令显示设计不匹配的详细信息。例如：

```
*****
Report:      design mismatches
Design:      mydesign
...
*****
Number of missing pins on logical library cells : 1

Design Object  Type    Mismatch
-----
top    inst1  cell    'wr' pin on 'inst1' cell in 'top' design
                        is missing from the 'SliceInit' reference.

Number of unconnectable nets due to missing pins : 1

Design Object  Type    Mismatch
-----
top    wr      net    Unable to connect 'wr' net to the
                        '*cell*6/wr' pin of 'inst1' cell in 'top'
                        design.

Number of linker anchor cells : 1

Design Object  Type    Mismatch
-----
```

```

top    DCLINKER cell    Linker anchor cell inserted for missing
                                pin on 'inst1' cell.

Number of reconstructed bus : 1

Design Object  Type      Mismatch
-----
top    U1        cell    Automatically reconstructed 'sel' bus
                                with width of 2 from { sel[0] ... sel[1]}
                                bit-blasted bus pins in 'my_design' reference.

```

### 参见

- 报告缺失约束
- 报告不可行路径
- 使用分类时序报告创建约束

## 4.8.2 报告缺失约束

### Reporting Missing Constraints

要报告当前设计中缺失的约束(例如缺失时钟、I/O latency、驱动单元与输出负载),使用 `report_missing_constraints` 命令。例如:

```

de_shell> report_missing_constraints
Information: Checking out the license 'DesignWare'. (SEC-104)
*****
Report : Missing Constraints
Design : des_top
...
*****
Missing Clock
-----
                clk_hi_1
-----
Total Missing Clock Definition found: 2.

Missing Input Delay
-----
                combi_hi[3]
                comb1_hi[2]
-----
The total number of missing input delay found is 2.

Missing Output Delay
-----
                out_filter_i1
-----

```

The total number of missing output delay found is 1.

默认情况下, DC Explorer 对缺失的 I/O delay 使用零值; 对于缺失的 driving cell 和 output load, 则从 logic library 中选择 inverter。可以为 missing constraint 指定具体数值, 而不采用工具推定的 constraint value。要在 current design 中创建 missing constraint, 请使用带相应选项的 create\_missing\_constraints 命令。

例如:

```
de_shell> create_missing_constraints -period 0.75 \
-input_delay 0.02 -output_delay 0.1 -output new1.sdc
```

### 参见

- 报告不可行路径
- 使用分类时序报告创建约束
- 为不可行路径创建例外

## 4.8.3 报告不可行路径

### Reporting Infeasible Paths

Infeasible timing path 可能由下列原因造成:

- 缺少 boundary condition
- input delay 的 timing goal 不切实际
- output delay 的 timing goal 不切实际

优化期间, compile\_exploration 命令会自动检测设计中的 infeasible path。为控制优化工作聚焦于其他 path, 该命令会暂时将这些 infeasible path 设置为 false path。优化结束时, 这些 false path 设置会被移除。

运行 compile\_exploration 之后, 可以报告暂时被设为 false path 的 infeasible path。为此, 对 create\_qor\_snapshot 与 query\_qor\_snapshot 命令指定 -infeasible\_paths 选项, 然后生成 HTML 格式的分类时序报告, 如下列命令序列所示。要设置要报告的不可行路径最大数量, 对 create\_qor\_snapshot 指定 -max\_paths 选项。

```
de_shell> compile_exploration
de_shell> create_qor_snapshot -max_paths 30 -infeasible_paths -name snap1
de_shell> query_qor_snapshot -name snap1 -infeasible_paths \
-columns {path_group infeasible_paths startpoint endpoint wns zero_path}\
-output_file snap1.htm
de_shell> sh firefox snap1.htm
```

图 4.13 给出由该命令序列生成的 HTML 格式分类时序报告。优化期间检测到的每条 infeasible path 都会在 Infeasible Path 列中标记为 YES。

也可通过对 report\_timing 使用 -attributes 选项生成时序报告来查看不可行路径。报告显示起点与终点标有 inf 属性的每条不可行路径。

(原书 *Figure 16: Categorized Timing Report in HTML Format*)  
 请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 4.13: HTML 格式的分类时序报告

#### 参见

- 为不可行路径创建例外
- 使用分类时序报告创建约束
- 分析结果质量

### 4.8.4 使用分类时序报告创建约束

#### Creating Constraints Using Categorized Timing Reports

可用 HTML 格式的分类时序报告创建下列约束：

- False paths
- Multicycle paths
- Maximum delays
- Input delays
- Output delays

#### 注意

必须先用类似下列命令序列生成分类时序报告：

```
de_shell> compile_exploration -scan
de_shell> create_qor_snapshot -nworst 3 -name my_snapshot
de_shell> query_qor_snapshot -name my_snapshot -output_file qor_report
de_shell> sh firefox qor_report.html
```

从图 4.14 所示的分类时序 HTML 报告中，可按下列步骤创建缺失约束与例外：

1. 单击 Append 按钮。
2. 单击相应参数按钮，例如 Multicycle Paths。
3. 为该参数输入数值。
4. 单击 Create Exceptions 按钮。

随即出现新的浏览器窗口，其中包含基于您所指定约束的一组命令。

(原书 *Figure 17: Categorized Timing Report in HTML Format*)  
 请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 4.14: HTML 格式的分类时序报告

5. 用这些命令更新您的 SDC 文件。

#### 参见

- 度量结果质量
- 报告不可行路径

### 4.8.5 为不可行路径创建例外

#### Creating Exceptions for Infeasible Paths

可用 HTML 格式的分类时序报告为不可行路径创建例外。必须先用类似下列命令序列生成包含设计中不可行路径的分类时序报告：

```
de_shell> compile_exploration
de_shell> create_qor_snapshot -max_paths 30 -infeasible_paths -name snap1
de_shell> query_qor_snapshot -infeasible_paths -name snap1 \
-columns {path_group infeasible_paths startpoint endpoint wns zero_path}\
-output_file snap1.html
de_shell> sh firefox snap1.html
```

图 4.15 给出由该命令序列生成的 HTML 格式分类时序报告。

(原书 *Figure 18: Categorized Timing Report in HTML Format*)  
 请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 4.15: HTML 格式的分类时序报告

从分类时序 HTML 报告中，可按下列步骤创建缺失约束与例外：

1. 单击 Append 按钮。
2. 单击 False Paths 按钮。
3. 在要为其创建例外的每条不可行路径的 Exceptions 列下勾选复选框。

#### 4. 单击 Create Exceptions 按钮。

随即出现新的浏览器窗口，其中包含基于您所指定例外的一组命令。例如，当选择前五条不可行路径时，弹出窗口中显示下列命令：

```
set_false_path -from u_clk2/X_out_reg/CLK -to out3;  
set_false_path -from in1 -to u_in1/A_out_reg/D;  
set_false_path -from in1 -to u_in2/A_out_reg/D;  
set_false_path -from in3 -to u_clk2/A_out_reg/D;  
set_false_path -from u_out3/X_out_reg/CLK -to out1;
```

#### 5. 用这些命令更新您的 SDC 文件。

##### 参见

- 报告不可行路径
- 报告结果质量

# Chapter 5

## 处理内存中的设计

### *Working With Designs in Memory*

DC Explorer 将设计从设计文件读入内存。任意时刻内存中都可以同时存在多个设计。设计读入后，可用多种方式修改，例如对子设计进行 group/ungroup，或更改子设计引用。

要了解如何打开与修改设计，见：

- 设计术语
- 打开设计
- 设置当前设计
- 链接设计
- 编辑设计
- 更改设计层次
- 使用设计对象属性
- 创建设计
- 复制设计
- 重命名设计
- 保存设计
- 移除设计
- 设计数据库与网表命名一致性

### 5.1 设计术语

#### **Design Terminology**

不同公司对设计及其组件使用不同术语。本节描述 Synopsys 综合工具使用的术语，以及设计实例与引用之间的关系。

本节包括：

- 设计
- 设计对象
- 设计、实例与引用之间的关系

### 5.1.1 设计

#### Designs

设计是执行逻辑功能的电路描述。设计以多种设计格式描述，如 VHDL 或 Verilog HDL。逻辑级设计表示为布尔方程组；门级设计（如网表）表示为互连单元。设计可彼此独立综合，也可用作更大设计中的子设计。

设计可以是平坦的或层次化的：

- 平坦设计（**Flat designs**）  
平坦设计不包含子设计，只有一层结构，仅包含库单元。
- 层次化设计（**Hierarchical designs**）  
层次化设计包含一个或多个设计作为子设计。每个子设计可进一步包含子设计，从而形成多层设计层次。包含子设计的设计称为父设计（**parent designs**）。

### 5.1.2 设计对象

#### Design Objects

图 5.1 给出名为 TOP 的设计中的设计对象。Synopsys 命令、属性与约束可指向特定设计对象。

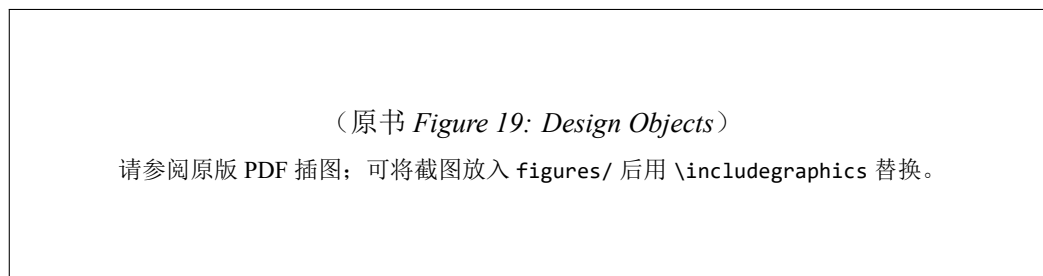


图 5.1: 设计对象

设计

#### Designs

设计由实例、网络、端口与引脚组成；可包含子设计与库单元。在图 5.1 中，设计为 TOP、ENCODER 与 REGFILE。活动设计（正在处理的设计）称为当前设计（**current design**）。多数命令特定于当前设计，即在当前设计的上下文中操作。

## 引用

### References

引用（reference）是可用作构建更大电路之元素的库组件或设计。引用的结构可以是简单逻辑门，也可以是更复杂的设计（如 RAM 核或 CPU）。一个设计可包含同一引用的多次出现；每次出现是一个实例。

reference 使您可以在某一设计内优化每个 cell（如 NAND 门），而不影响其他设计中的 cell。某一设计中的引用独立于不同设计中的相同引用。在图 5.1 中，引用为 INV、ENCODER 与 REGFILE。

## 实例或单元

### Instances or Cells

instance 是已加载到内存的 reference（库组件或设计）在电路中的一次实例化；每个 instance 都有唯一名称。一个设计可包含多个实例。多个实例可指向同一引用，但每个实例有唯一名称以与其他实例区分。实例也称为单元（cell）。

设计中另一设计的唯一实例称为层次实例（hierarchical instance）。设计中库单元的唯一实例称为叶单元（leaf cell）。某些命令在当前设计的层次实例上下文中工作。当前实例（current instance）为这些实例相关命令定义活动实例。在图 5.1 中，实例为 U1、U2、U3 与 U4。

## 端口

### Port

端口是设计的输入与输出。端口方向指定为 input、output 或 inout。

## 引脚

### Pins

引脚是设计中单元（如门与触发器）的输入与输出。子设计的端口在父设计中表现为引脚。

## 网络

### Nets

网络是将端口连接到引脚、以及将引脚彼此连接的连线。

## 5.1.3 设计、实例与引用之间的关系

### Relationships Between Designs, Instances, and References

图 5.2 给出设计、实例与引用之间的关系。

EXREF 设计包含两个引用：NAND2 与 MULTIPLIER。NAND2 被实例化三次，MULTIPLIER 被实例化一次。赋给 NAND2 三个实例的名称分别为 U1、U2 与 U3。EXREF 设计中 NAND2 与 MULTIPLIER 的引用独立于不同设计中的相同引用。

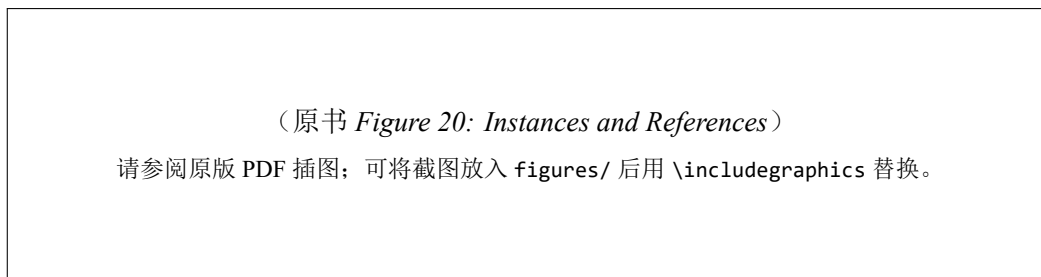


图 5.2: 实例与引用

## 参见

- 解析引用的过程

## 5.2 打开设计

### Opening Designs

打开设计之前，请确保设计采用下表所列格式之一。

表 5.1: 支持的输入格式

| 格式            | 说明                                       |
|---------------|------------------------------------------|
| .ddc          | Synopsys 内部数据库格式（推荐）                     |
| SystemVerilog | IEEE 标准 SystemVerilog（见 HDL Compiler 文档） |
| Verilog       | IEEE 标准 Verilog（见 HDL Compiler 文档）       |
| VHDL          | IEEE 标准 VHDL（见 HDL Compiler 文档）          |

要打开设计，使用下列两种方法之一：

- **analyze** 与 **elaborate** 命令

例如：

```
de_shell> analyze -format vhd1 my_design
de_shell> elaborate my_design
```

若 **analyze** 报告错误，请在 HDL 源文件中修复后再次运行 **analyze**。修改已分析的设计后，必须重新分析。

- **read\_file** 命令

例如：

```
de_shell> read_file -rtl vhd1 my_file
```

要了解两种方法的差异以及如何打开不同格式的设计，见：

- 两种读入方法的差异
- 打开 HDL 与 .ddc 文件

### 5.2.1 analyze 与 elaborate 命令

#### The analyze and elaborate Commands

**analyze** 命令执行下列任务：

- 读入 HDL 源文件
- 在不为设计构建逻辑的情况下检查错误
- 以与 HDL 无关的中间格式创建 HDL 库对象
- 将中间文件存储在您定义的位置

**elaborate** 命令执行下列任务：

- 使用通用技术（GTECH）库元素将中间设计转换为与工艺无关的逻辑设计
- 允许更改源码中定义的参数值
- 允许选择 VHDL architecture
- 用 DesignWare 组件替换代码中的 HDL 算术运算符
- 自动执行 link 命令以解析设计引用

更多信息亦见：

- *The HDL Compiler for Verilog User Guide*
- *The HDL Compiler for VHDL User Guide*

### 5.2.2 read\_file 命令

#### The read\_file Command

**read\_file** 命令：

- 可读入多种不同格式
- 在单一步骤中执行与 **analyze** 和 **elaborate** 相同的操作
- 为 VHDL 创建 **.mr** 与 **.st** 中间文件
- 不自动执行 link 命令
- 不为 Verilog 创建任何中间文件

要创建中间文件，将 **hdlin\_auto\_save\_templates** 变量设为 **true**。

对内存中的设计，DC Explorer 使用 **path\_name/design.ddc** 命名约定。**path\_name** 是原文件读入所在目录，**design** 是设计名。若随后从同名文件读入设计，DC Explorer 会覆盖原设计。为防止覆盖，对 **read\_file** 使用 **-single\_file** 选项。

若未指定设计格式，**read\_file** 根据文件扩展名推断格式。若未使用已知扩展名，DC Explorer 使用 **.ddc** 格式。用于自动推断的受支持扩展名不区分大小写。**read\_file** 可对除 **.ddc** 外的所有格式读入压缩文件（**.ddc** 写出时已在内部压缩）。要让 DC Explorer 自动推断压缩文件的格式，使用下列文件命名结构：**filename.format.gz**。

支持的扩展名包括：

- ddc: .ddc 扩展名
- db: .db、.sldb、.sdb、.db.gz、.sldb.gz 与 .sdb.gz 扩展名
- Verilog: .v、.verilog、.v.gz 与 .verilog.gz 扩展名
- SystemVerilog: .sv、.sverilog、.sv.gz 与 .sverilog.gz 扩展名
- VHDL: .vhd、.vhd1、.vhd.gz 与 .vhd1.gz 扩展名

#### 参见

- 链接设计

## 5.2.3 两种读入方法的差异

### Differences Between the Two Read Methods

表 5.2 汇总使用 `analyze/elaborate` 与使用 `read_file` 的差异。

表 5.2: `analyze/elaborate` 与 `read_file` 的对比

| 对比项          | <code>analyze</code> 与 <code>elaborate</code>                                                                            | <code>read_file</code>                                     |
|--------------|--------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| 输入格式         | 支持 VHDL 与 Verilog 格式。                                                                                                    | 支持所有格式。                                                    |
| 何时使用         | 综合 VHDL 或 Verilog 文件。                                                                                                    | 综合网表、预编译设计等。                                               |
| 参数           | 允许在 <code>elaborate</code> 命令行设置参数值。对于 <code>parameterized design</code> , 可用 <code>analyze/elaborate</code> 以非默认值构建新设计。 | 不能传递参数; 必须在 HDL 中使用 <code>directive</code> 。               |
| Architecture | 允许指定要 <code>elaborate</code> 的 <code>architecture</code> 。                                                               | 不能指定要 <code>elaborate</code> 的 <code>architecture</code> 。 |
| 链接过程         | <code>elaborate</code> 自动执行 <code>link</code> 以解析引用。                                                                     | 必须使用 <code>link</code> 命令解析引用。                             |

更多信息亦见:

- *The HDL Compiler for Verilog User Guide*
- *The HDL Compiler for VHDL User Guide*

## 5.2.4 打开 HDL 与 .ddc 文件

### Opening HDL and .ddc Files

可打开 HDL 与 .ddc 格式的设计:

- 打开 HDL 文件
- 打开 .ddc 文件

打开 HDL 文件

### Opening HDL Files

要打开 HDL 文件, 使用下列方法之一:

- **analyze** 与 **elaborate** 命令

使用此方法时，按自底向上顺序分析顶层设计与所有子设计，然后 **elaborate** 顶层设计以及需要赋值或覆盖参数的任何子设计。

例如：

```
de_shell> analyze -format vhd1 -library -work RISCTYPES.vhd
de_shell> analyze -format vhd1 -library -work {ALU.vhd STACK_TOP.vhd \
STACK_MEM.vhd ...}
de_shell> elaborate RISC_CORE \
-architecture STRUCT -library WORK -update
```

- **read\_file** 命令

例如：

```
de_shell> read_file -format verilog RISC_CORE.v
```

- **read\_verilog** 或 **read\_vhdl** 命令

例如：

```
de_shell> read_verilog RISC_CORE.v
```

也可使用 **read\_file -format VHDL** 或 **read\_file -format verilog** 命令。

打开 **.ddc** 文件

### Opening .ddc Files

要打开 **.ddc** 文件，使用 **read\_ddc** 命令或 **read\_file -format ddc** 命令。例如：

```
de_shell> read_ddc RISC_CORE.ddc
```

**.ddc** 格式向后兼容但不向前兼容；即，可读取用较旧版本 DC Explorer 创建的 **.ddc** 文件，但不能读取比当前所用版本更新的版本创建的文件。

## 5.3 设置当前设计

### Setting the Current Design

要将正在处理的设计设为当前设计，使用下列命令之一：

- **read\_file** 命令

**read\_file** 成功完成后，将当前设计设为刚读入的设计。例如：

```
de_shell> read_file -format ddc MY_DESIGN.ddc
Reading ddc file '/designs/ex/MY_DESIGN.ddc'
Current design is 'MY_DESIGN'
```

- **elaborate** 命令

- **current\_design** 命令

使用该命令将 **de\_shell** 内存中的任意设计设为当前设计。例如：

```
de_shell> current_design MY_DESIGN
Current design is 'MY_DESIGN'. {MY_DESIGN}
```

要显示当前设计的名称，输入：

```
de_shell> printvar current_design
current_design = "test"
```

为避免过度占用运行时间，避免编写包含大量 `current_design` 命令的脚本（例如在循环中）。

## 5.4 链接设计

### Linking Designs

要使设计完整，设计中的所有单元实例必须链接到所引用的库组件与设计。要链接设计，使用 `link` 命令。DC Explorer 使用由 `link_library` 变量、`search_path` 变量以及 `local_link_library` 属性设置的链接库来解析引用。

默认情况下，链接过程的大小写敏感性取决于引用的来源。要设置链接过程的大小写敏感性，设置 `link_force_case` 变量。

可执行的相关任务包括：

- 更改设计引用
- 使用搜索路径定位设计
- 查询设计引用
- 解析引用的过程

#### 参见

- 解析引用的过程

### 5.4.1 更改设计引用

#### Changing Design References

要更改单元或引用所链接到的组件或设计，使用 `change_link` 命令。当指定单元实例为对象时，更改该单元的链接；当指定引用为对象时，更改具有该引用的所有单元的链接。仅当指定的 `reference component` 或 `design` 与原 `reference` 的端口数量相同，且对应端口的位宽和方向均相同时，DC Explorer 才会更改 `link`。

运行 `change_link` 会将所有链接信息从旧设计复制到新设计。若旧设计是 `synthetic` 模块，旧 `synthetic` 模块的所有属性会移到新链接。运行 `change_link` 后，必须对设计运行 `link` 命令。

可使用 `-all_instances` 选项指定位于层次较低层、且其父设计不唯一的实例作为对象。同一父设计中的所有相似单元会自动链接到新引用设计；无需更改当前设计即可更改此类单元的链接。

示例 1 下列命令将当前设计中 U1 与 U2 单元的连接更改为 MY\_ADDER 设计：

```
de_shell> copy_design ADDER MY_ADDER
de_shell> change_link {U1 U2} MY_ADDER
```

示例 2 下列命令更改位于层次较低层的 U1 单元的连接：

```
de_shell> change_link top/sub_inst/U1 lsi_10k/AN3
```

示例 3 下列命令在父设计 bot 被实例化两次（mid1/bot1 与 mid1/bot2）时，更改 inv1 单元所有实例的连接：

```
de_shell> change_link -all_instances mid1/bot1/inv1 lsi_10k/AN3
Information: Changed link for all instances of cell 'inv1'
in subdesign 'bot'. (UID-193)

de_shell> get_cells -hierarchical -filter "ref_name == AN3"
{mid1/bot1/inv1 mid1/bot2/inv1}
```

示例 4 可指示工具解析与原单元引用等效但引脚名不同的单元引用。要在原引用与新单元引用之间映射引脚名，对 change\_link 使用 -pin\_map {{old\_pin1 new\_pin1} {old\_pin2 new\_pin2}...} 选项。

例如，下列命令将当前设计中的 U1 单元链接到 AN21 单元，并把旧引脚名（A1、A2 与 Z）映射到新引脚名（A、B 与 Y）：

```
de_shell> change_link [get_cells U1] AN21 \
    -pin_map {{A1 A} {A2 B} {Z Y}}
```

## 5.4.2 使用搜索路径定位设计

### Locating Designs by Using a Search Path

要查找设计文件位置，用 which 命令指定文件名。DC Explorer 使用 search\_path 变量定义的搜索路径查找设计文件，从该变量中最左侧目录开始，并报告找到的第一个设计文件。默认情况下，搜索路径包括当前工作目录与 \$SYNOPSYS/libraries/syn，其中 \$SYNOPSYS 是安装目录路径。例如：

```
de_shell> which my_design.ddc
{/usr/designers/example/my_design.ddc}
```

要在默认搜索路径之外指定其他目录，输入类似下列命令：

```
de_shell> lappend search_path project
```

## 5.4.3 查询设计引用

### Querying Design References

可用下列方法查询设计引用：

- 要报告当前实例或当前设计中所有引用的信息，使用 `report_reference` 命令。
- 要返回您指定的实例集合，使用 `get_references` 命令。  
例如，下列命令返回当前设计中具有 AN2 引用的实例集合：

```
de_shell> get_references AN2
{U2 U3 U4}
```

- 要查看引用名称，使用 `report_cell` 命令。例如：

```
de_shell> report_cell [get_references AN*]
Cell           Reference    Library    Area
Attributes
-----
U2              AN2         lsi_10k    2.000000
U3              AN2         lsi_10k    2.000000
U4              AN2         lsi_10k    2.000000
U8              AN3         lsi_10k    2.000000
```

5.4.4 解析引用的过程

The Process of Resolving References

DC Explorer 按下列步骤解析引用：

1. 确定当前设计及其层次中引用了哪些库组件与子设计。
2. 在链接库中搜索以定位引用。
  - (a) DC Explorer 首先搜索当前设计的 `local_link_library` 属性中定义的库与设计文件。  
在层次化设计中，DC Explorer 仅考虑顶层设计的本地链接库，忽略与子设计关联的本地链接库。
  - (b) 若 `link_library` 变量设为星号 (\*)，DC Explorer 在内存中搜索该引用。
  - (c) 然后 DC Explorer 搜索 `link_library` 变量中定义的库与设计文件。
3. 若在链接库中未找到引用，则在 `search_path` 变量指定的目录中搜索。
4. 将找到的引用链接（连接）到设计。

DC Explorer 使用它定位到的第一个引用。若定位到同名的其他引用，会生成警告消息标识被忽略的引用。若未找到引用，则发出警告。

图 5.3 中的箭头显示链接过程在实例、引用与链接库之间创建的连接。本例中，DC Explorer 在 LIBRARY\_2 逻辑库中找到 NAND2 组件，并在设计文件中找到 MULTIPLIER 子设计。

参见

- 使用搜索路径定位设计

(原书 *Figure 21: Resolving References*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 5.3: 解析引用

## 5.5 编辑设计

### Editing Designs

使用「设计编辑任务与命令」中描述的命令编辑设计。对于 `unique design`，这些 `netlist editing` 命令接受 `instance object`，即层次中较低层的 `cell`。可在层次化设计的任意层级操作，而无需使用 `current_design` 命令。例如，可输入下列命令在设计 `mid1` 中创建名为 `my_cell` 的单元：

```
de_shell> create_cell mid1/my_cell my_lib/AND2
```

连接或断开网络时，使用 `all_connected` 命令查看连接到网络、端口或引脚的对象。例如，下列命令序列用高功率反相器替换 U8 单元的引用：

```
de_shell> get_pins U8/*
{"U8/A", "U8/Z"}
de_shell> all_connected U8/A
{"n66"}
de_shell> all_connected U8/Z
{"OUTBUS[10]"}
de_shell> remove_cell U8
Removing cell 'U8' in design 'top'.
1
de_shell> create_cell U8 IVP
Creating cell 'U8' in design 'top'.
1
de_shell> connect_net n66 [get_pins U8/A]
Connecting net 'n66' to pin 'U8/A'.
1
de_shell> connect_net OUTBUS[10] [get_pins U8/Z]
Connecting net 'OUTBUS[10]' to pin 'U8/Z'.
```

也可使用 `change_link` 命令达到相同结果。例如：

```
de_shell> change_link U8 IVP
```

其他网表编辑命令包括 `size_cell`、`insert_buffer` 与 `remove_buffer`。

要查询或指定特定设计与对象，见：

- 设计编辑任务与命令
- 查询设计与对象

- 指定设计对象

参见

- 设计编辑任务与命令

5.5.1 设计编辑任务与命令

Design Editing Tasks and Commands

可使用下表所列 `de_shell` 命令对内存中的设计做增量编辑或更改网表。

表 5.3: 设计编辑任务与命令

| 对象    | 任务   | 命令                                    |
|-------|------|---------------------------------------|
| Cells | 创建单元 | <code>create_cell</code>              |
|       | 删除单元 | <code>remove_cell</code>              |
| Nets  | 创建网络 | <code>create_net</code>               |
|       | 连接网络 | <code>connect_net</code>              |
|       | 断开网络 | <code>disconnect_net</code>           |
|       | 删除网络 | <code>remove_net</code>               |
| Ports | 创建端口 | <code>create_port</code>              |
|       | 删除端口 | <code>remove_port</code>              |
|       |      | <code>remove_unconnected_ports</code> |
| Pins  | 连接引脚 | <code>connect_pin</code>              |
| Buses | 创建总线 | <code>create_bus</code>               |
|       | 删除总线 | <code>remove_bus</code>               |

参见

- 编辑设计

5.5.2 查询设计与对象

Querying the Design and Objects

要查明内存中有哪些设计,使用 `list_designs` 命令。若还想查明对应的设计文件名,使用 `-show_file` 选项。例如:

```
de_shell> list_designs
A (*)    B    C

de_shell> list_designs -show_file

/user1/designs/design_A/A.ddc
A (*)
```

```
/home/designer/dc/B.ddc
B      C
```

星号（\*）标记当前设计。B 与 C 设计都包含在 B.ddc 文件中。  
可使用表 5.4 所列命令列出当前设计中的各类对象。

表 5.4: 查询设计对象的命令

| 对象         | 命令                      | 任务                               |
|------------|-------------------------|----------------------------------|
| Instance   | list_instances          | 列出实例及其引用。                        |
|            | report_cell             | 显示实例信息。                          |
| Reference  | report_reference        | 显示引用信息。                          |
| Port       | report_port             | 显示端口信息。                          |
|            | report_bus              | 显示总线端口信息。                        |
|            | all_inputs              | 返回所有输入端口。                        |
|            | all_outputs             | 返回所有输出端口。                        |
| Net        | report_net              | 显示网络信息。                          |
|            | report_bus              | 显示总线网络信息。                        |
| Clock      | report_clock            | 显示时钟信息。                          |
|            | all_clocks              | 返回所有时钟。                          |
| Register   | all_registers           | 返回所有寄存器。                         |
| Collection | get_cells、get_designs 等 | 返回单元、设计、库与库单元、库单元引脚、网络、引脚或端口的集合。 |

5.5.3 指定设计对象

Specifying Design Objects

可通过下列方式指定设计对象：

- 使用相对路径
- 使用绝对路径

使用相对路径

Using a Relative Path

使用相对路径指定设计对象时，对象必须位于当前设计中，且路径相对于当前实例。当前实例是当前设计内的参照系。默认情况下，当前实例是当前设计的顶层。要更改当前实例，使用 `current_instance` 命令。

例如，要在 Count\_16 设计中用 `dont_touch` 属性标记 U1/U15 层次单元，使用下列命令序列之一：

- 参照系保持在 Count\_16 设计顶层

```
de_shell> current_design Count_16
Current design is 'Count_16'.
{Count_16}
de_shell> set_dont_touch U1/U15
```

- 参照系更改为 U1 实例

```
de_shell> current_design Count_16
Current design is 'Count_16'.
{Count_16}
de_shell> current_instance U1
Current instance is '/Count_16/U1'.
/Count_16/U1
de_shell> set_dont_touch U15
```

此后 DC Explorer 将所有未来对象规范解释为相对于 U1 实例。`current_instance` 命令指向当前实例。

要显示当前实例，输入：

```
de_shell> printvar current_instance
current_instance = "Count_16/U1"
```

要将当前实例复位到当前设计顶层，使用不带参数的 `current_instance` 命令。例如：

```
de_shell> current_instance
Current instance is the top-level of the design 'Count_16'
```

使用绝对路径

### Using an Absolute Path

使用绝对路径指定对象时，对象可以位于 `de_shell` 内存中的任意设计。指定对象的语法为：

```
[file:]design/object
```

其中

- `file` 是内存中设计文件的路径名，后跟冒号（:）  
当内存中多个设计同名时使用此参数。
- `design` 是 `de_shell` 内存中的设计名
- `object` 是设计对象名，包括其层次路径

若不同类型的多个对象同名且未指定对象类型，DC Explorer 使用命令允许的类型查找对象。要指定对象类型，使用 `get_*` 命令。

例如，要在 `Count_16` 设计中的 U1/U5 层次单元上放置 `dont_touch` 属性，输入：

```
de_shell> set_dont_touch /usr/designs/Count_16.ddc:Count_16/U1/U5
```

### 参见

- 查询设计与对象

## 5.6 更改设计层次

### Changing the Design Hierarchy

理想情况下，DC Explorer 中的设计层次应准确反映 HDL 描述中的设计划分。不过，可在 DC Explorer 中更改层次，以便在不修改原始 HDL 描述的情况下试验替代层次划分。

在做任何设计层次更改之前，使用 `report_hierarchy` 命令显示当前设计层次。可用下列方法在不修改 HDL 描述的情况下更改层次。

- 添加层次级别
- 移除层次级别
- 合并来自不同子设计的单元

做任何设计层次更改之后，使用 `report_hierarchy` 命令验证层次更改。

#### 5.6.1 添加层次级别

##### Adding Levels of Hierarchy

要通过将单元或相关组件分组到子设计中来创建一层层次，使用 `group` 命令。被分组的单元由引用新子设计的新实例替换。新子设计的端口以其在设计中所连接的网络命名。新子设计各端口的方向由对应网络的引脚确定。对 `cell` 执行 `group` 操作时，原 `cell` 的属性和约束可能无法全部保留。

下列示例将单元分组到子设计中：

要将两个单元分组到名为 `my_design`、实例名为 `U` 的新设计，输入：

```
de_shell> group {u1 u2} -design_name my_design -cell_name U
```

要将所有具有 `alu` 前缀的单元分组到名为 `uP`、实例名为 `UCCELL` 的新设计，输入：

```
de_shell> group "alu*" -design_name uP -cell_name UCCELL
```

要将具有唯一父设计 `mid1` 的 `bot1`、`bot2` 与 `j` 单元分组到名为 `my_design`、实例名为 `U1` 的新子设计，输入：

```
de_shell> group {mid1/bot1 mid1/bot2 mid1/j} \  
-cell_name U1 -design_name my_design
```

也可使用下列两条命令：

```
de_shell> current_design mid1  
de_shell> group {bot1 bot2 j} -cell_name U1 -design_name my_design
```

下列示例将相关组件分组到子设计中：

要将 `process ftj` 中 HDL 函数 `abc` 的所有单元分组到 `new_block` 设计，输入：

```
de_shell> group -hdl_block ftj/abc -design_name new_block
```

要将 `process ftj` 下所有总线化门分组到各自独立的层次级别，输入：

```
de_shell> group -hdl_block ftj -hdl_bussed
```

### 5.6.2 移除层次级别

#### Removing Levels of Hierarchy

DC Explorer 不会跨层次边界优化逻辑；因此，在某些设计中可能需要移除层次级别以改善时序。移除一层层次称为 **ungrouping**，即将某一层次级别的子设计合并到父设计中。标记了 **dont\_touch** 属性的设计、子设计与单元不能被 **ungroup**。

要 **ungroup** 层次，可使用下列命令：

- 优化前的 **ungroup** 命令  
见「优化前 Ungroup 层次」。
- 优化期间显式使用的 **set\_ungroup** 命令  
见「优化期间显式 Ungroup 层次」。
- 优化期间隐式使用的 **compile\_exploration** 命令  
见「优化期间自动 Ungroup 层次」。

参见

- Ungroup 期间的时序约束保留

#### Ungroup 期间的时序约束保留

#### Timing Constraint Preservation During Ungrouping

单元被 **ungroup** 时，层次引脚会被移除。DC Explorer 根据您是在优化前还是优化后 **ungroup** 层次，以不同方式处理放置在层次引脚上的时序约束。

表 5.5 汇总不同 **compile** 流程中 **ungrouping** 对时序约束的影响。

表 5.5: 保留层次引脚时序约束

| Compile 流程                                                                | 对层次引脚时序约束的影响                                                                             |
|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| 优化前使用 <b>ungroup</b> 命令 <b>ungroup</b> 层次                                 | 放置在层次引脚上的时序约束被保留。                                                                        |
| 优化期间使用 <b>set_ungroup</b> 后跟 <b>compile_exploration</b> <b>ungroup</b> 层次 | 默认不保留层次引脚上的时序约束。要保留时序约束，将 <b>auto_ungroup_preserve_constraints</b> 变量设为 <b>true</b> 。    |
| 优化期间使用 <b>compile_exploration</b> 自动 <b>ungroup</b> 层次                    | 要 <b>ungroup</b> 层次并保留时序约束，将 <b>auto_ungroup_preserve_constraints</b> 变量设为 <b>true</b> 。 |

保留时序约束时，DC Explorer 将时序约束重新赋给 **ungroup** 后仍留在同一网络上的相邻适当引脚。约束会沿同一网络向前或向后移动。注意：仅当驱动给定层次引脚的引脚不驱动其他引脚时，约束才能向后移动；否则约束必须向前移动。

当 DC Explorer 将约束移到叶单元时，通过用 **size\_only** 属性标记该单元，在 **compile** 期间保留约束。当向前与向后方向都可行时，DC Explorer 选择使需标记 **size\_only** 属性的叶单元数量最少的方向。

对未映射设计做 **ungroup** 时，DC Explorer 将层次引脚上的约束移到叶单元并用 **size\_only** 属性标记该单元。仅当未映射单元与目标库单元之间存在一对一匹配时，约束才会在 **compile** 过程中被

保留。

仅保留用下列命令设置的时序约束：

```
set_false_path
set_multicycle_path
set_min_delay
set_max_delay
set_input_delay
set_output_delay
set_disable_timing
set_case_analysis
create_clock
create_generated_clock
set_propagated_clock
set_clock_latency
```

#### 注意

`set_rtl_load` 约束不被保留。此外，仅保留当前设计的时序约束；对当前设计层次做 `ungroup` 可能导致其他设计中的时序约束丢失。

优化前 **Ungroup** 层次

### Ungrouping Hierarchies Before Optimization

使用 `ungroup` 命令在优化前 `ungroup` 一个或多个设计。可在层次化设计的任意层级使用该命令，而无需使用 `current_design` 命令。

#### 注意

若 `ungroup` 单元后使用 `change_names` 命令修改层次分隔符 (/)，可能会丢失属性与约束信息。

下列示例说明如何使用 `ungroup` 命令：

要 `ungroup` 单元列表，输入：

```
de_shell> ungroup {high_decoder_cell low_decoder_cell}
```

要 `ungroup` U1 单元并为新单元指定 U1: 前缀，输入：

```
de_shell> ungroup U1 -prefix "U1:"
```

要完全折叠当前设计的层次，输入：

```
de_shell> ungroup -all -flatten
```

要递归 `ungroup` 属于 CELL\_X 单元、且位于当前设计下三层层级的单元，输入：

```
de_shell> ungroup -start_level 3 CELL_X
```

要递归 `ungroup` 位于当前设计下三层层级、且属于 U1 与 U2 单元的单元（U1 与 U2 是当前设计的子单元），输入：

```
de_shell> ungroup -start_level 3 {U1 U2}
```

要递归 ungroup 位于当前设计下三层次的所有单元，输入：

```
de_shell> ungroup -start_level 3 -all
```

实例对象是较低层次的单元。下例说明当父设计唯一时，ungroup 如何接受实例对象。本例中，MID1/BOT1 是设计 BOT 的唯一实例化。该命令 ungroup MID1/BOT1/DES1 与 MID1/BOT1/DES2 单元。

```
de_shell> ungroup {MID1/BOT1/DES1 MID1/BOT1/DES2}
```

也可使用下列两条命令：

```
de_shell> current_instance MID1/BOT1
de_shell> ungroup {DES1 DES2}
```

#### 参见

- Ungroup 期间的时序约束保留

优化期间显式 Ungroup 层次

### Ungrouping Hierarchies Explicitly During Optimization

可通过 set\_ungroup 后跟 compile\_exploration 控制优化期间要 ungroup 的设计。指定要 ungroup 的单元或设计，set\_ungroup 会将 ungroup 属性赋给指定单元或被引用的设计。若在设计上设置该属性，则引用该设计的所有单元都会被 ungroup。

例如，要在优化期间 ungroup U1 单元，输入：

```
de_shell> set_ungroup U1
de_shell> compile_exploration
```

要查看对象是否设置了 ungroup 属性，使用 get\_attribute 命令：

```
de_shell> get_attribute object ungroup
```

要移除 ungroup 属性，使用 remove\_attribute 命令，或通过 set\_ungroup 将 ungroup 属性设为 false。例如：

```
de_shell> set_ungroup object false
```

#### 参见

- Ungroup 期间的时序约束保留

优化期间自动 Ungroup 层次

### Ungrouping Hierarchies Automatically During Optimization

要在优化期间自动 ungroup 层次，可使用 compile\_exploration 命令。DC Explorer 提供两种自动 ungroup 策略：基于面积的自动 ungrouping 与基于延时的自动 ungrouping。

默认情况下，`compile_exploration` 执行基于延时的自动 `ungrouping`，它沿关键路径 `ungroup` 层次，主要用于时序优化。此外，该命令在初始映射之前执行基于面积的自动 `ungrouping`。DC Explorer 估计未映射层次面积，并移除较小的子设计，以改善面积与时序 QoR。

要报告在 `compile_exploration` 过程中被 `ungroup` 的层次，使用 `report_auto_ungroup` 命令。该报告给出每个被 `ungroup` 层次的实例名、单元名与实例数。

更多信息亦见 *Design Compiler User Guide*。

#### 参见

- Ungroup 期间的时序约束保留

### 5.6.3 合并来自不同子设计的单元

#### Merging Cells From Different Subdesigns

要将不同子设计中的单元合并到新子设计中：

1. 将单元 `group` 到新设计中。
2. Ungroup 该新设计。

例如，下列命令序列创建包含最初位于 `u_add` 与 `u_mult` 子设计中之单元的新 `alu` 设计。

```
de_shell> group {u_add u_mult} -design alu
de_shell> current_design alu
de_shell> ungroup -all
de_shell> current_design top_design
```

## 5.7 使用设计对象属性

### Using Design Object Attributes

`attribute` 描述 `design database` 中对象的逻辑、电气、物理及其他特性。`attribute` 附着于 `design object`，并在写出 `design database` 时随之保存；但修改 `attribute` 设置后，`database` 不会自动写回。

DC Explorer 在下列类型的对象上使用属性：

- 整个设计
- 设计对象，如时钟、网络、引脚与端口
- 设计内的设计引用与单元实例
- 逻辑库、库单元与单元引脚

属性有名称、类型与值。属性类型可以是：字符串、数值或逻辑（`Boolean`）。

某些属性为只读，由 DC Explorer 设置这些属性值。不能更改只读属性的值，但可更改读/写属性的值。

要了解如何查询、更改、在设计对象上设置属性，见：

- 查询属性值
- 在对象上设置属性

- 更改对象属性
- 获取属性描述的命令
- 传播在单元实例上设置的用户属性
- 在时序单元上保留用户属性
- 对象搜索顺序

### 5.7.1 查询属性值

#### Querying Attribute Values

要查询设计对象属性的值，使用 `report_attribute` 命令、`get_attribute` 命令或 GUI。

所有属性

**All Attributes** 使用 `report_attribute` 命令查看所有属性。例如：

```
de_shell> report_attribute object_list
```

特定属性

**Specific Attributes** 使用 `get_attribute` 命令查看特定属性。下例报告 OUT7 端口上的 `max_fanout` 属性：

```
de_shell> get_attribute OUT7 max_fanout
Performing get_attribute on port 'OUT7'.
{3.000000}
```

可查询引脚与端口的边界框（bounding box）信息。例如，下列命令报告寄存器 `reg_1` 的 Q 引脚与当前设计的 `test_mode` 端口上的 `bbox` 属性：

```
de_shell> get_attribute [get_pins reg_1/Q] bbox
de_shell> get_attribute [get_ports test_mode] bbox
```

#### 注意

要查询引脚与端口的 `bbox` 属性，需通过将 `de_enable_physical_flow` 变量设为 `true` 来启用物理流程能力。

若某属性适用于多种对象类型，DC Explorer 在数据库中搜索指定对象。

使用 GUI

**Using the GUI** 可在 GUI 的 Properties 对话框中查看所选设计、设计对象或时序路径的属性及其他对象属性。也可对某些属性设置、更改或移除属性值。

有关使用 GUI 的更多信息，见 *Design Vision User Guide*。

**参见**

- 在对象上设置属性
- 更改对象属性
- 获取属性描述的命令
- 对象搜索顺序

### 5.7.2 在对象上设置属性

#### Setting Attributes on Objects

要在设计对象上设置属性，使用下列两种方法之一：

- 使用属性专用命令

使用属性专用命令在对象上设置与该命令关联的属性。下例用 `dont_touch` 属性标记 U1 单元。

```
de_shell> set_dont_touch U1
```

- 使用 `set_attribute` 命令

使用该命令设置任意属性的值，或定义新属性并设置其值。该命令强制执行预定义属性类型（如字符串或整数）。若以不正确类型的值设置属性，命令会发出错误消息。要查找属性的预定义类型，使用 `list_attributes` 命令。

在引用（子设计或库单元）上设置属性时，该属性适用于设计中具有该引用的所有单元。在实例（单元、网络或引脚）上设置属性时，该属性覆盖从实例引用继承的任何属性。

下例在 `lsi_10k/FJK3` 库单元上设置 `dont_touch` 属性：

```
de_shell> set_attribute lsi_10K/FJK3 dont_touch true
```

**参见**

- 查询属性值
- 更改对象属性
- 获取属性描述的命令
- 对象搜索顺序

### 5.7.3 更改对象属性

#### Changing Object Attributes

可更改、移除或保存对象上设置的属性：

- 使用 `remove_attribute` 命令移除对象上的特定属性

不能用该命令移除继承的属性。例如，若 `dont_touch` 属性赋给引用，应从引用而非从继承该属性的单元移除该属性。

下例从 OUT7 端口移除 `max_fanout` 属性：

```
de_shell> remove_attribute OUT7 max_fanout
```

- 使用 `reset_design` 命令从当前设计移除所有属性  
`reset_design` 移除所有 design information, 包括 clock、input/output delay、path group、operating condition、timing range 与 wire load model。运行该命令通常等效于重新开始设计。
- 更改属性值  
要更改属性值, 先移除属性, 再重新创建以设置类型。
- 使用 `write_script` 命令保存属性  
退出 `de_shell` 时, DC Explorer 不会自动保存属性值。要重新创建属性, 使用 `write_script` 生成 `de_shell` 脚本。默认情况下, `write_script` 将属性设置命令报告到屏幕。使用重定向运算符 (`>`) 将输出重定向到文件。例如:

```
de_shell> write_script > attr.scr
```

#### 注意

`write_script` 命令仅支持应用 (系统定义) 属性, 不支持用户定义属性。

#### 参见

- 查询属性值
- 更改对象属性
- 获取属性描述的命令
- 对象搜索顺序

## 5.7.4 获取属性描述的命令

### Commands to Get Attribute Descriptions

多数属性仅适用于有限选择的对象类型; 例如, `rise_drive` 属性仅适用于输入与 inout 端口。某些属性适用于多种对象类型; 例如, `dont_touch` 属性可适用于网络、单元、端口、引用或设计。某些属性预定义并由 DC Explorer 识别; 其他属性由用户定义。可使用下表所列命令获取预定义属性的详细信息。

#### 参见

- 查询属性值
- 在对象上设置属性
- 更改对象属性
- 对象搜索顺序

表 5.6: 获取属性描述的命令

| 对象类型          | 命令                          |
|---------------|-----------------------------|
| All           | man attributes              |
| Designs       | man design_attributes       |
| Cells         | man cell_attributes         |
| Clocks        | man clock_attributes        |
| Nets          | man net_attributes          |
| Pins          | man pin_attributes          |
| Ports         | man port_attributes         |
| Libraries     | man library_attributes      |
| Library cells | man library_cell_attributes |
| References    | man reference_attributes    |

### 5.7.5 传播在单元实例上设置的用户属性

#### Propagating User Attributes Set on Cell Instances

要将较低层设计单元实例上设置的用户定义属性传播到较高层次中的单元实例,使用 `propagate_user_attri` 命令。例如,在图 5.4 中:

- 当前设计设为 `mid`, 用户定义属性定义为 `XYZ`, 在 `mid` 设计的所有单元上值为 `false`。
- 用户定义属性设置在 `bot` 设计的 `B1` 与 `B2` 实例上, 这些实例包含单元实例 `U1`。

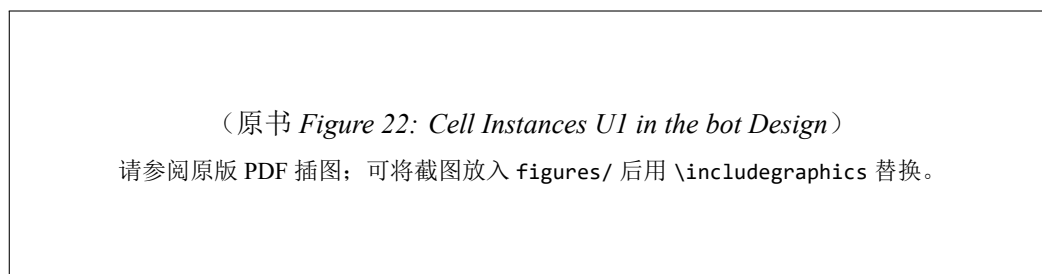


图 5.4: bot 设计中的单元实例 U1

当当前设计 (`mid`) 更改为 `Top` 时 (如示例 2 所示), 使用 `propagate_user_attributes` 命令将 `bot` 设计中所有单元实例上设置的所有用户定义属性传播到 `Top` 设计。

#### 示例 2

##### Example 2: Cell Instances U1 in the bot Design Propagate to the Top Design

```
prompt> current_design mid
Current design is 'mid'.
{mid}

prompt> define_user_attribute -type boolean -classes cell XYZ
XYZ
```

```

prompt> set_user_attribute [get_cells -hierarchical *] XYZ false
Information: Attribute 'XYZ' is set on 4 objects. (UID-186)
B1/U1 B2/U1 B1 B2

prompt> current_design Top
Current design is 'top'.
{top}

prompt> get_cells -hierarchical -filter "XYZ==false"
{M1/B1 M1/B2 M2/B1 M2/B2}

prompt> propagate_user_attributes {XYZ}
1

prompt> get_cells -hierarchical -filter "XYZ==false"
{M1/B1 M1/B1/U1 M1/B2 M1/B2/U1 M2/B1 M2/B1/U1 M2/B2 M2/B2/U1}

```

### 参见

- 在对象上设置属性

## 5.7.6 在时序单元上保留用户属性

### Preserving User Attributes on Sequential Cells

要指定 synthesis flow 中应在 sequential cell 上保留哪些 user-defined attribute, 使用 `define_preserve_user_attribute` 命令。

### 注意

- 属性列表仅在当前会话中保留。
- 若在同一会话中多次使用该命令，它会替换先前保留的属性列表。

要报告保留的属性列表，使用 `report_preserve_user_attribute` 命令。

下例说明如何指定要保留的属性并列出已指定属性：

```

de_shell> define_preserve_user_attribute {my_attribute_1 my_attribute_2}
1

de_shell> report_preserve_user_attribute
my_attribute_1 my_attribute_2

```

该功能有下列限制：

- 只能为时序单元保留属性，不能为引脚或网络保留。
- `optimize_registers` 与 `write_script` 命令不支持 `define_preserve_user_attribute` 命令。

## 5.7.7 对象搜索顺序

### Object Search Order

可在多种对象类型上设置属性的命令使用特定搜索顺序,以找到属性所适用的对象。例如,`set_dont_touch` 命令作用于单元、网络、引用与库单元。若对 `set_dont_touch` 指定名为 X 的对象,且有两个对象(如设计与单元)都名为 X,DC Explorer 将属性应用于找到的第一个对象类型。此时属性设置在设计上,而非单元上。

DC Explorer 一直搜索直到找到匹配对象;若未找到匹配对象则显示错误消息。

可使用 `get_*` 命令指定对象类型来搜索指定类型的对象。例如,假定当前设计包含同名为 `my_object` 的单元与网络。下列命令因默认搜索顺序而在单元上设置 `dont_touch` 属性:

```
de_shell> set_dont_touch my_object
```

要改为在网络上设置 `dont_touch` 属性,输入:

```
de_shell> set_dont_touch [get_nets my_object]
```

#### 参见

- 查询属性值
- 在对象上设置属性
- 更改对象属性
- 获取属性描述的命令

## 5.8 创建设计

### Creating Designs

要创建新设计,使用 `create_design` 命令。使用该命令在 `de_shell` 内存中创建空设计。下例创建名为 `my_design` 的新设计。内存文件名为 `my_design.db`,路径为当前工作目录。

```
de_shell> create_design my_design
de_shell> list_designs -show_file

/work_dir/mapped/test.ddc
test (*) test_DW01_inc_16_0 test_DW02_mult_16_16_1

/work_dir/my_design.db
my_design
```

用 `create_design` 创建的设计不包含设计对象。要向新设计添加设计对象,使用如 `create_clock`、`create_cell`、`create_bus`、`create_port` 与 `connect_net` 等命令。

#### 参见

- 编辑设计

## 5.9 复制设计

### Copying Designs

使用 `copy_design` 命令将内存中的设计复制为新设计。下例复制内存中的 `test` 设计，并将新设计命名为 `test_new`：

```
de_shell> copy_design test test_new
Information: Copying design /designs/test.ddc:to
designs/test.ddc:test_new

de_shell> list_designs -show_file
/designs/test.ddc
test (*) test_new
```

要手动创建可编辑的新子设计，并将实例链接到这些新子设计，使用 `copy_design` 与 `change_link` 命令。例如，假定设计有两个相同单元 `U1` 与 `U2`，都链接到 `COMP`。输入下列命令序列以创建新的、各自独立链接的子设计：

```
de_shell> copy_design COMP COMP1
Information: Copying design /designs/COMP.ddc:COMP to
designs/COMP.ddc:COMP1

de_shell> change_link U1 COMP1
Performing change_link on cell 'U1'.

de_shell> copy_design COMP COMP2
Information: Copying design /designs/COMP.ddc:COMP to
designs/COMP.ddc:COMP2

de_shell> change_link U2 COMP2
Performing change_link on cell 'U2'.
```

## 5.10 重命名设计

### Renaming Designs

可为设计指定新名称，或将设计列表移到文件。要重命名内存中的设计，使用 `rename_design` 命令。要保存重命名后的文件，使用 `write_file` 命令。可使用 `list_designs` 命令显示运行 `rename_design` 前后的设计。例如：

```
de_shell> list_designs -show_file
/designs/test.ddc
test(*) test_new

de_shell> rename_design test_new test_new_1
Information: Renaming design /designs/test.ddc:test_new to
/designs/test.ddc:test_new_1
```

```
de_shell> list_designs -show_file
/designs/test.ddc
test (*) test_new_1
```

可使用 `-prefix`、`-postfix` 与 `-update_links` 选项重命名设计，并为整个设计层次更新单元链接。例如，下列命令序列为 D 设计添加 `NEW_` 前缀，并更新其实例单元的链接：

```
de_shell> get_cells -hierarchical -filter "ref_name == D"
{b_in_a/c_in_b/d1_in_c b_in_a/c_in_b/d2_in_c}

de_shell> rename_design D -prefix NEW_ -update_links
Information: Renaming design /test_dir/D.ddc:D to
/test_dir/D.ddc:NEW_D. (UIMG-45)

de_shell> get_cells -hierarchical -filter "ref_name == D"
# no such cells!

de_shell> get_cells -hierarchical -filter "ref_name == NEW_D"
{b_in_a/c_in_b/d1_in_c b_in_a/c_in_b/d2_in_c}
```

本例中，第一步报告实例化 D 设计的单元。运行第二步后，实例链接到重命名后的引用设计 `NEW_D`。

## 5.11 保存设计

### Saving Designs

可随时以不同名称或格式保存（写入磁盘）设计及其层次中的子设计。修改设计后应手动保存，因为 DC Explorer 不会自动保存设计。

DC Explorer 支持下表所列设计文件格式。

表 5.7: 支持的输出格式

| 格式       | 说明                                                                                                                                                               |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| .ddc     | Synopsys 内部数据库格式                                                                                                                                                 |
| Verilog  | IEEE 标准 Verilog（见 HDL Compiler 文档）                                                                                                                               |
| svsim    | SystemVerilog 网表 wrapper。注意： <code>write_file -format svsim</code> 仅写出网表 wrapper，而不写出被测门级设计（DUT）本身。要写出门级 DUT，必须使用现有的 <code>write_file -format verilog</code> 命令。 |
| VHDL     | IEEE 标准 VHDL（见 HDL Compiler 文档）                                                                                                                                  |
| Milkyway | 在 DC Explorer 中写出 Milkyway 数据库的格式                                                                                                                                |

可用下列方法保存设计文件：

- 使用 `write_file` 命令将内存中的设计转换为指定格式并保存到磁盘  
默认情况下，`write_file` 仅保存顶层设计。要保存整个设计，指定 `-hierarchy` 选项。下例使用 `-hierarchy` 与 `-format` 选项将 `top` 设计层次中的所有设计写入 `.ddc` 文件：

```
de_shell> write_file -hierarchy -format ddc top
Writing ddc file 'top.ddc' Writing ddc file 'A.ddc' Writing ddc file
```

```
'B.ddc'
```

下例使用 `-output` 选项将多个设计写入 `test.ddc` 文件：

```
de_shell> write_file -format ddc -output test.ddc {ADDER MULT16}
Writing ddc file 'test.ddc'
```

- 使用 `write_milkyway` 命令写入 Milkyway 数据库  
`write_milkyway` 基于内存中的网表创建设计文件，并将当前设计的设计数据保存在该文件中。

### 参见

- 设计数据库与网表命名一致性
- 重命名设计
- 使用 Milkyway 数据库

## 5.12 移除设计

### Removing Designs

完成 `compile` 会话并保存优化后的设计后，可在打开另一设计之前从内存中删除该设计。要从内存移除设计，使用 `remove_design` 命令。默认情况下，`remove_design` 仅从 `de_shell` 内存中移除指定的 `design`。

若定义了引用设计对象的变量，从内存移除设计时 DC Explorer 会移除这些引用。这可防止后续命令对不存在的设计对象操作。

例如：

```
de_shell> set_app_var PORTS [all_inputs]
{"A0", "A1", "A2", "A3"}
de_shell> query_objects $PORTS
PORTS = {"A0", "A1", "A2", "A3"}
de_shell> remove_design
Removing design 'top'
1
de_shell> query_objects $PORTS
Error: No such collection '_sel2' (SEL-001)
```

## 5.13 设计数据库与网表命名一致性

### Design Database and Netlist Naming Consistency

在 `de_shell` 中写出网表之前，确保所有网络与端口名称符合布局工具的命名约定，并使用一致的总线命名风格。

某些 ASIC 与 EDA 厂商提供会创建 `.synopsys_dc.setup` 文件的程序，其中包含将名称转换为其约定的适当命令。若需要更改任何网络或端口名称，使用 `define_name_rules` 与 `change_names` 命令。

本节主题：

- .synopsys\_dc.setup 文件中的命名规则
- 解决命名问题
- 避免 SystemVerilog 与 VHDL 结构中的 bit-blasted 端口

### 5.13.1 .synopsys\_dc.setup 文件中的命名规则

#### Naming Rules in the .synopsys\_dc.setup File

使用 `define_name_rules` 命令定义设计对象命名规则集。.synopsys\_dc.setup 文件显示特定布局工具厂商提供的下列命名规则。这些命名规则：

- 将对象名限制为字母数字字符
- 将 DesignWare 单元名更改为有效名称（将 “\*cell\*” 改为 “U”，将 “\*-return” 改为 “RET”）

```
define_name_rules simple_names -allowed "A-Za-z0-9_" \
-last_restricted "_" \
-first_restricted "_" \
-map { {"\*cell\*", "U"}, {"*-return", "RET"}} }
```

例如，可指定 `-map` 选项以移除单元名的尾部下划线：

```
de_shell> define_name_rules naming_convention \
-map {{{_$, ""}} } -type cell
```

要列出可用命名规则，使用 `report_name_rules` 命令。

#### 注意

您的厂商可能使用不同命名约定。请与厂商确认需遵循的命名约定。

### 5.13.2 解决命名问题

#### Resolving Naming Problems

您可能在对象、输入输出文件与工具集中遇到命名约定冲突。在设计数据库文件中，可有许多设计对象（如端口、网络、单元、逻辑模块与逻辑模块引脚），各自有命名约定。此外，可能使用多种具有不同语法定义的输入输出文件格式。使用多家厂商的工具集可能引入更多命名问题。

正确命名有助于消除不匹配错误，并减少设计中对名称转义（`name escaping`）的需要。要解决命名问题，在写出任何文件之前使用 `change_names` 命令解决命名问题并在设计数据库文件中进行名称更改。初始流程为：

1. 读入设计 RTL 并施加约束。  
此处无需更改您的方法。
2. 综合设计以生成门级描述。  
按通常方式再次 `compile` 或优化设计，使用标准脚本集。
3. 应用名称更改并解决命名问题。在写出设计之前使用 `change_names` 及其 Verilog 或 VHDL 开关。

**注意**

每当要写出 Verilog 或 VHDL design 时，都应使用 `change_names -rules -[verilog|vhdl] -hierarchy` 命令，因为 design database 中的命名不符合 Verilog 或 VHDL 语法。例如：

```
de_shell> change_names -rules verilog -hierarchy
```

4. 使用 `write -format verilog` 命令将文件写入磁盘。  
留意报告的名称更改，这表示需要重复步骤 3 并细化命名规则。
5. 若所有适当的名称更改都已完成，输出文件与设计数据库文件匹配。输入下列命令并比较输出。

```
de_shell> write -format verilog -hierarchy -output "consistent.v"
de_shell> write -format ddc -hierarchy -output "consistent.ddc"
```

6. 为第三方工具写出文件。

要在实际不进行更改的情况下显示 `change_names` 命令的效果，使用 `report_names` 命令。

### 5.13.3 避免 SystemVerilog 与 VHDL 结构中的 bit-blasted 端口

#### Avoiding Bit-Blasted Ports in SystemVerilog and VHDL Structures

可使用 `change_names` 命令避免 SystemVerilog struct 与 VHDL record（packed 或 unpacked）中的 bit-blasted 端口。要启用此能力，对 `define_name_rules` 指定 `-preserve_struct_ports` 选项。可按如下方式将该选项添加到现有 Verilog 命名规则：

```
de_shell> define_name_rules verilog -preserve_struct_ports
de_shell> change_names -hierarchy -rules verilog
```

注意：使用 `define_name_rules` 的下列任一选项会覆盖 `-preserve_struct_ports` 选项：

- `-remove_port_bus`
- `-dont_change_bus_members`

如下列示例所示，Verilog 网表 2 使用前述两条命令指定的命名规则保留了 struct 端口：

- RTL

```
typedef struct {logic blue; logic red;} color;
module top (
    input  color i_color,
    output color o_color
);
assign o_color = i_color;
endmodule
```

- 具有 bit-blasted struct 端口的网表 1

```
module top ( i_color_blue_, i_color_red_, o_color_blue_,
o_color_red_ );
    input  i_color_blue_, i_color_red_;
    output o_color_blue_, o_color_red_;
```

```
wire  o_color_blue_, o_color_red_;  
assign o_color_blue_ = i_color_blue_;  
assign o_color_red_  = i_color_red_;  
endmodule
```

- 保留 struct 端口的网表 2

```
module top ( i_color, o_color );  
    input  [1:0] i_color;  
    output [1:0] o_color;  
assign o_color[1] = i_color[1];  
assign o_color[0] = i_color[0];  
endmodule
```

# Chapter 6

## 定义设计环境

### *Defining the Design Environment*

在优化设计之前，必须定义设计预期工作的设计环境，即指定工作条件与系统接口特性。工作条件包括温度、电压与工艺偏差。系统接口特性包括 input driver、input/output load 和 fanout load。设计环境模型会直接影响综合结果。

要了解如何定义设计环境，见：

- 工作条件
- 定义工作条件
- 对系统接口建模
- 在端口上设置逻辑约束
- 使用 UPF 的多电压设计

图 6.1 说明了用于定义设计环境的命令。

(原书 *Figure 23: Commands Used to Define the Design Environment*)

请参阅原版 PDF 插图；可将截图放入 figures/ 后用 \includegraphics 替换。

图 6.1: 用于定义设计环境的命令

## 6.1 工作条件

### **Operating Conditions**

在多数工艺中，工作温度、电源电压与制造工艺参数的变化会强烈影响电路性能。这些因素称为工作条件（operating conditions），一般具有下列特征：

- 工作温度变化

设计日常工作中温度变化不可避免。温度波动对性能的影响多数按线性缩放处理，但某些亚微米硅工艺需要非线性表征。

- 电源电压变化

设计供电电压在工作时可能偏离既定理想值。常使用复杂计算（结合阈值电压偏移），但逻辑级性能计算也会使用简单线性缩放因子。

- 工艺变化

该变化反映半导体制造工艺偏差。通常将工艺变化按性能计算中的百分比偏差处理。

进行时序分析时，DC Explorer 必须考虑工艺、温度与电压预期变化下的最坏情况与最好情况场景。

#### 参见

- 定义工作条件

## 6.2 定义工作条件

### Defining the Operating Conditions

多数逻辑库预定义了多组工作条件。设置工作条件之前，您可以：

- 使用 `report_lib` 命令列出逻辑库中定义的工作条件

逻辑库必须已加载到内存后才能运行 `report_lib`。要查看内存中已加载的库，使用 `list_libs` 命令。下例对存储在 `my_lib.db` 中的 `my_lib` 库生成报告：

```
de_shell> read_file my_lib.db
de_shell> report_lib my_lib
```

- 使用 `current_design` 命令列出当前设计已定义的工作条件

若逻辑库包含工作条件规范，可将其作为默认值。要显式指定 `operating condition` 并取代默认值，使用 `set_operating_conditions` 命令。下例将当前设计的工作条件设为最坏情况商用条件（`worst-case commercial`）：

```
de_shell> set_operating_conditions WCCOM -library my_lib
```

### 6.2.1 多角多模设计

#### Multicorner-Multimode Designs

设计通常需要在多个 `mode` 和多个 `operating condition`（也称 `corner`）下工作。此类设计称为 `multicorner-multimode design`。工具可并行跨多个 `mode` 和 `corner` 执行分析与优化。`multicorner-multimode` 功能可实现 DC Explorer flow 与 IC Compiler flow 的兼容。

要定义模式与 `corner`，使用 `create_scenario` 命令。`scenario` 定义包括指定 TLUPlus 库、工作条件与约束的命令。如何定义 `scenario`，见「定义 Scenario」。

## 参见

- 工作条件
- 优化多角多模设计

## 6.3 对系统接口建模

### Modeling the System Interface

要对设计与外部系统的交互建模，执行下列任务：

- 定义输入端口的驱动特性
- 定义输入与输出端口上的负载
- 定义输出端口上的扇出负载

#### 6.3.1 定义输入端口的驱动特性

##### Defining Drive Characteristics for Input Ports

为确定入站信号的延时与转换时间特性，DC Explorer 需要每个输入端口的外部驱动强度与负载信息。驱动强度是输出驱动电阻的倒数；输入端口上的转换延时等于驱动电阻与输入端口电容负载的乘积。

默认情况下，DC Explorer 假定输入端口驱动电阻为零，即无限驱动强度。要设置现实的驱动强度，使用下列命令之一：

- `set_driving_cell` 命令在由逻辑库单元驱动的端口上设置驱动特性。  
该命令将库引脚与输入端口关联，使延时计算器能准确建模外部驱动器的驱动能力。要移除端口上的 `driving cell` 属性，使用 `remove_driving_cell` 命令。
- `set_input_transition` 命令在输入驱动能力无法用逻辑库单元表征时，在顶层端口上设置信号转换时间。

`set_driving_cell` 与 `set_input_transition` 都会影响 `port transition delay`，但它们本身不会在 `input port` 上设置 `max fanout`、`max transition` 等 `design rule constraint`。但是，如果指定的 `driving cell` 带有 `design rule constraint`，`set_driving_cell` 会将这些约束施加到 `input port`。

图 6.2 给出层次化设计示例。顶层设计有两个子设计 U1 与 U2。顶层设计的 I1 与 I2 端口由外部系统驱动，驱动电阻为 1.5。

(原书 *Figure 24: Drive Characteristics*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 6.2: 驱动特性

要为本例设置驱动特性，按下列步骤操作：

1. 使用 `set_driving_cell` 命令定义驱动电阻。

```
de_shell> current_design top_level_design
de_shell> set_driving_cell -lib_cell IV {I1 I2}
```

2. 将当前设计改为 `sub_design2`，以描述该子设计端口上的驱动能力。

```
de_shell> current_design sub_design2
```

3. 使用 `set_driving_cell` 将 IV 单元的驱动能力与 I3 端口关联。

```
de_shell> set_driving_cell -lib_cell IV {I3}
```

4. AN2 单元不同弧的转换时间不同。检查 setup 违例时，指定最慢的 AN2 弧（假定最慢弧为从 B 到 Z）。

```
de_shell> set_driving_cell -lib_cell AN2 -pin Z -from_pin B {I4}
```

检查 setup violation 时，worst-case arc 是最慢 arc；检查 hold violation 时，worst-case arc 是最快 arc。

#### 注意

对重负载驱动端口（如时钟网络），请将驱动强度保持为零，以免 DC Explorer 对网络进行缓冲。时钟网络的缓冲应在时钟树综合中单独进行。

## 6.3.2 定义输入与输出端口上的负载

### Defining Loads on Input and Output Ports

默认情况下，DC Explorer 对输入与输出端口施加零电容负载。要在输入与输出端口上设置电容负载值，使用 `set_load` 命令。DC Explorer 用该值选择合适的输出 pad 驱动强度，并对输入 pad 上的转换时间建模。

下例在 `out1` 引脚上设置电容负载：

```
de_shell> set_load 30 {out1}
```

负载值的单位应与目标逻辑库一致。例如，若库以皮法（picofarads）表示负载，则用 `set_load` 设置的值也必须以皮法计。要列出库单位，使用 `report_lib` 命令。

## 6.3.3 定义输出端口上的扇出负载

### Defining Fanout Loads on Output Ports

要对一个或多个端口的外部扇出效应建模，使用 `set_fanout_load` 命令在输出端口上指定预期扇出负载值。例如：

```
de_shell> set_fanout_load 4 {out1}
```

DC Explorer 会尽量确保 output port 上的 fanout load 与连接到该 output port driver 的 cell fanout load 之和，小于 library、library cell 和 design 的 max fanout limit。

扇出负载是无量纲数值，表示对总扇出的数值贡献，可理解为连接到网络的标准单元输入个数；它不是电容值。

#### 参见

- 定义最大扇出

## 6.4 在端口上设置逻辑约束

### Setting Logic Constraints on Ports

为改善优化结果，可在端口上设置逻辑约束以消除冗余端口或反相器，方法如下：

- 设置逻辑等价

某些输入端口由逻辑相关信号驱动。要将两个 input port 定义为逻辑等价或逻辑相反，分别使用 `set_equal` 或 `set_opposite` 命令。

下例指定 `IN_X` 与 `IN_Y` 端口逻辑相等：

```
de_shell> set_equal IN_X IN_Y
```

- 为输入端口赋常值

为输入设置常值可使 DC Explorer 在优化期间简化周围逻辑，并生成更小设计。要为当前设计中的输入分别赋 don't care、逻辑 1 或逻辑 0，使用 `set_logic_dc`、`set_logic_one` 或 `set_logic_zero` 命令。图 6.3 给出简化输入端口逻辑的示例。

（原书 *Figure 25: Simplified Input Port Logic*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 6.3: 简化的输入端口逻辑

下例将 A、B 输入设为 don't care，将 IN 输入设为逻辑 1：

```
de_shell> set_logic_dc {A B}
de_shell> set_logic_one IN
```

要复位这些命令设置的值，使用 `remove_attribute` 命令。

- 指定未连接的输出端口

若某输出未使用（即未连接），驱动该输出的逻辑可在优化期间被最小化或消除，如图 6.4 所示。

要指定输出与外部逻辑未连接，使用 `set_unconnected` 命令。要复位该命令设置的属性，使用 `remove_attribute` 命令。

(原书 *Figure 26: Minimizing Logic Driving an Unconnected Output Port*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 6.4: 最小化驱动未连接输出端口的逻辑

## 6.5 使用 UPF 的多电压设计

### Multivoltage Designs Using UPF

使用 IEEE™ 1801 统一功耗格式 (Unified Power Format, UPF) 标准命令，可为多电压设计指定低功耗意图，以及各电源域的验证策略，包括该域的隔离单元、保持单元与电平移位单元。在 compile 期间，工具按所指定策略自动插入这些电源管理单元。

要在 DC Explorer 中使用 UPF command，必须将 `de_enable_upf_exploration` 从默认值 `true` 改为 `false`，并且必须具有 Power Compiler license。

在多电压设计中，子设计实例（块）工作在不同电压下。为降低功耗，使用 UPF 命令指定电源域。电源域中的块可独立于其他电源域的电源状态上电/下电，但存在相对 `always-on` 关系的两个电源域除外。特别地，可定义电源域，并用电平移位单元与隔离单元调整电源域之间的电压差，以及隔离关断电源域。

电源域定义为共享下列内容的一个或多个层次块的分组：

- primary supply 的电压状态或电压范围（即相同 operating voltage）
- power net hookup 要求
- power-down control 与 acknowledge signal（若有）
- power-switching style
- 相同的 process、voltage、temperature (PVT) operating condition 值（power domain 中除 level shifter 外的所有 cell）
- 相同的一组 nonlinear delay model (NLDM) target library，或该集合的子集

电源域不是 voltage area。电源域是逻辑层次的分组，而对应的 voltage area 是将电源域层次中的单元放入的物理布局区域。这种对应并非自动建立；您需要将层次与 voltage area 对齐。

除 power domain 外，还需使用 UPF 命令为 power intent 指定 supply network、power strategy、power state table 和 operating voltage 等 power information。

要实现使用 UPF 的多电压设计，见：

- *Power Compiler User Guide*
- *Synopsys Multivoltage Flow User Guide*

要使用最小 UPF 运行 RTL UPF 设计探索，见第 八 章「UPF 探索」。

# Chapter 7

## 定义设计约束

### *Defining Design Constraints*

约束是以可度量的电路特性（如时序、面积与电容）定义设计目标的声明。DC Explorer 需要这些约束才能有效优化设计。

要了解定义设计约束所需的概念与任务，见：

- DC Explorer 约束类型
- 定义设计规则约束
- 对特殊网络禁用设计规则修复
- 在层次化设计中传播约束

### 7.1 DC Explorer 约束类型

#### DC Explorer Constraint Types

DC Explorer 使用下列两类约束优化设计：

- 设计规则约束（**Design Rule Constraints**）  
logic library 定义这些 implicit constraint。这些约束是设计正确工作必须满足的要求，适用于使用该 library 的任何 design。默认情况下，design rule constraint 的优先级高于 optimization constraint。
- 优化约束（**Optimization Constraints**）  
由您显式定义。优化约束在整个 de\_shell 会话期间作用于当前处理的设计，表示设计目标。

可在命令行交互指定约束，或在约束文件中指定。DC Explorer 会尝试同时满足设计规则约束与优化约束，但设计规则约束优先于优化约束。

图 7.1 给出 DC Explorer 的主要设计规则约束与优化约束，以及 de\_shell 接口中用于设置约束的命令。

(原书 *Figure 27: Major DC Explorer Constraints*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 7.1: DC Explorer 主要约束

#### 参见

- 约束优先级
- 设计规则约束的优先顺序
- 生成约束报告

### 7.1.1 设计规则约束

#### Design Rule Constraints

设计规则约束反映设计按预期功能工作所必须满足的工艺相关要求。设计规则约束作用于设计的网络，但作为与逻辑库单元引脚关联的属性维护。多数逻辑库指定默认设计规则。

设计规则约束类型包括：

- 最大转换时间 (Maximum Transition Time)
- 最大扇出 (Maximum Fanout)
- 最大电容 (Maximum Capacitance)
- 最小电容 (Minimum Capacitance)
- 单元退化 (Cell Degradation)

不能移除逻辑库中设置的 `max_transition`、`max_fanout`、`max_capacitance` 与 `min_capacitance` 属性，因为它们是逻辑库的要求；但可设置更严格的值以覆盖默认值。

#### 最大转换时间

##### Maximum Transition Time

最大转换时间是设计规则约束。网络的最大转换时间是其驱动引脚改变逻辑值所需的最长时间。许多逻辑库对引脚的最大转换时间有限制，从而为使用这些库的设计创建隐式转换时间约束。若设计使用多个逻辑库且各库对 `max_transition` 属性的默认值不同，DC Explorer 在全局使用最严格的值。

## 最大扇出

### Maximum Fanout

最大扇出是设计规则约束。许多逻辑库对驱动引脚有扇出限制，从而为使用这些库的设计中的驱动引脚创建隐式扇出约束。DC Explorer 通过将 `fanout_load` 属性与各输入引脚关联、将 `max_fanout` 属性与单元各输出（驱动）引脚关联来建模扇出限制，如图 7.2 所示。

（原书 *Figure 28: The fanout\_load and max\_fanout Attributes*）  
请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 7.2: `fanout_load` 与 `max_fanout` 属性

要评估驱动引脚（如图 7.2 中的 X 引脚）的扇出，DC Explorer:

1. 计算由 X 引脚驱动的所有输入的 `fanout_load` 属性之和。  
扇出负载是无量纲数，不是电容；它表示对总有效扇出的数值贡献。
2. 将 `max_fanout` 属性的指定值与该和比较。  
若和小于指定值，则满足扇出约束；否则 DC Explorer 尝试通过选择更高驱动的组件来满足约束。

## 最大电容

### Maximum Capacitance

最大电容是设计规则约束。它作为引脚级属性设置，定义输出引脚可驱动的最大总电容负载。也就是说，该引脚不能连接到总电容（负载引脚电容与互连电容）大于或等于该引脚所定义最大电容的网络。

DC Explorer 通过将网络线电容与连接到该网络的引脚电容相加来计算网络电容。要判断网络是否满足电容约束，DC Explorer 将计算出的电容值与驱动该网络的引脚的 `max_capacitance` 属性值比较。

## 最小电容

### Minimum Capacitance

最小电容是设计规则约束。某些逻辑库指定最小电容。最小电容约束规定单元可驱动的最小负载。优化期间，DC Explorer 确保单元驱动负载满足该单元的最小电容约束。发生违例时，DC Explorer 通过对驱动器调尺寸（sizing）来修复。可对连接到输入端口的网络设置最小电容，以判断输入边界处驱动单元是否出现违例。若 `compile` 后报告违例，可通过重新综合驱动这些端口的模块来修复。

## 单元退化

### Cell Degradation

单元退化是设计规则约束。某些逻辑库包含单元退化表。这些表给出 cell 可驱动的 maximum capacitance, 其值是 cell input transition time 的函数。compile 期间, 若逻辑库指定了单元退化表, DC Explorer 会尽量保证网络电容值小于指定值。若逻辑库未指定单元退化表, 可使用 `set_cell_degradation` 命令在输入端口上显式设置 `cell_degradation` 属性。默认情况下, 端口没有 `cell_degradation` 属性。

#### 参见

- 约束优先级
- 设计规则约束的优先顺序
- 定义设计规则约束

## 7.1.2 优化约束

### Optimization Constraints

优化约束表示您希望达到、但对设计工作未必至关重要的时序与面积目标及限制。时序约束的优先级高于面积约束。默认情况下, 优化约束次于设计规则约束。

优化约束包括:

- 输入与输出延时 (时序约束)
- 最大与最小延时 (时序约束)
- 最大面积

### 定义时序约束

#### Defining Timing Constraints

定义时序约束时, 应考虑设计可能同时包含同步路径与异步路径。

- 要约束同步路径, 使用 `create_clock` 命令在设计中指定时钟。指定时钟后, 还应使用 `set_input_delay` 与 `set_output_delay` 命令指定输入与输出端口时序规范。
- 要约束 asynchronous path, 使用 `set_max_delay` 与 `set_min_delay` 命令指定 point-to-point maximum/minimum delay。

### 最大延时

**Maximum Delay** 最大延时是优化约束。DC Explorer 内置静态时序分析器以评估时序约束。静态时序分析器根据逻辑库中的时序值与工作条件, 由本地门与互连延时计算路径延时, 但不仿真设计。DC Explorer 时序分析器执行关键路径追踪, 检查设计中每条时序路径的最小与最大延时。在时序设计中, 最关键路径不一定是最长组合路径, 因为路径可能相对于路径起点与终点处的不同时钟。

DC Explorer 综合考虑 clock waveform 与 skew、library setup time、external delay、multicycle/false-path specification, 以及 `set_max_delay` 设置的值, 确定设计中每条 timing path 的 maximum-delay target。负载、驱动、工作条件及其他因素也会被考虑。

## 最小延时

**Minimum Delay** 尽管最小延时是优化约束, DC Explorer 在修复设计规则违例时也会修复最小延时约束违例。最小延时约束由 `set_min_delay` 显式设置, 或因 hold 时间要求隐式设置。到引脚或端口的最小延时必须大于目标延时。

## 最大面积

**Maximum Area** 最大面积是优化约束。最大面积表示设计中的门数, 而非设计占据的物理面积。设计的面积要求通常表述为满足性能目标的最小设计。默认情况下, DC Explorer 在时序优化完成后自动对面积进行优化。您不能为设计定义最大面积约束。

DC Explorer 在计算电路面积时忽略下列组件:

- 未知组件
- 面积未知的组件
- 与工艺无关的通用 (generic) 单元

单元面积与工艺相关, 取自逻辑库。

更多信息见 *Synopsys Timing Constraints and Optimization User Guide*。

## 7.1.3 约束优先级

### Constraint Priorities

DC Explorer 尝试最小化总约束代价。每类约束有相对代价, 从而将更多努力用于满足更重要的约束, 必要时可能牺牲其他约束。表 7.1 给出默认优先级顺序。默认情况下, 设计规则约束的优先级高于优化约束。

当出现多个设计规则违例时, DC Explorer 从最高优先级开始修复。可能时, 它也会评估并选择能减少其他设计规则违例的替代方案。对优化延时违例, DC Explorer 采用相同方法。必要时, DC Explorer 可违犯优化约束以避免违犯设计规则约束。

#### 参见

- 设计规则约束的优先顺序
- 优化约束
- 定义设计规则约束
- 生成约束报告

表 7.1: 约束优先级的默认顺序

| 优先级（降序）            | 说明     |
|--------------------|--------|
| connection classes |        |
| min_capacitance    | 设计规则约束 |
| max_transition     | 设计规则约束 |
| max_fanout         | 设计规则约束 |
| max_capacitance    | 设计规则约束 |
| cell_degradation   | 设计规则约束 |
| max_delay          | 优化约束   |
| min_delay          | 优化约束   |
| power              | 优化约束   |
| area               | 优化约束   |
| cell count         |        |

### 7.1.4 设计规则约束的优先顺序

#### Precedence of Design Rule Constraints

DC Explorer 按下列优先级降序解决 design rule constraint 之间的冲突。

1. 最小电容
2. 最大转换时间
3. 最大扇出
4. 最大电容
5. 单元退化

有关设计规则约束优先顺序的细节如下：

- 最大转换时间优先于最大扇出。若最大扇出约束未满足，请检查是否存在冲突的最大转换时间约束。
- DC Explorer 按库的不同以两种方式计算网络转换时间。
  - 对使用 CMOS 延时模型的库，DC Explorer 使用驱动单元的驱动电阻与网络电容负载计算转换时间。
  - 对使用 nonlinear delay model 的 library，DC Explorer 通过 table lookup 和 interpolation 计算 transition time；该值是 output pin capacitance 和 input transition time 的函数。
- 取决于逻辑库，set\_driving\_cell 命令行为不同。
  - 对使用 CMOS 延时模型的库，驱动电阻为常数。
  - 对使用非线性延时模型的库，set\_driving\_cell 根据表中的负载动态计算转换时间。

set\_driving\_cell 命令影响端口转换时间；该命令在受影响端口上放置设计规则约束。

`set_load` 命令在端口或网络上放置负载。该负载的单位必须与逻辑库一致。该值用于 `timing optimization`，不用于 `max-fanout optimization`。

#### 参见

- 约束优先级
- 定义设计规则约束

## 7.1.5 生成约束报告

### Generating Constraint Reports

要检查设计规则与优化目标，使用 `report_constraint` 命令为当前设计生成约束报告。

对当前设计中的每个约束，约束报告列出：

- 约束是否满足或违例，以及违例程度
- 最严重违例者对应的设计对象

约束报告按优先级顺序汇总约束。设计中不存在的约束不会出现在报告中。

### 生成最大电容报告

### Generating Maximum Capacitance Reports

要仅报告违例网络上的最大电容，按如下方式使用 `report_constraint`：

```
de_shell> report_constraint -all_violators -max_capacitance
...
      Required      Actual
Net      Capacitance  Capacitance      Slack
-----
n_6      0.0014226    0.0128121    -0.0113895 (VIOLATED)
n_16     0.0014226    0.0115580    -0.0101354 (VIOLATED)
n_121    0.0014226    0.0114569    -0.0100343 (VIOLATED)
n_21     0.0014226    0.0113311    -0.0099085 (VIOLATED)
div_18_17/n_2 0.0014226    0.0044171    -0.0029945 (VIOLATED)
div_18_17/n_4 0.0014226    0.0042607    -0.0028381 (VIOLATED)
...
```

要报告所有网络上的最大电容，按如下方式使用 `report_net`：

```
de_shell> report_net -max_capacitance
...
Net:      Required Capacitance  Actual Capacitance  Slack
-----
CMD[0]    1.25                  0.25                1.0(Met)
CMD[1]    2.50                  4.50                -2.0(Violated)
...
```

这些命令报告网络的下列细节：

- 用户定义或由库推导的要求电容

- 工具计算的实际电容
- Slack（要求电容减去实际电容）

#### 参见

- 约束优先级

## 7.2 定义设计规则约束

### Defining Design Rule Constraints

定义设计规则约束时，应考虑下列典型设计规则场景：

- `set_max_fanout` 与 `set_max_transition` 命令
- `set_max_fanout` 与 `set_max_capacitance` 命令

通常，逻辑库会为 `max_transition` 或 `max_capacitance` 属性之一指定默认值，但不会两者都指定。为获得最佳结果，不要同时为设计指定 `max_transition` 与 `max_capacitance` 属性。

要了解如何设置设计规则约束，见：

- 定义最大转换时间
- 定义最大扇出
- 定义最大电容
- 定义最小电容
- 定义单元退化
- 定义连接类

#### 7.2.1 定义最大转换时间

##### Defining Maximum Transition Time

要在时钟组、端口或设计上指定最大转换时间，使用 `set_max_transition` 命令设置 `max_transition` 属性。compile 期间，DC Explorer 确保 net transition time 小于指定值；例如，可在 driving gate 的输出端插入 buffer。

例如，要将 adder 设计的最大转换时间设为 3.2 单位，输入：

```
de_shell> set_max_transition 3.2 [get_designs adder]
```

要复位 `set_max_transition` 设置的值，使用 `remove_attribute` 命令：

```
de_shell> remove_attribute [get_designs adder] max_transition
```

若设置的 `max_transition` 属性在库中也有定义，DC Explorer 尝试满足更严格的值。

指定基于时钟的最大转换时间

### Specifying Clock-Based Maximum Transition Time

`max_transition` 属性的值可随单元工作频率变化。该频率定义为驱动某一 logic cone 的 register 所使用的最高 clock frequency。要对包含多个时钟频率的设计中特定时钟组的引脚设置 `max_transition` 属性，使用 `set_max_transition` 命令。

例如，下列命令将 Clk 时钟组中所有引脚的 `max_transition` 属性设为 5 单位：

```
de_shell> set_max_transition 5 [get_clocks Clk]
```

DC Explorer 在确定 `max_transition` 属性值时遵循下列规则：

- 当在设计或端口以及时钟组上都设置了 `max_transition` 属性时，采用最严格约束。
- 若多时钟启动同一路径，采用最严格约束。
- 若逻辑库中已指定 `max_transition` 属性，DC Explorer 在 compile 期间尝试满足这些约束。

#### 参见

- 设计规则约束
- 约束优先级

## 7.2.2 定义最大扇出

### Defining Maximum Fanout

要为一个或多个输入端口指定最大扇出负载，使用 `set_max_fanout` 命令设置 `max_fanout` 属性。可对整个库设置更保守的扇出约束，或为库中个别单元的特定引脚定义扇出约束。若指定的 `max_fanout` 属性在库中也有定义，DC Explorer 尝试满足更严格的值。

例如，要对名为 ADDER 的设计中每个驱动引脚与输入端口设置最大扇出约束，输入：

```
de_shell> set_max_fanout 8 [get_designs ADDER]
```

图 7.3 说明对设计施加的最大扇出约束。

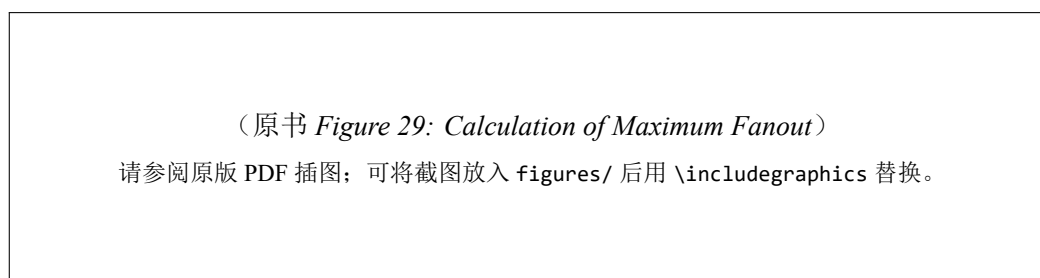


图 7.3: 最大扇出的计算

要检查 Z 驱动引脚是否满足最大扇出约束，DC Explorer 将 `max_fanout` 属性的指定值与扇出负载比较。本例中，总扇出负载为  $1.0 + 1.0 + 3.0 + 2.0 = 7.0$ ，小于 8；因此该网络满足约束。

被驱动 cell (U3) 施加的 fanout load 不一定为 1.0。library 可以定义更高的 fanout load，以建模 cell 的内部 fanout effect。也可在 output port (OUT1) 上设置 fanout load，以建模 external fanout effect。

要复位在输入端口或设计上设置的最大扇出值，使用 `remove_attribute` 命令。例如：

```
de_shell> remove_attribute [get_ports port_name] max_fanout
de_shell> remove_attribute [get_designs design_name] max_fanout
```

为输出端口定义预期扇出

### Defining Expected Fanout for Output Ports

要在输出端口上指定外部扇出负载，使用 `set_fanout_load` 命令设置 `fanout_load` 属性。DC Explorer 将该扇出负载值加到驱动各指定端口的引脚上的所有其他负载，并尽量使总负载小于该引脚的最大扇出负载。

例如，下列命令为所有输出设置 2 单位的扇出负载：

```
de_shell> set_fanout_load 2 [all_outputs]
```

要复位 `set_fanout_load` 在输出端口上设置的值，使用 `remove_attribute` 命令。例如：

```
de_shell> remove_attribute port_name fanout_load
```

要确定扇出负载，使用 `get_attribute` 命令。下例查找 `libA` 库中 `AND2` 库单元输入引脚上的扇出负载：

```
de_shell> get_attribute "libA/AND2/i" fanout_load
```

要查找逻辑库 `libA` 上设置的默认扇出负载，输入：

```
de_shell> get_attribute libA default_fanout_load
```

#### 参见

- 设计规则约束
- 约束优先级

## 7.2.3 定义最大电容

### Defining Maximum Capacitance

要为连接到具名端口的网络或所有网络指定最大电容，使用 `set_max_capacitance` 命令设置 `max_capacitance` 属性。可用 `set_max_capacitance` 在输入端口或设计上指定电容值。该值应小于或等于驱动该网络的引脚的 `max_capacitance` 属性值。

例如，要将 `adder` 设计的最大电容设为 3 单位，输入：

```
de_shell> set_max_capacitance 3 [get_designs adder]
```

要复位 `set_max_capacitance` 设置的值，使用 `remove_attribute` 命令：

```
de_shell> remove_attribute [get_designs adder] max_capacitance
```

**参见**

- 设计规则约束
- 约束优先级

## 7.2.4 定义最小电容

### Defining Minimum Capacitance

要为连接到输入或双向端口的网络指定最小电容,使用 `set_min_capacitance` 命令设置 `min_capacitance` 属性。若库 `min_capacitance` 属性与输入端口 `min_capacitance` 属性同时存在, DC Explorer 尝试满足更严格的值。默认情况下,最小电容约束的优先级高于最大转换时间、最大扇出与最大电容约束。

下例在 `high_drive` 端口上设置最小电容值为 12.0 单位:

```
de_shell> set_min_capacitance 12.0 high_drive
```

要仅报告最小电容约束信息,对 `report_constraint` 使用 `-min_capacitance` 选项。要获取当前端口设置信息,使用 `report_port` 命令。要复位 `set_min_capacitance` 的设置,使用 `remove_attribute` 命令。

**参见**

- 设计规则约束
- 约束优先级

## 7.2.5 定义单元退化

### Defining Cell Degradation

要在输入端口上显式设置 `cell_degradation` 属性,使用 `set_cell_degradation` 命令。`cell_degradation` 属性要求网络电容值小于单元退化值。

可将单元退化约束与其他约束一起使用,但最大电容约束的优先级高于单元退化约束。

**注意**

使用 `set_cell_degradation` 命令需要 DC Ultra 许可证。

下例在 `late_riser` 端口上设置最大电容值为 2.0 单位:

```
de_shell> set_cell_degradation 2.0 late_riser
```

要仅报告单元退化约束信息,对 `report_constraint` 使用 `-cell_degradation` 选项。要移除 `cell_degradation` 属性,使用 `remove_attribute` 命令。

**参见**

- 设计规则约束
- 约束优先级

## 7.2.6 定义连接类

### Defining Connection Class

端口上的连接类约束描述给定工艺的连接要求。只有具有相同连接类标签的负载与驱动器才能合法连接。该约束可在库中指定。要在端口上设置连接类约束，使用 `set_connection_class` 命令。

例如，下列命令在当前设计的 `xyz` 端口上设置名为 `internal` 的连接类：

```
de_shell> set_connection_class internal xyz
```

#### 参见

- 设计规则约束
- 约束优先级

## 7.3 对特殊网络禁用设计规则修复

### Disabling Design Rule Fixing on Special Nets

可使用 `set_auto_disable_drc_nets` 命令对时钟、常数或扫描网络启用或禁用设计规则修复。可对当前设计中给定类型的所有网络这样做，例如所有时钟网络、所有常数网络、所有扫描网络，或这三类网络的任意组合。被设置为禁用设计规则修复的网络会标记 `auto_disable_drc_nets` 属性。默认情况下，工具对时钟与常数网络禁用设计规则修复，但不对扫描网络禁用。

#### 注意

时钟网络默认是理想网络。使用 `set_auto_disable_drc_nets` 启用设计规则修复不会影响时钟网络的理想时序属性。必须使用 `set_propagated_clock` 命令更改时钟网络的理想时序。

不能使用 `set_auto_disable_drc_nets` 命令覆盖带有 `ideal_networks` 属性的理想网络上已禁用的设计规则修复。该命令从不覆盖由 `set_ideal_net` 或 `set_ideal_network` 命令指定的设置。

#### 参见

- 设计规则约束
- 定义设计规则约束

## 7.4 在层次化设计中传播约束

### Propagating Constraints in Hierarchical Designs

层次化设计由子设计组成。可使用下列方法沿层次向上或向下传播约束：

- 表征子设计（**Characterizing Subdesigns**）

表征方法捕获特定单元实例的环境信息，并将信息作为属性赋给这些单元所链接的设计。

- 建模 (**Modeling**)  
建模方法将已表征设计创建为库单元。
- 沿层次向上传播约束  
该方法将时钟、时序例外与禁用时序弧从较低层子设计传播到当前设计。

### 7.4.1 表征子设计

#### Characterizing Subdesigns

当分别综合子设计时，边界条件（如输入驱动强度、输入信号延时（到达时间）与输出负载）可从父设计推导并设置到各子设计上。可使用 `set_driving_cell`、`set_input_delay`、`set_output_delay` 与 `set_load` 命令完成。

多数情况下，表征设计会移除先前表征的效果并替换相关信息。但在后标注（`set_load`、`set_resistance`、`read_timing`、`set_annotated_delay`、`set_annotated_check`）情况下，表征会移除标注，且不能覆盖子设计上已有标注。此时必须在再次表征前使用 `reset_design` 命令显式移除子设计上的标注。

表征子设计的端口信号接口

#### Characterizing Port Signal Interfaces of Subdesigns

可使用下列命令手动设置端口信号信息：

```
create_clock
set_clock_latency
set_clock_uncertainty
set_driving_cell
set_input_delay
set_load
set_max_delay
set_min_delay
set_output_delay
set_propagated_clock
```

组合设计示例

**Combinational Design Example** 图 7.4 给出名为 `top` 的组合设计中的子设计块。



图 7.4: 表征组合设计的驱动、时序与负载值

下列脚本给出如何手动设置端口接口属性的示例：

```
current_design top
set_input_delay 0 A
set_max_delay 10 -to B
current_design block1
set_driving_cell -lib_cell INV {C}
set_input_delay 3.3 C
set_load 1.3 D
set_max_delay 9.2 -to D
current_design top
```

该脚本将 n2 网络的到达时间设为 3.3 单位，将 U3 的负载设为 1.3 单位，并将继承的最大延时设为  $10 - 0.8 = 9.2$  单位（每个反相器 0.4）。

时序设计示例

**Sequential Design Example** 图 7.5 给出名为 top 的时序设计中的子设计块。

（原书 *Figure 31: Characterizing Sequential Design Drive, Timing, and Load Values*）  
请参阅原版 PDF 插图；可将截图放入 figures/ 后用 \includegraphics 替换。

图 7.5: 表征时序设计的驱动、时序与负载值

下列脚本给出如何手动设置端口接口信息的示例：

```
current_design top
create_clock -period 10 -waveform {0 5} clock
current_design block
create_clock -name clock -period 10 -waveform {0 5} clock1
create_clock -name clock_bar -period 10 \
-waveform {5 10} clock2
set_input_delay -clock clock 1.8 in1
set_output_delay -clock clock 1.2 out1
set_driving_cell -lib_cell INV -input_transition_rise 1 in1
set_driving_cell -lib_cell CKINV clock1
set_driving_cell -lib_cell CKBUF clock2
set_load 0.85 out1
current_design top
```

该脚本将从 ff1/CP 到 u5/in1 的输入延时设为 1.8 单位，将经 u4 加上 ff2 setup 时间的输出延时设为 1.2 单位，并将 out1 网络上的外部负载设为 0.85 单位。

表征子设计逻辑端口连接的示例

**Example of Characterizing Logical Port Connections of Subdesigns** 图 7.6 给出名为 top 的设计中的子设计块。

(原书 *Figure 32: Characterizing Port Connection Attributes*)  
 请参阅原版 PDF 插图；可将截图放入 figures/ 后用 \includegraphics 替换。

图 7.6: 表征端口连接属性

可使用下列脚本手动设置逻辑端口连接：

```
current_design block
set_opposite    { E F }
set_logic_zero  { G }
set_unconnected { J }
```

## 7.4.2 沿层次向上传播约束

### Propagating Constraints Up the Hierarchy

对层次化设计,若先综合较低层块再综合较高层块(自底向上综合),可使用 `propagate_constraints` 命令将时钟、时序例外与禁用时序弧从较低层 .ddc 文件传播到当前设计。该命令将约束从较低层块传播到当前设计。若不使用该命令,则无法将较低层块上设置的约束传播到实例化它的较高层块。

#### 注意

使用 `propagate_constraints` 命令可能增加内存使用。

下列脚本给出向上传播约束的方法。假定 A 为顶层块, B 为较低层块。

```
current_design B
source constraints.tcl
compile
current_design A
propagate_constraints -design B
report_timing_requirements
compile
report_timing
```

要生成所有向上传播约束的报告,使用 `-verbose` 与 `-dont_apply` 选项并将输出重定向到文件。例如:

```
de_shell> propagate_constraints \
  -design name -verbose -dont_apply -output report.cons
```

使用 `-format ddc` 选项保存带已传播约束的 .ddc 文件,这样在重启新的 `de_shell` 会话时无需重复传播步骤。

设计之间的冲突

### Conflicts Between Designs

lower-level block 与 top-level block 发生冲突时，适用下列规则：

- 时钟名冲突  
当较低层时钟与当前设计或另一块的时钟同名时，该时钟不传播。DC Explorer 发出警告。
- 时钟源冲突  
当较低层块的时钟源已被定义为较高层块的时钟源时，较低层块不传播。DC Explorer 发出警告。
- 来自或指向未传播时钟的例外  
该时钟可以是 virtual clock，也可以是因冲突而未从该 block 传播的 clock。
- 例外  
较低层例外会覆盖在同一路径上定义的较高层例外。

# Chapter 8

## UPF 探索

### *UPF Exploration*

DC Explorer 接受 minimal UPF，并根据 power state table 和 domain supply voltage 自动生成 isolation strategy 与 level-shifter strategy。DC Explorer 生成的 RTL UPF 可用于 floorplanning、synthesis 和 physical implementation。

要运行 UPF 探索流程，请确保 `de_enable_upf_exploration` 变量设为 `true`（默认值）。

下列主题描述 UPF 探索流程：

- UPF 探索流程概览
- 最小 UPF 文件
- 带 UPF 的探索综合
- 报告功耗探索结果

有关 UPF 多电压设计实现，请参阅 *Power Compiler User Guide*。

### 8.1 UPF 探索流程概览

#### **UPF Exploration Flow Overview**

使用仅包含 power domain 和 power state 定义的 minimal UPF 文件，DC Explorer 会生成完整的 RTL UPF 文件和 floorplan；该 RTL UPF 包含 isolation strategy 与 level-shifter strategy，且新增 strategy 使用 RTL name。该文件可作为 Design Compiler 与 IC Compiler synthesis/implementation flow 的 golden UPF。也可修改该文件，以更改工具创建的默认 isolation/level-shifter strategy，或添加 retention strategy。流程概要见图 8.1。

要了解如何运行 UPF 探索流程，见：

- 带 UPF 的设计探索
- 运行带 UPF 的设计探索
- 隔离与电平移位插入

(原书 *Figure 33: Design Exploration, Synthesis, and Implementation Flow With UPF*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 8.1: 带 UPF 的设计探索、综合与实现流程

更多信息见 *Synopsys Multivoltage Flow User Guide*。

#### 参见

- SolvNet 文章 24582, 《UPF in DC Explorer Application Note》
- 第六章「使用 UPF 的多电压设计」

### 8.1.1 带 UPF 的设计探索

#### Design Exploration With UPF

为 DC Explorer 指定 UPF 功耗意图,与为 Design Compiler 或 IC Compiler 指定的方式不同。对于设计探索,您只需使用 `create_power_domain` 命令指定电源域,并使用 `add_power_state`、`create_pst` 与 `add_pst_state` 命令指定电源状态。这一完整功耗意图的基本子集称为最小 UPF (minimal UPF)。

利用 minimal UPF, DC Explorer 会根据是否存在 power-down domain 以及 domain 之间的 supply-voltage difference, 自动生成设计所需的 isolation strategy 与 level-shifter strategy。工具以完整形式写出修改后的 UPF, 可在 Design Compiler 与 IC Compiler 中用作 design power intent specification。

#### DC Explorer 使用的输入数据

##### Input Data Used by DC Explorer

DC Explorer 在 UPF 流程中使用的输入数据类型如下:

- **Design RTL** — exploration flow 使用的 RTL 不应包含任何 power-management cell (isolation、level shifter 与 retention cell), 也不应包含这些 cell 的 control signal。若存在此类 cell, 工具会将其作为普通 standard cell 处理, 并忽略其 power-management function。
- **Synopsys design constraint** — SDC 文件必须包含 `set_voltage` 命令, 为各 power domain 的每条 supply net 设置电压; 还必须包含 `set_operating_conditions` 命令, 以设置 top-level operating condition。
- **Minimal UPF** — minimal UPF 文件必须包含定义 power domain 与 power state 的命令, 但不应包含 isolation、level-shifter 或 retention 命令; 若包含, 工具将忽略这些命令。

## 面向综合与物理实现的输出数据

### Output Data for Synthesis and Physical Implementation

带 UPF 的设计探索流程为综合与物理实现提供下列数据：

- **RTL UPF** —可将 RTL UPF 用作综合与物理实现流程的 golden UPF。可按需修改 RTL UPF。例如，探索流程对隔离策略使用 `location self`；若更偏好 `location parent`，可自行更改。
- **Floorplan** —可用 floorplan 指导 Design Compiler topological 模式中的综合，以及 IC Compiler 中的物理实现。

exploration flow 会自动创建 isolation control signal，并可选地创建用于驱动这些信号的 dummy controller block (black box)。开始 logic synthesis 前，必须提供真实 controller logic 的 RTL，以替换 dummy controller block。controller logic 根据设计中各 domain 当前的 power-down state 使能或禁用 isolation cell。可以在 RTL UPF 中编辑这些 control signal。

带 UPF 的一般探索流程概要见图 8.2。

(原书 Figure 34: Design Exploration Flow With UPF)  
请参阅原版 PDF 插图；可将截图放入 figures/ 后用 \includegraphics 替换。

图 8.2: 带 UPF 的设计探索流程

#### 参见

- 运行带 UPF 的设计探索
- 隔离与电平移位插入

### 8.1.2 运行带 UPF 的设计探索

#### Running Design Exploration With UPF

按下列步骤运行带 UPF 的一般设计探索流程。

1. 读入设计的 RTL 描述；其中不应包含任何电源管理单元、此类单元的控制信号或电源控制器电路。
2. 读入 Synopsys 设计约束 (SDC 文件)，包括 `set_voltage` 命令，并设置顶层工作条件。
3. 使用 `load_upf` 命令读入最小 UPF 文件；其中不应包含任何隔离、电平移位或保持策略。
4. 使用 `generate_rtl_upf` 命令写出完整 UPF 设计意图 (RTL UPF)。

对于输出馈入 active domain 的 power-down domain，DC Explorer 会自动生成 isolation strategy；对于 supply voltage 不同的相邻 power domain，工具会自动生成 level-shifting strategy。（工具不生成也不实现 retention strategy。）

如果没有 operating voltage 合适的 basic logic gate 可用于 link，工具会 link 到 operating voltage 不同的 cell。

5. 使用 `compile_exploration` 命令综合设计。

DC Explorer 按 RTL UPF 文件中的完整 power intent 插入 isolation cell、level shifter 和 always-on buffer。

6. 报告设计的面积、时序与功耗。

可使用新的最小 UPF 文件重复步骤 1 至 6，以探索其他可能的供电架构。

7. 使用 `write_file` 命令写出包含 PG connection 的 gate-level netlist。

8. 使用 `save_upf -supplemental` 命令写出 supplemental UPF 文件。该文件包含设计优化期间被重命名的对象。

9. 执行 floorplan 探索并为设计生成 floorplan，见「执行 Floorplan 探索」。

#### 注意

电源管理单元只能由 `compile_exploration` 命令插入；不支持 `insert_mv_cells` 命令。尽管 DC Explorer 不支持增量 compile，但可将包含电源管理单元的已综合设计以 `.ddc` 格式重新读入工具进行分析。

探索流程完成后，可将生成的 RTL UPF 与 floorplan 用于逻辑综合与物理实现。

#### 参见

- 带 UPF 的设计探索
- 隔离与电平移位插入

### 8.1.3 隔离与电平移位插入

#### Isolation and Level Shifter Insertion

下例说明 DC Explorer 在探索流程中如何插入电源管理单元。

在最小 UPF 文件中，仅指定电源域与电源状态表信息，例如：

```
create_power_domain PD1 -elements U1
create_power_domain PD2 -elements U2
create_power_domain PD3 -elements U3

add_power_state PD1.primary -state HIGH12 \
  {-supply_expr {power == `{FULL_ON, 1.2}}}}
add_power_state PD1.primary -state OFFX \
  {-supply_expr {power == `{OFF}}}}
add_power_state PD1.ground -state GND \
  {-supply_expr {ground == `{FULL_ON, 0.00}}}}

add_power_state PD2.primary -state HIGH12 \
  {-supply_expr {power == `{FULL_ON, 1.2}}}}
add_power_state PD2.primary -state LOW08 \
```

```

    {-supply_expr {power == `{FULL_ON, 0.8}}}}
add_power_state PD2.primary -state OFFX \
    {-supply_expr {power == `{OFF}}}}
add_power_state PD2.ground -state GND \
    {-supply_expr {ground == `{FULL_ON, 0.00}}}}

add_power_state PD3.primary -state HIGH12 \
    {-supply_expr {power == `{FULL_ON, 1.2}}}}
add_power_state PD3.primary -state LOW08 \
    {-supply_expr {power == `{FULL_ON, 0.8}}}}
add_power_state PD3.ground -state GND \
    {-supply_expr {ground == `{FULL_ON, 0.00}}}}

create_pst MY_PST -supplies \
    {PD1.primary.power PD2.primary.power PD3.primary.power}
add_pst_state S1 -pst MY_PST -state {HIGH12 LOW08 LOW08 }
add_pst_state S2 -pst MY_PST -state {OFFX OFFX LOW08 }
add_pst_state S3 -pst MY_PST -state {HIGH12 HIGH12 HIGH12}

```

读入 RTL 设计、SDC 约束与最小 UPF 文件后，使用 `generate_rtl_upf` 命令自动写出包含隔离与电平移位策略的完整 RTL UPF。

使用 `compile_exploration` 命令时，DC Explorer 遵循所生成策略，插入图 8.3 所示的电源管理单元。

(原书 *Figure 35: Isolation and Level Shifter Cells Inserted by DC Explorer*)  
 请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 8.3: DC Explorer 插入的隔离与电平移位单元

工具在每个可关断 domain 的 output 处插入 isolation cell；当 driver domain 的 output 可能以不同于 receiver domain 的电压工作时，插入 level shifter。如果某个 output 同时需要 isolation 和 level-shifter 功能，工具会插入 enable level shifter (ELS)。

在某些情况下，对关断域所有输出插入隔离单元会导致插入实际不需要的隔离单元。例如，插入 U2 块中的 ELS 单元的隔离功能并不需要，因为驱动域 PD2 仅在接收域 PD1 也关断时才关断。

若信号必须经第三电源域从某一电源域传到另一电源域，且连接上需要缓冲器，工具会在网络上插入 always-on buffer，如图 8.4 所示。本例中，always-on buffer 使用信号接收域 PD3 的电源供电。

```

create_power_domain PD1 -elements U1
create_power_domain PD2 -include_scope
create_power_domain PD3 -elements U3

```

(原书 *Figure 36: Always-On Buffer Inserted by the DC Explorer Tool*)  
请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 8.4: DC Explorer 工具插入的 Always-On Buffer

#### 参见

- 带 UPF 的设计探索
- 运行带 UPF 的设计探索

## 8.2 最小 UPF 文件

### Minimal UPF File

在设计探索流程中仅指定最小 UPF，可快速理解、分析并比较各种供电架构决策的影响，例如是否对某区域下电，或使用不同供电电压。可以及早获得各 domain 及整个系统的 area、power 和 leakage 估计。

最小 UPF 文件可能包含下列内容：

- 必需命令
- 可选命令
- 被忽略的命令
- 被忽略的供电网络域依赖性

### 8.2.1 必需命令

#### Required Commands

最小 UPF 需要下列内容：

- 由 `create_power_domain` 命令指定的电源域定义
- 由 `add_power_state` 或 `add_port_state` 命令指定的各电源域主供电的电源状态
- 由 `create_pst` 与 `add_pst_state` 命令指定的、定义所有可能电源状态的电源状态表

除提供指定电源域、电源状态与电源状态表的 UPF 命令外，还需使用 `set_voltage` 命令指定主供电电压，并使用 `set_operating_conditions` 命令指定顶层工作条件。

## 8.2.2 可选命令

### Optional Commands

最小 UPF 文件可包含指定供电网络、供电集合、供电端口、供电连接与电源开关的命令。DC Explorer 支持下列 UPF 命令，但在设计探索中使用这些命令是可选的：

```
associate_supply_set
connect_supply_net
create_power_switch
create_supply_net
create_supply_port
create_supply_set
load_upf
name_format
set_port_attributes
set_related_supply_net
set_scope
```

`create_power_domain` 命令会为指定域自动创建默认隔离供电集合句柄 `default_isolation`。DC Explorer 将该供电集合句柄用于该域的隔离策略，并自动生成 `set_voltage` 命令以设置该供电集合的电压。（在其他工具中，可使用 `set_isolation` 命令显式指定其他供电集合或供电网络。）

为 `default_isolation` 句柄指定电源状态是可选的。若未指定这些电源状态，工具会根据主供电的电源状态表自动生成。

`set_port_attributes` 命令的 `-repeater_supply` 选项被忽略。

## 8.2.3 被忽略的命令

### Ignored Commands

最小 UPF 文件不应包含隔离、电平移位或保持策略命令。DC Explorer 忽略下列 UPF 命令：

```
map_isolation_cell
map_level_shifter_cell
map_power_switch
map_retention_cell
set_isolation
set_isolation_control
set_level_shifter
set_retention
set_retention_control
```

工具会读入并接受这些命令，但不会使用它们。使用 `generate_rtl_upf` 生成 RTL UPF 文件时，工具将这些命令写为注释行。例如，若最小 UPF 文件包含：

```
set_isolation IS01 -domain PD1
```

工具在 RTL UPF 文件中写出下列注释行：

```
# set_isolation IS01 -domain PD1
```

## 8.2.4 被忽略的供电网络域依赖性

### Ignored Domain Dependency of Supply Nets

在 DC Explorer 中，隔离单元的自动推断依赖于在所有电源域中可用的供电集合句柄与隐式供电集合。工具不支持 domain-dependent supply set；在 Design Compiler 与 IC Compiler 中，此类 supply set 由带 -domain 选项的 create\_supply\_net 命令创建。

例如，最小 UPF 文件包含下列使用 -domain 选项将供电网络 VDD1 的使用限制在电源域 PD1 的命令：

```
create_supply_net VDD1 -domain PD1
```

运行 generate\_rtl\_upf 时，DC Explorer 在 RTL UPF 文件中先将带 -domain 选项的 create\_supply\_net 写为注释行，再写出去掉该选项的修改后命令。例如：

```
#create_supply_net VDD1 -domain PD1
create_supply_net VDD1
```

## 8.3 带 UPF 的探索综合

### Exploration Synthesis With UPF

在 DC Explorer 中，generate\_rtl\_upf 命令写出包含设计完整意图（含隔离与电平移位策略）的 UPF 文件。该文件称为 RTL UPF，因为它用 RTL 名称表示自动生成的隔离与电平移位策略所对应的设计功耗意图。

compile\_exploration 命令对设计执行逻辑综合，包括以下电源管理任务：隔离插入、电平移位插入与 always-on buffer 插入。有关带 UPF 的探索综合的更多信息，见 *UPF in DC Explorer Application Note*。

#### 注意

设计探索流程不实现保持策略。探索流程完成后，可向 RTL UPF 添加保持策略，供 Design Compiler 与 IC Compiler 使用。

不支持 well-bias 功能。在 Design Compiler 中，该功能用于实现 well-bias supply 与 primary supply，并通过将 enable\_bias design attribute 设为 true 启用。

在自底向上流程中，优化各块时需定义该块的边界条件。对于 UPF，使用 set\_related\_supply\_net、set\_port\_attributes -driver\_supply 与 set\_port\_attributes -receiver\_supply 命令指定边界条件。这些命令指定块输入处外部驱动器的供电，以及块输出处外部负载的供电。

#### 参见

- SolvNetPlus 文章 24582，《UPF in DC Explorer Application Note》

## 8.4 报告功耗探索结果

### Reporting Power Exploration Results

可随时使用 `check_mv_design` 命令检查供电架构。使用 `compile_exploration` 综合拟议设计后，可使用 `report_area`、`report_timing` 与 `report_mv_qor` 命令估计面积、时序与功耗。

要单独获取隔离单元或电平移位单元的漏电功耗，使用 `report_mv_qor` 命令。默认情况下，`report_mv_qor` 报告所有电源域中的总漏电功耗、面积与单元数。指定 `-verbose` 选项时，命令分别报告隔离单元、电平移位单元、always-on 单元、宏、I/O pad 与标准单元的漏电功耗。

#### 8.4.1 Design Vision GUI

##### Design Vision GUI

可使用 Design Vision GUI（由 `gui_start` 命令打开）查看设计的 UPF 意图并分析设计问题。要选择 GUI 分析工具，使用 Power 下拉菜单。

有关 GUI 分析功能的更多信息，见 *Design Vision User Guide*。

# Chapter 9

## 使用 Floorplan 物理约束

### *Using Floorplan Physical Constraints*

使用 floorplan 物理约束的主要原因，是在优化期间考虑这些约束，从而改善与 IC Compiler 等布局后工具的时序相关性。可通过下列方法之一向 DC Explorer 提供 floorplan 物理约束：

- 从 IC Compiler 以 DEF 文件或 Tcl 脚本导出 floorplan 信息，并导入 DC Explorer
  - 手动创建这些约束
- 本章包括以下主题：
- 导入 Floorplan 信息
  - 物理约束概览
  - 保存物理约束
  - 以 ASCII 格式保存供 IC Compiler II 使用
  - 包含仅物理单元（Physical-Only Cells）
  - 相对布局（Relative Placement）

### 9.1 导入 Floorplan 信息

#### Importing Floorplan Information

可通过下列方法之一将 floorplan 信息导入 DC Explorer：

- 使用 **DEF** 文件  
在 IC Compiler 中用 `write_def` 导出 floorplan，再用 `extract_physical_constraints` 导入 DC Explorer。
  - 在 IC Compiler 中使用 `write_def`
  - 在 DC Explorer 中读取 DEF 信息
- 使用 **Tcl** 脚本

- 在 IC Compiler 中保存 Floorplan 信息
- 在 DC Explorer 中读取 Floorplan 脚本

#### 参见

- 导入的 DEF 信息
- 导入的物理约束

### 9.1.1 在 IC Compiler 中使用 write\_def 命令

#### Using the write\_def Command in IC Compiler

为改善 DC Explorer 与 IC Compiler 之间的时序、面积与功耗相关性，可将已映射的 DC Explorer 网表读入 IC Compiler，在 IC Compiler 中创建基本 floorplan，再导出该 floorplan 并读回 DC Explorer。

在 IC Compiler 中使用 write\_def 将 floorplan 导出为 Design Exchange Format (DEF) 文件，以便读入 DC Explorer。下例将 floorplan 写入 my\_physical\_data.def:

```
icc_shell> write_def -version 5.7 -rows_tracks_gcells -macro -pins \
    -blockages -specialnets -vias -regions_groups -verbose \
    -output my_physical_data.def
```

### 9.1.2 在 DC Explorer 中读取 DEF 信息

#### Reading DEF Information in DC Explorer

要从 DEF 文件导入 floorplan，使用 extract\_physical\_constraints 命令。该命令从指定 DEF 导入 floorplan 并施加到设计；已施加的 floorplan 会保存在 .ddc 中，后续 de\_shell 会话读入该 .ddc 时无需重新施加。

从多个 DEF 导入物理约束：

```
de_shell> extract_physical_constraints {des_1.def des2.def ... des_N.def}
```

默认以增量模式运行：处理多个 DEF 时保留设计上已有物理注释。增量模式下，布局区域基于当前 core area 与 DEF 中的 site row 导入；禁用增量模式时，布局区域仅基于 DEF 中的 site row。冲突按下列方式解决：

- 只能有一个取值的物理约束会被最新 DEF 的值覆盖（如端口位置、宏位置）。
- 可累加取值的物理约束会重新计算（如 core area 可基于已有值与最新 DEF 的 site row 定义重算）。不同 DEF 的 placement keepout 会累加，最终 keepout 几何在综合期间内部计算。

图 9.1 给出 extract\_physical\_constraints 增量提取示例。

```
de_shell> extract_physical_constraints DEF1.def DEF2.def
```

要禁用增量模式，对 extract\_physical\_constraints 指定 -no\_incremental。

有关导入的 DEF 信息以及端口与宏名称匹配，见：

- 导入的 DEF 信息
- extract\_physical\_constraints 的名称匹配

(原书 *Figure 37: Incremental Extraction With the extract\_physical\_constraints Command*)  
请参阅原版 PDF 插图；可将截图放入 figures/ 后用 \includegraphics 替换。

图 9.1: 用 extract\_physical\_constraints 进行增量提取

- 解决 Site 名称不匹配

### 9.1.3 导入的 DEF 信息

#### Imported DEF Information

DC Explorer 支持高层 floorplan 物理约束，例如 die area、core area 与形状、端口位置、宏位置与方向、keepout margin、placement blockage、preroute、bound、via、track、voltage area 与 wiring keepout:

- Die Area
- Placement Area
- Macro Location and Orientation
- Hard、Soft 与 Partial Placement Blockage
- Wiring Keepout
- Placement Bound
- Port Location
- Preroute
- Site Array Information
- Via
- Routing Track
- Keepout Margin

要直观检查提取的物理约束，使用 Design Vision layout 窗口中的 layout 视图；从 DEF 提取的全部物理约束会自动加入 layout 视图。

#### 注意

Voltage area 不在 DEF 中定义。因此对多电压设计，须用 create\_voltage\_area 定义 voltage area。若以 IC Compiler 做 floorplanning，可用 write\_floorplan 与 read\_floorplan 获取自动包含 voltage area 的 floorplan，无需手动定义。

## Die Area

### Die Area

`extract_physical_constraints` 支持从 DEF 自动提取 die area。DEF 由 floorplanning 工具生成，供 DC Explorer 导入物理约束。工具可提取 DEF 中定义的矩形与折线形 die area。Die area 亦称 cell boundary，表示芯片硅边界，包围焊盘、I/O 引脚与单元等全部对象。

例 9.1 给出 DEF 中的 die area 定义；例 9.2 给出用 `write_floorplan -all` 写出时 DC Explorer 译成的 Tcl。

```
UNITS DISTANCE MICRONS 1000 ;
DIEAREA ( 0 0 ) ( 0 60000 ) ( 39680 60000 ) ( 39680 40000 ) \
        ( 59360 40000 ) ( 59360 0 ) ;
```

代码 9.1: DEF 中的 Die Area 定义

```
de_shell> create_die_area -polygon { { 0.000 0.000 } { 0.000 60.000 } \
    { 39.680 60.000 } { 39.680 40.000 } { 59.360 40.000 } \
    { 59.360 0.000 } { 0.000 0.000 } }
```

代码 9.2: 写出物理约束时的等价 Tcl

## Placement Area

### Placement Area

Placement area 计算为 site row 的矩形包围盒。

### 宏位置与方向

### Macro Location and Orientation

使用 `extract_physical_constraints` 时，对 DEF 中带位置且指定 `FIXED` 属性的每个单元，DC Explorer 在设计中对应单元上设置位置。例 9.3 中 E/W 分别表示东/西旋转；等价 Tcl 见例 9.4。

```
COMPONENTS 2 ;
- macro_cell_abx2 + FIXED ( 4350720 8160 ) E ;
- macro_cell_cdy1 + FIXED ( 4800 8160 ) W ;
END COMPONENTS
```

代码 9.3: DEF 宏位置与方向

```
de_shell> set_cell_location macro_cell_abx2 \
    -coordinates { 4350.720 8.160 } -orientation E -fixed
de_shell> set_cell_location macro_cell_cdy1 \
    -coordinates { 4.800 8.160 } -orientation W -fixed
```

代码 9.4: 等价 Tcl

## Hard、Soft 与 Partial Placement Blockage

### Hard, Soft, and Partial Placement Blockages

`extract_physical_constraints` 可导入 DEF 中定义的 hard、soft 与 partial placement blockage。

#### 注意

DEF 5.7 之前的版本不支持 partial blockage。此外，若 floorplanning 工具生成 DEF 5.6，需手动添加 `#SNPS_SOFT_BLOCKAGE` pragma 以指定 soft blockage。

例 9.5 为 hard blockage 的 DEF；例 9.6 为等价 Tcl。

```
BLOCKAGES 50 ;
...
- PLACEMENT RECT ( 970460 7500 ) ( 3247660 129940 )
...
END BLOCKAGES
```

代码 9.5: DEF hard placement blockage

```
de_shell> create_placement_blockage -name def_obstruction_23 \
-bbox { 970.460 7.500 3247.660 129.940 }
```

代码 9.6: 等价 Tcl

对 soft blockage: DEF 5.7 见例 9.7；DEF 5.6 见例 9.8；等价 Tcl 见例 9.9。

```
BLOCKAGES 50 ;
...
- PLACEMENT + SOFT RECT ( 970460 7500 ) ( 3247660 129940 ) ;
...
END BLOCKAGES
```

代码 9.7: DEF 5.7 soft placement blockage

```
BLOCKAGES 50 ;
...
- PLACEMENT RECT ( 970460 7500 ) ( 3247660 129940 ) ;
#SNPS_SOFT_BLOCKAGE
...
END BLOCKAGES
```

代码 9.8: DEF 5.6 soft placement blockage

```
de_shell> create_placement_blockage -name def_obstruction_23 \
-bbox { 970.460 7.500 3247.660 129.940 } -type soft
```

代码 9.9: soft blockage 等价 Tcl

对 partial blockage (例 9.10)，等价 Tcl 见例 9.11。

```
BLOCKAGES 50 ;
...
```

```
- PLACEMENT + PARTIAL 80 RECT ( 970460 7500 ) ( 3247660 129940 ) ;
...
END BLOCKAGES
```

代码 9.10: DEF partial placement blockage

```
de_shell> create_placement_blockage -name def_obstruction_23 \
-bbox { 970.460 7.500 3247.660 129.940 } -type partial \
-blocked_percentage 80
```

代码 9.11: partial blockage 等价 Tcl

## Wiring Keepout

### Wiring Keepouts

对 DEF 中定义的 wiring keepout, DC Explorer 在设计上创建 wiring keepout。例 9.12 给出 DEF 信息。

```
BLOCKAGES 30 ;
...
- LAYER METAL6 RECT ( 0 495720 ) ( 4050 1419120 );
...
END BLOCKAGES
```

代码 9.12: DEF wiring keepout

## Placement Bound

### Placement Bounds

若 DEF 中存在定义 bound 的 REGIONS, `extract_physical_constraints` 会导入 placement bound。若相关 GROUP 中有单元附着到该 region, 则在这些单元与设计中的单元之间进行模糊名称匹配, 并以两种方式附着到 bound:

- 若设计中已有与 DEF 同名的 region, 则以增量模式用 `update_bounds` 将相关 group 中的单元附着到该 region。
- 若设计中不存在该 region, 则用 `create_bounds` 按 DEF 同名创建, 并附着匹配单元。

```
REGIONS 1 ;
- c20_group ( 201970 40040 ) ( 237914 75984 ) + TYPE FENCE ;
END REGIONS
GROUPS 1 ;
- c20_group
  cell_abc1
  cell_sm1
  cell_sm2
  + SOFT
  + REGION c20_group ;
END GROUPS
```

代码 9.13: 导入的 placement bound 信息

```
de_shell> create_bounds -name "c20_group" \
  -coordinate {201970 40040 237914 75984} \
  -exclusive {cell_abc1 cell_sm1 cell_sm2}
```

代码 9.14: 等价 Tcl

端口位置

## Port Location

对 DEF 中指定位置的每个端口，DC Explorer 在设计对应端口上设置位置。

```
PINS 2 ;
-OUT1 + NET Out1 + DIRECTION OUTPUT + USE SIGNAL +
  LAYER M3 (0 0) (4200 200) + PLACED (80875 0) N;
-Sel0 + NET Sel0 + DIRECTION INPUT + USE SIGNAL +
  LAYER M4 (0 0) (200 200) + PLACED (135920 42475) N;
END PINS
```

代码 9.15: 导入的端口位置

```
de_shell> set_port_location Out1 -coordinate {80.875 0.000} \
  -layer_name M3 -layer_area {0.000 0.000 4.200 0.200}
de_shell> set_port_location Sel0 -coordinate {135.920 0.000} \
  -layer_name M4 -layer_area {0.000 0.000 0.200 0.200}
```

代码 9.16: 等价 Tcl

支持更名端口与多层端口。例 9.17 与例 9.18 给出此类情况；也支持端口方向（例 9.19）。

```
PINS 2 ;
- sys_addr\[23\].extra2 + NET sys_addr[23] + DIRECTION INPUT +USE SIGNAL
+ LAYER METAL4 ( 0 0 ) ( 820 5820 ) + FIXED ( 1587825 2744180 ) N ;
- sys_addr[23] + NET sys_addr[23] + DIRECTION INPUT + USE SIGNAL + LAYER
  METAL3 ( 0 0 ) ( 820 5820 ) + FIXED ( 1587825 2744180 ) N ;
END PINS
```

代码 9.17: 更名与多层端口的 DEF

```
de_shell> set_port_location sys_addr[23] \
  -coordinate { 1587.825 2744.180 } \
  -layer_name METAL3 -layer_area {0.000 0.000 0.820 5.820}
de_shell> set_port_location sys_addr[23] \
  -coordinate { 1587.825 2744.180 } \
  -layer_name METAL4 -layer_area {0.000 0.000 0.820 5.820} -append
```

代码 9.18: 多层端口等价 Tcl

```
PINS 1;
- OUT + NET OUT + DIRECTION INPUT + USE SIGNAL
+ LAYER m4 ( -120 0 ) ( 120 240 )
+ FIXED ( 4557120 1726080 ) S ;
END PINS
de_shell> set_port_location OUT -coordinate { 4557.120 1726.080 } \
-layer_name m4 -layer_area {-0.120 -0.240 0.120 0.000}
```

代码 9.19: 带方向的端口 DEF 与 Tcl

## Preroute

### Preroutes

DC Explorer 提取 DEF 中定义的 preroute。

```
SPECIALNETS 2 ;
- vdd
+ ROUTED METAL3 10000 + SHAPE STRIPE ( 10000 150000 ) ( 50000 * )
+ USE POWER ;
...
END SPECIALNETS
```

代码 9.20: 导入的 preroute

```
de_shell> create_net_shape -type path -net vdd -datatype 0 -path_type 0 \
-route_type pg_strap -layer METAL3 -width 10.000 \
-points {{10.000 150.000} {50.000 150.000}}
```

代码 9.21: 等价 Tcl

## Site Array 信息

### Site Array Information

DC Explorer 导入 DEF 中定义的 site array; site array 定义布局区域。

```
ROW ROW_0 core 0 0 N DO 838 BY 1 STEP 560 0;
de_shell> create_site_row -name ROW_0 -coordinate {0.000 0.000} \
-kind core -orient 0 -dir H -space 0.560 -count 838
```

代码 9.22: Site array DEF 与等价 Tcl

## Via

### Vias

extract\_physical\_constraints 提取 DEF 中定义的 via, 并以与其他物理约束相同的方式存入 .ddc。用 write\_floorplan -all 写出时的 Tcl 示例如下:

```
de_shell> create_via -type via -net VDD -master VIA67 \
  -route_type pg_strap -at {746.61 2279} -orient N
de_shell> create_via -type via_array -net VDD -master FATVIA45 \
  -route_type pg_strap -at {1491.79 2127.8} -orient N -col 5 -row 2 \
  -x_pitch 0.23 -y_pitch 0.23
```

## 布线 Track

### Routing Tracks

`extract_physical_constraints` 提取 DEF 中的 track 信息。Track 定义基于标准单元设计的布线网格，可用于布线；track 支持可增强 DC Explorer 中的拥塞评估与报告，使拥塞布线更精确并与 IC Compiler 更接近。Track 信息以与其他物理约束相同方式存入 .ddc。若 floorplan 含 track 类型与位置等信息，在 floorplan exploration 期间会传给 IC Compiler。

DEF 中的 track 数据示例：

```
TRACKS X 330 DO 457 STEP 660 LAYER METAL1 ;
TRACKS Y 280 DO 540 STEP 560 LAYER METAL1 ;
```

```
de_shell> create_track -layer metal1 -dir X -coord 0.100 -space 0.200 \
  -count 11577 -bounding_box {{0.000 0.000} {2315.400 2315.200}}
de_shell> create_track -layer metal1 -dir Y -coord 0.200 -space 0.200 \
  -count 11575 -bounding_box {{0.000 0.000} {2315.400 2315.200}}
```

## Keepout Margin

### Keepout Margins

`extract_physical_constraints` 提取 DEF 中定义的 keepout margin，并以与其他物理约束相同方式存入 .ddc。

```
COMPONENTS 2 ;
- U542 OAI21XL + FIXED( 80000 80000 ) FN + HALO 10000 10000 50000 50000 ;
- U543 OAI21XL + FIXED( 10000 20000 ) FN + HALO SOFT 15000 15000 15000
  15000 ;
END COMPONENTS
```

### 参见

- 在 DC Explorer 中读取 DEF 信息
- 定义物理约束

## 9.1.4 extract\_physical\_constraints 的名称匹配

### Name Matching of extract\_physical\_constraints

`extract_physical_constraints` 施加物理约束时，将 DEF 中的宏与端口与内存中的宏与端口匹配；找不到精确匹配时使用智能名称匹配。

该命令读取的 DEF 可能来自与内存网表对象名不同的网表；名称不匹配可能由自动 ungrouping 与 change\_names 引起，常见来源是层次分隔符与总线记号。

例如，compile\_exploration 自动 ungrouping 后再执行 change\_names，可能把正斜杠 (/) 分隔符替换为下划线(\_)。因此 RTL 中名为 a/b/c/macro\_name 的宏在已映射网表中可能变为 a/b\_c\_macro\_name。从 DEF 提取时，命令通过智能名称匹配自动消解这些差异。

默认下列字符视为等价：

- 层次分隔符 { / \_ . }

例如单元 a.b\_c/d\_e 自动匹配 DEF 中的 a/b\_c.d/e。

- 总线记号 { [ ] \_\_ ( ) }

例如单元 a [4] [5] 自动匹配 DEF 中的 a\_4\_\_5\_。

要禁用智能名称匹配，可对 extract\_physical\_constraints 指定 -exact，要求内存网表对象与 DEF 中对应对象精确匹配。

使用 -verbose 时，工具显示如下信息消息：

```
Information: Fuzzy match cell %s in netlist with instance
%s in DEF.
```

要用 set\_query\_rules 定义智能名称匹配所用规则。

#### 参见

解决 Site 名称不匹配（下文）。

### 9.1.5 解决 Site 名称不匹配

#### Resolving Site Names Mismatches

用 extract\_physical\_constraints 读取 DEF 时，DC Explorer 自动将 floorplan 中的 site 名与 Milkyway 参考库中的 tile 名匹配。Milkyway 参考库通常默认将 site 名设为 unit。但若 site 尺寸与 unit 不匹配，或 DEF 使用多种 site 类型，仍需定义名称映射。使用 mw\_site\_name\_mapping 变量，例如：

```
de_shell> set mw_site_name_mapping \
{{def_site_name1 mw_ref_lib_site_name} \
{def_site_name2 mw_ref_lib_site_name} }
```

可为该变量指定多对取值。

#### 参见

extract\_physical\_constraints 的名称匹配（上文）。

### 9.1.6 在 IC Compiler 中保存 Floorplan 信息

#### Saving the Floorplan Information in IC Compiler

为改善相关性，可将已映射网表读入 IC Compiler、创建基本 floorplan、导出后再读回 DC Explorer。

可在 IC Compiler 中用 `write_floorplan` 导出 floorplan，写成可读入 DC Explorer 以重建指定设计 floorplan 的 Tcl 脚本。下例将所有已布局 standard cell 写入名为 `placed_std.fp` 的文件：

```
icc_shell> write_floorplan \
    -placement {io terminal hard_macro soft_macro} -create_terminal \
    -row -create_bound -preroute -track floorplan_for_DC.fp
```

### 9.1.7 在 DC Explorer 中读取 Floorplan 脚本

#### Reading the Floorplan Script in DC Explorer

使用 `read_floorplan` 读入由 `write_floorplan` 生成、包含描述 floorplan 命令的脚本文件。也可用 `source` 导入，但 `source` 会报告对 DC Explorer 不适用的错误与警告；`read_floorplan` 会去除这些不必要消息。此外，使用 `source` 时需手动启用模糊名称匹配；`read_floorplan` 会自动启用。

要直观检查导入的物理约束，使用 Design Vision layout 窗口中的 layout 视图。

导入的物理约束会自动保存到 `.ddc`。要单独保存，提取后使用 `write_floorplan`，以便再次读回 DC Explorer。

不应用 `read_floorplan` 做增量 floorplan 修改：它导入从 IC Compiler 用 `write_floorplan` 导出的完整 floorplan，并覆盖任何现有 floorplan；`write_floorplan` 输出通常包含移除现有 floorplan 的命令。

有关导入的物理约束、floorplan 修改问题以及端口与宏名称匹配，见：

- 导入的物理约束
- `read_floorplan` 的名称匹配

### 9.1.8 导入的物理约束

#### Imported Physical Constraints

从 floorplan 文件导入的部分物理约束包括：

- Voltage area
- Die Area — 芯片硅边界，包围焊盘、I/O 引脚与单元等
- Placement Area
- Macro Location and Orientation — 对带位置且指定 `FIXED` 的单元设置位置
- Hard 与 Soft Placement Blockage — 在设计上创建 placement blockage
- Wiring Keepout — 由 `create_route_guide` 创建
- Placement Bound — 按与 DEF 导入类似的两种方式提取（`update_bounds` 增量附着，或 `create_bounds` 新建）
- Port Location — 支持更名与多层端口
- Preroute — 以 `create_net_shape` 表示
- User Shape — 导入并包含在 preroute 段

- Site Array Information 一定义布局区域
- Via 一导入并包含在 preroute 段
- Track

### 9.1.9 read\_floorplan 的名称匹配

#### Name Matching of read\_floorplan

在 topographical 模式下, read\_floorplan 施加物理约束时自动匹配 floorplan 文件与内存中的宏与端口;找不到精确匹配时使用智能名称匹配。名称不匹配原因及默认等价字符与 extract\_physical\_constraints 相同(层次分隔符与总线记号)。

要用 set\_query\_rules 定义智能名称匹配规则。

#### 注意

不推荐使用 source。若仍用 source 导入 floorplan, 须在 source 之前将 enable\_rule\_based\_query 设为 true 以手动启用模糊名称匹配, 例如:

```
de_shell> set enable_rule_based_query true
de_shell> source design.fp
de_shell> set enable_rule_based_query false
```

## 9.2 物理约束概览

### Physical Constraints Overview

无法从 DEF 或 read\_floorplan 获得信息时, 可手动定义物理约束。在 Tcl 脚本中定义后, 用 source 施加; 应在施加用户指定物理约束之前先读入设计。

要了解如何手动定义物理约束, 见:

- 定义物理约束
- 用于定义物理约束的命令

要报告施加到设计的任何物理约束, 使用 report\_physical\_constraints。读入新的或已修改的 floorplan 之前, 用 reset\_physical\_constraints 重置全部物理约束。要用 compute\_polygons 报告恰好覆盖对输入多边形执行布尔运算后所得区域的多边形集合。

#### 参见

- 在 DC Explorer 中读取 DEF 信息
- 在 DC Explorer 中读取 Floorplan 脚本

## 9.2.1 定义物理约束

### Defining Physical Constraints

若未通过 floorplanning 工具提供任何 floorplan 物理约束，DC Explorer 使用下列默认物理约束：

- 纵横比 (aspect ratio) 1.0 (正方形布局区域)
- 利用率 (utilization) 0.6 (core area 中百分之四十为空闲空间)

也可按下列主题手动定义物理约束：

- 定义 Die Area
- 定义 Placement Area
- 定义 Core Placement Area
- 定义端口位置
- 定义宏位置
- 定义 Placement Blockage
- 定义 Voltage Area
- 创建 Placement Bound
- 创建 Wiring Keepout
- 创建 Preroute
- 创建 User Shape
- 为引脚定义物理约束
- 创建 Via
- 创建布线 Track
- 创建 Keepout Margin
- Placement Bound 概览

### 定义 Die Area

#### Defining the Die Area

可用 `create_die_area` 在 DC Explorer 内定义 die area (cell boundary)。设计中应只有一个 die area。矩形可用 `-coordinate`；折线形必须用 `-polygon` (`-polygon` 也可指定矩形，但不能用 `-coordinate` 指定折线形)。下列命令创建相同矩形 die area：

```
de_shell> create_die_area -coordinate { 0 0 100 100 }
de_shell> create_die_area -coordinate { {0 0} {100 100} }
de_shell> create_die_area -polygon { {0 0} {100 0} {100 100} {0 100} }
```

`get_die_area` 返回包含当前设计 die area 的集合；未定义则返回空字符串。用 `get_attribute` 获取 `object_class`、`bbox`、`boundary`、`name` 等；也可用 `report_attribute` 浏览属性。

```

de_shell> create_die_area -polygon {{0 0} {0 400} {200 400} {200 200} \
{400 200} {400 0}}
de_shell> get_attribute [get_die_area] object_class
die_area
de_shell> get_attribute [get_die_area] bbox
{0.000 0.000} {400.000 400.000}
de_shell> get_attribute [get_die_area] boundary
{0.000 0.000} {0.000 400.000} {200.000 400.000} {200.000 200.000}
{400.000 200.000} {400.000 0.000} {0.000 0.000}
de_shell> get_attribute [get_die_area] name
{my_design_die_area}

```

## 定义 Placement Area

### Defining Placement Area

若尚未用 `create_die_area` 定义 die area、用 `create_site_row` 定义 floorplan 区域，或从 floorplanning 工具导入 floorplan，可用 `set_aspect_ratio` 与 `set_utilization` 估算布局区域。纵横比是块的高宽比，定义块形状；利用率指定希望单元在块内放置的密度。提高利用率会减小 core area。

(原书 *Figure 38: Using the set\_aspect\_ratio and set\_utilization Commands*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 9.2: 使用 `set_aspect_ratio` 与 `set_utilization`

## 定义 Core Placement Area

### Defining the Core Placement Area

用 `create_site_row` 定义 core placement area: 包含所有 row、表示标准单元可放置区域的盒子。Core area 小于 cell boundary (die area)。焊盘、I/O 引脚与顶层电源/地环通常在 core 外；标准单元、宏与线轨通常在 core 内。设计中应只有一个 core area。

Site 是单元可放置的预定义有效位置；site array 是布局 site 的阵列，定义布局 core area。创建左下角位于 (10,10)、间距 5、含 100 个 CORE\_2H site 的水平行：

```

de_shell> create_site_row -count 100 -kind CORE_2H \
-space 5 -coordinate {10 10}

```

用 `report_area -physical` 报告 floorplan 信息 (core area、纵横比、利用率、固定/可移动单元总面积等)。

## 定义端口位置

### Defining Port Locations

可用 `set_port_side` 或 `set_port_location` 限制端口布局与尺寸；可指定相对或精确约束。

## 定义相对端口位置

### Defining Relative Port Locations

```
de_shell> set_port_side port_name -side port_side
```

有效边为 `left (L)`、`right (R)`、`top (T)` 或 `bottom (B)`。端口可置于指定边任意位置；若提供 `port side` 约束则吸附到该边，否则默认吸附到粗布局器分配位置最近的边。

(原书 *Figure 39: Setting Relative Port Sides*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 9.3: 设置相对端口边

## 定义精确端口位置

### Defining Exact Port Locations

```
de_shell> set_port_location port_name -coordinate {x y}
de_shell> set_port_location -coordinate {100 5000} Z
de_shell> set_port_location -layer_name METAL1 \
-layer_area {-5 -10 5 10} Z
```

`-layer_name` 与 `-layer_area` 可定义金属层几何，从而定义长端口 (`long port`)，使基于虚拟布局的优化可在指定金属层任意点连接。长端口可以是输入、输出或双向端口；用大于最小金属面积的尺寸指定。

图 9.4 说明不同类型：

- **Long port** — 单层金属面积，仅触及块 `core` 边界一侧
- **Flushed port** — 单层金属面积，触及边界两侧
- **Multiple port** 一对块有多个连接，可触及两边或更多边，可用一层或多层；用 `set_port_location` 的 `-append` 添加多个形状

## 定义宏位置

### Defining Macro Locations

```
de_shell> set_cell_location cell_name -coordinates {x y} -fixed \
-orientation orientation_value
```

(原书 *Figure 40: Types of Port Dimension Specifications*)  
 请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 9.4: 端口尺寸规格类型

`-coordinates` 为单元包围盒左下角坐标（微米，相对块方向）。方向取值为旋转：N（默认）、S、E、W、FN、FS、FE、FW。

## 定义 Placement Blockage

### Defining Placement Blockages

用 `create_placement_blockage` 定义 placement blockage:

```
de_shell> create_placement_blockage -name Blockage1 \
  -type hard -bbox {50 160 300 280}
```

(原书 *Figure 41: Defining a Hard Placement Blockage*)  
 请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 9.5: 定义 hard placement blockage

`get_placement_blockages` 返回当前设计中满足指定条件的 placement blockage 集合；若没有匹配对象，则返回空字符串。该命令的结果可作为其他命令的参数，也可赋给变量。

要返回指定区域内的全部 blockage，可使用 `-within region`；region 边界可以是矩形或多边形。下例中，第一个坐标对 {2 2} 是矩形左下角，第二个坐标对 {25 25} 是右上角：

```
de_shell> get_placement_blockages * -within {{2 2} {25 25}}
{"PB#5389"}
```

要用表达式过滤集合，可使用 `-filter expression`。下例返回面积大于 900 的全部 placement blockage:

```
de_shell> get_placement_blockages * -filter "area > 900"
{"PB#4683"}
```

## 定义 Voltage Area

### Defining Voltage Area

用 `create_voltage_area` 在芯片 `core` 上创建 `voltage area`，并与层次单元关联。工具将 `voltage area` 视为独占的 `hard move bound`，尽量把关联单元放在 `voltage area` 内，并把未关联单元放在其外。

```
de_shell> create_voltage_area -name my_area \
    -coordinate {100 100 200 200} INST_1
```

代码 9.23: 约束实例到 `voltage area`

```
de_shell> create_voltage_area -power_domain PD1 \
    -coordinate {100 100 200 200}
```

代码 9.24: 约束电源域单元到 `voltage area`

用 `read_floorplan` 从 IC Compiler 导入 `floorplan` 时会自动定义 `voltage area`；从 DEF 导入时需手动定义。可在 Design Vision layout 窗口的 `visual mode` 中检查。

#### 参见

在 DC Explorer 中读取 Floorplan 脚本。

## 创建 Placement Bound

### Creating Placement Bounds

为有效使用 `placement bound`，`bound` 中定义的单元数应相对设计总单元数较少；定义 `bound` 会缩小布局器可达最优结果的解空间。

建议按下列顺序创建：

1. 浮动 `group bound`（位置与尺寸由工具优化；不指定尺寸）
2. 固定尺寸的浮动 `group bound`（指定尺寸）
3. 固定位置与尺寸的 `fixed move bound`（指定坐标）

指南：

- 少用 `soft/hard move bound`
- 避免将单元放入多个 `bound`
- 注意 `group bound` 中少量固定单元可能移动 `bound`
- 不要把 `placement bound` 当作 `keepout`
- 在芯片上保持均匀单元密度分布

用 `create_bounds` 指定各类 `bound`：

```
# Soft group bound
de_shell> create_bounds -dimension {100 100} -name temp1 INST1
# Soft move bound
de_shell> create_bounds -coordinate {0 0 10 10} -name temp2 INST2
# Hard group bound
```

```
de_shell> create_bounds -dimension {100 100} \
    -type hard -name temp3 INST3
# Hard move bound
de_shell> create_bounds -coordinate {0 0 10 10} \
    -type hard -name temp4 INST4
# Exclusive move bound
de_shell> create_bounds -coordinate {0 0 10 10} \
    -type hard -exclusive -name temp5 INST5
```

`get_bounds` 返回当前设计中的 `bound` 集合。该命令可在命令提示符下直接使用，也可嵌套为 `report_bounds` 等其他命令的参数，或将结果赋给变量。在命令提示符下执行时，其行为如同调用 `report_bounds` 来报告集合中的对象；默认最多显示 100 个对象，可用 `collection_result_display_limit` 更改此上限。下例返回名称以 `my_bound` 开头的全部 `bound`：

```
de_shell> get_bounds my_bound*
```

### 参见

Placement Bound 概览（下文）。

## 创建 Wiring Keepout

### Creating Wiring Keepouts

```
de_shell> create_route_guide -name "my_keepout_1" \
    -no_signal_layers "METAL1" -coordinate {12 12 100 100}
```

## 创建 Preroute

### Creating Preroutes

可在布线其余网络之前对时钟网等做 preroute。全局布线时工具在计算拥塞图时考虑这些 preroute；该图与 IC Compiler 一致，有助于缓解因 floorplan 信息不一致导致的相关性问题。

用 `create_net_shape` 定义 preroute。图 9.6 给出三类 net shape: path、wire（水平/垂直）与 rectangle。

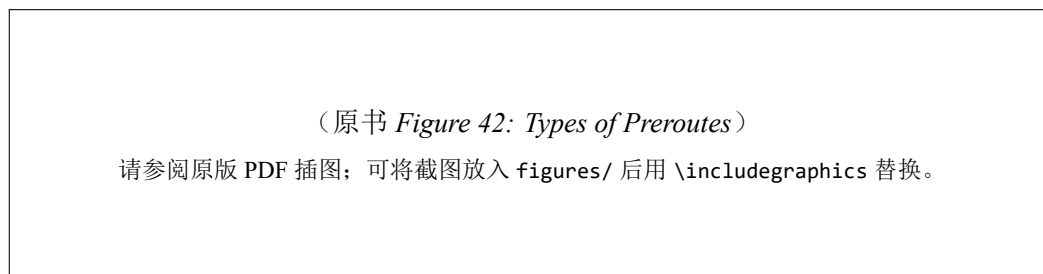


图 9.6: Preroute 类型

```
de_shell> create_net_shape -type wire -net VSS \
    -bbox {0 60 80 80} -layer METAL5 -route_type pg_strap \
    -net_type ground
de_shell> create_net_shape -type wire -net VSS \
```

```
-origin {50 100} -width 20 -length 80 -layer METAL4 \
-route_type pg_strap -vertical
```

代码 9.25: 创建 wire net shape

```
de_shell> create_net_shape -type path -net VSS \
-points {70 30 110 30 110 90 150 90 150 110} \
-width 0.20 -layer M3 -route_type pg_std_cell_pin_conn
```

代码 9.26: 创建 path net shape

```
de_shell> create_net_shape -type rect -net VSS \
-bbox {{80 140} {160 180}} -layer METAL1 -route_type pg_strap
```

代码 9.27: 创建 rectangle net shape

## 创建 User Shape

### Creating User Shapes

用 `create_user_shape` 创建不与网络关联的金属形状。`-type` 可为 `wire` (水平/垂直)、`path`、`rectangle`、`polygon`、`trapezoid`。未指定 `-type` 时按优先级判定：`-origin` → `wire`；`-bbox` 且带 `-path_type/-route_type/-vertical` → `wire`，否则 `rectangle`；`-points` → `path`。

```
de_shell> create_user_shape -type wire \
-origin {0 0} -length 10 -width 2 -layer M1
```

用 `write_floorplan -user_shape` 保存 user shape；或用 `write_floorplan -preroute` 同时保存 user shape 与 net shape。`create_user_shape` 命令保存在 floorplan 文件的 Preroute 段。用 `remove_user_shape` \* 移除设计中的全部 user shape。

可在 DC Explorer Graphical 的 layout 窗口中查看；在 View Settings 面板控制可见性。详见 Design Vision Help/User Guide 中“Examining Preroutes”。

为引脚定义物理约束

### Defining Physical Constraints for Pins

可用 `set_pin_physical_constraints` 与 `create_pin_guide` 控制 I/O 单元与 terminal 的布局。

#### 注意

DC Explorer 仅支持顶层设计端口布局，不支持 IC Compiler design planning 所支持的 plan group 或宏引脚布局。

```
de_shell> set_pin_physical_constraints -pin_name {CCEN} -width 1.2 \
-depth 1.0 -side 1 -offset 10 -order 2
de_shell> create_pin_guide -bbox {{-20 50} {20 800}} \
[get_ports xyz*] -name abc
```

`set_pin_physical_constraints` 可为指定 pin 设置 width、depth 等物理约束。在上例中，CCEN 的几何宽度为 1.2 micron、深度为 1.0 micron；该 pin 邻接 side 1（最左侧的竖直边），offset 为 10 micron，相对 placement order 为 2，因此被放在最左侧边上。

`create_pin_guide` 创建 pin guide，将 pin assignment 限制在指定 bounding box 内；上例为名称以 xyz 开头的全部 port 创建名为 abc 的 pin guide。

`compile_exploration` 会遵守这些约束。默认布局引脚时将端口吸附到 track；将 `dct_port_dont_snap_onto_tracks` 设为 true 可禁止吸附。

引脚约束保存在 .ddc 中，新会话读入后恢复。`write_floorplan` 与 `report_physical_constraints` 不报告引脚约束；可在 layout 窗口 View Settings 中选择 Pin Guide 可见性验证。DC Explorer 与 floorplan exploration 之间不互相传递引脚约束。

## 创建 Via

### Creating Vias

若工艺文件中已定义 master via，可用 `create_via` 创建 via：

```
de_shell> create_via -type via -net vdd -master via1 \
  -route_type pg_strap -at {213 215} -orient E
de_shell> create_via -type via_array -net vdd -master via4 \
  -route_type signal_route -at {215 215} -orient W -col 4 -row 3 \
  -x_pitch 0.4 -y_pitch 0.5
```

若 via 在 DEF 中定义但不在工艺文件中，会在 Milkyway 设计库中创建 FRAM 视图保存 via master；实例化为 preroute 时保存为引用该 master 的 via cell。用 `remove_via` 移除。

#### 注意

`write_floorplan` 无法写出供新会话使用的 via master 定义；FRAM 中的 via master 不会传到新会话，因此 IC Compiler 与 DC Explorer 无法 source 使用工艺文件中未定义 via master 的 `create_via`。

可在 Design Vision layout 窗口查看 preroute via（默认可见）。

## 创建布线 Track

### Creating Routing Tracks

用 `create_track` 为 routing layer 与 polygon layer 创建一组 track，供 router 执行 detail routing。Track 覆盖整个 die area；若 die area 为多边形，track 以矩形覆盖并会延伸到 polygon die area 之外。必须为 track 指定 polygon layer 或 routing layer；在 polygon layer 上创建 track 并不表示支持在该层布线。

```
de_shell> create_track -layer MET3 -dir Y -coord 0.000 -space 0.290 \
  -count 4827 -bounding_box {{0.000 0.000} {1460.000 1400.000}}
de_shell> create_track -layer MET3 -dir X -coord 0.000 -space 0.290 \
  -count 5034 -bounding_box {{0.000 0.000} {1460.000 1400.000}}
```

`get_tracks` 返回当前设计中满足选择条件的 track 集合。例如，`-within rectangle` 返回指定矩形内的全部 track；矩形格式为 {{l1x l1y} {urx ury}}，坐标单位由 technology file 指定：

```
de_shell> get_tracks * -within {{2 2} {25 25}}
{"USER_TRACK_5389"}
```

```
de_shell> get_tracks "*TRACK_*"
{"DEF_TRACK_4683 USER_TRACK_5389"}
```

`report_track`（不带选项）报告 metal layer、metal direction、起点、track 数量、metal pitch 与属性来源。

表 9.1 汇总 track 相关命令。`create_track` 与 `remove_track` 由 `write_floorplan` 写出；表中其他命令不会。

表 9.1: 布线 Track 命令摘要

| 命令                        | 说明                      |
|---------------------------|-------------------------|
| <code>create_track</code> | 为布线层与 polygon 层创建 track |
| <code>remove_track</code> | 从当前设计移除 track           |
| <code>report_track</code> | 报告指定层或全部层的布线 track      |
| <code>get_tracks</code>   | 返回满足选择条件的 track 集合      |

## 创建 Keepout Margin

### Creating Keepout Margins

用 `set_keepout_margin` 为指定 cell 或 library cell 创建 keepout margin;也可指定 `-tracks_per_macro_pin` 自动推导。推导值根据 track width、macro pin 数量和指定的 track-to-pin ratio 计算；该比率通常设为接近 0.5，值越大，keepout margin 越大：

```
de_shell> set_keepout_margin -tracks_per_macro_pin .6 \
-min_padding_per_macro .1 -max_padding_per_macro 0.2
```

推导的 keepout margin 始终为 hard(忽略 `-type`);`-all_macros`、`-macro_masters`、`-macro_instances` 不允许用于推导 margin。推导 margin 受 `-min_padding_per_macro` 与 `-max_padding_per_macro` 限制。

用 `remove_keepout_margin` 移除;用 `report_physical_constraints` 或 `report_keepout_margin` 报告：

```
de_shell> report_keepout_margin -type hard MY_CELL
```

## 9.2.2 Placement Bound 概览

### Placement Bounds Overview

Placement bound 是在其内放置单元与层次单元的矩形或折线形区域，用于将时钟门控单元或其他时序关键单元组聚在一起。布局期间工具确保所分组的单元保持在一起、不被其他逻辑拆开。

建议尽量不强加 bound 约束，以便工具充分优化时序与可布线性；当 QoR 不满足要求时，可用 `create_bounds` 改善。

可指定 move bound 与 group bound。Move bound 可为 soft、hard 或 exclusive；group bound 可为 soft 或 hard：

- **Soft** —指定布局目标，不保证单元落在 bound 内；时序或拥塞代价过高时可能放在区域外（默认）

- **Hard** —强制指定单元放在 bound 内
- **Exclusive** —强制指定单元在 bound 内，所有其他单元必须在外

表 9.2: Bound 命令摘要

| 命令            | 说明                          |
|---------------|-----------------------------|
| create_bounds | 创建矩形与折线形 move/group bound   |
| update_bounds | 通过添加或移除内容更新 bound           |
| get_bounds    | 返回当前设计中的 bound 集合           |
| remove_bounds | 移除用 create_bounds 设置的 bound |
| report_bounds | 报告 bound 与 bound ID         |

参见

创建 Placement Bound（上文）。

9.2.3 用于定义物理约束的命令

Commands for Defining Physical Constraints

表 9.3 列出用于定义物理约束的命令。

表 9.3: 定义物理约束的命令

| 要定义的物理约束                     | 使用的命令                                              |
|------------------------------|----------------------------------------------------|
| Die Area                     | create_die_area                                    |
| 未知精确面积时的 floorplan 估算        | set_aspect_ratio 与 set_utilization                 |
| 精确 core area                 | create_site_row                                    |
| 相对端口位置                       | set_port_side                                      |
| 精确端口位置                       | set_port_location                                  |
| 宏位置与方向                       | set_cell_location                                  |
| Placement keepout (blockage) | create_placement_blockage                          |
| Voltage area                 | create_voltage_area                                |
| Placement bound              | create_bounds                                      |
| Wiring keepout               | create_route_guide                                 |
| Preroute                     | create_net_shape                                   |
| User shape                   | create_user_shape                                  |
| 引脚物理约束                       | set_pin_physical_constraints 与<br>create_pin_guide |
| Via                          | create_via                                         |
| Track                        | create_track                                       |
| Keepout margin               | set_keepout_margin                                 |

要用 report\_physical\_constraints 报告已施加约束；用 reset\_physical\_constraints 在读入新/修改 floorplan 前重置；用 compute\_polygons 报告布尔运算后的多边形集合。

**参见**

物理约束概览（上文）。

## 9.3 保存物理约束

### Saving Physical Constraints

可用 `write_floorplan` 以 IC Compiler 或 IC Compiler II 格式保存 floorplan 信息。

#### 9.3.1 IC Compiler 格式

##### IC Compiler Format

默认情况下, `write_floorplan` 以 IC Compiler 格式写出物理约束信息。输出为包含 `bound`、`placement blockage`、`route guide`、`plan group`、`voltage area` 等 floorplan 信息的命令脚本文件。

从 DC Explorer 写出 floorplan 时, 应使用 `-all` 写出完整 floorplan。该命令也提供写出部分 floorplan 的选项。

要在 IC Compiler 中用该输出创建 floorplan, 必须从设计顶层用 `read_floorplan` 读入。也可用 `source`, 但会报告对 DC Explorer 不适用的错误与警告; `read_floorplan` 会去除这些消息, 并自动启用模糊名称匹配 (`source` 需手动启用)。

#### 9.3.2 IC Compiler II 格式

##### IC Compiler II Format

要以 IC Compiler II 格式保存, 使用 `write_floorplan -format icc2`。输出为包含 Tcl 与 DEF 格式文件的文件夹。

指定 `-format icc2` 时, 工具写出完整 floorplan; 允许写出部分 floorplan 的选项会被忽略。要在 IC Compiler II 中使用, 须从顶层设计 `source` 输出文件夹中的 `floorplan.tcl`, 以将全部约束加载到 IC Compiler II 会话。

下列文件由该命令生成并传递约束:

- `floorplan.def` —Die area、row、track、component、pin、via、special net、keepout margin 等
- `floorplan.tcl` —Route guide、placement blockage、bound、层布线方向
- `voltage_area.tcl` —Voltage area
- `rp.tcl` —Relative placement group
- `routing_rule.layer.tcl` —网络上的布线层约束
- `routing_rule.tcl` —非默认布线规则定义
- `routing_rule.net.tcl` —网络上的布线规则

## 9.4 以 ASCII 格式保存供 IC Compiler II 使用

### Saving in ASCII Format for IC Compiler II

可用 `write_icc2_files` 生成将设计加载到 IC Compiler II 所需的全部文件。该命令将当前设计的设计文件与 Tcl 脚本写入 `-output` 指定的目录，可包括：

- Verilog 格式网表
- 传播的开关活动信息（backward SAIF）
- Tcl floorplan 与 DEF 文件
- 设计功耗意图（UPF 命令脚本）
- SCANDEF 格式扫描链信息
- SDC 格式约束
- 时序上下文（scenario）

此外还写出：

- 以 IC Compiler II 格式表示的 DC Explorer 设置
- 在 IC Compiler II 中加载全部所生成数据的顶层脚本

`write_icc2_files` 以可被 IC Compiler II 读入、用以创建与 DC Explorer 中相同 scenario 集的格式写出时序上下文（scenario）。

所生成脚本不包含在 IC Compiler II 中创建库的脚本；需先创建与 DC Explorer 所用库一致的库，再 `source` 顶层脚本。

与 `set_host_options` 配合时，`write_icc2_files` 可在同一计算机上最多使用用户指定数量的 CPU 核并行执行。详见「使用多核技术」。

```
# Perform synthesis
de_shell> compile_exploration -scan -gate_clock
# Change names if needed
de_shell> change_names -rules verilog -hierarchy
# Save the file for IC Compiler II
de_shell> write_icc2_files -output ./icc2_files
de_shell> quit
```

代码 9.28: 从 DC Explorer 以 IC Compiler II 格式写出设计信息

```
# Perform Library setting in IC Compiler II
icc2_shell> source -echo -verbose icc2_lib_setting.tcl
# Load the design using script created by write_icc2_files
icc2_shell> source -echo -verbose icc2_files/ORCA.icc2_script.tcl
# Continue with the flow
icc2_shell> commit_upf
icc2_shell> place_opt
```

代码 9.29: 将设计读入 IC Compiler II

若运行命令时出现警告或错误，工具发出信息消息指向含警告/错误信息的日志：

```
de_shell> write_icc2_files -output ./icc2_files
Information: During the execution of write_icc2_files error messages were
issued, please check ./icc2_files/write_icc2_files.log files. (DCT-228)
```

### 9.4.1 限制

#### Limitations

`write_icc2_files` 当前有下列限制:

- 不支持层次化流程。
- 仅包含下列施加在库单元上的约束:
  - SDC: `set_timing_derate` 与 `set_driving_cell`
  - UPF: `map_isolation_cell`、`map_retention_cell` 等
- 下列命令部分支持:
  - `set_congestion_options --max_util` 仅对全局设置包含, 不对区域设置包含
  - `set_multi_vth_constraint` 仅传递库中定义的阈值电压组; 用户自定义组的设置不包含
- 下列变量部分支持:
  - `power_default_toggle_rate_type` 等价 IC Compiler II 变量仅支持 `fastest_clock;absolute` 设置不传递
  - `placer_channel_detect_mode` 等价变量仅支持 `true/false`; `auto` 不传递
  - `bus_naming_style` 仅支持 IC Compiler II 中定义的分隔符: `[ ]`、`{ }`、`( )`、`< >`

## 9.5 包含仅物理单元

### Including Physical-Only Cells

可在 floorplan 中包含无逻辑功能的 physical cell, 称为 physical-only cell, 例如 filler cell、tap cell、flip-chip pad cell、endcap cell 和 decap cell。虽无逻辑功能, 它们会创建工具在优化与拥塞分析中考虑的 placement blockage。

下列各节描述对仅物理单元的支持:

- 指定仅物理单元
- 从 DEF 提取仅物理单元
- 报告仅物理单元

#### 9.5.1 指定仅物理单元

##### Specifying Physical-Only Cells

手动指定仅物理单元的步骤:

1. 对 `create_cell` 使用 `-only_physical` 创建仅物理单元，例如：

```
de_shell> create_cell -only_physical MY_U_PO_CELL MY_FILL_CELL
Information: create physical-only cell 'MY_U_PO_CELL'
Warning: The newly created cell does not have location.
         Timing will be inaccurate. (DCT-004)
```

这会创建引用物理库单元 `MY_FILL_CELL` 的 `MY_U_PO_CELL`，并在其上设置 `is_physical_only` 属性。

2. 为单元分配位置。与有逻辑功能的物理单元不同，综合期间 DC Explorer 不会为仅物理单元分配位置。可用下列方法之一：

- 通过 **DEF** 分配位置 — 在 DEF 中定义位置后，用 `extract_physical_constraints` 施加：

```
COMPONENTS 1 ;
- MY_U_PO_CELL MY_FILL_CELL + FIXED ( 3100000 700000 ) N ;
END COMPONENTS
```

- 通过 **Tcl** 命令分配位置 — 在 Tcl floorplan 文件中定义，并用 `read_floorplan` 读入。必须设置 `is_fixed`，否则工具忽略位置：

```
de_shell> set obj [get_cells {"MY_U_PO_CELL"} -all]
de_shell> set_attribute -quiet $obj orientation N
de_shell> set_attribute -quiet $obj origin {3100.000 700.000}
de_shell> set_attribute -quiet $obj is_fixed TRUE
```

- 使用 **set\_cell\_location**：

```
de_shell> set_cell_location -coordinates {3100.00 700.00} \
    -orientation N -fixed MY_U_PO_CELL
```

## 参见

- 从 DEF 提取仅物理单元
- 报告仅物理单元

## 9.5.2 从 DEF 提取仅物理单元

### Extracting Physical-Only Cells From a DEF File

要对 `extract_physical_constraints` 使用 `-allow_physical_cells`。DEF 中的仅物理单元定义必须包含 `+FIXED` 属性，否则工具忽略位置。

```
de_shell> extract_physical_constraints \
    -allow_physical_cells my_design.def
```

```
COMPONENTS 2 ;
- fill_1 FILL_CELL + FIXED ( 3100000 700000 ) N ;
- fill_2 FILL_CELL + FIXED ( 3000000 600000 ) N ;
END COMPONENTS
```

## 代码 9.30: 两个 filler 单元的 DEF 定义

下列类型视为仅物理单元：使用 `-allow_physical_cells` 时工具以 `is_physical_only` 标记：

- 标准单元 filler
- Pad filler
- Corner cell
- Chip cell
- Cover cell
- Tap cell
- 仅含电源与地端口的单元

此外，该选项还会提取 DEF 中未被识别为仅物理单元的某些单元（如 ROM/RAM 等逻辑单元）；工具用这些新逻辑单元更新当前设计，并包含在 Verilog 网表中。

## 注意

DEF 中的仅物理单元定义必须包含单元位置。综合期间 DC Explorer 不会为仅物理单元分配位置。

## 9.5.3 报告仅物理单元

## Reporting Physical-Only Cells

默认情况下，工具在所有 physical-only cell 上设置 `is_physical_only` 属性。检查某 cell 是否为 physical-only cell：

```
de_shell> get_attribute [get_cells -all $cell_name] is_physical_only
```

报告方式：

- `report_cell -only_physical` — 报告当前设计的 physical-only cell, 例如 `report_cell -only_physical`
- `report_physical_constraints` — 报告含仅物理单元信息的 floorplan
- `all_physical_only_cells` — 返回设计中全部仅物理单元的集合

## 9.5.4 保存仅物理单元

## Saving Physical-Only Cells

用 `write_floorplan` 保存（含仅物理单元信息）；用 `read_floorplan` 读回。仅物理单元信息及其他约束（如时序）保存在 `.ddc` 中，可再次读入 DC Explorer。

## 注意

与所有其他 floorplan 信息类似，仅物理单元信息不写出到 ASCII 网表或 Milkyway 接口，也不能通过 `.ddc` 传给 IC Compiler。将 floorplan 传给 IC Compiler 的唯一方式是使用 DC Explorer 中的 floorplan exploration。

## 9.6 相对布局

### Relative Placement

DC Explorer 的相对布局能力允许创建指定实例相对列/行位置的结构，称为相对布局结构（relative placement structure），属于布局约束。在布局与合法化期间这些结构被保留，结构中的单元作为单一实体放置。相对布局亦称物理 datapath 或结构化布局。

下列主题描述相对布局流程及如何执行相对布局：

- 相对布局概览
- 运行相对布局流程
- 指定相对布局约束
- 使用 HDL Compiler 指令创建相对布局

#### 9.6.1 相对布局概览

##### Relative Placement Overview

相对布局常用于 datapath 与寄存器，但也可用于任何单元，以控制门级逻辑组的精确相对布局拓扑并定义电路版图。可用相对布局探索更短线长、更低拥塞、更好时序、skew 控制、更少 via、更好良率以及更低动态/漏电功耗等 QoR 收益。

相对布局约束

##### Relative Placement Constraints

可用下列方式之一指定相对布局约束：

- 指定相对布局约束 — 使用与 IC Compiler 类似的专用 Tcl 命令集
- 使用 **HDL Compiler** 指令 — 在 RTL、GTECH 网表或已映射门级网表中使用 HDL compiler directive

创建并注释相对布局约束以生成实例矩阵结构并控制其布局。在物理优化中使用已注释网表时，工具保留该结构并将其作为单一实体或组放置，如图 9.7。

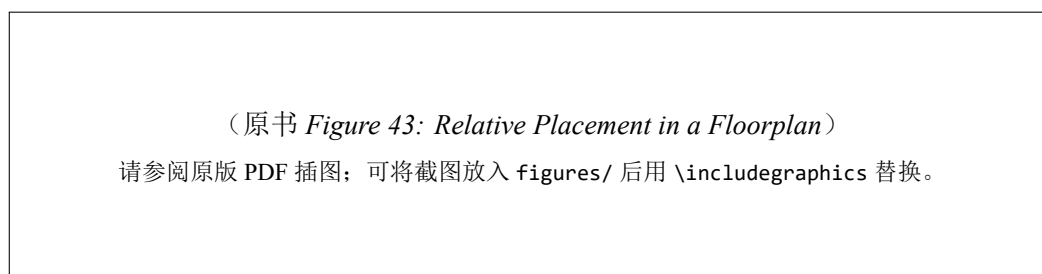


图 9.7: Floorplan 中的相对布局

相对布局的收益

### Benefits of Relative Placement

- 为遗留或 IP 设计维持结构化布局的方法
- 缩小设计关键区域的布局搜索空间，提高 QoR（线长、时序、功耗）可预测性
- 与 IC Compiler 更好相关
- 可最小化拥塞并改善可布线性

相对布局注意事项

### Considerations for Relative Placement

- 相对布局约束由 `uniquify` 与 `ungroup` 保留。
- DC Explorer 自动在每个相对布局单元上放置 `size_only` 属性以保留相对布局结构。
- 确保相对布局适用于您的设计。设计可同时含结构化与非结构化元素；`datapath` 与流水线设计更适合结构化布局。对更适合由工具布局的单元指定相对布局约束可能产生差结果。
- 相对布局约束保留在 `.ddc` 中，不会自动传到 IC Compiler。可用 `write_rp_groups` 写出可读入 IC Compiler 的 Tcl 脚本。另见 *IC Compiler Implementation User Guide*。

参见

指定相对布局约束（下文）。

## 9.6.2 运行相对布局流程

### Running the Relative Placement Flow

图 9.8 给出 DC Explorer 中相对布局流程概览。

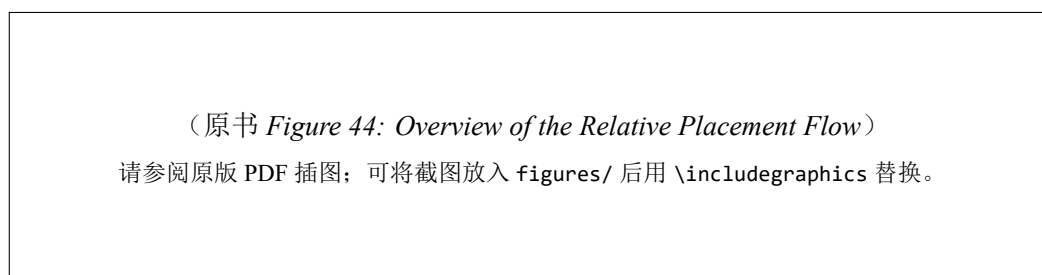


图 9.8: 相对布局流程概览

步骤如下：

1. 在 DC Explorer 中读入下列文件之一或多项：
  - 带 HDL compiler directive 相对布局约束的 RTL、GTECH 或已映射门级网表
  - 无相对布局约束的已映射门级网表（在步骤 2 用 Tcl 指定）
2. 对无相对布局约束的已映射门级网表，用 Tcl 定义约束：

(a) 用 `create_rp_group` 创建相对布局组

(b) 用 `add_to_rp_group` 向组添加相对布局对象

工具用相对布局约束注释网表，并对这些单元施加隐式 `size_only`。

3. 读入 `floorplan` 信息，例如：

```
de_shell> extract_physical_constraints floorplan.def
```

4. 用 `check_rp_groups` 检查相对布局。

该命令仅报告步骤 2 中定义的约束；由 HDL compiler directive 指定的相对布局数据在 `compile_exploration` 之后才会报告。

5. 用 `compile_exploration` 编译并优化设计。

6. 在 Design Vision layout 视图中直观验证布局。

Layout 视图显示用 `extract_physical_constraints` 或其他 Tcl 读入的 `floorplan` 约束。需链接全部适用设计与库以获得准确 `floorplan`。

#### 注意

必须将 `de_enable_physical_flow` 设为 `true` 以启用物理流程能力。

7. 用 `write_rp_groups` 将相对布局约束与数据写出到文件。

要导入 IC Compiler，source 该文件，或对 `import_designs` 使用 `-rp_constraint`。

更多信息另见 *HDL Compiler for SystemVerilog User Guide*、*HDL Compiler for VHDL User Guide*，以及 *Design Vision User Guide* 中的 Viewing the Floorplan 主题。

#### 参见

指定相对布局约束（下文）。

### 9.6.3 指定相对布局约束

#### Specifying Relative Placement Constraints

相对布局允许创建指定实例相对列/行位置的结构；布局与优化期间这些结构被保留，结构中的单元作为单一实体放置。

要了解如何指定，见：

- 相对布局命令摘要
- 创建相对布局组
- 锚定相对布局组
- 向组添加对象
- 在列内对齐单元
- 查询相对布局组
- 检查相对布局约束

- 保存相对布局信息
- 移除相对布局组、对象与属性

## 相对布局命令摘要

### Summary of Relative Placement Commands

表 9.4 汇总 DC Explorer 中可用的相对布局命令。

表 9.4: 相对布局命令

| 命令                                   | 说明                                                                   |
|--------------------------------------|----------------------------------------------------------------------|
| <code>create_rp_group</code>         | 创建新的相对布局组                                                            |
| <code>add_to_rp_group</code>         | 向相对布局组添加项                                                            |
| <code>set_rp_group_options</code>    | 设置相对布局组属性                                                            |
| <code>report_rp_group_options</code> | 报告相对布局组属性                                                            |
| <code>get_rp_groups</code>           | 创建匹配条件的相对布局组集合                                                       |
| <code>write_rp_groups</code>         | 写出指定组的相对布局信息                                                         |
| <code>all_rp_groups</code>           | 返回指定相对布局组及其层次中全部子组的集合                                                |
| <code>all_rp_hierarchicals</code>    | 返回作为指定组祖先的层次相对布局组集合                                                  |
| <code>all_rp_inclusions</code>       | 返回包含指定组的层次相对布局组集合                                                    |
| <code>all_rp_instantiations</code>   | 返回实例化指定组的层次相对布局组集合                                                   |
| <code>all_rp_references</code>       | 返回包含指定单元（叶单元或含已实例化相对布局组的层次单元）的相对布局组集合                                |
| <code>check_rp_groups</code>         | 检查相对布局约束并报告失败                                                        |
| <code>remove_rp_groups</code>        | 移除相对布局组列表                                                            |
| <code>remove_rp_group_options</code> | 移除指定相对布局组的属性                                                         |
| <code>remove_from_rp_group</code>    | 从指定相对布局组移除单元、组或 keepout                                              |
| <code>rp_group_inclusions</code>     | 返回直接包含的组（用 <code>add_to_rp_group -hierarchy</code> 添加）的集合            |
| <code>rp_group_instantiations</code> | 返回直接实例化的组（用 <code>add_to_rp_group -hierarchy -instance</code> 添加）的集合 |
| <code>rp_group_references</code>     | 返回直接嵌入的叶单元、直接包含且含层次实例化单元的单元，或二者                                      |

## 创建相对布局组

### Creating Relative Placement Groups

相对布局组是单元、组与 keepout 的关联，由所用行数与列数定义。用 `create_rp_group` 创建。

命令创建名为 `design_name::group_name` 的组（`design_name` 由 `-design` 指定）。在其他相对布局命令中引用该组时必须使用此名称或相对布局组集合。未使用 `-design` 时默认为当前设计；未指定任何选项时创建一行一列且无对象的组。用 `add_to_rp_group` 添加对象。

创建六列六行的 `designA::rp1`:

```
de_shell> create_rp_group rp1 -design designA -columns 6 -rows 6
```

在该图中：

（原书 *Figure 45: Relative Placement Column and Row Positions*）  
 请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 9.9: 相对布局的列与行位置

- 列从列 0（最左列）计数
- 行从行 0（最底行）计数
- 列宽由该列最宽单元决定
- 行高由该行最高单元决定
- 不必使用结构中的全部位置（例如位置 0 3 与 4 1 未使用）

锚定相对布局组

### Anchoring Relative Placement Groups

默认工具可将相对布局组放在 `core area` 内任意位置。可通过锚定控制顶层相对布局组的布局。

对 `create_rp_group` 或 `set_rp_group_options` 使用 `-x_offset` 与 `-y_offset`（微米浮点值，相对芯片原点）。若同时指定 `x` 与 `y`，组锚定在该位置；若只指定一个坐标，工具可沿未指定坐标滑动组以确定布局。

```
de_shell> create_rp_group misc1 -design block1 \
  -columns 3 -rows 10 -x_offset 100 -y_offset 100
```

向组添加对象

### Adding Objects to a Group

可向用 `create_rp_group` 创建的组添加叶单元、其他相对布局组与 `keepout`。添加时确保：目标组已存在；对象必须添加到组中的空位置。

添加叶单元

**Adding Leaf Cells** 在相对布局组中，占据多列或多行位置的叶单元称为 `leaf cell straddling`。通过 `add_to_rp_group` 的 `-num_columns` 或 `-num_rows`（默认各为 1）指定：

```
de_shell> add_to_rp_group rp_group_name -leaf cell_name \
  -column 0 -num_columns 2 -row 0 -num_rows 1
```

限制：

- 不应在 `straddling` 叶单元同一位置放置相对布局 `keepout`；`straddling` 仅用于叶单元，不用于层次组或 `keepout`

- 不应向具有多列、多行或两者的 **straddling** 叶单元施加 **compression**；可对多行 **straddling** 叶单元施加右对齐或引脚对齐，但不能对多列单元施加

用 **-orientation** 指定叶单元布局方向；用 **-alignment** 或 **-pin\_align\_name** 指定列内对齐方法。

### 参见

在列内对齐叶单元（下文）。

## 添加相对布局组

**Adding Relative Placement Groups** 层次相对布局允许将相对布局组嵌入其他相对布局组，以简化约束，无需为重复模式多次提供相对布局信息。嵌入组像叶单元一样处理。用 **add\_to\_rp\_group** 时有两种方法：

- 包含（**include**）相对布局组 — 若组与其父组在同一设计中，则为 **included group**。可在平坦或层次设计中包含。包含后如同直接嵌入父组。同一设计的同一组中，某个 **included group** 只能使用一次；但包含 **included group** 的组可进一步包含在同设计另一组中，或实例化到不同设计的组中。
- 实例化（**instantiate**）相对布局组 — 若待添加组位于其父组的子设计实例中，则为 **instantiated group**。只能在层次设计中实例化。使用 **-hierarchy** 指定组时，必须用 **-instance** 指定参考设计实例，且该实例必须与正在实例化指定组的层次组位于同一设计。实例化组可跨设计的多个实例复制相对布局信息，并在这些实例之间创建相对布局关系。

例 9.31 创建包含 **rp1** 三个实例的层次组 **rp2**。**rp1** 在 **pair\_design** 中并含叶单元 **U1**、**U2**；**rp2** 在 **mid\_design** 中三次实例化 **rp1**（**mid\_design** 必须至少含三个 **pair\_design** 实例）。**rp2** 当作叶单元处理，可按 **mid\_design** 在网表中的实例化次数再实例化。结果见图 9.10。

```
create_rp_group rp1 -design pair_design -columns 2 -rows 1
  add_to_rp_group pair_design::rp1 -leaf U1 -column 0 -row 0
  add_to_rp_group pair_design::rp1 -leaf U2 -column 1 -row 0
create_rp_group rp2 -design mid_design -columns 1 -rows 3
  add_to_rp_group mid_design::rp2 \
-hierarchy pair_design::rp1 -instance I1 -column 0 -row 0
  add_to_rp_group mid_design::rp2 \
-hierarchy pair_design::rp1 -instance I2 -column 0 -row 1
  add_to_rp_group mid_design::rp2 \
-hierarchy pair_design::rp1 -instance I3 -column 0 -row 2
```

代码 9.31: 在层次组中实例化组

（原书 Figure 46: *Instantiating Groups in a Hierarchical Group*）

请参阅原版 PDF 插图；可将截图放入 **figures/** 后用 **\includegraphics** 替换。

图 9.10: 在层次组中实例化组

## 添加 **Keepout**

### Adding Keepouts

```
de_shell> add_to_rp_group TOP::misc -keepout gap1 \
    -column 0 -row 2 -width 15 -height 1
```

在列内对齐叶单元

### Aligning Leaf Cells Within a Column

控制单元对齐可改善时序与可布线性。可用下列对齐方法：

- 对齐到左下角
- 对齐到右下角
- 对齐到引脚位置
- 覆盖引脚对齐

对齐到左下角

**Aligning to the Bottom-Left Corner** 对 `create_rp_group` 或 `set_rp_group_options` 使用 `-alignment bottom-left`（未指定对齐方法时的默认）：

```
de_shell> set_rp_group_options -alignment bottom-left [get_rp_groups *]
```

```
create_rp_group rp1 -design pair_design -columns 1 -rows 4
add_to_rp_group pair_design::rp1 -leaf U1 -column 0 -row 0
add_to_rp_group pair_design::rp1 -leaf U2 -column 0 -row 1
add_to_rp_group pair_design::rp1 -leaf U3 -column 0 -row 2
add_to_rp_group pair_design::rp1 -leaf U4 -column 0 -row 3
```

（原书 *Figure 47: Bottom-Left Aligned Relative Placement Group*）  
 请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 9.11: 左下对齐的相对布局组

对齐到右下角

**Aligning to the Bottom-Right Corner** 使用 `-alignment bottom-right`：

```
de_shell> set_rp_group_options -alignment bottom-right \
    [get_rp_groups *]
```

**注意**

对层次相对布局组，右下对齐不会沿层次传播。

```
create_rp_group rp1 -design pair_design -columns 1 -rows 4 \
  -alignment bottom-right
  add_to_rp_group pair_design::rp1 -leaf U1 -column 0 -row 0
  add_to_rp_group pair_design::rp1 -leaf U2 -column 0 -row 1
  add_to_rp_group pair_design::rp1 -leaf U3 -column 0 -row 2
  add_to_rp_group pair_design::rp1 -leaf U4 -column 0 -row 3
```

(原书 *Figure 48: Bottom-Right Aligned Relative Placement Group*)  
 请参阅原版 PDF 插图；可将截图放入 figures/ 后用 \includegraphics 替换。

图 9.12: 右下对齐的相对布局组

## 对齐到引脚位置

**Aligning to a Pin Location** 使用 `-alignment bottom-pin` 与 `-pin_align_name:`

```
de_shell> set_rp_group_options -alignment bottom-pin \
  -pin_align_name align_pin
```

工具在列内每个单元中查找指定对齐引脚：

- 找到则对齐到该引脚位置
- 未在该单元中找到则对齐到左下角并发出信息消息
- 在任何单元中都未找到则发出警告

若同时指定引脚对齐与单元方向，冲突解决如下：

- 用户指定的单元方向优先于 DC Explorer 所做的引脚对齐
- DC Explorer 所做的引脚对齐优先于 DC Explorer 所做的单元方向优化

```
create_rp_group rp1 -design pair_design -columns 1 -rows 4 \
  -pin_align_name A
  add_to_rp_group pair_design::rp1 -leaf U1 -column 0 -row 0
  add_to_rp_group pair_design::rp1 -leaf U2 -column 0 -row 1
  add_to_rp_group pair_design::rp1 -leaf U3 -column 0 -row 2
  add_to_rp_group pair_design::rp1 -leaf U4 -column 0 -row 3
```

## 覆盖引脚对齐

(原书 *Figure 49: Relative Placement Group Aligned by Pin A*)  
 请参阅原版 PDF 插图；可将截图放入 figures/ 后用 \includegraphics 替换。

图 9.13: 按引脚 A 对齐的相对布局组

**Overriding Pin Alignment** 为组指定对齐引脚时，该方法适用于列中全部单元。要对组中特定单元覆盖，在 `add_to_rp_group` 上使用 `-pin_align_name` 指定另一引脚。

#### 注意

从相对布局层次浏览器添加单元时，不能指定单元专用对齐引脚。

```
create_rp_group misc1 -design block1 -columns 3 -rows 10 \
  -pin_align_name A
  add_to_rp_group block1::misc1 -leaf I3 -column 0 -row 0
  add_to_rp_group block1::misc1 -leaf I4 -column 0 -row 1
  add_to_rp_group block1::misc1 -leaf I5 -column 0 -row 2 \
    -pin_align_name B
  add_to_rp_group block1::misc1 -leaf I6 -column 0 -row 3 \
    -pin_align_name B
  add_to_rp_group block1::misc1 -leaf I7 -column 0 -row 4
  add_to_rp_group block1::misc1 -leaf I8 -column 0 -row 5
```

(原书 *Figure 50: Relative Placement Group Aligned by Pins*)  
 请参阅原版 PDF 插图；可将截图放入 figures/ 后用 \includegraphics 替换。

图 9.14: 按引脚对齐的相对布局组

## 查询相对布局组

### Querying Relative Placement Groups

表 9.5 列出用于查询相对布局组以注释、编辑与查看的命令。

```
de_shell> get_rp_groups r*::g*
{ripple::grp_ripple}
de_shell> set_rp_group_options [all_rp_groups] -utilization 0.95
```

表 9.5: 查询相对布局组的命令

| 要返回的集合                                                      | 使用的命令                              |
|-------------------------------------------------------------|------------------------------------|
| 匹配某些条件的相对布局组                                                | <code>get_rp_groups</code>         |
| 全部或指定相对布局组及其层次中包含与实例化的组                                     | <code>all_rp_groups</code>         |
| 含 <code>included</code> 或 <code>instantiated</code> 组的相对布局组 | <code>all_rp_hierarchicals</code>  |
| 含 <code>included</code> 组的相对布局组                             | <code>all_rp_inclusions</code>     |
| 含 <code>instantiated</code> 组的相对布局组                         | <code>all_rp_instantiations</code> |
| 直接嵌入叶单元或实例化组的相对布局组                                          | <code>all_rp_references</code>     |

## 检查相对布局约束

### Checking Relative Placement Constraints

用 `check_rp_groups` 检查相对布局约束并报告失败。检查下列失败：

- 相对布局组不能作为单一实体放置
- 相对布局组的高度或宽度大于 `core area` 的高度或宽度
- 无法满足用户指定的方向

该命令不检查相对布局组中 `keepout` 未正确创建时的失败。

生成的报告将关键与非关键失败分节列出：若失败阻止组按相对布局约束作为单一实体放置，则为关键；若不阻止布局但导致相对布局约束违例，则为非关键。

用 `-all` 检查全部组，或指定组名；默认显示到屏幕，用 `-output` 保存到文件；用 `-verbose` 显示详细报告：

```
de_shell> check_rp_groups "compare17::seg7 compare17::rp_group3" \
    -output rp_failures.log
de_shell> check_rp_groups -all -verbose
```

## 保存相对布局信息

### Saving Relative Placement Information

用 `write_rp_groups` 写出相对布局约束：

```
de_shell> get_rp_groups
{mul::grp_mul ripple::grp_ripple example3::top_group}
de_shell> write_rp_groups -all -output my_groups.tcl
1
de_shell> remove_rp_groups -all -quiet
1
de_shell> get_rp_groups
Error: Can't find objects matching '*'. (UID-109)
de_shell> source my_groups.tcl
{example3::top_group}
de_shell> get_rp_groups
{example3::top_group ripple::grp_ripple mul::grp_mul}
```

默认写出创建指定相对布局组以及向这些组添加叶单元、层次组与 **keepout** 的命令，但不写出在层次组内生成子组的命令。命令写出更新后的相对布局约束，并包含 **uniquification** 或 **ungrouping** 带来的名称变更。

若用 **-num\_columns** 或 **-num\_rows** 为单元指定多个列/行位置，**write\_rp\_groups** 会写出该多位置单元。

移除相对布局组、对象与属性

## Removing Relative Placement Groups, Objects, and Attributes

移除相对布局组

**Removing Relative Placement Groups** 用 **remove\_rp\_groups** 的 **-all** 或指定组名移除。指定要移除的组列表时，工具仅移除指定组，不移除其中包含或实例化的组；要一并移除，须指定 **-hierarchy**。

```
de_shell> remove_rp_groups ripple::grp_ripple
Removing rp group 'ripple::grp_ripple'
1
de_shell> remove_rp_groups -all
```

从相对布局组移除对象

**Removing Objects From a Relative Placement Group** 用 **remove\_from\_rp\_group** 移除叶单元 (**-leaf**)、included 组 (**-hierarchy**)、instantiated 组 (**-hierarchy -instance**) 与 **keepout** (**-keepout**):

```
de_shell> remove_from_rp_group ripple::grp_ripple -leaf carry_in_1
```

若单元占据多个列/行位置，该命令从全部位置移除该单元。

移除相对布局组属性

**Removing Relative Placement Group Attributes** 用 **remove\_rp\_group\_options**；必须指定组名与至少一个选项，否则无效：

```
de_shell> remove_rp_group_options block1::misc1 -x_offset
{block1::misc1}
```

命令返回属性已更改的相对布局组集合；若无更改则返回空字符串。

## 9.6.4 使用 HDL Compiler 指令创建相对布局

### Creating Relative Placement Using HDL Compiler Directives

DC Explorer 支持嵌入在 Verilog 或 VHDL 描述中的相对布局信息。该能力由可指定与修改相对布局信息的 HDL compiler directive 启用。使用这些指令指定相对布局可增加设计灵活性并简化相对布局，因为无需再更新设计中许多单元的位置。

利用嵌入的 HDL compiler directive，可在 RTL、GTECH 网表或已映射网表中放置相对布局约束。有关在 Verilog 与 VHDL 中用 HDL compiler directive 指定相对布局（含指南与限制），见 HDL Compiler 文档。

更多信息另见:

- *HDL Compiler for SystemVerilog User Guide*
- *HDL Compiler for VHDL User Guide*

# Chapter 10

## 优化

### *Optimization*

DC Explorer 提供快速综合以加速高质量 RTL 开发。相对完整综合，运行时间显著更短，便于对多种配置做 what-if 分析，以满足功能、时序与面积要求。使用 `compile_exploration` 综合设计。DC Explorer 并发优化全部 endpoint，并使用不同优化引擎，使运行时间相对设计规模近似线性。

本章包括以下主题：

- Compile 策略
- 执行自顶向下 Compile
- 执行自底向上 Compile
- 面向高性能设计的优化
- 执行手动高扇出综合
- 解析设计引用的多实例
- 定义库子集限制
- 优化多角多模式设计
- 带 Floorplan 物理约束的优化
- 执行功耗优化
- 流水线逻辑 Retiming
- 其他优化功能

## 10.1 Compile 策略

### Compile Strategies

对层次化设计可采用多种 compile 策略。基本策略为：

- 自顶向下（**Top-down**）**compile** — 顶层设计及其全部子设计一起 compile

- 自底向上（**Bottom-up**）**compile** —从层次底部开始分别 **compile** 各子设计，再逐级向上直至顶层

以图 10.1 所示简单设计说明自顶向下策略。表 10.1 给出该设计的顶层/全局规格，由例 10.1 中的脚本定义；这些规格适用于 TOP 及其全部子设计。



图 10.1: 用于说明 Compile 策略的设计

表 10.1: 设计 TOP 的设计规格

| 规格类型                 | 取值            |
|----------------------|---------------|
| Operating condition  | WCCOM         |
| Clock frequency      | 40 MHz        |
| Input delay time     | 3 ns          |
| Output delay time    | 2 ns          |
| Input drive strength | drive_of (IV) |
| Output load          | 1.5 pF        |

```
set_operating_conditions WCCOM
create_clock -period 25 clk
set_input_delay 3 -clock clk \
    [remove_from_collection [all_inputs] [get_ports clk]]
set_output_delay 2 -clock clk [all_outputs]
set_load 1.5 [all_outputs]
set_driving_cell -lib_cell IV [all_inputs]
set_drive 0 clk
```

代码 10.1: 设计 TOP 的约束文件（defaults.com）

注意

为防止时钟网络被缓冲，脚本将时钟端口（`clk`）的输入驱动电阻设为 0（无限驱动强度）。

- 参见
- 执行自顶向下 Compile
  - 执行自底向上 Compile

## 10.2 执行自顶向下 Compile

### Performing a Top-Down Compile

对不受内存或 CPU 限制的设计，可使用自顶向下策略。内存受限的顶层设计常可先用 block abstraction 替换部分子设计后再采用该策略；用 block abstraction 替换子设计可降低顶层中该实例的内存需求。

优点：

- 提供一键式（push-button）方法
- 自动处理块间依赖

缺点：需要更多内存；对超过约 100K 门的设计可能运行时间更长。

步骤：

1. 加载整个设计。
2. 在顶层施加属性与约束（实现设计规格）。

#### 注意

也可向子设计分配局部属性与约束，但须相对于顶层设计定义。

3. Compile 设计。

#### 注意

若顶层含一个或多个 block abstraction，使用第 十一章「使用层次化模型」中的 compile 流程。

例 10.2 给出 TOP 的自顶向下 compile 脚本；约束通过包含「Compile 策略」示例中的约束文件施加。

```
/* read in the entire design */
read_verilog E.v
read_verilog D.v
read_verilog C.v
read_verilog B.v
read_verilog A.v
read_verilog TOP.v
current_design TOP
link
report_design_mismatch
/* apply constraints and attributes */
source defaults.con
/* compile the design */
compile_exploration
```

代码 10.2: 自顶向下 Compile 脚本

**参见**

- Compile 策略
- 第十一章「使用层次化模型」
- 第五章「处理内存中的设计」
- 第六章「定义设计环境」
- 第七章「定义设计约束」

## 10.3 执行自底向上 Compile

### Performing a Bottom-Up Compile

推荐策略是自顶向下 compile。但为应对设计与运行时间挑战，或采用分而治之的综合方法，可使用自底向上 compile（亦称层次化 compile 流程）：分别 compile 子块，再并入顶层。

DC Explorer 可读取下列类型的层次块：

- 在 DC Explorer 中生成的网表
- 在 DC Explorer 或 IC Compiler 中生成的 block abstraction

可将子块 compile 后以完整 .ddc 网表或 block abstraction 提供给顶层；也可在 IC Compiler 中继续处理子块以创建 block abstraction 再提供给顶层。子块的时序与物理信息会传播到顶层供物理综合使用；还可在顶层综合期间为子块提供布局位置以保持与 IC Compiler 的相关性。

在层次化/自底向上流程中，用 `compile_exploration -scan -gate_clock` 做顶层集成。工具自动将块级时序与布局传播到顶层并驱动优化；也可指定子块布局位置。顶层优化时考虑布局，并可使用与后端工具相同的物理约束。

要进一步了解，见：

- 自底向上 Compile 概览
- Compile 子块
- 在顶层 Compile 设计

**参见**

- Compile 策略
- 第十一章「层次化模型概览」

### 10.3.1 自底向上 Compile 概览

#### Overview of Bottom-Up Compile

在自底向上流程中，compile 子块并将已映射子块保存为 .ddc 网表，再在 IC Compiler 中生成 block abstraction；也可将已映射子块直接保存为 .ddc 中的 block abstraction。

顶层综合可接受：

- DC Explorer 综合的 .ddc 网表
- DC Explorer 创建的 block abstraction
- IC Compiler 创建的 block abstraction

#### 注意

IC Compiler 不能接受 DC Explorer 的 block abstraction。

图 10.2 给出含 block abstraction 设计的层次化流程概览。

(原书 *Figure 52: Overview of the Hierarchical Flow for Designs Containing Block Abstractions*)

请参阅原版 PDF 插图；可将截图放入 figures/ 后用 \includegraphics 替换。

图 10.2: 含 Block Abstraction 设计的层次化流程概览

主要任务：

1. Compile 子块—分别 compile 各子设计
2. 在顶层 Compile 设计—读入顶层及尚未在内存中的已 compile 子设计，再 compile 顶层

### 10.3.2 Compile 子块

#### Compiling the Subblock

1. 指定逻辑库与物理库。
2. 读入子块（Verilog、VHDL、网表或 .ddc），并将当前设计设为该子块。
3. 施加块级时序约束与功耗约束。
4. (可选) 提供物理约束。
5. (可选) 在 Design Vision layout 窗口中直观验证综合前 floorplan；需链接全部适用设计与库。
6. 用 `compile_exploration -scan -gate_clock` 对子块做 test-ready 与时钟门控 compile。
7. 运行 `uniquify -force`，为每个单元实例创建唯一设计名，避免顶层集成时名称冲突。
8. 写出设计数据前，用 `change_names -rules verilog -hierarchy` 对全部设计对象应用 Verilog 命名规则。

此后按目标继续：

- 创建完整 .ddc 网表—用 `write_file -format ddc` 保存；顶层综合时读入或加入 `link_library`
- 为子块生成 **Block Abstraction**—见下文

- 在 **IC Compiler** 中为子块生成 **Block Abstraction** 一见下文

#### 参见

- 指定逻辑库 / 物理库
- 使用 Floorplan 物理约束
- 在顶层 Compile 设计

为子块生成 **Block Abstraction**

#### Generating a Block Abstraction for the Subblock

完成「Compile 子块」步骤后继续：

1. 运行 `create_block_abstraction` 一识别当前设计接口逻辑并注释到内存中的设计
2. 用 `write_file` 保存，例如：

```
de_shell> write_file -hierarchy -format ddc -output myfile.mapped.ddc
```

应在创建后立即保存；`write_file` 将完整设计（含 block abstraction 信息）写入 `.ddc`。

然后按「在顶层 Compile 设计」集成。

在 **IC Compiler** 中为子块生成 **Block Abstraction**

#### Generating a Block Abstraction for the Subblock in IC Compiler

1. 在 DC Explorer 中用 `write_file -format ddc` 写出块级设计。
2. 在 IC Compiler 中设置设计与 Milkyway 库。
3. 读入已映射子块的 `.ddc` 以及 topographical 模式生成的块 SCANDEF。
4. 用 IC Compiler 命令创建 floorplan，或用 `read_def` 从 DEF 读入。
5. 运行 `place_opt`。
6. 运行 `clock_opt`。
7. 运行 `route_opt`。
8. 运行 `create_block_abstraction`。
9. 运行 `save_mw_cel` 创建 CEL 视图。

### 10.3.3 在顶层 Compile 设计

#### Compiling the Design at the Top Level

1. 指定逻辑库与物理库。若使用 IC Compiler 创建的 block abstraction，须将含该 CEL 视图的 Milkyway 设计库加入顶层 Milkyway 参考库列表。

2. 若使用 DC Explorer 或 IC Compiler 的 block abstraction, 用 `set_top_implementation_options` 指定应以 block abstraction 集成的块:

- DC Explorer (.ddc): 在打开块之前使用 `-block_references`; 否则加载完整块网表
- IC Compiler (CEL): 设置顶层链接选项; 否则无法链接到 IC Compiler block abstraction。若已将含 CEL 的 Milkyway 库加入参考库列表, 链接顶层时自动加载

#### 注意

要在顶层综合期间忽略完全位于 block abstraction 内的时序路径, 见第 十一章「忽略 Block Abstraction 内的时序路径」。

3. (可选) 将 `timing_ignore_paths_within_block_abstraction` 设为 true, 忽略完全位于 block abstraction 内的寄存器到寄存器路径。
4. 读入顶层设计 (Verilog、VHDL、网表或 .ddc)。
5. 读入已映射子块, 可为: 完整 .ddc 网表; DC Explorer block abstraction (须先设 `set_top_implementation_options`, 仅加载接口逻辑); IC Compiler block abstraction (须在链接前设置选项)。

#### 注意

可将 .ddc 加入 `link_library` 而非直接读入; 链接到顶层时工具自动加载。

6. 将当前设计设为顶层。
7. (可选) 对完整 .ddc 网表用 `set_physical_hierarchy` 将子块当作物理子块; 此时顶层综合保留块内逻辑结构与单元布局。若不希望当作物理块, 则省略该命令。
8. 用 `link` 将子块链接到顶层。
9. 施加顶层时序与功耗约束。
10. (可选) 提供物理约束 (`set_cell_location` 或从 DEF `extract_physical_constraints`)。
11. (可选) 直观验证 floorplan。
12. 运行 `compile_exploration -scan -gate_clock`。
13. 用 `write_file -format ddc` 写出顶层网表 (与原始 .ddc 合并变更并写出完整块)。

#### 参见

- 指定逻辑库 / 物理库
- 忽略 Block Abstraction 内的时序路径
- 使用 Floorplan 物理约束

## 10.4 面向高性能设计的优化

### Performing Optimization for High-Performance Designs

可用 `compile_exploration` 获得更好的 QoR，尤其适用于时序约束非常紧的高性能设计。该命令允许在 `compile` 期间应用尽可能合适的一组 `timing-centric variable`，以优化 `critical delay` 并改善 `area` QoR。

默认 `compile` 对多数设计可给出良好结果。若设计探索后仍未达优化目标，可尝试：

- Datapath 优化
- 面向最小面积的优化

#### 10.4.1 Datapath 优化

##### Datapath Optimization

Datapath 设计常见于含大量数据操作的应用（3-D、多媒体、DSP）。Datapath 提取将加、减、乘等算术算子变换为 `datapath` 块，由 `datapath` 生成器实现，利用进位保存算术改善 QoR。

运行 `compile_exploration` 时默认启用 `datapath` 优化。DC Explorer 使用 `datapath` 生成器构建算术组件，并在优化期间考虑位级时序上下文。要将 `datapath` 生成器与 `compile_exploration` 一起使用，请将 `dw_foundation.sldb` 列入 `synthetic library` 列表；必要时执行 `set synthetic_library dw_foundation.sldb`。需要 DesignWare 许可证。

默认若 `synthetic library` 列表中无 `dw_foundation.sldb` 但 DesignWare 许可证已成功检出，则仅对当前命令自动将该库加入列表；用户指定的 `synthetic/link library` 列表不受影响。

DC Explorer 启用 `datapath` 提取，并在 `compile` 期间探索多种 `datapath` 与资源共享选项。收益包括：共享 `datapath` 算子；提取 `datapath`；探索可能涉及不同资源共享配置的更好解；在资源共享与 `datapath` 优化之间做更好折中。

#### 10.4.2 面向最小面积的优化

##### Optimizing for Minimum Area

若设计有时序约束，时序始终优先于面积。对面积极关键设计，指定现实约束。微调面积时可保留层次，并让 DC Explorer 默认执行边界优化；要进一步减小面积，可能需要移除层次边界。

若未满足面积约束，可尝试跨层次边界优化：

- 边界优化（Boundary Optimization）
- 层次移除（Hierarchy Removal）

边界优化

##### Boundary Optimization

默认 DC Explorer 跨层次边界优化。也可：

```
de_shell> set_boundary_optimization subdesign true
```

启用后，工具传播常量、未连接引脚与互补信息。对许多常量（VCC/GND）连接到子设计输入的设计，传播可减小面积（见图 10.3）。

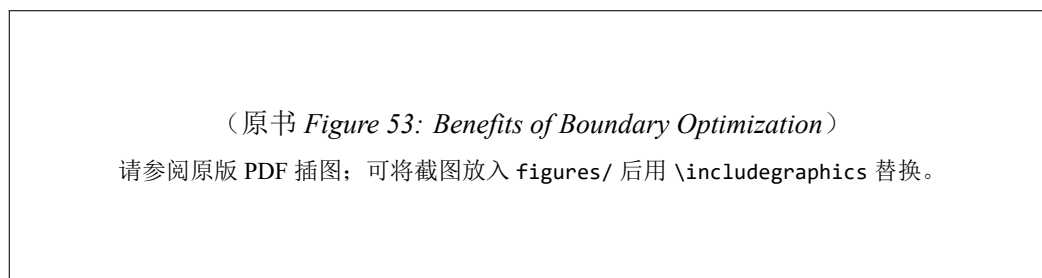


图 10.3: 边界优化的收益

若禁用边界优化仍需减小面积,将 `compile_optimize_unloaded_seq_logic_with_no_bound_opt` 设为 `true`（默认 `false`）以移除未连接逻辑；跨层次移除未连接寄存器；通过多比特寄存器传播未连接逻辑以移除未使用位并推断更小尺寸的多比特寄存器。

层次移除

### Hierarchy Removal

默认第一遍 `compile` 自动 `ungroup` 小设计层次；第二遍用基于延时的 `auto-ungrouping` 沿关键路径 `ungroup`。Ungrouping 移除层次边界，使工具可在更多门上优化。也可用 `set_ungroup` 在优化前指定要 `ungroup` 的单元。

默认第二遍中全部 DesignWare 层次无条件 `ungroup`。将 `compile_ultra_ungroup_dw` 设为 `false`（默认 `true`）可阻止。

参见

移除层次级别。

## 10.5 执行手动高扇出综合

### Performing Manual High-Fanout Synthesis

用 `create_buffer_tree` 手动运行高扇出综合以控制缓冲树的创建与移除。默认 `compile_exploration` 会自动做高扇出综合。

注意

必须已有已映射设计才能运行手动高扇出综合。

`create_buffer_tree` 移除设计中已有缓冲树并重新创建。要保留已有缓冲树,指定 `-incremental`。此时仅在 `-from` 指定的各网上按需构建缓冲树以降低高扇出，不移除现有缓冲器/反相器。

步骤：

1. 用 `report_buffer_tree` 报告已映射网表上的当前缓冲树实现
2. 识别 `compile_exploration` 产生的次优缓冲

### 3. 用 `remove_buffer_tree` 与 `create_buffer_tree` 按需移除并创建缓冲树

插入的缓冲器/反相器可能被其他优化命令进一步优化；要在优化期间保留特定缓冲器/反相器，对其设置 `size_only` 或 `dont_touch` 等限制属性。

## 10.6 解析设计引用的多实例

### Resolving Multiple Instances of a Design Reference

在层次化设计中，子设计常被多个单元实例引用。例如图 10.4 中设计 TOP 对设计 C 引用两次（U2/U3 与 U2/U4）。

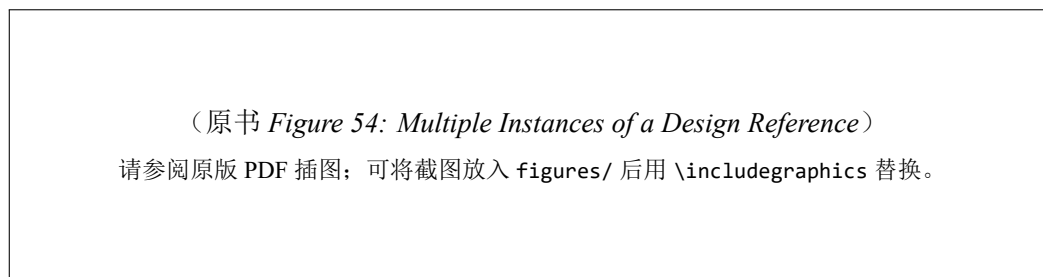


图 10.4: 设计引用的多实例

对 `check_design` 使用 `-multiple_designs` 报告与多重实例化设计相关的信息消息；列出全部多重实例化设计及实例名与属性（`dont_touch`、`black_box`、`ungroup`）。

处理方法：

- **Uniquify** 方法 — `compile` 过程自动 `uniquify`；仍可在 `compile` 前手动 `uniquify`，但这会增加运行时间（`compile` 时会再次 `uniquify`）。不能关闭 `uniquify` 过程。
- **Compile-Once-Don't-Touch** 方法 — 用 `set_dont_touch` 在其余设计 `compile` 时保留已 `compile` 的子设计
- **Ungroup** 方法 — 用 `ungroup` 移除层次

#### 参见

保留子设计。

### 10.6.1 Uniquify 方法

#### Uniquify Method

`Uniquify` 过程复制并重命名任何多重引用的设计，使每个实例引用唯一设计；创建新唯一设计后从内存移除原始设计，原始设计及其集合不再可访问。

工具在 `compile` 过程中自动 `uniquify`，可解析当前设计层次中的多重引用（带 `dont_touch` 的除外）。之后可基于各单元实例的独特环境优化每个设计副本。

也可用 `uniquify` 的 `-reference` 或 `-cell` 为特定引用/单元创建唯一副本；即使标有 `dont_touch` 也会复制。`-cell` 接受实例对象；指定下层实例时会 `uniquify` 到该实例的完整路径，例如：

```
de_shell> uniquify -cell mid1/bot1
```

命名约定由 `uniquify_naming_style` 指定，默认 `%s_%d`（%s 为原 subdesign 名或 `-base_name` 指定的名称；%d 为形成唯一 subdesign 名的最小整数）。

对图中设计 C 的多重实例，自动 `uniquify` 创建 `C_0` 与 `C_1`：

```
de_shell> current_design top
de_shell> compile_exploration
```

（原书 *Figure 55: Uniquify Results*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 10.5: Uniquify 结果

相对 `compile-once-don't-touch`：需要更多内存；`compile` 时间更长。

## 10.6.2 Compile-Once-Don't-Touch 方法

### Compile-Once-Don't-Touch Method

若多重引用设计各实例周围环境足够相似，可先用某一实例的环境 `compile`，再用 `set_dont_touch` 在后续优化中保留该子设计。

步骤：

1. `Compile` 被引用的子设计
2. 对引用已 `compile` 子设计的全部实例设置 `dont_touch`
3. `Compile` 整个设计

例如（假设 `U2/U3` 环境最差）：

```
de_shell> current_design top
...
de_shell> current_design C
de_shell> compile_exploration
de_shell> current_design top
de_shell> set_dont_touch {U2/U3 U2/U4}
de_shell> compile_exploration
```

优点：引用设计只 `compile` 一次；比 `uniquify` 更省内存与时间。主要缺点：表征可能不适用于全部实例；带 `dont_touch` 的对象不能 `ungroup`。

参见

保留子设计。

(原书 *Figure 56: Compile-Once-Don't-Touch Results*)  
 请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 10.6: Compile-Once-Don't-Touch 结果

### 10.6.3 Ungroup 方法

#### Ungroup Method

效果与 `uniquify` 类似 (创建唯一副本)，并额外移除层次级别；用 `ungroup` 产生平坦网表。Ungroup 实例后可重新 `compile` 顶层：

```
de_shell> current_design B
de_shell> ungroup {U3 U4}
de_shell> current_design top
de_shell> compile_exploration
```

(原书 *Figure 57: Ungroup Results*)  
 请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 10.7: Ungroup 结果

特点：比 `compile-once-don't-touch` 更耗内存与时间；通常提供最佳综合结果。缺点：移除用户定义的设计层次。

#### 参见

移除层次级别。

### 10.6.4 保留子设计

#### Preserving Subdesigns

用 `set_dont_touch` 在优化期间保留子设计：在单元、网络、引用与设计上放置 `dont_touch`，防止被修改或替换。

#### 注意

设计中的任何 `block abstraction` 会自动标记 `dont_touch`；其单元也标记为 `don't touch`。

`dont_touch` 不阻止或禁用穿过该设计的时序。注意：

- 在层次单元上设置会对其下全部单元隐含 `don't touch`
- 在库单元上设置会对该单元全部实例隐含 `don't touch`
- 在网络上设置仅对连接到该网的已映射组合单元隐含 `don't touch`；若仅连到 `generic logic`，优化可能移除该网
- 在引用上设置会在后续优化中对使用该引用的全部单元隐含 `don't touch`
- 在设计上设置仅当该设计作为层次实例化在另一设计中时有效；对顶层设计设置无效
- 不能用 `ungroup` `ungroup` 带 `dont_touch` 的对象；自动 `ungrouping` 也不影响 `don't touch` 对象

#### 注意

对 `synthetic part` 单元以及连有未映射单元的网络，`dont_touch` 被忽略；`compile` 时会连到未映射单元（`generic logic`）的 `don't touch` 网络发出警告。

用 `remove_attribute` 或将 `set_dont_touch` 设为 `false` 移除该属性。

#### 参见

第十一章「使用层次化模型」。

## 10.7 定义库子集限制

### Defining Library Subset Restrictions

可将 `sequential cell` 与 `instantiated combinational cell` 的 `mapping` 和 `optimization` 限制到 `target library` 中用户指定的 `library cell` 子集。对 `define_libcell_subset` 使用 `-family_name` 定义一个或多个子集；仅当库单元满足下列条件时才归入子集：

- 必须是 `sequential cell` 或 `instantiated combinational cell`
- 不能已属于另一子集
- 必须具有相同的 `functional identification`

分组后，这些 `library cell` 在 `compile` 期间不再用于一般 `mapping`；DC Explorer 仅使用它们映射由 `set_libcell_subset` 指定的 `sequential cell` 与 `instantiated combinational cell`。用 `-object_list` 指定要优化的 `cell` 列表；指定 `cell` 必须尚未 `mapping`，或已实例化为该子集中的某个 `library cell`。

运行这两条命令后，全部优化（含单元尺寸调整与交换）限制在该库子集中的单元；该限制仅适用于优化期间创建、映射或修改的单元，不适用于未优化的设计部分。

```
de_shell> define_libcell_subset -libcell_list {SDFLOP1 SDFLOP2} \
    -family_name specialflops
de_shell> set_libcell_subset -libcell_list [get_cells {reg01 reg02}] \
    -family_name specialflops
```

用 `report_libcell_subset` 报告；用 `remove_libcell_subset` 移除约束或用户定义子集：

```
de_shell> report_libcell_subset -object_list [get_cells reg01]
de_shell> remove_libcell_subset -object_list [get_cells {reg01 reg02}]
de_shell> remove_libcell_subset -family_name specialflops
```

## 10.8 优化多角多模式设计

### Optimizing Multicorner-Multimode Designs

DC Explorer 可跨多个 mode 与多个 corner 并发分析和优化 multicorner-multimode (MCMM) 设计。由于用户界面相似, 该功能在 DC Explorer 与 IC Compiler flow 之间提供了易用性和兼容性。

下列主题描述 MCMM 设计优化:

- 多角多模式概念
- 多角多模式设计不支持的功能
- 基本多角多模式流程
- 在多角多模式流程中处理库
- 定义 Scenario
- 管理 Scenario
- 并发多角多模式优化与时序分析
- 多角多模式设计的报告命令
- 多角多模式设计支持的 SDC 命令
- 在多角多模式设计中使用 Block Abstraction

#### 10.8.1 多角多模式概念

##### Multicorner-Multimode Concepts

在多个工作条件 (corner) 与多个 mode 下工作的设计称为多角多模式设计。

常用术语:

- **Corner** 一由一组工艺、电压与温度 (PVT) 条件组成的工作条件。Corner 不依赖功能设置, 用于捕获制造工艺变化以及芯片工作环境中预期的电压与温度变化。
- **Mode** 一由一组 clock、supply voltage、timing constraint 和 library 定义; 也可包含 SDF 或 parasitics file 等 annotation data。MCMM 设计可工作于 test、mission、standby 等多种 mode。
- **Scenario** 一modal 约束与 corner 规格的组合。工具以 scenario 或 scenario 集作为 MCMM 分析与优化的单位。某些约束可同时属于 mode 与 corner 规格。优化 MCMM 设计涉及管理设计的 scenario。

参见

管理 Scenario (下文)。

## 10.8.2 多角多模式设计不支持的功能

### Unsupported Features for Multicorner-Multimode Designs

DC Explorer 对 MCMM 设计不支持：

- 功耗驱动的时钟门控—使用 `compile_exploration -gate_clock` 时，时钟门插入与 `scenario` 无关
- 时钟树估计
- k-factor scaling —MCMM 设计库不支持 k-factor scaling，为各 `scenario` 指定的工作条件必须匹配 `link library` 列表中某一库的标称工作条件
- 针对个别 `scenario` 的 `set_min_library` —该命令定义的库设置适用于全部 `scenario`
- `set_scenario_options` 的 `-leakage_power` 与 `-dynamic_power` —若指定，工具忽略并发出警告

## 10.8.3 基本多角多模式流程

### Basic Multicorner-Multimode Flow

图 10.8 给出基本 MCMM 流程。关键步骤是创建 `scenario` 定义。至少应包括 `scenario` 名、工作条件、TLUPlus 库与约束；可创建多个 `scenario`。对每个 `scenario` 指定该 `mode` 特有的约束与该 `corner` 特有的工作条件。

(原书 *Figure 58: Basic Multicorner-Multimode Flow*)  
 请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 10.8: 基本多角多模式流程

```
#.....path settings.....
set search_path ". $DESIGN_ROOT $lib_path/dbs \
  $lib_path/mwlibs/macros/LM"
set target_library "stdcell.setup.ftyp.db \
  stdcell.setup.typ.db stdcell.setup.typhv.db"
set link_library [concat * $target_library \
  setup.ftyp.130v.100c.db setup.typhv.130v.100c.db \
  setup.typ.130v.100c.db]
set_min_library stdcell.setup.typ.db -min_version stdcell.hold.typ.db
#.....MW setup.....
#.....load design.....
create_scenario s1
set_operating_conditions WORST -library stdcell.setup.typ.db:stdcell_typ
set_tlu_plus_files -max_tluplus design.tlup -tech2itf_map layermap.txt
read_sdc s1.sdc
set_scenario_options -scenarios s1 -setup false -hold false
```

```

create_scenario s2
set_operating_conditions BEST -library stdcell.setup.ftyp.db:stdcell_ftyp
set_tlu_plus_files -max_tluplus design.tlup -tech2itf_map layermap.txt
read_sdc s2.sdc

create_scenario s3
set_operating_conditions NOM -library stdcell.setup.ftyp.db:stdcell_ftyp
set_tlu_plus_files -max_tluplus design.tlup -tech2itf_map layermap.txt
read_sdc s3.sdc

set_active_scenarios {s1 s2}
report_scenarios
compile_exploration -scan -gate_clock
report_qor
report_constraint
report_timing -scenarios [all_scenarios]
...
compile_exploration -scan

```

代码 10.3: 运行多角多模式流程的基本脚本

## 10.8.4 在多角多模式流程中处理库

### Handling Libraries in the Multicorner-Multimode Flow

使用具有相同 PVT 标称值的 **Link Library**

#### Using Link Libraries With the Same PVT Nominal Values

Link library 包含全部 scenario 链接设计所用的库。因一个 scenario 常使用多个库（标准单元库与宏库等），工具自动将库分组为集合，并识别各 scenario 应链接哪一集合。

具有相同 PVT 值的库分到同一集合。工具用 scenario 定义中最大工作条件的 PVT 值选择适当集合。若在 link library 中找不到合适单元则报错；应在优化前验证工作条件与库设置。

表 10.2 示例给出 link library 列表、标称 PVT 与库中工作条件。

表 10.2: 具有 PVT 值与工作条件的 Link Library

| Link library (顺序)   | 标称 PVT     | 库中工作条件 (PVT)       |
|---------------------|------------|--------------------|
| Combo_cells_slow.db | 1/0.85/130 | WORST (1/0.85/130) |
| Sequentials_fast.db | 1/1.30/100 | None               |
| Macros_fast.db      | 1/1.30/100 | None               |
| Macros_slow.db      | 1/0.85/130 | None               |
| Combo_cells_fast.db | 1/1.30/100 | BEST (1/1.3/100)   |
| Sequentials_slow.db | 1/0.85/130 | None               |

下例创建 scenario s1，单元实例链接到 Combo\_cells\_slow、Macros\_slow 与 Sequentials\_slow:

```

de_shell> create_scenario s1
de_shell> set_operating_conditions -max WORST -library slow

```

对 `set_operating_conditions` 使用 `-library` 有助于工具识别正确 PVT。链接时用最大工作条件的 PVT 在 `link library` 列表中找到匹配。该方法可链接无工作条件定义的库，并支持标准单元与宏分属不同库文件。

防止不一致的库

**Preventing Inconsistent Libraries** 对多库设计，若同名库单元功能、引脚名或引脚顺序不一致，工具发出警告。应在运行 MCMM 流程前用 `check_library` 检查：

```
de_shell> set_check_library_options -mcmm
de_shell> check_library -logic_library_name {a.db b.db}
```

指定 `-mcmm` 时，`check_library` 执行 MCMM 专用检查（如工作条件或不一致的下电功能）；检测到不一致时生成报告并发出信息消息。

在库单元上设置 `dont_use`

**Setting the dont\_use Attribute on Library Cells** 若在某库单元上设置 `dont_use`，MCMM 流程要求该单元的全部表征都有该属性，否则可能视为库不一致。可用通配符：

```
de_shell> set_dont_use */AN2
```

带 `dont_use` 的库单元若与各 `corner` 库单元引脚顺序不同，工具继续流程且不报错/警告；若移除这些单元上的 `dont_use`，则发出 MV-087 错误。

使用不同的 PVT 值

Using Distinct PVT Values

为确保单元实例正确链接，为工作条件使用不同的标称 PVT。工具用 PVT 将 `link library` 分组（如不同 `corner` 的最大库为一组）。当各 `scenario` 的最大库没有不同的 PVT 时，单元实例可能错误链接到该集合中第一个类型匹配的单元，即使 `set_operating_conditions ... -library` 指定了 `scenario` 专用库。

例 10.4 与表 10.3/10.4 说明：工具将 `Ftyp.db` 与 `TypHV.db` 分到同一集合且 `Ftyp.db` 在前，导致 `scenario s2` 的单元错误链接到 `Ftyp.db` 而非预期的 `TypHV.db`。

表 10.3: 具有标称 PVT 与工作条件的 Link Library

| Link library | 标称 PVT     | 工作条件（PVT）          |
|--------------|------------|--------------------|
| Ftyp.db      | 1/1.30/100 | WORST (1/1.30/100) |
| Typ.db       | 1/0.85/100 | WORST (1/0.85/100) |
| TypHV.db     | 1/1.30/100 | WORST (1/1.30/100) |
| HoldTyp.db   | 1/0.85/100 | BEST (1/0.85/100)  |

```
create_scenario s1
set_operating_conditions WORST -library Typ.db:Typ
create_scenario s2
set_operating_conditions WORST -library TypHV.db:TypHV
```

表 10.4: Scenario 及其工作条件

|            | s1             | s2               | s3              | s4                |
|------------|----------------|------------------|-----------------|-------------------|
| Max Opcond | WORST (Typ.db) | WORST (TypHV.db) | WORST (Ftyp.db) | WORST (Typ.db)    |
| Min Opcond | None           | None             | None            | BEST (HoldTyp.db) |

```
create_scenario s3
set_operating_conditions WORST -library Ftyp.db:Ftyp
create_scenario s4
set_operating_conditions \
    -max WORST -max_library Typ.db:Typ \
    -min BEST -min_library HoldTyp.db:HoldTyp
```

代码 10.4: 导致链接问题的不清晰 PVT 值

模糊库警告

**Ambiguous Libraries Warning** 若设计使用含同名单元且标称 PVT 相同的库，工具发出警告：库模糊，并指明使用/忽略哪些库。

定义最小库

Defining Minimum Libraries

用 `set_operating_conditions` 为各 scenario 定义最小库。若用 `set_min_library` 为某一 scenario 定义最小库，即使未对全部 scenario 定义，工具也会将该库用作全部 scenario 的最小库。

一个最小库可关联多个最大库（表 10.5）。但不能将两个不同最小库关联到两个 scenario 中的同一最大库（表 10.6）；此时工具对两个 scenario 都应用第一个最小库。

表 10.5: 支持的库配置

|                 | Scenario s1 | Scenario s2 |
|-----------------|-------------|-------------|
| Maximum library | Slow.db     | SlowHV.db   |
| Minimum library | Fast_0yr.db | Fast_0yr.db |

表 10.6: 不支持的多重最小库配置

|                 | Scenario s1 | Scenario s2  |
|-----------------|-------------|--------------|
| Maximum library | Slow.db     | Slow.db      |
| Minimum library | Fast_0yr.db | Fast_10yr.db |

## 自动检测驱动单元库

### Automatic Detection of Driving Cell Libraries

要用 `set_driving_cell` 的 `-library` 为不同 scenario 指定不同库以构建驱动单元时序弧。工具在 link library 集合中搜索；若不存在则报错。

若不指定 `-library`，工具搜索匹配工作条件的全部库：使用集合中第一个匹配工作条件的库；若无匹配则使用含匹配库单元的的第一个库；若仍无则报错。

## 10.8.5 定义 Scenario

### Defining Scenarios

为 MCMM flow 设置设计时，必须为每个 scenario 指定 TLUPlus file、operating condition 和 SDC constraint。DC Explorer 按 nominal process、voltage 和 temperature (PVT) 值将 library 分组；PVT 相同的 library 归入同一集合，并使用各 scenario 的 maximum operating condition 的 PVT 值选择相应集合。

创建 scenario 定义：

1. 用 `create_scenario` 创建名称。创建第一个 scenario 时，先前全部 scenario 专用约束从设计移除：

```
de_shell> create_scenario s1
Warning: Any existing scenario-specific constraints
are discarded. (MV-020)
Current scenario is: s1
```

2. 用 `set_operating_conditions` 指定工作条件。
3. 用 `set_tlu_plus_files` 为每个 scenario 指定 TLUPlus file。要启用 temperature scaling，TLUPlus file 必须包含 GLOBAL\_TEMPERATURE 和 CRT1；CRT2 可选。
4. 用例如 `read_sdc s1.sdc` 指定设计约束。
5. (可选) 用 `set_scenario_options` 设置 setup/hold 延时优化等选项。
6. (可选) 用 `read_sdf` 指定正确的网络 RC 与引脚到引脚延时。

#### 参见

管理 Scenario (下文)。

## 10.8.6 管理 Scenario

### Managing Scenarios

#### 设置当前 Scenario

#### Setting the Current Scenario

用 `current_scenario` 指定当前 scenario。定义多个 scenario 时，工具跨全部 scenario 并发分析与优化，与当前 scenario 设置无关。

## 定义活动 Scenario

### Defining Active Scenarios

并发分析与优化期间，可将活动 scenario 限制为必要或主导（dominant）者，以降低内存与运行时间。主导 scenario 是对其至少一个受约束对象（如与引脚关联的延时约束、网络的设计规则约束、漏电功耗）在全部 scenario 中具有最差 slack 的 scenario。

用 `set_active_scenarios` 定义活动 scenario；相关命令还有 `all_scenarios` 与 `all_active_scenarios`。

## 启用 Scenario Reduction

### Enabling Scenario Reduction

默认工具用当前 scenario 优化。要启用 scenario reduction，先将 `de_enable_physical_flow` 设为 `true` 以启用物理流程。

#### 注意

运行 scenario reduction 需要 DC-Extension 许可证。

在物理流程中，将并发分析与优化限制到主导 scenario 子集通常不会导致显著 QoR 差异。工具分析全部活动 scenario，基于违例 endpoint 数量确定主导集合，再针对时序、功耗与设计规则优化。

可用 `set_preferred_scenario` 选择首选 scenario；工具将其视为最具约束性的 scenario，仍执行 reduction，但把首选当作最紧约束。

## 指定 Scenario 选项

### Specifying Scenario Options

用 `set_scenario_options` 为优化定义特定约束选项；用 `-scenarios` 可一次施加到多个 scenario（活动或非活动）。未指定 scenario 时仅作用于当前 scenario。

`-setup` 启用/禁用最大延时优化（默认 `true`）；`-hold` 启用/禁用最小延时优化（默认 `true`；设为 `false` 时忽略 hold/最小延时违例）。

用 `report_scenario_options` 报告；用 `remove_scenario` 移除指定 scenario（删除其中全部 scenario 专用约束）。

## 10.8.7 并发多角多模式优化与时序分析

### Concurrent Multicorner-Multimode Optimization and Timing Analysis

工具可跨全部 scenario 对最差违例并发优化，消除顺序方法中的收敛问题。时序分析在全部 scenario 上并发进行，代价按全部 scenario 的时序与设计规则衡量；因此时序与约束报告显示跨全部 scenario 的最差情况。

运行时序分析可用：

- 传统 min-max 分析：`set_operating_conditions -analysis_type bc_wc`
- Early-late 分析：`set_operating_conditions -analysis_type on_chip_variation`（PrimeTime 亦用 OCV）

## 10.8.8 多角多模式设计的报告命令

### Reporting Commands for Multicorner-Multimode Designs

用 `report_scenarios` 报告全部已定义 `scenario` 的设置（逻辑库、工作条件、TLUPlus 等）；用 `report_scenario_options` 报告由 `set_scenario_options` 设置的选项（默认报告当前活动 `scenario`）。

支持 `-scenario` 的报告命令包括：`report_timing`、`report_path_group`、`report_timing_derate`、`report_extraction_options`、`report_constraint`、`report_tlu_plus_files`、`report_clock`。

报告当前 `scenario` 详情（报告头含当前 `scenario` 名）的命令包括：`report_net`、`report_delay_estimation_options`、`report_annotated_check`、`report_transitive_fanout`、`report_annotated_transition`、`report_disable_timing`、`report_annotated_delay`、`report_attribute`、`report_power_calculation`、`report_case_analysis`、`report_ideal_network`、`report_timing_requirements`、`report_internal_loads`、`report_transitive_fanin`、`report_clock_gating_check`、`report_crpr`、`report_delay_calculation`、`report_clock_timing` 等。

## 10.8.9 多角多模式设计支持的 SDC 命令

### Supported SDC Commands for Multicorner-Multimode Designs

MCMM 流程支持的 SDC 命令包括（节选）：`all_clocks`、`create_clock`、`create_generated_clock`、`get_clocks`、`group_path`、`set_annotated_delay`、`set_case_analysis`、`set_clock_gating_check`、`set_clock_latency`、`set_clock_transition`、`set_clock_uncertainty`、`set_data_check`、`set_disable_timing`、`set_drive`、`set_false_path`、`set_fanout_load`、`set_input_delay`、`set_input_transition`、`set_load`、`set_max_capacitance`、`set_max_delay`、`set_max_transition`、`set_min_delay`、`set_multicycle_path`、`set_output_delay`、`set_propagated_clock`、`set_resistance`、`set_timing_derate`。

## 10.8.10 在多角多模式设计中使用 Block Abstraction

### Using Block Abstractions in Multicorner-Multimode Designs

Block abstraction 与 MCMM scenario 兼容：可将 MCMM 约束施加到 block abstraction，并在顶层使用。

#### 注意

在 MCMM 设计中使用 block abstraction 时，必须将 `de_enable_physical_flow` 设为 `true`。

指南：

- 对每个顶层 MCMM scenario，所用各 block abstraction 块中必须存在同名 scenario。不匹配时用 `select_block_scenario` 映射；用 `-reset` 重置用户指定映射。
- Block abstraction 可有顶层未使用的额外 scenario。
- 无 MCMM scenario 的顶层默认只能与无 MCMM scenario 的 block abstraction 一起使用；用 `select_block_scenario` 映射后，即使顶层无 MCMM 也可使用 MCMM block abstraction。
- 对所用每个 TLUPlus，block abstraction 存储提取数据与指定工作条件；顶层不能为已有 TLUPlus 使用额外文件或定义额外温度 corner。

- 有 MCMM scenario 的顶层可与无 MCMM 定义的 block abstraction 一起使用；此时全部顶层 scenario 使用相同块数据，无日志消息。

在顶层将 **Block Abstraction** 与 **Scenario** 一起使用

### Using Block Abstractions With Scenarios at the Top Level

1. 将当前设计设为顶层。
2. 用 `remove_scenario -all` 移除全部 scenario。
3. 为顶层定义 scenario（须与设计中的 block abstraction 块的 scenario 定义匹配；须在下一步前全部定义完）。
4. 用 `compile_exploration` 执行优化。  
`compile` 开始时检查顶层与 block abstraction 之间无 scenario 不匹配；遇下列情况终止：数量不匹配（顶层多于块时也报错终止）；定义不一致；顶层未定义 scenario 而块有定义时也报错终止。也可用 `check_scenarios` 检查一致性。

#### 参见

- 在多角多模式设计中使用 Block Abstraction
- 第十一章「使用层次化模型」

## 10.9 带 Floorplan 物理约束的优化

### Performing Optimization With Floorplan Physical Constraints

使用 floorplan 物理约束的主要原因是准确表示布局区域，并改善与布局后设计的时序相关性。运行以布局为驱动的优化可改善 DC Explorer 与 Design Compiler topographical 模式之间的 QoR 与相关性，尤其对复杂 floorplan 的设计。

默认优化不包含任何 floorplan 信息。可提供高层 floorplan 物理约束以确定 core area 与形状、端口位置、宏位置与方向、voltage area、placement blockage 与 placement bound。这些约束可源自 IC Compiler floorplan、从 DEF 提取，或手动创建。

步骤：

1. 设置物理库。
2. 将 `de_enable_physical_flow` 设为 true 以启用物理流程：

```
de_shell> set_app_var de_enable_physical_flow true
```

- 3.（可选）提供 floorplan 物理约束；未指定时工具自动推导 floorplan 信息。
4. 运行 `compile_exploration`。

## 参见

- 指定物理库
- 第九章「使用 Floorplan 物理约束」
- 物理流程中的拥塞报告

## 10.10 执行功耗优化

### Performing Power Optimization

执行功耗优化时，可启用时钟门控、运行漏电功耗优化，并注释设计对象的开关活动信息。DC Explorer 不支持 IEEE 1801 UPF，但可处理先前已实例化的电源管理单元（如隔离单元与电平移位单元）。

- 启用时钟门控：

```
de_shell> compile_exploration -gate_clock
```

- 启用漏电功耗优化，用 `set_multi_vth_constraint`，例如：

```
de_shell> set_multi_vth_constraint -lvth_groups {lvt svt} \
    -lvth_percentage 15 -type soft -cost cell_count
```

在 DC Explorer 中应对 `-cost` 指定 `cell_count`，表示设计中低阈值电压单元的百分比。

- 用 `set_switching_activity` 注释网络开关活动，例如：

```
de_shell> set_switching_activity [get_net net1] \
    -static_probability 0.2 -toggle_rate 10 -period 1000
```

## 注意

DC Explorer 不支持 SAIF 文件。

更多信息见 *Power Compiler User Guide* 与 *Design Vision User Guide*。

## 10.11 流水线逻辑 Retiming

### Pipelined-Logic Retiming

用 `set_optimize_registers` 启用 pipelined-logic retiming。在逻辑综合前用 RTL 描述电路时，寻找最佳 register 位置并将其编码到 HDL 中非常困难且耗时。通过 register retiming，可自动调整 sequential design 中 flip-flop 的位置，使各 pipeline stage 的 delay 尽可能均衡。

`set_optimize_registers` 所用技术对 pipelined datapath block 特别有用：先指定所需 pipeline stage 数量，并在 RTL 中把全部 pipeline register 放在 combinational logic 的输入端或输出端；retiming 期间，工具将这些 register 移入 combinational logic、平衡各 stage 的 delay，并根据给定 clock period 获得最佳 timing 和 area 结果。

**Retiming** 在主输入与主输出处保持电路行为不变（除非添加流水线级或选择不保留复位状态的特殊选项），因此不必更改为原始 RTL 开发的仿真 `testbench`。

但 **retiming** 会改变设计中寄存器的位置、内容与名称；使用内部寄存器输入/输出作为参考点的验证策略将不再适用。**Retiming** 也可能改变层次单元功能，并向其接口添加时钟、清零、置位与使能引脚。

### 10.11.1 流水线逻辑 **Retiming** 命令

#### Pipelined-Logic Retiming Command

在 `compile_exploration` 之前运行 `set_optimize_registers`，以便在综合期间启用 **retiming**。该命令在指定设计或当前设计上设置 `optimize_registers` 属性，使 `compile` 时自动调用并在优化期间 **retiming**。亦称 one-pass retiming，因由 `compile_exploration` 一步完成。

### 10.11.2 寄存器 **Retiming** 示例

#### Register Retiming Example

**Retiming** 期间寄存器在组合逻辑中前移或后移。图 10.9 与图 10.10 给出通过后移寄存器降低延时的示例：**retiming** 前关键路径有四级组合逻辑与一个终点寄存器；**retiming** 后寄存器被四个寄存器替换并后移两级逻辑，关键路径变为两级，各级关键路径延时小于初始单级。延时降低常伴随寄存器数量增加，但通常增幅不大。

（原书 *Figure 59: Circuit Before Retiming*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 10.9: Retiming 前的电路

（原书 *Figure 60: Circuit After Retiming*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 10.10: Retiming 后的电路

### 10.11.3 含 Path Group 约束的寄存器 Retiming

#### Retiming Registers Containing Path Group Constraints

带 path group 约束的寄存器可被 retiming，前提是新寄存器继承原寄存器约束，且除 path group 上定义的例外外无其他时序例外。属于不同 path group 的寄存器在流水线逻辑 retiming 期间分开考虑。

允许的情况包括：

- 前移 — 仅当寄存器属于相同 path group；否则不 retiming。前移产生的寄存器继承原 path group 约束。
- 后移 — 后移产生的重复寄存器继承原 path group 约束。
- **MCMM** 设计中的 **retiming** — 工具识别在全部 scenario 中属于相同 path group 的寄存器并 retiming；不允许跨 path group。共享全部 scenario 相同 path group 规格的寄存器（如 reg\_0/reg\_1）分到一组；不共享的（如 reg\_2/reg\_3）保持分离。
- **MCMM** 中的前移 — 仅当寄存器跨全部 scenario 共享相同 path group 规格时允许。

不允许：

- 用寄存器引脚而非寄存器创建 path group，例如 create\_path\_group -to A\_reg/D
- Path group 包含始于并终于同一寄存器的路径，例如：

```
de_shell> create_path_group \
    -from A [get_cells ff_reg*] -to [get_cells ff_reg*]
```

## 10.12 其他优化功能

### Additional Optimization Features

可用下列技术改善优化结果：

- 保留子设计
- 修复多端口网络
- 优化多比特寄存器
- 物理流程中的拥塞报告
- 控制 Path Group 创建

更多信息见 *Design Compiler User Guide*。

#### 10.12.1 保留 Subdesign

##### Preserving Subdesigns

要在 optimization 期间保留 subdesign，使用 set\_dont\_touch。该命令在当前设计的 cell、net、reference 和 design 上设置 dont\_touch 属性，防止这些对象在 optimization 期间被修改或替换。

**注意**

设计中的任何 block abstraction 都会自动标记 dont\_touch; block abstraction 中的 cell 也会标记为 don't touch。

可用 set\_dont\_touch 指定不与设计层次其余部分一起优化的 subdesign。dont\_touch 属性不会阻止或禁用穿过该 design 的 timing。使用该命令时应注意:

- 在 hierarchical cell 上设置 dont\_touch, 会对该 cell 以下的全部 cell 隐式设置 don't touch。
- 在 library cell 上设置该属性, 会对该 library cell 的全部 instance 隐式设置 don't touch。
- 在 net 上设置该属性, 仅对连接到该 net 的 mapped combinational cell 隐式设置 don't touch。若该 net 仅连接 generic logic, optimization 仍可能移除该 net。
- 在 reference 上设置该属性, 会在后续 design optimization 中对使用该 reference 的全部 cell 隐式设置 don't touch。
- 在 design 上设置该属性, 仅当该 design 作为一个层次级实例化于另一 design 中时才有效; 此时该层次以下的全部 cell 都受 dont\_touch 约束。在 top-level design 上设置无效, 因为 top-level design 不会实例化在其他 design 中。
- 不能用 ungroup 对标有 dont\_touch 的对象执行 ungroup; DC Explorer 的 automatic ungrouping 也不会影响 don't-touch object。

**注意**

对 synthetic part cell (例如从 HDL description 读入的许多 cell) 以及连接有 unmapped cell 的 net, dont\_touch 属性会被忽略。Compile 期间, 对于连接到 unmapped cell (generic logic) 的 don't-touch net, 工具会发出警告。

要移除 dont\_touch 属性, 使用 remove\_attribute, 或用 set\_dont\_touch 将其设为 false。

**参见**

第十一章「使用层次化模型」。

### 10.12.2 修复多端口网络

#### Fixing Multiple-Port Nets

用 set\_fix\_multiple\_port\_nets 向连接到多个端口的网络插入缓冲器/反相器、复制逻辑或重布线。图 10.11 所示多端口网络包括: 穿越网络 (输入端口馈入输出端口); 连接到多个输出端口的网络 (逻辑等价输出)。

这些多端口网络在门级网表中以 assign 语句表示, 可能在后端工具中引起设计规则违例。防止步骤:

1. 将 compile\_advanced\_fixed\_multiple\_port\_nets 设为 true (DC Explorer 默认已为 true), 以启用修复多端口网络与常量驱动端口。
2. 用适当选项对设计设置 fix\_multiple\_port\_nets 属性, 例如:

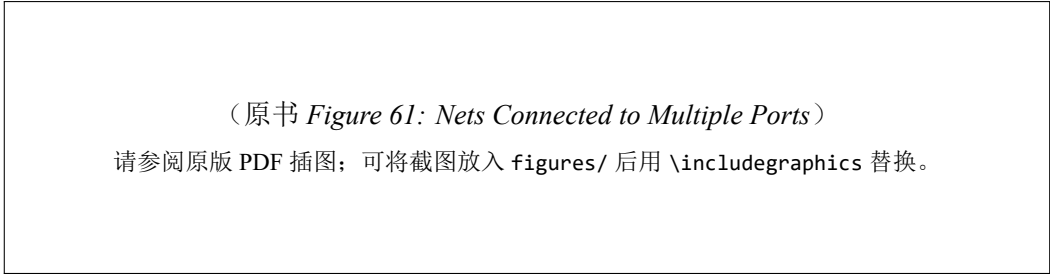


图 10.11: 连接到多个端口的网络

```
de_shell> set_fix_multiple_port_nets -all -buffer_constants
```

用 `-exclude_clock_network` 排除时钟网络；用 `-no_rewire` 禁用缓冲插入前的重布线。

3. 运行 `compile_exploration`。工具按选项通过插入缓冲器、复制逻辑或重布线修复带该属性的设计中的多端口网络。

修复顺序：

- 1. 重布线连接，确保每个此类网络只连到一个层次端口（先重布线以避免不必要缓冲；因默认启用边界优化，也会跨层次重布线）。
- 2. 对未由重布线修复的网络插入缓冲器/反相器。

图 10.12 显示通过跨层次重布线修复设计 B 中穿越的情况。



图 10.12: 通过跨层次重布线修复穿越

表 10.7 汇总 `set_fix_multiple_port_nets` 选项。

表 10.7: `set_fix_multiple_port_nets` 命令选项

| 目的                   | 选项                                  |
|----------------------|-------------------------------------|
| 禁用修复多端口网络            | <code>-default</code>               |
| 插入缓冲器以防止穿越网络         | <code>-feedthroughs</code>          |
| 使每个驱动单元只驱动一个输出端口     | <code>-outputs</code>               |
| 复制逻辑常量使每个常量只驱动一个输出端口 | <code>-constants</code>             |
| 等价于上述三者              | <code>-all</code>                   |
| 对逻辑常量插入缓冲器而非复制逻辑     | <code>-buffer_constants</code>      |
| 禁用修复前的重布线            | <code>-no_rewire</code>             |
| 排除时钟网络中的多端口网络        | <code>-exclude_clock_network</code> |

### 10.12.3 优化多比特寄存器

#### Optimizing Multibit Registers

若逻辑库中有多比特寄存器单元，DC Explorer 可用其替换单比特寄存器，以降低时钟树功耗、时钟树缓冲器数量、时钟树线长与面积。

多比特寄存器

#### Multibit Registers

工具用单比特寄存器表示寄存器位。在 RTL 总线推断流程中，可指示工具将多个寄存器位组织为多比特组件，并目标实现为多比特寄存器（例如八位可实现为一个 8 比特库寄存器或两个 4 比特库寄存器）。

如图 10.13，用多比特单元替换单比特可降低：时钟树功耗与缓冲器数；时钟树总线长；因共享晶体管与优化晶体管级版图而减小的面积。须与布局/布线优化灵活性的损失权衡。

（原书 *Figure 63: Replacing Two Multiple Single-Bit Register Cells With One Multibit Register*）

请参阅原版 PDF 插图；可将截图放入 figures/ 后用 \includegraphics 替换。

图 10.13: 用一个多比特寄存器替换多个单比特寄存器

运行 RTL 总线推断流程

#### Running the RTL Bus Inference Flow

运行该流程时，工具将属于各总线（按 RTL 定义）的寄存器位分到多比特组件；实际替换发生在 compile\_exploration 期间。也可用 create\_multibit 手动分组。

步骤：

1. 设置 hdl\_infer\_multibit 指导如何从 RTL 总线推断，例如：

```
de_shell> set_app_var hdl_infer_multibit default_all
```

2. 读入 RTL。
3. （可选）用 set\_multibit\_options 指定推断模式与多比特寄存器最小宽度。
4. （可选）用 create\_multibit 手动将特定单比特单元/实例分到多比特组件（需已映射设计或 GTECH；实例不能已属于另一多比特组件）：

```
de_shell> create_multibit x_reg[0] x_reg[1] -name my_multi
```

5. （可选）用 remove\_multibit 排除某些单比特寄存器：

```
de_shell> remove_multibit y_reg
```

6. 用 `compile_exploration compile`；指定 `-scan` 时先将单比特替换为扫描单元再做多比特综合：

```
de_shell> compile_exploration -gate_clock -scan
```

7. 用 `report_multibit` 报告全部多比特组件（推断或手动创建）。

Compile 期间的非连续多比特映射

Noncontiguous Multibit Mapping During Compile

`compile_exploration` 可将多比特组件的非连续位映射到多比特寄存器。如图 10.14，将非连续的 `A_reg[0]` 与 `A_reg[2]` 打包到两比特寄存器 `A_reg[2,0]`；`A_reg[1]` 因 `size_only` 保留为单比特。

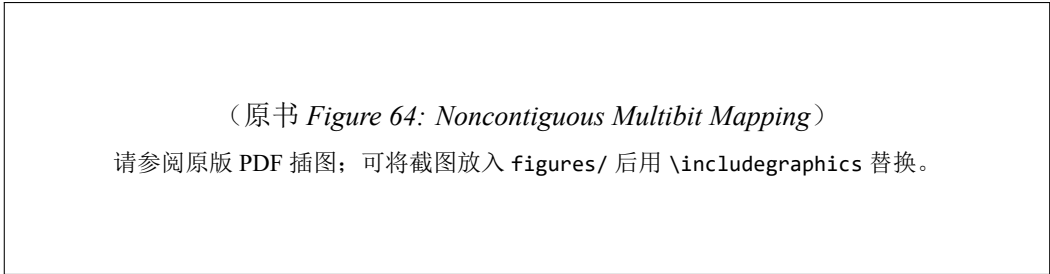


图 10.14: 非连续多比特映射

更多信息见 *Multibit Register Synthesis and Physical Implementation Application Note*。

10.12.4 物理流程中的拥塞报告

Congestion Reporting in the Physical Flow

DC Explorer 在物理流程中执行拥塞驱动布局以降低布线拥塞。将 `de_enable_physical_flow` 设为 `true`（默认 `false`）以启用物理流程。拥塞驱动布局可改善与 Design Compiler topographical 模式的相关性，尤其对复杂 floorplan。

表 10.8 列出物理流程中报告拥塞状态的命令；运行时工具用 Zroute 估计拥塞信息。

表 10.8: 拥塞报告命令

| 命令                                     | 说明          |
|----------------------------------------|-------------|
| <code>report_congestion</code>         | 生成全局布线拥塞报告  |
| <code>report_congestion_options</code> | 报告当前设计的拥塞选项 |
| <code>set_congestion_options</code>    | 设置拥塞优化选项    |
| <code>remove_congestion_options</code> | 移除指定拥塞选项    |

表 10.9 列出用于设置/了解拥塞估计路由选项的命令。

表 10.9: 路由选项命令

| 命令                              | 说明                                          |
|---------------------------------|---------------------------------------------|
| set_route_zrt_common_options    | 设置全局布线、track assignment 与详细布线的公共选项          |
| get_route_zrt_common_options    | 返回指定公共路由选项的值                                |
| report_route_zrt_common_options | 报告全部公共路由选项设置                                |
| set_route_zrt_global_options    | 设置全局布线选项                                    |
| get_route_zrt_global_options    | 返回指定全局路由选项的值                                |
| report_route_zrt_global_options | 报告全部全局路由选项设置                                |
| create_route_guide              | 配合利用率选项 创建带水平/垂直 track 利用率及层设置的 route guide |

注意

DC Explorer 不支持 set\_congestion\_optimization 命令。

若报告显示严重拥塞，可在 GUI 中生成拥塞图进一步分析，识别高拥塞区域并判断设计是否可布。详见 *Design Vision User Guide*。

读写单元扩展数据

Reading and Writing Cell Expansion Data

拥塞优化期间，拥塞区域中的 cell 会扩展以占据更多空间，从而形成较低密度区域。工具生成 cell expansion data，以改善与 IC Compiler 和 IC Compiler II 的 congestion correlation；数据包括每个 expanded cell 的 area、width 和 height。分别使用 read\_cell\_expansion 和 write\_cell\_expansion 读取和写出：

```
de_shell> write_cell_expansion out.exp
```

参见

带 Floorplan 物理约束的优化（上文）。

10.12.5 控制 Path Group 创建

Controlling Path Group Creation

用 create\_auto\_path\_groups 自动为 I/O、宏与集成时钟门控单元创建特定 path group。

改善 QoR

Improving QoR

在 elaboration 之后、优化之前对 create\_auto\_path\_groups 指定 -mode rtl，可指示工具自动创建 path group 以改善 QoR：

```
de_shell> create_auto_path_groups -mode rtl
de_shell> compile_exploration
de_shell> remove_auto_path_groups
```

```
de_shell> report_qor
```

命令在自动创建 path group 前保存用户创建的 path group: RTL 模式下每个层次一个 path group, 多层层次则每层都创建。要用仅含用户指定 path group 的 QoR 报告, 用 `remove_auto_path_groups` 移除工具创建的 path group (保留用 `group_path` 创建的)。

防止 **Path Group** 创建

### Preventing Path Group Creation

用 `-exclude` 列出 IO、macro、ICG 的组合以防止创建特定 path group:

```
de_shell> create_auto_path_groups -mode rtl -exclude {IO macro}
```

在 RTL 模式下, 用 `-min_regs_per_hierarchy` 防止为寄存器数少的层次创建 path group (默认 10); 寄存器数低于该值的层次不创建 path group。

参见

第十三章「报告结果质量」。

# Chapter 11

## 使用层次化模型

### *Using Hierarchical Models*

在大型设计上执行顶层优化时，可使用层次化模型（**hierarchical models**）以减少设计对象数量与内存需求。术语「层次化模型」指 **block abstraction**（块抽象）与物理层次（**physical hierarchy**）。

本章包括以下主题：

- 层次化模型概览
- 关于 Block Abstraction

### 11.1 层次化模型概览

#### Overview of Hierarchical Models

在 DC Explorer 中，层次化模型用于在大型设计的顶层优化期间减少设计对象数量与内存需求。

对于 interface logic model (ILM)，block 的 gate-level netlist 被建模为 partial gate-level netlist，其中仅包含该 block 所需的 interface logic，以及可能由用户手动关联到 interface logic 的逻辑；其余逻辑均被移除。但 DC Explorer 不支持 ILM，而是使用 block abstraction。

Block abstraction 是 ILM 的扩展。ILM 会移除 internal logic，并将其保存到单独的 .ddc 文件；而使用 block abstraction 时，完整 design 与 block abstraction 信息保存在同一个 .ddc 文件中。Interface logic 会被标记；读取 block abstraction 时，工具仅加载 interface logic。DC Explorer 可创建 block abstraction，并支持在 IC Compiler 中创建的 block abstraction；但 IC Compiler 仅支持在 IC Compiler 中创建的 block abstraction。

图 11.1 给出一个 block 及其 hierarchical model，其中保留下列位置之间的逻辑：

- 每条 timing path 的 input port 与第一个 register 之间
- 每条 timing path 的最后一个 register 与 output port 之间

与纯组合 input-port-to-output-port timing path（A 到 X）相关的逻辑也会保留。到被保留 register 的 clock connection 同样保留；register-to-register logic 则被丢弃。

DC Explorer 支持下列使用层次化模型的流程：

(原书 *Figure 65: A Block and Its Hierarchical Model*)  
 请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 11.1: 块及其层次化模型

- 多角多模式 (**Multicorner-multimode**)  
工具自动检测多个 `corner` 与多个 `mode`，并保留各 `scenario` 接口时序路径所涉及的接口逻辑。
- 具有多层物理层次的设计  
工具仅通过 `block abstraction` 支持多层物理层次。  
要进一步了解层次化模型，见：
- 层次化模型中使用的信息
- 多角多模式流程中的层次化模型
- 在 GUI 中查看层次化模型

#### 参见

自底向上 `compile`，见第 十 章「优化」。

### 11.1.1 层次化模型中使用的信息

#### Information Used in Hierarchical Models

默认情况下，层次化模型包含下列网表对象：

- 下列 critical timing path 中的 leaf cell 与 macro cell：input port 到 output port (combinational input-to-output path)、input port 到 edge-triggered register，以及 edge-triggered register 到 output port
- Abstracted clock tree，包括到 interface register 的 clock path、也可能连接到 noninterface register 的 minimum/maximum clock path，以及属于这些 clock path 的 clock tree synthesis exception (CTS exception)

DC Explorer 将 generated clock 视为 clock port，并保留由 generated clock 驱动的 interface register；但不会保留由 generated clock 驱动的 internal register。当 design 包含 generated clock 时，master clock 与 generated clock 之间 timing path 上的逻辑也会保留。

此外还包括：

- Clock-gating circuitry——仅当其由 external port 驱动
- Master clock 与由其派生的任何 generated clock 之间 timing path 上的逻辑
- 所有 net 的 side-load cell  
Side-load cell 是可能影响 interface path timing、但不直接属于 interface logic 的 cell。

**注意**

默认情况下，latch 按 combinational logic 处理。

除网表对象外，层次化模型还存储下列信息：

- Cell 或 pin location 信息
- Parasitic 信息
- Timing constraint
- Internal net 的 pin-to-pin delay
- UPF constraint

图 11.2 给出层次化模型中包含的元素，并突出时钟树：深色阴影元素位于时钟路径上；浅色阴影为 side-load cell；未阴影元素不包含在模型中。

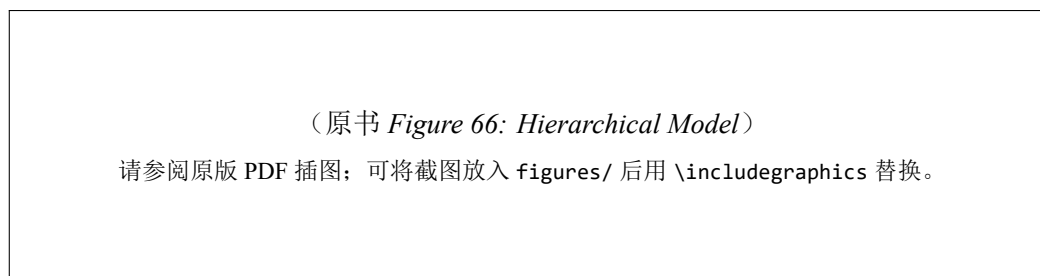


图 11.2: 层次化模型

### 11.1.2 多角多模式流程中的层次化模型

#### Hierarchical Models in a Multicorner-Multimode Flow

可将 MCMM constraint 施加到 hierarchical model，并在 top-level design 中使用这些 constraint。对使用层次化模型建模的块、并配合多角多模式 scenario，适用下列要求：

- 对每个 top-level MCMM scenario，design 中使用的每个 block 都必须存在同名 scenario。若名称不匹配，使用 `select_block_scenario` 将 block 中的 scenario 映射到 top-level design。Block 可以包含 top level 未使用的额外 scenario。
- 具有 MCMM scenario 的 top-level design 可以与没有 MCMM scenario 的 block 一起使用；无需执行 `select_block_scenario`。工具自动对 top level 定义的每个 active scenario 使用相同的 block-level data，例如 timing 和 power data。
- 默认情况下，没有 MCMM scenario 的 top-level design 只能使用同样没有 MCMM scenario 的 block。要在 non-MCMM top-level design 中使用 MCMM block，必须用 `select_block_scenario` 指定 scenario mapping。
- 对每个 TLUPlus file，block 按指定 temperature（属于 operating condition）存储 extraction data。为使 top-level parasitic extraction 获得准确结果，top level 的 TLUPlus file 和 temperature corner 应与 block-level implementation 使用的 TLUPlus 和 temperature corner 匹配。不过，top level 可以使用额外的 TLUPlus 和 temperature，但会损失精度。

**注意**

DC Explorer 支持创建多个 scenario，但仅在 current scenario 中执行 optimization；工具不执行 scenario reduction。

### 11.1.3 在 GUI 中查看层次化模型

#### Viewing Hierarchical Models in the GUI

在 DC Explorer GUI 中，层次化模型实例显示为一级物理层次：带填充图案的矩形或折线形（rectilinear）形状。View Settings 面板提供对 block abstraction 显示的额外控制。

- 要显示层次化模型，选择 **Cell > ILM/Block Abstraction** 可见性选项。
- 要显示层次化模型引脚，选择 **Pin** 可见性选项。
- 要展开层次化模型以显示其中的叶单元、引脚、网络与宏，在“Level”框中输入或选择正整数。
- 要将显示重置为默认未展开视图，在“Level”框中输入或选择 0。

更多信息另见 *Design Vision User Guide* 中的“Examining Physical Hierarchy Blocks”主题。

## 11.2 关于 Block Abstraction

### About Block Abstractions

使用 block abstraction 的 top-level flow 允许优化 top-level logic。

Block abstraction 是 interface logic model (ILM) 的扩展。ILM 会移除 internal logic，并将其保存到单独的 .ddc 文件；而使用 block abstraction 时，完整 design 与 block abstraction 信息保存在同一个 .ddc 文件中。在 top level 将 design 作为 block abstraction 读入时，工具仅从该 .ddc 文件加载 interface logic。

**注意**

DC Explorer 不支持 ILM，应改用 block abstraction。

图 11.3 说明 ILM 与 block abstraction 的差异，并展示 Design Compiler topographical 如何支持使用 block abstraction 的顶层优化。在 DC Explorer 中不能对 block abstraction 进行优化。

(原书 *Figure 67: ILM and Block Abstraction Comparison*)  
请参阅原版 PDF 插图；可将截图放入 figures/ 后用 \includegraphics 替换。

图 11.3: ILM 与 Block Abstraction 对比

要了解层次化流程以及 block abstraction 的限制，见：

- Block Abstraction 层次化流程
- 限制

### 11.2.1 Block Abstraction 层次化流程

#### Block Abstraction Hierarchical Flow

DC Explorer 可创建 block abstraction，并支持在 DC Explorer、Design Compiler 或 IC Compiler 中创建的 block abstraction。但 IC Compiler 仅支持在 IC Compiler 中创建的 block abstraction。

要运行 block abstraction 层次化流程，请按下列主题顺序完成任务：

1. 创建并保存 Block Abstraction
2. 设置顶层设计选项
3. 控制为 Block Abstraction 加载的逻辑范围
4. 加载 Block Abstraction
5. 检查 Block Abstraction 就绪性
6. 执行顶层综合

要在顶层综合期间忽略完全位于 block abstraction 内的时序路径，见「忽略 Block Abstraction 内的时序路径」。

更多信息另见 *IC Compiler Implementation User Guide*。

#### 参见

限制，见下文「限制」。

### 11.2.2 创建并保存 Block Abstraction

#### Creating and Saving Block Abstractions

创建并保存 block abstraction 时，它们存储在单个包含完整设计网表的 .ddc 文件中。您无需维护两个 .ddc（一个用于接口逻辑、一个用于完整网表）。

要为当前设计生成 block abstraction，请在综合结束时使用 `create_block_abstraction` 命令。该命令将 block abstraction 信息添加到内存中的设计。

要保存 block abstraction，必须在创建后立即使用 `write_file` 命令：

```
de_shell> create_block_abstraction
de_shell> write_file -hierarchy -format ddc \
  -output ${DESIGN_NAME}.mapped.ddc
```

该 .ddc 文件同时包含完整设计与 DC Explorer block abstraction。

要用 `set_top_implementation_options` 指定将链接到 block abstraction 并与顶层设计集成的块引用列表，见「设置顶层设计选项」。

#### 参见

Block Abstraction 层次化流程（上文）。

### 11.2.3 设置顶层设计选项

#### Setting Top-Level Design Options

要在顶层使用 block abstraction, 对 `set_top_implementation_options` 使用 `-block_references` 选项, 指定要链接到 block abstraction 的块引用列表。对 DC Explorer 与 IC Compiler 的 block abstraction 均需如此设置。

请在将含 block abstraction 的 .ddc 读入内存之前使用 `set_top_implementation_options`; 否则会加载完整块网表。链接时, 工具仅将接口逻辑加载到顶层。

使用 IC Compiler block abstraction 时, 请在 DC Explorer 中运行任何 link 命令之前使用 `set_top_implementation_options`。链接顶层设计时, 工具会从 Milkyway 参考库列表自动读取 IC Compiler block abstraction。

下例为 blk1 与 blk2 块设置顶层实现选项:

```
de_shell> set_top_implementation_options -block_references {blk1 blk2}
```

可使用多条 `set_top_implementation_options` 命令为顶层全部 block abstraction 指定选项。例 11.1 与例 11.2 中的命令设置等价。

```
de_shell> set_top_implementation_options -block_references {blk1 blk2}
```

代码 11.1: 在一条命令中为 Block Abstraction 指定选项

```
de_shell> set_top_implementation_options -block_references {blk1}
de_shell> set_top_implementation_options -block_references {blk2}
```

代码 11.2: 用多条命令为 Block Abstraction 指定选项

要验证 `set_top_implementation_options` 设置的选项并确认 block abstraction 正确加载, 使用 `report_top_implementation_options` 命令。

要重置顶层实现选项, 对 `set_top_implementation_options` 使用 `-reset` 选项。这将为该块加载完整设计网表, 而非加载 block abstraction。

下例重置 blk2 的顶层实现选项。此例中加载 blk1 的 block abstraction, 并加载 blk2 的完整网表:

```
de_shell> set_top_implementation_options -block_references {blk1 blk2}
de_shell> set_top_implementation_options -block_references {blk2} -reset
```

#### 参见

Block Abstraction 层次化流程 (上文)。

### 11.2.4 控制为 Block Abstraction 加载的逻辑范围

#### Controlling the Extent of Logic Loaded for Block Abstractions

在顶层将块作为 block abstraction 加载时, 可通过 `set_top_implementation_options` 的 `-load_logic` 选项控制加载完整接口 (full interface) 或紧凑接口 (compact interface)。

选择 compact interface 时, 仅保留 min/max 和 rise/fall interface logic。对于 port 扇出到大量 boundary register 的 block, 使用 compact interface 可改善 runtime 与 memory usage。用 `create_block_abstraction` 创建 block abstraction 时, 工具会同时识别 full interface logic 与 compact interface logic。

用下列选项控制加载范围：

- **-load\_logic full\_interface:** 加载 full interface（默认）。下列逻辑标记为 interface logic：
  - 从输入端口到寄存器或输出端口的全部逻辑
  - 从寄存器或输入端口到输出端口的全部逻辑
- **-load\_logic compact\_interface:** 加载 compact interface，仅包含每个 input/output port 的 timing-critical path 逻辑。下列逻辑标记为 interface logic：
  - 属于从输入端口到寄存器或输出端口的 min/max rise/fall 逻辑
  - 属于从寄存器或输入端口到输出端口的 min/max rise/fall 逻辑

无论加载完整接口还是紧凑接口，下列时钟电路均标记为接口逻辑：

- 从主时钟到生成时钟路径上的任何逻辑
- 驱动接口寄存器的时钟树（含时钟树中的任何逻辑）
- 从时钟端口出发的最长与最短时钟路径
- 所有 non-ideal 且未因 DRC 而 disabled 的 net 的 side-load cell

无论加载完整接口还是紧凑接口，工具在 block abstraction 中保留下列 IEEE 1801 Unified Power Format (UPF) 对象：

- 所有 retention 寄存器（无论是否属于接口逻辑）
- 属于接口逻辑的电平移位单元与隔离单元
- 所保留 retention 寄存器、隔离单元与电平移位单元的控制逻辑

当工具移除 UPF netlist object 时，会移除或更新关联的 UPF constraint，并确保 block abstraction 中存储的 UPF strategy 与 UPF netlist object 一致。

要将 block abstraction 中的 UPF 对象传播到顶层，运行 `propagate_constraints -power_supply_data` 命令。

#### 参见

Block Abstraction 层次化流程（上文）。

## 11.2.5 加载 Block Abstraction

### Loading Block Abstractions

在使用 `set_top_implementation_options` 之后，使用 `read_ddc` 命令加载将链接到顶层设计的 DC Explorer block abstraction：

```
de_shell> read_ddc {DesignA.mapped.ddc DesignB.mapped.ddc}
```

读入块 .ddc 时，下列消息表示正在加载 DC Explorer block abstraction：

```
Information: Reading block abstraction for design 'blk1' and its
hierarchy. (DDC-27)
```

确保您的 Milkyway 参考库包含 IC Compiler block abstraction 设计。链接顶层设计时，IC Compiler block abstraction 会从 Milkyway 参考库列表自动加载。

链接顶层设计时，下列消息表示正在加载 IC Compiler block abstraction:

```
Information: Auto loading blk2.CEL (version 1) from Milkyway library
blk2.mw ...(ILM-102)
```

#### 参见

Block Abstraction 层次化流程（上文）。

## 11.2.6 检查 Block Abstraction 就绪性

### Checking Block Abstraction Readiness

要确定已加载的 block abstraction 是否已为顶层综合就绪，可执行下列任务：

#### 报告 Block Abstraction

##### Reporting Block Abstractions

要报告链接到当前设计的 block abstraction，使用 `report_block_abstraction` 命令。对 IC Compiler 创建的 block abstraction，该命令还会报告 UPF 信息。

#### 查询 Block Abstraction

##### Querying Block Abstractions

要用 `get_cells` 创建 block abstraction 集合：

```
de_shell> get_cells -hierarchical -filter is_block_abstraction==true
{blk1 blk2}
```

#### 检查 Block Abstraction

##### Checking Block Abstractions

要在优化前检查 block abstraction 就绪性，使用 `check_block_abstraction` 命令。该命令检查链接到顶层设计的块，确保其已为顶层综合就绪。

该命令检查下列内容：

- MCMC block 中的 scenario consistency
  - Block 与 cell 上是否存在 dont\_touch 设置
- 所有对象都必须具有 dont\_touch 属性，以确保不会被修改。

也可使用 `compile_exploration -check_only` 验证设计与库是否具备 compile\_exploration 成功运行所需的全部数据。

#### 参见

Block Abstraction 层次化流程（上文）。

## 11.2.7 执行顶层综合

### Performing Top-Level Synthesis

在 block abstraction 流程中执行顶层综合之前，应完成下列任务：

- 创建 block abstraction
- 指定将其链接到顶层设计的设置
- 运行报告以检查顶层实现设置
- 运行报告以获取 block abstraction 详情
- 检查 block abstraction 就绪性
- （可选）忽略完全位于 block abstraction 内的时序路径

下列脚本给出顶层综合流程的一部分。顶层设计包含 DesignA 与 DesignB 块及其对应的 block abstraction (DesignA.mapped.ddc 与 DesignB.mapped.ddc)：

```
set BLOCK_ABSTRACTION_DESIGNS "DesignA DesignB"
set_top_implementation_options -block_references \
    ${BLOCK_ABSTRACTION_DESIGNS}
... # Read in the top-level RTL
read_ddc {DesignA.mapped.ddc DesignB.mapped.ddc}
current_design top
link
report_block_abstraction
get_cells -hierarchical -filter is_block_abstraction==true
report_top_implementation_options
... # Read in constraints
check_block_abstraction
compile_exploration -check_only
compile_exploration
```

#### 参见

- Block Abstraction 层次化流程（上文）
- 忽略 Block Abstraction 内的时序路径（下文）
- 第十章「优化」中的自底向上 compile

## 11.2.8 忽略 Block Abstraction 内的时序路径

### Ignoring Timing Paths Within Block Abstractions

Block abstraction 可包含完全位于块内的寄存器到寄存器路径。这些路径源于 block abstraction 中由输入到寄存器与寄存器到输出路径共享的组合逻辑。

这些路径在块级优化期间已优化。要忽略这些路径，将 timing\_ignore\_paths\_within\_block\_abstraction 变量设为 true（默认值为 false）。例如：

```
de_shell> set_app_var timing_ignore_paths_within_block_abstraction true
```

该变量设置不影响始于 block abstraction 内、穿出后再返回该 block abstraction 的路径。

参见

Block Abstraction 层次化流程（上文）。

### 11.2.9 限制

#### Limitations

Block abstraction 具有下列限制：

- IC Compiler 不支持在 DC Explorer 中创建的 block abstraction。
- 在 DC Explorer 中不能修改或优化 block abstraction。

参见

Block Abstraction 层次化流程（上文）。

# Chapter 12

## 使用 GUI

### *Working With the GUI*

Design Vision 是 Synopsys 逻辑综合环境的图形用户界面（GUI）。它提供在通用技术（GTECH）级与门级查看并分析设计的分析工具，并具备 DC Explorer 工具的全部综合能力。

GUI 提供多种可视化与分析功能，可用于可视化设计数据并分析结果。GUI 窗口为最常用的综合功能提供菜单命令与对话框；此外，也可在 GUI 或 shell 的命令行中输入任意 `de_shell` 命令。

在使用 GUI 分析与排查设计之前，应先熟悉其操作及所提供的各类工具。下列各节给出首次使用 GUI 前需要了解的一般与具体信息：

- 功能与收益
- 使用 GUI 窗口
- 执行 Floorplan 探索
- 使用 IC Compiler II 进行 Floorplan 探索
- 分析 RTL
- RTL 交叉探测（RTL Cross-Probing）

## 12.1 功能与收益

### Features and Benefits

Design Vision GUI 提供下列功能：

- 面向 Synopsys 逻辑综合的窗口与菜单驱动界面
- 分析可视化能力，包括：
  - hierarchy browser，用于在设计层次中导航并探索设计结构，包括查看 area attribute
  - 直方图（histogram），用于可视化各类度量的趋势，例如 slack 与电容
  - 时序状态摘要与路径检查器（path inspector），用于详细时序路径分析

- 路径原理图 (path schematic)，用于直观检查时序路径、扇入/扇出逻辑以及网连接
  - 原理图视图 (schematic view)，用于直观检查高层次与低层次设计连接
  - 属性查看器 (properties viewer)，用于查看对象属性并编辑属性值
  - 版图视图 (layout view)，用于查看并分析物理设计中的 floorplan 元素
  - 报告功能，可将报告中的对象与图形视图关联
  - 集成于 GUI 的 command-line interface，支持以 Design Compiler tool command language (dctcl) 的全部命令编写脚本
  - 基于 HTML 的文档浏览器，用于查看 man 页与联机帮助页
- 借助 Design Vision GUI 的功能，您可以：
- 执行 DC Explorer 综合与报告
  - 探索设计结构
  - 用 Design Vision 图形分析可视化整体时序
  - 对正在综合的 block 执行 timing analysis
  - 对时序路径与相连逻辑做详细可视化分析
  - 在物理设计版图中直观检查并调试时序路径及某些 floorplan 约束

## 12.2 使用 GUI 窗口

### Using GUI Windows

GUI 提供灵活的工作环境，包含多个用于不同任务的窗口。这些 GUI 窗口在您的 X Windows 环境中彼此独立运行，但共享内存中的同一设计、相同的当前时序信息，以及工具中的全局选择数据。

Design Vision 提供下列 GUI 窗口：

- Design Vision 主窗口：从 `de_shell` 打开 GUI 时自动出现。  
默认在 Design Vision 主窗口中显示 hierarchy browser (logic hierarchy view) 与 command console (console panel)。
- 版图窗口 (layout window)：用于可视化设计的物理方面。  
要打开版图窗口，选择 **Window > New Layout Window**。

每个 GUI 窗口顶部包含标题栏、菜单栏与若干工具栏，底部为状态栏。工具栏与状态栏之间的工作区显示视图窗口与面板。视图窗口提供设计信息的图形或文本视图；面板提供用于设置选项或执行常用任务的交互工具。

可用 Linux 桌面的窗口管理工具调整、移动、最小化或最大化各 GUI 窗口。更多细节见 Design Vision Help。也可打开同一 GUI 窗口的多个实例，以便并排比较视图或同一视图中的不同设计信息。窗口在整个会话中按序编号。

图 12.1 给出读入设计并为顶层设计打开设计原理图后的 Design Vision 主窗口示例。

可在状态栏中显示菜单命令所执行操作的简要说明：将指针悬停在命令名上即可。要确定工具栏按钮的功能，将指针悬停在按钮上；ToolTip 显示按钮名称，状态栏显示其用途的简要说明。

(原书 *Figure 68: Design Vision Main Window*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 12.1: Design Vision 主窗口

对于也可通过工具栏按钮或键盘快捷键使用的菜单命令，菜单会显示这些替代方式的示意。部分常用菜单命令也出现在各视图的弹出菜单上；要使用弹出菜单，右键单击并选择命令。

可分别配置每个窗口的工具栏、视图窗口与面板。工具栏始终附着于窗口边缘；可将个别面板附着于窗口边缘，或使其在工作区内/外浮动。多数面板与特定类型的视图关联，并对活动视图操作；例外是控制台，它包含命令行及其自身视图。

可通过调整视图窗口与面板大小来改善工作环境；可最小化个别视图窗口，或最大化视图窗口以填满工作区；还可在工作区内平铺排列已打开的视图窗口。

要在下次会话中以相同配置恢复 GUI，必须在关闭 GUI 或退出 DC Explorer 工具之前保存当前窗口配置。窗口配置包括主窗口与版图窗口的大小与位置，以及各工具栏与面板的可见性与位置。

关于 GUI 窗口的更多信息，见下列主题：

- 输入命令并查看结果
- 查看 Man 页
- 保存窗口或视图的图像
- 在 GUI 中获取帮助

### 12.2.1 输入命令并查看结果

#### Entering Commands and Viewing Results

打开 GUI 时，控制台面板默认停靠在状态栏上方。可用控制台：

- 在控制台命令行输入 `de_shell` 命令
- 点击命令行上方的选项卡，查看 `session transcript (log view)` 或 `command history list (history view)`
- 复制、编辑或重用命令，并搜索、选择与保存命令或消息

可在控制台命令行输入任意 `de_shell` 命令。发出命令后（按 `Return`，或点击命令行左侧的 `prompt` 按钮），`console` 与 `de_shell` 都会回显命令输出，包括处理消息以及所有 `warning` 或 `error message`。例如，若输入 `get_selection` 命令，控制台 `log` 视图会显示所有所选对象的名称列表。

可用方向键在控制台命令行上显示、编辑并重新发出命令：左右移动插入点，或上下滚动命令栈。

`log` 视图显示会话信息转录，包括自打开 GUI 以来运行的命令及其输出与消息。可用这些信息：

- 在执行功能后检查工具状态
- 排查遇到的问题

- 在转录中搜索以往功能相关信息
- 将转录、所选文本或仅错误与警告消息保存为文本文件

可像在 Linux shell 中一样，在 log 视图中复制文本并粘贴到命令行：用鼠标左键选择文本，用中键粘贴。

history 视图列出当前 GUI 会话中使用过的菜单、对话框与 `de_shell` 命令。可对该列表：

- 查看已使用的命令
- 查找并重新发出已使用的命令
- 保存列表供以后使用

可在 history 视图中选择命令，并在命令行上编辑或重新发出。

更多信息亦见 *Design Vision User Guide* 中下列主题：

- Viewing the Session Log
- Viewing the Command History
- Entering Commands in the Console

## 12.2.2 查看 Man 页

### Viewing Man Pages

GUI 提供基于 HTML 的浏览器窗口，用于查看命令、变量与错误消息的 man 页。可用它：

- 显示 man 页
- 在当前查看的 man 页中搜索文本
- 打印当前查看的 man 页
- 在已查看的 man 页之间前后浏览

要打开 man 页查看器：

- 选择 **Help > Man Pages**。

主页显示不同 man 页类别（commands、variables、messages）的链接列表。点击要查看的类别链接，再点击要查看的 man 页标题链接。

也可在控制台命令行使用 `man` 或 `gui_show_man_page` 命令，或在 GUI 已打开时在 `de_shell` 中使用 `gui_show_man_page` 命令，在 man 页查看器中显示 man 页。

更多信息亦见 *Design Vision User Guide* 中的“Viewing Man Pages”主题。

## 12.2.3 保存窗口或视图的图像

### Saving an Image of a Window or View

可将 GUI 窗口或视图窗口的图像保存为图像文件。图像显示屏幕上所见窗口外观，但不含窗口边框或标题栏。例如，若保存活动原理图视图的图像，图像显示当前缩放级别与平移位置下原理图的可见部分。

要保存活动 GUI 窗口或视图窗口的图像：

1. 选择 **View > Save Screenshot As**。

出现 Save Screenshot As 对话框。

2. 指定文件名与图像格式。

图像格式可为 PNG（默认）、BMP、JPEG 或 XPM。要指定非默认格式，使用该格式的文件扩展名。

- 3.（可选）若要保存活动视图窗口而非当前工作的 GUI 窗口的图像，选中“Grab screenshot of active view only”选项。

4. 点击 **Save**。

也可使用 `gui_write_window_image` 命令，指定文件名、图像格式与窗口名。窗口实例名出现在窗口标题栏与 Window 菜单上。例如，要将活动原理图视图保存为名为 `schem_1.jpg` 的 JPEG 图像，输入：

```
de_shell> gui_write_window_image -file schem_1.jpg
```

不能保存对话框或其他 GUI 元素（如工具栏或面板）的图像。

更多信息亦见 *Design Vision User Guide* 中的“Saving an Image of a Window”主题。

## 12.2.4 在 GUI 中获取帮助

### Getting Help in the GUI

Design Vision Help 可在 DC Explorer GUI 中使用。关于 GUI 的多数所需信息可在面向 DC Explorer 的 Design Vision Help 中找到。可从 Design Vision 主窗口或版图窗口的 Help 菜单访问 Design Vision Help。

要查看面向 DC Explorer 的 Design Vision Help：

- 选择 **Help > Online Help**。

要查看关于版图窗口中交互工具用法的 Design Vision Help 提示：

- 选择 **Help > Layout Help**。

帮助系统包含解释可执行任务细节的主题。例如，若用户指南中某步骤需要帮助，可在 Design Vision Help 中找到所需信息。

Design Vision Help 中的信息按下列类别组织：

- Feature 主题：概述 Design Vision 窗口组件与工具
- How-to 主题：提供完成综合与分析任务的步骤
- Reference 主题：解释视图、工具栏按钮、菜单命令与对话框选项

Design Vision Help 是基于浏览器的 HTML Help 系统，设计用于在 Firefox 与 Mozilla Web 浏览器中查看。

#### 注意

在 Design Vision 内访问 Design Vision Help 之前，Web 浏览器可执行文件必须列在 Linux 的 `$PATH` 变量中。

Design Vision Help 大量使用 JavaScript 与级联样式表（CSS）。若浏览器显示 Design Vision Help 时

遇到问题, 请打开浏览器首选项, 确保已启用 JavaScript 与样式表, 且安全首选项未阻止 JavaScript。

关于 GUI 的其他信息来源包括 `gui_*` 命令 man 页与 SolvNetPlus 知识库。  
更多信息亦见 *Design Vision User Guide* 中的 “Using This Help System” 主题。

## 12.3 执行 Floorplan 探索

### Performing Floorplan Exploration

可在综合环境内, 通过 IC Compiler 版图窗口中的 IC Compiler 设计规划工具, 执行 floorplan 分析、创建与修改等 floorplanning 任务。DC Explorer 中的 floorplan 探索与 IC Compiler 版图窗口之间的接口是透明的, 可在 Design Vision 与 IC Compiler 版图窗口之间无缝切换。

要了解如何执行 floorplanning 任务, 见:

- 启用 Floorplan 探索
- 启用数据流分析
- 使用 Floorplan 探索 GUI
- 保存 Floorplan 或丢弃更新
- 退出会话
- 在 Floorplan 变更后执行综合
- 在批处理模式下使用 Floorplan 探索
- 黑盒、物理层次与块抽象

#### 12.3.1 启用 Floorplan 探索

##### Enabling Floorplan Exploration

DC Explorer 允许您使用 IC Compiler 版图窗口中的 IC Compiler 设计规划工具执行 floorplan 探索。DC Explorer 中的 floorplan 探索与 IC Compiler 版图窗口之间的接口是透明的。可在下列设计阶段执行 floorplan 探索:

- 综合之前

要执行 floorplan 探索, 确保变量 `enable_presynthesis_floorplanning` 设为 true (默认)。

##### 注意

必须有逻辑库以及至少包含 AND 门、反相器与缓冲器的物理库, 才能在 IC Compiler 中启动设计规划。

- 综合之后

在执行 floorplan 探索之前, 先在 DC Explorer 中读入 RTL 文件、指定物理与逻辑库、定义 Synopsys 设计约束、指定 floorplan (若有), 并综合设计。综合后, 可评估结果质量, 并按需使用 floorplan 探索创建新 floorplan 或改进现有 floorplan。

要设置并启动 floorplan 探索会话：

1. (可选) 若要在综合前调用 floorplan 探索，将变量 `enable_presynthesis_floorplanning` 设为 `true`。
2. 在 DC Explorer GUI 中，选择 **Floorplan > Set Design Planning Options**。  
出现 Set Design Planning Options 对话框，如图 12.2 所示。
3. 在 Work Directory 框中，输入工作目录名及其相对于当前目录的路径。  
此处存放 floorplanning 脚本、floorplan 及其他必要文件。
4. 在 ICC Executable 框中，输入 IC Compiler 可执行文件的名称与位置。  
默认情况下，DC Explorer 使用 `$path` 变量指定的 `icc_shell` 可执行文件。
5. 在 File Name Prefix 框中，输入要用于任何生成文件（如 floorplanning 脚本与 floorplan 文件）文件名的前缀。  
默认文件前缀命名风格为 `%s_%d`，其中 `%s` 为设计名，`%d` 为进程 ID。
6. 点击 **OK**。
7. 选择 **Floorplan > Start Design Planning**。  
出现警告，显示下列消息：

Warning: Starting floorplan exploration will **close** the GUI. Do you want to **continue**?

8. 点击 **OK**。  
DC Explorer 在 IC Compiler layout window 中启动 floorplan exploration，并使用简化的 floorplanning menu。

(原书 Figure 69: Set Design Planning Options Dialog Box)  
请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 12.2: Set Design Planning Options 对话框

作为步骤 1 至 8 的替代，可使用下列 `de_shell` 命令：

```
de_shell> set_icc_dp_options -work_dir "./dcg_link" \
  -icc_executable "*/bin/icc_shell" \
  -file_name_prefix "%s_%d" -keep_files
de_shell> start_icc_dp
```

更多信息亦见：

- *Design Vision User Guide* 中的 “Using Floorplan Exploration Tools” 主题
- IC Compiler Help

## 参见

在批处理模式下使用 Floorplan 探索

### 12.3.2 启用数据流分析

#### Enabling Data Flow Analysis

Floorplan 探索允许您使用 IC Compiler 设计规划中的数据流分析工具，分析并改进物理设计中的数据流。可用数据流分析器窗口执行下列数据流分析：

- **logic connectivity analysis:** 从 logic data flow 视角查看 macro 连接
- **高级飞线分析 (advanced flyline analysis):** 通过检查宏之间以及宏与 I/O 单元之间的连接来识别数据流
- **交互式宏编辑:** 修改宏布局、创建宏组与宏阵列

图 12.3 给出从 DC Explorer 到 IC Compiler 设计规划的早期 floorplanning 概览。

(原书 *Figure 70: Early Floorplanning Using Data Flow Analysis*)  
请参阅原版 PDF 插图；可将截图放入 figures/ 后用 \includegraphics 替换。

图 12.3: 使用数据流分析的早期 Floorplanning

要启动数据流分析会话：

1. 按「启用 Floorplan 探索」中的步骤调用 IC Compiler 版图窗口。  
出现 IC Compiler 版图窗口。
2. 选择 **Floorplan > Analyze Logic Connectivity**。  
出现数据流分析器窗口。
3. 使用 IC Compiler floorplanning 工具执行 floorplanning 任务，包括创建、修改与分析 floorplan。
4. 选择 **Floorplan > Update DC Floorplan**，将 floorplanning 更新保存回 DC Explorer。
5. (可选) 按「使用 Floorplan 物理约束执行优化」中的步骤执行优化。

更多信息亦见：

- IC Compiler Help 中的 “Analyzing the Data Flow for Macro Placement” 主题
- *IC Compiler Implementation User Guide*

## 参见

- 启用 Floorplan 探索
- 使用 Floorplan 物理约束执行优化

- SolvNetPlus 文章 24224, “Design Compiler Graphical Floorplan Exploration Application Note”

### 12.3.3 使用 Floorplan 探索 GUI

#### Using the Floorplan Exploration GUI

DC Explorer 中的 floorplan 探索使用 IC Compiler 版图窗口。默认情况下, 启用 floorplan 探索时, DC Explorer 会配置 IC Compiler 版图窗口, 使其仅包含执行 DC Explorer floorplanning 所需的菜单与菜单命令。图 12.4 给出简化 floorplanning 菜单结构。

(原书 *Figure 71: IC Compiler Simplified Layout Window Features*)  
请参阅原版 PDF 插图; 可将截图放入 figures/ 后用 \includegraphics 替换。

图 12.4: IC Compiler 简化版图窗口功能

若您是有经验的 IC Compiler 用户, 可使用完整的设计规划菜单集。

可通过选择 **File > Task > Design Planning** 访问完整的 IC Compiler 设计规划菜单集, 如图 12.5 所示。要从完整设计规划菜单结构切换回简化菜单结构, 选择 **File > Task > DCG-DP Link**。

(原书 *Figure 72: Switching Between Simplified and Full Design Planning Menu Structures*)

请参阅原版 PDF 插图; 可将截图放入 figures/ 后用 \includegraphics 替换。

图 12.5: 在简化与完整设计规划菜单结构之间切换

使用完整 IC Compiler 设计规划菜单集完成的任何工作, 在切换回简化 floorplanning 菜单后仍然可用。

#### 创建与编辑 Floorplan

**Creating and Editing Floorplans** 无论使用简化的 DC Explorer floorplanning 菜单还是完整的 IC Compiler 设计规划菜单, IC Compiler 版图窗口都提供可用于创建或修改 floorplan 的交互工具与命令。您可以:

- 使用编辑工具移动、调整大小、复制、对齐、分布、展开、分割、重塑或移除对象
- 使用编辑命令旋转、对齐、分布、展开或扩展对象, 或更改单元方向

最常用的编辑操作提供快捷键; 每个编辑工具都提供联机帮助页。

此外，可用 Floorplan 菜单上的命令创建物理对象，例如 placement blockage、bound 与 routing keepout。创建对象时，可指定其坐标，或在版图视图中绘制该对象。

更多信息亦见：

- IC Compiler Help
- *IC Compiler Design Planning User Guide*

### 12.3.4 保存 Floorplan 或丢弃更新

#### Saving the Floorplan or Discarding Updates

可随时从 IC Compiler floorplanning 会话中保存或丢弃 DC Explorer 中的 floorplan 更新。要保存或丢弃 floorplan，执行下列步骤：

1. 选择 **Floorplan > Update DC Floorplan**。

出现 Update DC Floorplan 对话框，如图 12.6 所示。

#### 注意

“Update DC Floorplan”命令仅在面向 DC Explorer 的简化 floorplanning 菜单结构中可用。关于从完整设计规划菜单集切换到简化 floorplanning 菜单结构的信息，见「使用 Floorplan 探索 GUI」。

2. 选择适当选项。
3. 点击 **OK**。

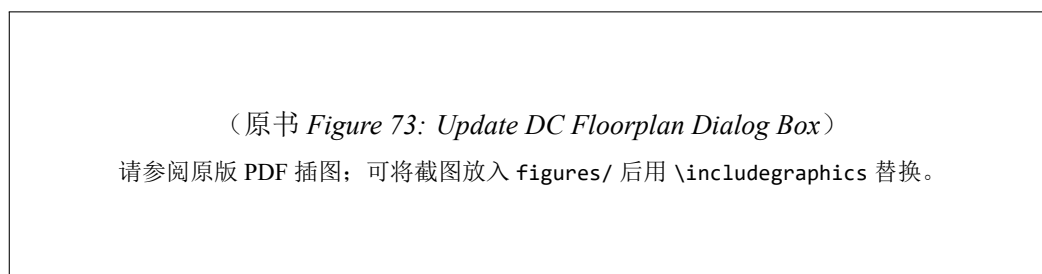


图 12.6: Update DC Floorplan 对话框

也可将 floorplan 保存在下列文件中：

- Tcl 脚本文件  
选择 **Floorplan > Write Floorplan**。也可在工具提示符下使用 `write_floorplan` 命令。
- DEF 文件  
选择 **File > Export > Write DEF**。也可在工具提示符下使用 `write_def` 命令。

### 12.3.5 退出会话

#### Exiting the Session

要退出 floorplan 探索会话并保存 floorplan 供综合使用，或丢弃 floorplan，执行下列步骤：

1. 选择 **File > Exit**。  
出现 Exit IC Compiler 对话框，如图 12.7 所示。
2. 选择适当选项。
3. 点击 **OK**。

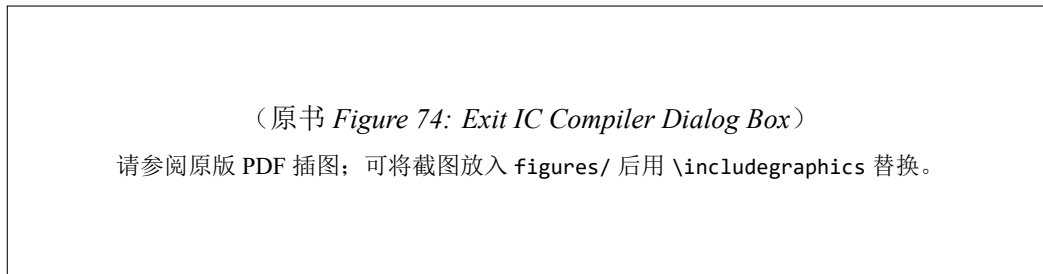


图 12.7: Exit IC Compiler 对话框

若更新了 floorplan，工具会关闭原来的 Design Vision 版图窗口。要查看 floorplan 变更，请打开新的 Design Vision 版图窗口。

### 12.3.6 在 Floorplan 变更后执行综合

#### Performing Synthesis After Floorplan Changes

要在 floorplan 变更后准备完整综合：

1. 必须运行 `write_floorplan` 命令，将 floorplan 保存为 Tcl 脚本文件。例如：

```
de_shell> write_floorplan -all Design.fp
```

对使用新的或已修改的 floorplan 重新综合设计，建议使用 `-all` 选项。

2. 在 DC Explorer 中用 `read_floorplan` 命令读入 floorplan 以供综合。
3. 读入 RTL 文件、SDC 文件与新的 floorplan 文件后，用 `compile_exploration` 命令执行完整综合。

#### 参见

使用 Floorplan 物理约束

### 12.3.7 在批处理模式下使用 Floorplan 探索

#### Using Floorplan Exploration in Batch Mode

也可在批处理模式下使用 floorplan 探索。批处理模式允许您在命令行用脚本创建并修改 floorplan。

在 DC Explorer 中用下列命令以批处理模式执行 floorplan 探索：

1. 使用 `set_icc_dp_options` 命令指定 IC Compiler 可执行文件及其他设置要求。例如：

```
de_shell> set_icc_dp_options -icc_executable icc_image
```

2. (可选) 使用 `report_icc_dp_options` 命令报告 `set_icc_dp_options` 所指定的选项。  
若未对 `set_icc_dp_options` 指定任何选项, `report_icc_dp_options` 报告默认设置。
3. 使用 `start_icc_dp` 命令启动 floorplan 探索会话。  
`start_icc_dp` 使用 `set_icc_dp_options` 指定的 IC Compiler 可执行文件。要在批处理模式下 `source` 脚本, 对 `start_icc_dp` 使用 `-f` 选项。例如:

```
de_shell> start_icc_dp -f ../../scripts/script_file_name.tcl
```

### 参见

启用 Floorplan 探索

## 12.3.8 黑盒、物理层次与块抽象

### Black Boxes, Physical Hierarchies, and Block Abstractions

DC Explorer 支持对包含黑盒 (black box)、物理层次 (physical hierarchy) 与块抽象 (block abstraction) 的网表进行 floorplan 探索。

- 黑盒

若设计含有黑盒, 无需执行额外步骤或修改 floorplan 探索流程。在 DC Explorer 中定义或创建的黑盒会自动传输到 IC Compiler 设计规划。使用 IC Compiler 工具, 可在 floorplanning 会话中编辑含黑盒的 floorplan, 并将修改传回 DC Explorer。

- 物理层次与块抽象

在 floorplan 探索期间, DC Explorer 会自动将物理层次与块抽象传输到 IC Compiler 设计规划。物理层次与 block abstraction 作为 macro 传输, block 内包含 quick timing model。

在将物理层次与块抽象传输到 IC Compiler 工具以进行设计规划时, DC Explorer 会自动在实例上将 `dct_physical_hier` 属性设为 `true`。

### 查询层次块与引脚连接

**Querying Hierarchical Blocks and Pin Connections** 要返回所有 DC Explorer 层次块 (包括物理层次与块抽象), 对 `get_cells` 命令使用 `dct_physical_hier` 属性。将物理层次与块抽象传回 DC Explorer 时会予以保留。

例如:

```
icc_shell> get_cells -hierarchical "dct_physical_hier==true"
{GPRs Multiplier}
```

要查询连接到层次块引脚的网, 对 `get_nets` 使用 `-boundary_type lower` 或 `-boundary_type upper` 选项, 分别针对层次块内部或外部的网。要同时查询两者, 指定 `both` 关键字。若不指定 `-boundary_type` 选项, 命令返回层次块外部的网。使用该选项时, 必须用 `-of_objects` 选项指定引脚。

例如, 下列两条命令返回连接到 u2 层次块外部 in 引脚的 topnet 网:

```
de_shell> get_nets -of_objects u2/in
{topnet}
de_shell> get_nets -of_objects u2/in -boundary_type upper
{topnet}
```

下列命令返回连接到 u2 层次块内部 in 引脚的 u2/in 网：

```
de_shell> get_nets -of_objects u2/in -boundary_type lower  
{u2/in}
```

## 12.4 使用 IC Compiler II 进行 Floorplan 探索

### Floorplan Exploration With IC Compiler II

DC Explorer 允许您使用 IC Compiler II 版图窗口中的 IC Compiler II 设计规划工具执行 floorplan 探索。综合后，可评估结果质量，并在 IC Compiler II 中使用 floorplan 探索创建 floorplan 或改进现有 floorplan。DC Explorer 中的 floorplan 探索与 IC Compiler II 版图窗口之间的接口是透明的。

下列主题说明如何用 IC Compiler II 执行 floorplan 探索：

- Floorplan 探索的先决条件
- 运行 Floorplan 探索流程
- 向 IC Compiler II 传输数据
- 以 ASCII 格式保存以供 IC Compiler II 使用

参见

执行 Floorplan 探索

#### 12.4.1 Floorplan 探索的先决条件

##### Prerequisites for Floorplan Exploration

在 IC Compiler II 中执行 floorplan 探索之前，必须完成下列任务：

- 综合设计  
不能对未综合（即 unmapped）的设计运行 IC Compiler II。
- 生成 **IC Compiler II** 参考库  
由于参考库不会由 DC Explorer 自动转换，必须使用 IC Compiler II Library Manager 生成库。
- 为多电压设计生成 **RTL UPF** 文件  
为处理设计的功耗意图，运行 RTL UPF 探索流程以生成 RTL UPF 文件。

参见

UPF 探索

#### 12.4.2 运行 Floorplan 探索流程

##### Running the Floorplan Exploration Flow

下图给出从 DC Explorer 到 IC Compiler II 设计规划的 floorplan 探索概览：

下列各节说明先决条件，以及以交互模式或以 `de_shell` 脚本批处理模式运行的 floorplan 探索流程。

(原书 *Figure 75: From DC Explorer to Floorplan Exploration in IC Compiler II*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 12.8: 从 DC Explorer 到 IC Compiler II 中的 Floorplan 探索

## 先决条件

**Prerequisites** 在 floorplan 探索之前，必须完成下列任务：

1. 用 `compile_exploration` 命令综合设计。
2. 用 `set_icc2_options` 命令指定 IC Compiler II 参考库。例如：

```
de_shell> set_icc2_options -icc2_executable ../icc2_shell \
-ref_libs "lib1.ndm lib2.ndm"
```

## 交互模式

**In Interactive Mode** 用 `start_icc2` 命令运行 floorplan 探索流程需要 IC Compiler II 许可证。

要启动 floorplan 探索流程：

1. 在 DC Explorer 中，用 `start_icc2` 命令启动 IC Compiler II 设计规划会话。  
`start_icc2` 调用 `icc2_shell>` 提示符并打开 IC Compiler II 版图窗口。
2. 在 IC Compiler II 中执行 floorplan 探索。  
可用 IC Compiler II 版图窗口中的设计规划工具创建 floorplan 或编辑现有 floorplan。
3. 在 `icc2_shell` 中用 `update_dc_floorplan` 命令生成更新后的 floorplan。  
IC Compiler II 中的 floorplan 数据会传输到 DC Explorer，并 `reset placement`。若不执行此步骤，在 `icc2_shell` 中所做的 floorplan 变更不会更新到 `de_shell`，并将丢失。
4. (可选) 要在步骤 3 中保留 floorplan 文件，对 `set_icc2_options` 指定 `-keep_files` 选项。
5. 在 `icc2_shell` 中输入 `exit` 命令以结束设计规划会话。  
返回 `de_shell>` 提示符。

### 注意

运行 `start_icc2` 命令时，若指定了 `TMPDIR` Linux 环境变量，则调用 IC Compiler II 所需的文件会写入 `$TMPDIR/dcg_unique_string` 工作目录。要更改默认目录，对 `set_icc2_options` 指定 `-work_dir` 选项。

## 批处理模式

**In Batch Mode** 要从批处理脚本运行 floorplan 探索，在启动 DC Explorer 时对 `start_icc2` 指定 `-f script_file` 选项。在批处理模式下，`start_icc2` 调用 `icc2_shell` 并执行指定脚本，但不打开 GUI。

下列脚本给出与 IC Compiler II 配合的一般 floorplan 探索流程示例：

```
#Perform synthesis using compile_exploration
compile_exploration -scan -gate_clock

#Set the IC Compiler II options
set_icc2_options -icc2_executable ../icc2_shell \
-ref_libs "lib1.ndm lib2.ndm"

#Launch IC Compiler II session in batch mode
start_icc2 -f my_icc2.tcl
```

示例批处理脚本 my\_icc2.tcl：

```
#This example script displaces the macro I_RAM by 10 microns
set_placement_status placed [get_cells I_RAM]
move_objects -delta {10 10} [get_cells I_RAM]
set_placement_status fixed [get_cells I_RAM]

#Update the floorplan changes before exit
update_dc_floorplan
exit
```

### 12.4.3 向 IC Compiler II 传输数据

#### Data Transfer to IC Compiler II

在 DC Explorer 内启动 IC Compiler II 时，设计数据会以 IC Compiler II 兼容格式自动传输到 IC Compiler II。表 12.1 列出从 DC Explorer 传输到 IC Compiler II 的设计与环境设置。

## 12.5 分析 RTL

### Analyzing the RTL

可在早期设计阶段通过分析设计的整体时序与逻辑概况，来调试时序与拥塞问题。DC Explorer GUI 使您能生成此类概况，以识别最差时序路径、逻辑级数最多的路径、不可行路径、多周期路径等。要分析设计，可生成直方图以显示落在各 slack 范围或逻辑级范围（称为 bin）中的路径分布。选定某个 bin 后，可将路径载入路径数据表，并在 RTL 浏览器中对其单元做交叉探测。

#### 注意

要使用此分析功能，设计必须已 mapped 或已 compile。

在 DC Explorer GUI 中，可执行下列 RTL 分析：

- 生成路径 **slack** 直方图
  1. 选择 **AnalyzeRTL > Analyze Path Slack**。
  2. 选择 **Single Clock Paths**、**Cross Clock Paths**、**Infeasible Paths** 或 **Multicycle Paths**。

表 12.1: 传输到 IC Compiler II 的设计与环境设置

| 传输  | 设计与环境设置                                                                                                                                                                                                                                                                                                                                            |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 已传输 | <ul style="list-style-type: none"><li>• Verilog 网表</li><li>• IC Compiler II 兼容格式的时序约束</li><li>• DEF 文件与 IC Compiler II 兼容格式 Tcl floorplan 文件中的 floorplan 约束</li><li>• 属性，例如 dont_touch、size_only、dont_use</li><li>• Technology 文件</li><li>• TLUPlus 文件</li><li>• UPF 文件</li><li>• 与层相关的约束，例如网布线层约束与首选布线方向</li><li>• 忽略层（ignored layer）设置</li></ul> |
| 未传输 | <ul style="list-style-type: none"><li>• hierarchical model: DC Explorer 与 IC Compiler 的 block abstraction 和 physical hierarchy 在 icc2_shell 中被视为 unresolved reference</li><li>• 物理黑盒的估算：这些物理单元由 estimate_fp_black_boxes 命令创建</li><li>• 相对布局约束：这些约束在 DC Explorer 中创建</li></ul>                                                                      |

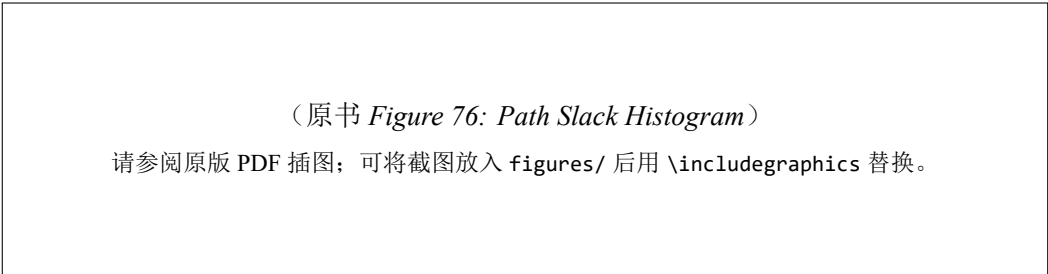


图 12.9: 路径 Slack 直方图

可创建违及时序约束的同时钟路径、跨时钟路径、不可行路径或多周期路径的直方图。默认情况下，**path slack histogram** 使用 10 个 bin 显示所有路径的 slack 值分布，每个 bin 的最小区间宽度为 100 ps。各 bin 表示路径数量（ $y$  轴）相对于其 slack 值（ $x$  轴）。

选定某个 bin 后，可选择该 bin 中的路径、在新的路径 slack 或逻辑级直方图中显示这些路径，或将路径载入路径数据表。

- 生成逻辑级直方图

1. 选择 **AnalyzeRTL > Analyze Logic Levels**。
2. 选择 **Single Clock Paths**、**Cross Clock Paths**、**Infeasible Paths** 或 **Multicycle Paths**。

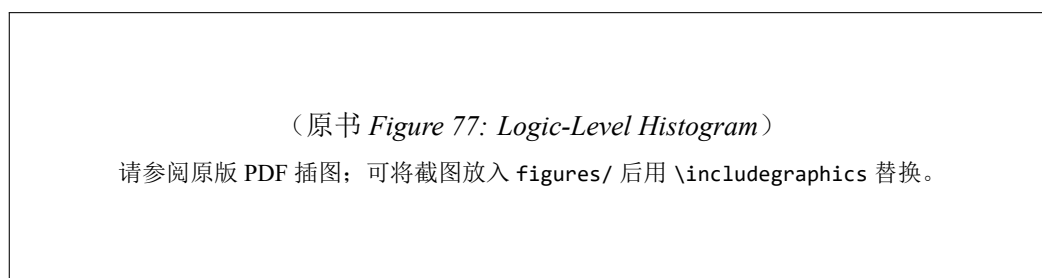


图 12.10: 逻辑级直方图

可创建违及时序约束的同时钟路径、跨时钟路径、不可行路径或多周期路径的直方图。默认情况下，**logic-level histogram** 使用 8 至 16 个 bin 显示所有路径的 logic level 分布。各 bin 表示路径数量（ $y$  轴）相对于逻辑级数（ $x$  轴）。

选定某个 bin 后，可选择该 bin 中的路径、在新的路径 slack 或逻辑级直方图中显示这些路径，或将路径载入路径数据表。

- 生成路径数据表

1. 在路径 slack 或逻辑级直方图中选择一个或多个 bin。  
所选 bin 的颜色变为黄色。
2. 右键单击并选择 **Create Data Table**。

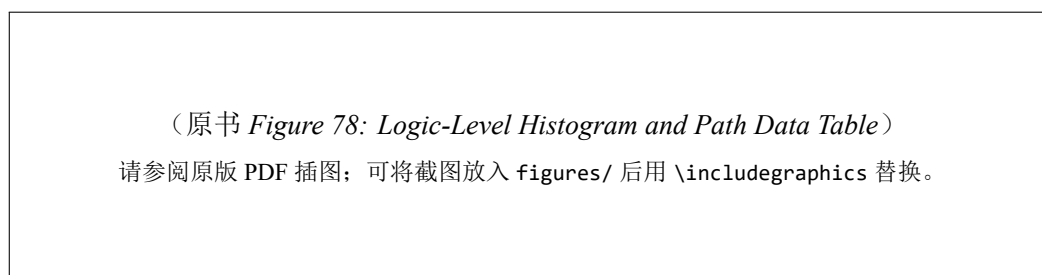


图 12.11: 逻辑级直方图与路径数据表

- 对单元做交叉探测

1. 选择一个或多个单元、引脚或端口。  
可在原理图或版图视图、层次浏览器、对象列表中选择对象，或通过 **Select** 菜单上的命令选择。

2. 用下列任一方式做交叉探测：

- 选择 **AnalyzeRTL > Cross Probe to Source**。
- 右键单击所选对象并选择 **Cross Probe to Source**。

工具定位单元来源的 RTL 文件，并在 RTL 浏览器中打开这些文件。

- 对时序路径做交叉探测

1. 在路径数据表中选择一条或多条路径。

2. 用下列任一方式做交叉探测：

- 选择 **AnalyzeRTL > Cross Probe to Source**。
- 右键单击所选路径并选择 **Cross Probe to Source**。

工具为每条路径上的所有单元创建集合，定位单元来源的 RTL 文件，并在新的 RTL 浏览器窗口中打开这些文件。

(原书 *Figure 79: Cross-Probing Between Path Data Table and RTL Browser*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 12.12: 路径数据表与 RTL 浏览器之间的交叉探测

可在下列各设计视图与 RTL 浏览器之间做交叉探测：

- 原理图视图
- 路径 slack 或逻辑级直方图（经路径数据表）
- 路径检查器（path inspector）
- 拥塞图（congestion map）

更多信息亦见 *Design Vision User Guide* 中的“Analyzing RTL”主题。

## 12.6 RTL 交叉探测

### RTL Cross-Probing

DC Explorer 在 GUI 中提供 cross-probing 功能，使您能在 RTL browser 中审阅并验证设计对象与 RTL 源码之间的对应关系。可从某一视图（如原理图、单元列表、版图或时序视图）交叉探测设计中的单元、引脚或时序路径，并检查对应的 RTL 文件。这使您能识别可修改以改善时序、拥塞或 datapath 提取的代码行。

#### 注意

要使用交叉探测能力，必须有已 elaborate 或已综合的设计。

可从路径检查器选择关键路径，工具会交叉探测 RTL 以帮助调试最差情况时序路径。拥塞图与 RTL 浏览器之间的交叉探测有助于识别导致拥塞的 RTL 代码。也可在路径数据表与 RTL 浏览器之间做交叉探测；路径数据表显示您在路径 slack 或逻辑级直方图检查时序路径的详细信息。

工具不支持下列交叉探测能力：

- 从各种设计视图到 UPF 文件
- 选择网作为交叉探测对象

图 12.13 给出 Design Vision 版图视图与 RTL 之间的交叉探测：

(原书 *Figure 80: Cross-Probing Between the Layout View and the RTL Browser*)

请参阅原版 PDF 插图；可将截图放入 figures/ 后用 \includegraphics 替换。

图 12.13: 版图视图与 RTL 浏览器之间的交叉探测

### 12.6.1 重复 RTL 交叉探测

#### Repeating RTL Cross-Probing

每次工具为交叉探测读取 RTL 时，会将 RTL 行号与文件路径存入数据库。若在 cross-probing 后将 RTL 文件移到其他位置，数据库中保存的文件路径将失效；再次执行同一 cross-probing 序列时，工具无法定位该文件并会发出错误消息。

要重复交叉探测序列或引用先前的交叉探测数据，必须先查明曾被交叉探测的文件状态。可通过运行下列命令获取状态：

- `report_cross_probing_files`

该命令报告已交叉探测文件的状态，包括 OK、Missing、Modified 与 No Permissions。

- `update_cross_probing_files`

该命令更新曾被交叉探测的文件位置，并列出文件的先前路径与当前路径。若文件已被修改，该命令不会更新已交叉探测的文件。

更多信息亦见 *Design Vision User Guide* 中下列主题：

- Analyzing RTL
- Viewing Path Data Tables

#### 参见

- 分析 RTL
- 使用命令进行 RTL 交叉探测

# Chapter 13

## 分析并解决设计问题

### *Analyzing and Resolving Design Problems*

使用 DC Explorer 生成的报告来分析并调试设计。可在 `compile` 设计之前与之后生成报告。`compile` 前生成报告，以检查属性、约束与设计规则是否设置正确；`compile` 后生成报告，以分析结果并调试设计。

要了解如何分析并解决设计问题，见：

- 修复由不受支持的 **Technology** 文件属性引起的错误
- 比较 DC Explorer 与 IC Compiler 环境
- 检查设计一致性
- 检查未映射单元
- 指定输出端口方向不正确时的严重级别
- 分析设计问题
- 分析面积
- 分析时序
- 生成结果质量（QoR）
- 调试带有 `dont_touch` 属性的单元与网
- 使用命令进行 RTL 交叉探测
- 在批处理模式下报告逻辑级

### 13.1 修复由不受支持的 **Technology** 文件属性引起的错误

#### **Fixing Errors Caused by Unsupported Technology File Attributes**

有时 IC Compiler 会新增支持某些 DC Explorer 尚未支持的 `technology` 文件属性。在这些情况下，使用同一 `technology` 文件时，DC Explorer 会发出 TFCHK-009 错误消息，而 IC Compiler 不会。

若读入 `technology` 文件时出现 TFCHK-009 错误,请检查属性拼写并确保拼写正确。若仍有 TFCHK-009 错误,可用下列两种方法之一消除这些错误。在使用任一方法之前,请确认可以安全移除或忽略这些新属性而不影响工具功能。多数情况下,新属性与新的布线规则相关,不应影响 DC Explorer 的功能。

- 从 `technology` 文件的指定节中移除这些新属性。
- 若新属性可安全忽略,可将变量 `ignore_tf_error` 设为 `true`,使工具忽略不受支持的属性。这会使工具忽略所有 TFCHK-009 错误,但错误仍会写入日志文件。

## 13.2 比较 DC Explorer 与 IC Compiler 环境

### Comparing DC Explorer and IC Compiler Environments

有时 DC Explorer 与 IC Compiler 的环境设置不同,这些差异可能导致相关性问题。为帮助修复 DC Explorer 与 IC Compiler 之间的相关性问题,可在 Linux shell 中使用 `consistency_checker` 命令比较各自环境。

在比较两个环境之前,需要确定每个工具的环境。为此,使用 `write_environment` 命令,如示例 13.1 与示例 13.2 所示。在 DC Explorer 中,先运行 `write_environment`,再运行 `compile_exploration`;在 IC Compiler 中,先运行 `write_environment`,再运行 `place_opt`。

```
de_shell> write_environment -consistency -output DCE.env
```

代码 13.1: 在 DC Explorer 中使用 `write_environment`

```
icc_shell> write_environment -consistency -output ICC.env
```

代码 13.2: 在 IC Compiler 中使用 `write_environment`

获得各工具环境后,用 `consistency_checker` 命令比较环境,如示例 13.3 所示。

```
%consistency_checker -file1 DCE.env -file2 ICC.env \  
-folder temp-html html_report | tee cc.log
```

代码 13.3: 比较 DC Explorer 与 IC Compiler 环境

生成的 HTML 输出报告列出不匹配的命令与变量。在示例 13.3 中,HTML 输出报告写入 `./html_report` 目录。要访问该报告,在任意 Web 浏览器中打开 `./html_report/index.html`。为减少相关性问题,请在继续优化之前修复这些不匹配。

除 HTML 输出报告外,工具在运行 `consistency_checker` 时还会向 shell 打印摘要报告。可如示例 13.3 所示使用 Linux `tee` 命令将该报告保存到文件;本例中保存为 `./cc.log`。

#### 注意

`./temp` 目录存放工具执行 `consistency_checker` 时使用的中间文件。命令完成后可移除 `./temp` 目录。

### 13.2.1 压缩格式的环境文件

#### Environment Files in Compressed Format

要生成 gzip 格式的工具环境输出，对 `write_environment` 指定 `-compress` 选项。可将输出文件再 `source` 回 IC Compiler 工具。对于一致性检查，必须先解压用 `-compress` 生成的输出文件；在使用 `consistency_checker` 之前，用 Linux `gunzip` 命令解压。

下列示例生成压缩文件 `DCE.tcl.gz`，并在 Linux shell 中解压以供一致性检查：

```
de_shell> write_environment -consistency -output DCE.tcl -compress
Terminals:                145
Hard Macros:               38
Site Rows:                 351
Bounds:                   3
Placement Blockages:      22
Route Guides:              3
Preroutes:                20

%gunzip DCE.tcl.gz
```

#### 参见

SolvNetPlus 文章 31000, “Using the Consistency Checker to Automatically Compare Environment Settings Between Design Compiler and IC Compiler”

## 13.3 检查设计一致性

### Checking for Design Consistency

设计不包含下列错误时，称为一致：未连接端口、常量值端口、无输入或输出引脚的 `cell`、`cell` 与其 `reference` 不匹配、`multiple-driver net`、`connection-class violation` 或递归层次定义。

DC Explorer 会对所有被 `compile` 的设计运行 `check_design -summary` 命令。也可显式运行该命令以验证设计一致性。命令按如下方式报告警告与错误消息列表：

- 若发现问题是 DC Explorer 无法解决的，则报告错误。例如，递归层次（设计引用自身）是错误。不能对 `check_design` 报告有错误的设计进行 `compile`。
- 若发现问题表明设计损坏或设计错误，但严重程度不足以导致 `compile` 失败，则报告警告。

报告默认以标准输出格式显示。报告开头为摘要节，显示检查类型与已检测到的违例数量；随后列出错误与警告消息。

可用 `check_design` 的 `-html_file_name` 选项指定 HTML 格式报告。指定该选项与文件名时，DC Explorer 在当前目录创建 HTML 文件。

图 13.1 给出 HTML 格式 `check_design` 报告示例。报告将信息组织为 `Input/Outputs`、`Cells`、`Designs` 等节；每节列出检查类型与违例数量。若有违例，数量为可点击的 HTML 链接以显示更多信息。图中报告显示五个黑盒违例；点击数字 5 可展开报告以显示五个高亮违例。

默认情况下，在 `compile` 或执行 `check_design` 期间，若三态总线由非三态驱动器驱动，DC Explorer 发出警告消息。可将变量 `check_design_allow_non_tri_drivers_on_tri_bus` 设为 `false`，使 DC



图 13.1: HTML 格式 `check_design` 报告示例

Explorer 改为显示错误消息；默认值为 `true`。当变量为 `false` 时，`compile_exploration` 在报告错误后停止；但 `check_design` 在报告错误后继续运行。

可对 `check_design` 使用选项，按检查类型过滤消息，例如与端口、网、单元相关的警告，以及与多重实例化设计相关的信息消息等。例如，若指定 `-multiple_designs` 选项，报告显示多重实例化设计列表以及实例名与相关属性（如 `dont_touch`、`black_box`、`ungroup`）。默认情况下，与多重实例化设计相关的消息被抑制。

### 13.4 检查未映射单元

#### Checking for Unmapped Cells

可通过以下方式对 `check_design` 指定 `-unmapped` 选项，报告设计中的未映射单元：

- `-unmapped` 选项  
指定该选项时，`check_design` 报告未映射单元总数，并显示带有未映射单元名及其设计名的 LINT-61 警告消息。默认情况下这些消息被抑制。

例如：

```
de_shell> check_design -unmapped
*****
check_design summary:
...
*****
Name                               Total
-----
Cells                               2
Cell is unmapped (LINT-61)         2
-----
Warning: In design 'unit_2', cell 'A_out_reg' is unmapped (LINT-61)
Warning: In design 'unit_0', cell 'I_2' is unmapped (LINT-61)
```

- 带有 LINT-61 警告消息 ID 的 `-unmapped` 选项  
当指定 `-unmapped {LINT-61}` 时，如下例所示，命令显示关于未映射单元的格式化详细报告：

```
de_shell> check_design -unmapped {LINT-61}
*****
Report:      LINT-61
Total Count: 2
```

```
Severity:    Warning
Message:     In design '', cell '' is unmapped
...
*****
Design      Cell
-----
unit_2      A_out_reg
unit_0      I_2
-----
```

### 参见

检查设计一致性

## 13.5 指定输出端口方向不正确时的严重级别

### Specifying the Severity Level for Incorrect Output Port Directions

当输出端口方向未按设计中的指定使用时，可指定 `compile` 与 `check_design` 命令发出的严重级别。为此，使用变量 `check_design_allow_inconsistent_output_port`。

将该变量设为下列值之一时：

- `true`（默认）  
`compile` 与 `check_design` 发出 LINT-68 警告消息且不退出。
- `false`  
`compile` 以 LINT-70 错误消息退出，而 `check_design` 发出相同错误消息但不退出。

## 13.6 分析设计问题

### Analyzing Design Problems

可使用表 13.1 所列命令分析设计问题。

## 13.7 分析面积

### Analyzing Area

使用 `report_area` 命令显示当前设计或实例的面积信息与统计。该命令报告组合面积、非组合面积与总面积。若已设置当前实例，则对该实例所属设计生成报告；否则对当前设计生成报告。使用 `-hierarchy` 选项可报告层次中各单元所用面积。

示例 13.4 给出 `report_area` 命令生成的报告：

```
de_shell >report_area
...
Library(s) Used:
test_lib (File: test_lib.db)
```

表 13.1: 分析设计对象的命令

| 对象         | 命令                                                                                                                           | 说明                                           |
|------------|------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| Design     | report_area<br>report_design<br>report_design_mismatch<br>report_hierarchy<br>report_missing_constraints<br>report_resources | 报告设计大小与对象计数；设计特性；设计不匹配；设计层次；当前设计中缺失的约束；资源实现。 |
| Instances  | report_cell                                                                                                                  | 显示关于实例的信息。                                   |
| References | report_reference                                                                                                             | 显示关于引用的信息。                                   |
| Pins       | report_transitive_fanin<br>report_transitive_fanout                                                                          | 报告扇入逻辑；报告扇出逻辑。                               |
| Ports      | report_port<br>report_bus<br>report_transitive_fanin<br>report_transitive_fanout                                             | 显示端口信息；总线端口信息；扇入/扇出逻辑。                       |
| Nets       | report_net<br>report_bus<br>report_transitive_fanin<br>report_transitive_fanout                                              | 报告网特性；总线网特性；扇入/扇出逻辑。                         |
| Clocks     | report_clock                                                                                                                 | 显示关于时钟的信息。                                   |

```

Number of ports:                565
Number of nets:                 9654
Number of cells:                8120
Number of combinational cells:  6594
Number of sequential cells:     1526
Number of macros:               0
Number of buf/inv:              1439
Number of references:           163

Combinational area:             562404.432061
Noncombinational area:          6720840.351067
Net Interconnect area:          undefined (Wire load has zero net area)

Total cell area:                 7283244.783128
Total area:                      undefined

```

代码 13.4: report\_area 报告示例

### 13.7.1 将块抽象中单元面积报告为宏单元面积

#### Reporting the Area of Cells in a Block Abstraction as a Macro Cell Area

DC Explorer 将块抽象中单元的面积按标准单元面积计算，这可能导致 `report_area` 报告更高的利用率。

可将变量 `block_abstraction_compute_area_as_macro` 设为 `true`，使工具将块抽象中单元的面积计算并报告为宏单元面积；默认值为 `false`。

## 13.8 分析时序

### Analyzing Timing

使用 `report_timing` 命令为当前设计或当前实例生成时序报告。默认情况下，该命令列出每个 `path group` 中最长的 `max-delay timing path` 的完整路径。DC Explorer 根据控制 `endpoint` 的 `clock` 对路径分组；所有未与 `clock` 关联的路径均属于 `default path group`。也可用 `group_path` 命令创建路径组。

在开始调试时序问题之前，请确保已：

- 定义工作条件（`operating conditions`）
- 指定现实的约束
- 适当预算时序约束
- 正确约束各路径
- 正确描述 `clock skew`

若运行物理流程，可对 `report_timing` 使用 `-physical` 选项，在路径报告中报告引脚位置以及路径中引脚与网的电容负载。当变量 `de_enable_physical_flow` 设为 `true` 时启用物理流程。

生成初始已映射网表后，使用 `report_constraint` 命令检查设计性能。

表 13.2 列出时序分析命令。

表 13.2: 时序分析命令

| 命令                                      | 分析任务说明                          |
|-----------------------------------------|---------------------------------|
| <code>report_design</code>              | 显示工作条件、模式、时序范围、内部输入/输出以及禁用的时序弧。 |
| <code>check_timing</code>               | 检查未约束时序路径与时钟门控逻辑。               |
| <code>report_port</code>                | 显示未约束的输入/输出端口与端口负载。             |
| <code>report_timing_requirements</code> | 显示施加于设计的所有时序例外。                 |
| <code>report_clock</code>               | 检查时钟定义与时钟偏斜信息。                  |
| <code>report_path_group</code>          | 显示设计中所有时序路径组。                   |
| <code>report_timing</code>              | 检查设计时序。                         |
| <code>report_constraint</code>          | 检查设计约束。                         |
| <code>report_delay_calculation</code>   | 报告延时弧计算的细节。                     |

## 13.9 生成结果质量

### Generating Quality of Results

可使用 `create_qor_snapshot`、`query_qor_snapshot` 与 `report_qor` 等报告命令，为设计在当前状态下生成结果质量(QoR)报告。`create_qor_snapshot` 从 `timing`、`design rule`、`area`、`power`、`congestion`、`clock tree synthesis`、`routing` 等方面度量并报告设计质量，并将 QoR 信息存入一组 `snapshot` 文件。随后可用 `query_qor_snapshot` 检索并报告该快照；也可选择性地检索、排序并显示所需信息。`report_qor` 显示当前设计的 QoR 信息与统计。

要了解如何生成结果质量，见：

- 度量结果质量
- 分析结果质量
- 报告结果质量

#### 13.9.1 度量结果质量

##### Measuring Quality of Results

要用一组报告文件度量设计当前状态的 QoR 并存储质量信息，使用 `create_qor_snapshot` 命令。可用不同优化策略或在设计不同阶段捕获 QoR 信息并比较结果质量。例如，可在首次 `compile` 设计后、DFT 插入后，以及增量 `compile` 后再创建快照。

可用命令选项指定分析条件，例如零线负载、每个时序组的最大路径数，以及每个端点的最大路径数。

使用 `create_qor_snapshot` 创建快照时，必须至少用 `-name` 选项指定快照名称。例如：

```
de_shell> create_qor_snapshot -name my_snapshot1
```

报告写入当前工作目录下名为 `snapshot` 的目录中的一组文件；这组文件充当快照报告的数据库。无需访问这些文件，也不得修改它们。随后运行 `query_qor_snapshot` 并用创建快照时相同的名称指定 `-name`（本例中为 `my_snapshot1`）时，该命令以文本或 HTML 格式显示结果。

#### 13.9.2 分析结果质量

##### Analyzing Quality of Results

`query_qor_snapshot` 读取先前由 `create_qor_snapshot` 生成的 QoR 报告，并通过收集每条路径的信息来分析结果。可用各种搜索查询所收集的信息，并以文本或 HTML 格式显示结果。HTML 报告使您能快速找到有大扇出或转换退化等问题的路径。

命令选项允许过滤与排序搜索结果，并以文本或 HTML 格式显示；可指定要显示的数据列。例如，可针对指定数值范围（如 `WNS`、`logic level` 等）生成 `set_false_path constraint`。该命令不对设计执行额外分析，仅处理已保存在快照报告文件中的报告信息；因此其执行远快于 `create_qor_snapshot`。

可用 `-filters` 选项过滤路径。例如，下列命令报告 `WNS` 小于 0 的所有路径：

```
de_shell> query_qor_snapshot -name my_snapshot -filters "-wns ,0"
```

下列示例报告满足任一指定过滤要求的所有路径：

```
de_shell> query_qor_snapshot -name my_snapshot \
  -filters "-wns -4.0-fanout 20"
```

若不指定 `-filters` 选项，`query_qor_snapshot` 自动应用一组默认过滤器。最大条件下的默认过滤设置如下例所示：

```
de_shell> query_qor_snapshot -name my_snapshot \
  -filters "-wns ,0 -zero_path ,0 -fanout 40 -logic_levels 50"
```

此示例分析名为 `my_snapshot1` 的快照，并报告最差负 slack 在 `-4.0` 与 `-3.0` 时间单位之间的路径。

```
de_shell> query_qor_snapshot -name my_snapshot1 \
  -filters "-wns -4.0,-3.0"
```

代码 13.5: 报告具有最差负 Slack 的路径

此示例报告 slack 差于 `-1.0` 时间单位且扇出大于 40 的路径。

```
de_shell> query_qor_snapshot -name my_snapshot2 \
  -filters "-wns ,-1.0 -fanout 40"
```

代码 13.6: 按 Slack 与扇出报告路径

此示例读取名为 `max_logic_level` 的快照，并指定 `-filters` 以报告逻辑级在 2 与 14 之间的所有路径。

```
de_shell> query_qor_snapshot -name max_logic_level \
  -filters "-logic_level 2,15"
```

代码 13.7: 报告指定逻辑级之间的所有路径

注意：逻辑级为 2 的路径包含在报告中，但逻辑级为 15 的路径被排除。

有时 `worst-timing path` 从某一 `hierarchical block` 开始并在另一 `hierarchical block` 结束；识别这些路径并对其分组可能很困难。`-hierarchy` 选项启用 `top-level view`，并自动查找跨越 `hierarchy boundary` 的潜在 `path grouping`，以便更高效地分析问题路径。使用 `-hierarchy` 时，必须指定 `-to`、`-from` 或 `-through` 选项。下列示例查找从模块 A 开始、在模块 B 结束、跨越硬宏的违例路径：

```
de_shell> query_qor_snapshot -name snap_shot3 -hierarchy \
  -from {A/*} -to {B/*}
```

代码 13.8: 报告跨越层次边界的时序违例

`query_qor_snapshot` 自动生成提取时序模型（ETM）与硬宏之间所有违例路径的报告。可应用过滤器，或通过指定 `-to`、`-from` 与 `-through` 展开顶层路径。下列示例查找经过模块 A、B 或 C 的违例路径：

```
de_shell> query_qor_snapshot -name snap_shot4 -hierarchy \
  -through {A B C}
```

代码 13.9: 报告经过指定模块的违例路径

此示例使用 `-incremental` 选项，保留先前分析结果（经过模块 A、B 与 C），并应用关于从模块 A 或 B 开始、在模块 A 或 B 结束的路径的额外查询。

```
de_shell> query_qor_snapshot -hierarchy -incremental \
    -from {A/* B/*} -to {A/* B/*}
```

代码 13.10: 增量报告

此示例报告从模块 C 开始、在模块 B 结束，且最差负 slack 差于  $-3.0\text{ ns}$  或扇出大于 40 的路径。

```
de_shell> query_qor_snapshot -name my_snapshot5 \
    -hierarchy -from top/A/C/* -to top/B/* \
    -filters "-wns , -3.0 -fanout 40"
```

代码 13.11: 按 WNS 与扇出报告经过指定模块的路径

### 13.9.3 报告结果质量

#### Reporting Quality of Results

`report_qor` 命令显示当前设计的结果质量信息及其他统计。它报告时序路径组细节、单元计数与当前设计统计（包括组合、非组合与总面积）。该命令还报告静态功耗、设计规则违例与 compile 时间细节。注意：`report_qor` 不属于 `create_qor_snapshot` 与 `query_qor_snapshot` 命令集。

此示例给出 `report_qor` 生成的报告。在 Cell Count 节中，报告显示设计中的宏数量。要成为宏单元，单元必须是非层次的，并且在其库单元上设置了 `is_macro_cell` 属性。

```
de_shell> report_qor
...
Timing Path Group 'clk'
-----
Levels of Logic:           1.00
Critical Path Length:      0.02
Critical Path Slack:       -0.68
Critical Path Clk Period:  8.00
Total Negative Slack:      -393.61
No. of Violating Paths:    1066.00
Worst Hold Violation:      0.00
Total Hold Violation:      0.00
No. of Hold Violations:    0.00
-----

Cell Count
-----
Hierarchical Cell Count:   7
Hierarchical Port Count:   2360
Leaf Cell Count:           43978
Buf/Inv Cell Count:        6764
CT Buf/Inv Cell Count:     4
Combinational Cell Count:  37212
Sequential Cell Count:     6773
Macro Count:               0
-----
```

```

Area
-----
Combinational Area:   562404.432061
Noncombinational Area:
                        6720840.351067
Net Area:              0.000000
Net XLength           :    2836623.25
Net YLength           :    2555007.75
-----
Cell Area:            7283244.783128
Design Area:          7283244.783128
Net Length           :    5391631.00

```

#### Design Rules

```

-----
Total Number of Nets:      47207
Nets With Violations:      5
Max Trans Violations:      3
-----

```

Hostname: machine

#### Compile CPU Statistics

```

-----
Overall Compile Time:      631.32
Overall Compile Wall Clock Time:  288.11

```

代码 13.12: report\_qor 报告示例

## 13.9.4 通过控制路径组创建改善 QoR

### Improving QoR by Controlling Path Groups Creation

可指示工具自动创建路径组以改善 QoR。为此,在 elaborate 之后、优化之前对 `create_auto_path_groups` 指定 `-mode rtl` 选项。例如:

```

de_shell> create_auto_path_groups -mode rtl
de_shell> compile_exploration
de_shell> remove_auto_path_groups
de_shell> report_qor

```

工具为每个 hierarchy 创建一个 path group; 若设计包含多个 hierarchy level, 则为每一层创建 path group。要用仅包含用户指定路径组的 QoR 报告, 使用 `remove_auto_path_groups` 移除工具创建的路径组。该命令移除所有路径组, 但保留您用 `group_path` 创建的路径组。

#### 参见

创建报告与缺失约束

## 13.10 调试带有 dont\_touch 属性的单元与网

### Debugging Cells and Nets With the dont\_touch Attribute

若 DC Explorer 未优化或移除某个单元或网,原因往往是对象上的 dont\_touch 属性。显式 dont\_touch 属性较易调试;但要确定对象为何具有隐式 dont\_touch 可能较难,尤其当对象上存在多种重叠的隐式 dont\_touch 时。即使移除对象上的 dont\_touch 设置,仍可能看到对象被报告为具有 dont\_touch;这发生在对象因多种原因具有 dont\_touch 的情况。

为帮助调试单元或网中的对象为何被标记为 don't touch, DC Explorer 提供下列各节所述命令。

#### 13.10.1 报告 Don't Touch 单元与网

##### Reporting Don't Touch Cells and Nets

要报告设计中所有 dont\_touch 单元与网以及对每个报告对象适用的 dont\_touch 类型,使用 report\_dont\_touch 命令。对完整设计层次报告 dont\_touch 对象。

非 dont\_touch 的单元或网会被跳过。也可对 report\_dont\_touch 指定单元或网的集合。

下列示例显示特定网的 dont\_touch 类型报告。本例用 report\_dont\_touch 调查 Multiplier/Product[0] 网上的 dont\_touch 类型。该网与其上所有类型的 dont\_touch 一并列出。表图例为所报告的每种 dont\_touch 类型提供详细说明。可能的 dont\_touch 类型很多,但仅列出相关类型。

```
de_shell> report_dont_touch [get_nets Multiplier/Product[0]]
...
Description for Net dont_touch Types:
  inh_parent    - Net inherits dont_touch from parent
  mv_iso        - Prevents buffering between isolation cells and
                  power domain boundary

Object          Class      Types
-----
Multiplier/Product[0]  net      inh_parent, mv_iso
-----
```

如示例所示, Multiplier/Product[0] 网从父单元继承了一个 dont\_touch。该网位于某一层块内部, 其父层次单元上设置了显式 dont\_touch。该网还因多电压综合而具有隐式 dont\_touch。

下列示例显示设计中所有单元的 dont\_touch 报告:

```
de_shell> report_dont_touch -class cell -nosplit
...
Description for Cell dont_touch Types:
  inh_ref        - Cell inherits dont_touch from reference design or
                  library cell
  ph_fixed_placement - Cell with fixed placement

Object          Class      Types
-----
headerfooter5_HDRDID1BWPHVT_R5_C0    cell      ph_fixed_placement
headerfooter6_HDRDID1BWPHVT_R6_C0    cell      ph_fixed_placement
u_des_soft_macro/u_8/u_mem           cell      inh_ref
```

```
-----
Total 3 dont_touch cells
```

```
1
```

要返回当前定义的单元与网 `dont_touch` 类型及其说明的按字母排序列表,使用 `list_dont_touch_types` 命令。在构建 `dont_touch` 对象集合时查找可用类型,该命令很有用。默认情况下,`list_dont_touch_types` 列出 `cell` 与 `net` 对象类的所有 `dont_touch` 类型。

使用 `-class class_name` 选项可将列表限制为 `cell` 或 `net` 对象类; `class_name` 值可为 `cell` 或 `net`。

下列示例给出默认 `list_dont_touch_types` 输出的摘录, 列出网与单元的所有 `dont_touch` 类型:

```
de_shell> list_dont_touch_types
...
*****
Class  Type          Description
-----
net    cts_synthesized Net is synthesized with clock tree synthesis
net    dft_scanbuf     Net connected to input pin of scan compression
                        buffer cell
net    dtn            Net in dont_touch network set by the
                        set_dont_touch_network command
net    dtn_charz      Net in dont_touch network through
                        characterization
net    fp_abutted     Net is floorplan abutted net
net    fp_border      Net is dangling on floorplan border
net    idn            Net in ideal network set by the set_ideal_network
                        command
...
-----
1
```

### 13.10.2 创建 Don't Touch 单元与网的集合

#### Creating a Collection of Don't Touch Cells and Nets

可分别使用 `get_dont_touch_cells` 与 `get_dont_touch_nets` 命令, 创建当前设计中相对于当前实例的 `dont_touch` 单元与网集合。若没有单元或网匹配指定条件, 命令返回空字符串。

可在命令提示符下使用这些命令, 或将其嵌套为另一命令 (如 `query_objects`) 的参数; 也可将命令结果赋给变量。

从命令提示符发出时, 这些命令的行为如同已调用 `query_objects` 以显示集合中的对象。默认最多显示 100 个对象; 可通过设置变量 `collection_result_display_limit` 更改该值。

下列示例查询 `InstDecode` 块下因 `ideal_network` 设置而具有 `dont_touch` 的单元:

```
de_shell> get_dont_touch_cells -type idn InstDecode/*
{InstDecode/reset_UPF_LS InstDecode/U38 InstDecode/U36}
```

下列示例查询名为 `block1` 的块中以 `NET` 开头的多电压类型 `dont_touch` 网:

```
de_shell> get_dont_touch_nets -type mv* block1/NET*
{block1/NET1QNX block1/NET2QNX}
```

## 13.11 使用命令进行 RTL 交叉探测

### RTL Cross-Probing Using Commands

为改善时序、拥塞或 datapath 提取，可在命令行使用 RTL 交叉探测能力检查 RTL 设计，并识别需修改的代码行。

要使用下列 cross-probing 命令，设计必须已 analyze 并 elaborate，或已综合：

- **get\_cross\_probing\_info**

该命令以字符串形式返回指定单元或端口的交叉探测数据，说明单元或端口在何处创建，以及对象来自哪个文件与行号。设计中某些对象可能已合并，因而返回多个源位置。

每个对象的结果用一对花括号分隔各源位置以及各对象。例如，若指定两个对象 obj1 与 obj2，其中 obj1 有一个源位置、obj2 有两个源位置，工具按下列格式返回结果：

```
{ {obj1_file1:obj1_line1} } { {obj2_file1:obj2_line1}
{obj2_file2:obj2_line2} }
```

要防止结果中出现重复的 file:linenumber 组合,对 get\_cross\_probing\_info 使用 -unique\_source 选项。此时工具按下列格式返回结果：

```
obj1_file1:obj1_line1 obj2_file1:obj2_line1 obj2_file2:obj2_line2
```

下列示例显示带 -unique\_source 选项的 get\_cross\_probing\_info 结果：

```
prompt> get_cross_probing_info [get_cells -hierarchical] \
-unique_source
/usr/joe/my_design/m.v:21 /usr/joe/my_design/m.v:22
/usr/joe/my_design/m.v:28 /usr/joe/my_design/m.v:12
/usr/joe/my_design/m.v:13 /usr/joe/my_design/m.v:14
/usr/joe/my_design/m.v:4
```

- **cross\_probing\_filter**

该命令允许通过指定源文件与行号按下列格式过滤集合：

```
"file_pattern[:line_pattern]"
```

下列示例显示 cross\_probing\_filter 的结果，返回源自以 m.v 结尾的源文件第 21 行的单元：

```
prompt> cross_probing_filter [get_cells -hierarchical] -source \
"*m.v:21"
{mid1/bot1/c1 mid1/bot2/c1}
```

设计中因合并而具有多个源位置的对象，只要任一源位置匹配任一指定源模式，就会返回在集合中。

- **report\_cross\_probing**

该命令以报告形式报告指定单元或端口的交叉探测数据。若对象由工具生成，则文件与行号不适用，以连字符（-）表示。此时报告会说明对象来源是 UPF、DFT 引擎还是其他来源。已合并的对象在报告中按每个源位置显示多次。

#### 参见

RTL 交叉探测

## 13.12 在批处理模式下报告逻辑级

### Reporting Logic Levels in Batch Mode

作为在 GUI 中显示逻辑级直方图的替代，可在命令行或脚本中使用 **report\_logic\_levels** 命令报告逻辑级。

#### 注意

必须有已映射设计才能运行逻辑级报告。

要将逻辑级报告与现有脚本集成，在批处理模式下使用 **report\_logic\_levels**。可用该命令报告整个设计或选定路径组的逻辑级。该命令报告所有路径类型的 logic level，包括 same-clock path、cross-clock path、infeasible path 与 multicycle path。报告的 logic-level 数量不计 buffer 和 inverter。

要报告逻辑级：

1. (可选) 用 **set\_analyze\_rtl\_logic\_level\_threshold** 命令设置不同于默认的特定逻辑级阈值。默认情况下，工具根据路径所需的延时值推导逻辑级阈值。

- 要为整个设计设置逻辑级阈值，输入：

```
de_shell> set_analyze_rtl_logic_level_threshold threshold_value
```

- 要为特定路径组设置逻辑级阈值，输入：

```
de_shell> set_analyze_rtl_logic_level_threshold \  
-group path_group threshold_value
```

路径组设置覆盖整个设计的全局阈值。

- 要复位所有阈值，输入：

```
de_shell> set_analyze_rtl_logic_level_threshold -reset
```

- 要复位特定组设置，输入：

```
de_shell> set_analyze_rtl_logic_level_threshold \  
-group path_group -reset
```

2. (可选) 通过设置变量 **logic\_level\_report\_summary\_format** 指定报告摘要节中的字段。默认值为 **{%group %period %wns %num\_paths %max\_level}**。例如：

```
de_shell> set_app_var logic_level_report_summary_format \  
    {%group %num_paths %max_level %min_level %avg_level}  
...
```

3. (可选) 通过设置变量 `logic_level_report_group_format` 指定报告路径组节中的字段。默认值为 `{%slack %l1 %l1threshold %startpoint %endpoint}`。例如:

```
de_shell> set_app_var logic_level_report_group_format \  
    {%slack %l1 %startclk %endclk %infeas %startpoint %endpoint}  
...
```

4. 使用 `report_logic_levels` 命令报告逻辑级。下列示例生成 `clk_i` 路径组的逻辑级报告:

```
de_shell> report_logic_levels -group clk_i
```

逻辑级报告包含三个节: 摘要、逻辑级分布与逻辑级路径报告。

参见

分析 RTL

# Chapter 14

## 使用 Milkyway 数据库

### *Using a Milkyway Database*

可在 DC Explorer 内写出 Milkyway 数据库，以便与其他基于 Synopsys Milkyway 数据库的工具（如 IC Compiler）配合使用。为此使用 `write_milkyway` 命令。

使用 Milkyway database 时，无需借助 Verilog 或 VHDL 等中间 netlist file exchange format，即可与其他基于 Synopsys Milkyway database 的工具交换数据。在 DC Explorer 内使用 Milkyway 数据库之前，必须准备设计库与参考库，并了解下列概念与任务：

- 关于 Milkyway 数据库
- 使用 Milkyway 数据库的指南
- 创建 Milkyway 设计库
- 写出 Milkyway 数据库

### 14.1 关于 Milkyway 数据库

#### About the Milkyway Database

Milkyway 数据库将设计数据存储在 Milkyway 设计库中，将物理库数据存储在 Milkyway 参考库中。

- **Milkyway 设计库**

用于存储设计数据（即 uniquified、mapped netlist 及 constraint）的 Milkyway 目录结构称为 Milkyway design library。通过将变量 `mw_design_library` 设为根目录路径，为当前会话指定 Milkyway 设计库。

- **Milkyway 参考库**

用于存储物理库数据的 Milkyway 目录结构称为 Milkyway 参考库。参考库包含标准单元、宏单元与 pad 单元。关于创建参考库的信息，见 Milkyway 文档。

通过将变量 `mw_reference_library` 设为根目录路径，为当前会话指定 Milkyway 参考库。列表

顺序决定 reference conflict resolution 的优先级；若多个 reference library 中存在同名 cell，则列表中的第一个 reference library 优先。

### 14.1.1 所需许可证与文件

#### Required License and Files

使用 Milkyway 数据库之前，需要下列所需许可证与文件：

- Milkyway-Interface 许可证  
DC Explorer 提供此许可证，用于运行 `write_milkyway` 命令。
- 逻辑库源（.lib）
- 已编译数据库
  - logic library（.db），包含 standard cell 的 timing、power、function、test 等信息
  - Milkyway library（FRAM），包含 technology data

### 14.1.2 调用 Milkyway Environment 工具

#### Invoking the Milkyway Environment Tool

要调用 Milkyway Environment 工具，输入：

```
% Milkyway -galaxy
```

该命令检出 Milkyway-Interface 许可证。Milkyway Environment 工具是图形用户界面（GUI），用于操作 Milkyway 库。可用该工具维护 Milkyway 设计库，例如删除设计的未使用版本。

关于使用 Milkyway Environment 工具的信息，见 Milkyway 文档。

#### 参见

- 使用 Milkyway 数据库的指南
- 使用 Milkyway 格式的限制
- 写出 Milkyway 数据库

## 14.2 使用 Milkyway 数据库的指南

### Guidelines for Using the Milkyway Databases

使用 `write_milkyway` 命令时，请遵循下列指南。

- 确保 Milkyway 参考库中存在的所有单元在时序库中都有对应单元。Milkyway 参考库中单元的端口方向由时序库中单元的端口方向设置。若单元存在于 Milkyway 参考库但不在时序库中，则参考库中这些单元的端口方向不会被设置。
- 在运行 `write_milkyway` 之前运行 `uniquify` 命令。

- 必须确保 logic library 与 Milkyway technology file 使用一致的单位。

SDC 文件不包含单位信息。若 logic library 与 Milkyway technology file 的单位不一致, `write_milkyway` 无法自动完成单位换算。例如, 若逻辑库使用飞法 (femtofarad) 作为电容单位, 而 Milkyway technology 文件使用皮法 (picofarad), 则 `write_sdc` 的输出会显示不同的网负载值。

在下列示例中, 逻辑库与 Milkyway technology 文件中的电容单位不一致。运行 `write_milkyway` 之前, 网 `gpdhi_word_d_21_` 显示下列 `set_load` 信息:

```
set_load 1425.15 [get_nets {gpdhi_word_d_21_}]
```

运行 `write_milkyway` 后, 生成的 SDC 文件显示:

```
set_load 8.36909 [get_nets {gpdhi_word_d_21_}]
```

- DC Explorer 区分大小写。可在运行 `write_milkyway` 之前通过下列任务以不区分大小写模式使用工具:
  - 准备 link library 中所用库的大写版本。
  - 使用 `change_names` 命令确保网表为大写。

## 14.3 创建 Milkyway 设计库

### Creating a Milkyway Design Library

使用 Milkyway 设计库指定物理库并以 Milkyway 格式保存设计。创建 Milkyway 设计库所需的输入为: Milkyway 参考库与 Milkyway technology 文件。

要创建 Milkyway 设计库:

1. 用 `create_mw_lib` 命令创建 Milkyway 设计库, 如下例所示:

```
de_shell> create_mw_lib -technology $mw_tech_file \
  -mw_reference_library $mw_reference_library $mw_design_library_name
```

2. 用 `open_mw_lib` 命令打开已创建的 Milkyway 库, 如下例所示:

```
de_shell> open_mw_lib $mw_design_library_name
```

3. (可选) 用 `set_tlu_plus_files` 命令附着 TLUPlus 文件, 如下例所示:

```
de_shell> set_tlu_plus_files -max_tluplus $max_tlu_file \
  -min_tluplus $min_tlu_file -tech2itf_map $prs_map_file
```

4. 在后续 `de_shell` 会话中用 `open_mw_lib` 打开 Milkyway 库。若使用 TLUPlus 文件进行 RC 估算, 用 `set_tlu_plus_files` 附着这些文件, 如下例所示:

```
de_shell> open_mw_lib $mw_design_library_name
de_shell> set_tlu_plus_files -max_tluplus $max_tlu_file \
  -min_tluplus $min_tlu_file -tech2itf_map $prs_map_file
```

## 参见

关于 Milkyway 数据库

## 14.4 写出 Milkyway 数据库

### Writing the Milkyway Database

要将设计数据保存在 Milkyway 设计库中, 使用 `write_milkyway` 命令。该命令将内存中的 netlist 与 floorplan 信息写为 Milkyway design library 格式, 重建 hierarchy preservation 信息, 并将 current design 的数据保存到 design file。设计文件路径为 `design_dir/CEL/file_name:version`, 其中 `design_dir` 是变量 `mw_design_library` 中指定的位置。

例如, 用 `-output` 选项指定文件名:

```
de_shell> write_milkyway -output my_file -overwrite
```

要将内存中的设计信息写入名为 `testmw` 的 Milkyway 库并将设计文件命名为 `TOP`, 输入:

```
de_shell> set_app_var mw_design_library testmw
de_shell> write_milkyway -output TOP
```

示例 14.1 给出设置并写出 Milkyway 数据库的脚本。

```
set_app_var search_path "$search_path ./libraries"
set_app_var link_library "* max_lib.db"
set_app_var target_library "max_lib.db"

create_mw_lib -technology $mw_tech_file -mw_reference_library \
    $mw_reference_library$mw_lib_name
open_mw_lib $mw_lib_name
read_file -format ddc design.ddc
current_design TopDesign
link
write_milkyway -output myTop
```

代码 14.1: 设置并写出 Milkyway 数据库的脚本

## 参见

- 关于 `write_milkyway` 命令
- 使用 Milkyway 数据库的指南
- 使用 Milkyway 格式的限制
- 使用层次模型

#### 14.4.1 关于 `write_milkyway` 命令

##### About the `write_milkyway` Command

使用 `write_milkyway` 命令时, 请记住下列要点:

- 必须在运行 `write_milkyway` 之前运行 `create_mw_lib` 命令。
- 若 design file 已存在（即先前已使用同一 output directory 对该设计运行 `write_milkyway`），该命令会另建一个 design file version，并递增 version number。必须确保在 Milkyway 中打开正确版本；默认情况下 Milkyway 打开最新版本。为避免创建额外版本，指定 `-overwrite` 选项以覆盖设计文件的当前版本并节省磁盘空间。
- 该命令不修改内存中的数据。
- 对于内存中设计上的 attribute，只有在 Milkyway 中存在等价 attribute 时才会转换；design database 中的 attribute 并非全部都会转换。
- 用 `write_milkyway` 转换的层次网表在其 Milkyway 数据库中保留层次。

#### 参见

写出 Milkyway 数据库

### 14.4.2 使用 Milkyway 格式的限制

#### Limitations of Using Milkyway Format

以 Milkyway 格式写出设计时适用下列限制：

- 设计必须已映射。  
由于 Milkyway 格式描述物理信息，它仅支持已映射设计。不能用 Milkyway 格式存储未映射设计的设计数据。
- 设计不得包含 multiply-instantiated design。  
以 Milkyway 格式保存设计之前，必须先执行 `uniquify`。使用 `check_design -multiple_designs` 报告 multiply-instantiated design 的相关信息。
- `write_milkyway` 将整个层次设计保存在单个 Milkyway 设计文件中。不能为每个子设计生成单独的设计文件。

#### 参见

- 关于 Milkyway 数据库
- 写出 Milkyway 数据库

# 附录 A

## 面向综合的设计文件管理

### *Design File Management for Synthesis*

设计（设计描述）存储在设计文件中。设计文件必须具有唯一名称。若设计是层次化的，每个子设计引用另一个设计文件，该文件也必须具有唯一名称。注意：不同设计文件可包含同名子设计。

本章包括以下节：

- 管理设计数据
- 面向综合的划分（Partitioning）
- 面向综合的 HDL 编码

### A.1 管理设计数据

#### **Managing the Design Data**

使用系统化的组织方法管理设计数据。管理设计数据的两个基本要素是：

- 设计数据控制
- 设计数据组织

#### **A.1.1 设计数据控制**

##### **Design Data Control**

随着设计新版本的创建，必须维护某种归档与记录方法，以提供设计演进历史，并在数据丢失时能够重新启动设计过程。建立数据创建、维护、覆盖与删除的控制是基本的设计管理问题。建立文件命名约定是数据创建最重要的规则之一。表 A.1 列出各设计数据类型的推荐文件扩展名。

表 A.1: 文件名扩展名

| 设计数据类型 | 扩展名  | 说明               |
|--------|------|------------------|
| 设计源代码  | .v   | Verilog          |
|        | .vhd | VHDL             |
|        | .con | Constraint       |
| 综合脚本   | .scr | Script           |
| 报告与日志  | .rpt | Report           |
|        | .log | Log              |
| 设计数据库  | .ddc | Synopsys 内部数据库格式 |

A.1.2 设计数据组织

Design Data Organization

建立并遵守数据组织方法，比选择何种方法更重要。在将基本设计数据置于一致控制之下后，即可创建有意义的数据组织。为简化数据交换与搜索，设计者应遵守该数据组织系统。

可用层次目录结构解决数据组织问题。您的 compile 策略会影响目录结构。图 A.1 给出基于自顶向下 compile 策略的目录结构，图 A.2 给出自底向上 compile 策略。



图 A.1: 自顶向下 Compile 目录结构

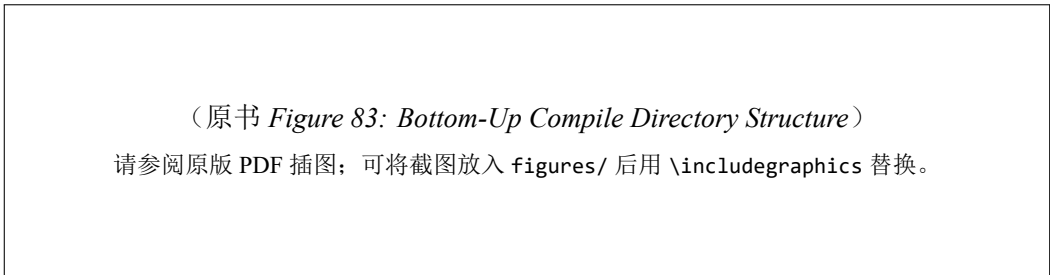


图 A.2: 自底向上 Compile 目录结构

参见

Compile 策略

## A.2 面向综合的划分

### Partitioning for Synthesis

有效划分设计可提升综合结果、缩短 `compile` 时间，并简化约束与脚本文件。

划分影响块大小；尽管 DC Explorer 没有固有的块大小限制，仍应谨慎控制块大小。若块过小，可能造成限制有效优化的人为边界；若创建过大块，`compile` 运行时间可能很长。

使用下列策略划分设计并改善优化与运行时间：

- 为设计重用而划分
- 将相关组合逻辑放在一起
- 对块输出寄存器化
- 按设计目标划分
- 按 `compile` 技术划分
- 将可共享资源放在一起
- 将用户定义资源与其所驱动的逻辑放在一起
- 隔离特殊功能，例如 `pad`、时钟、边界扫描与异步逻辑

下列各节描述每种策略。

#### A.2.1 为设计重用而划分

##### Partitioning for Design Reuse

设计重用通过减少设计、集成与测试工作来缩短上市时间。

重用现有设计时，应划分设计以便实例化这些设计。

为使设计可被重用，在划分与块设计期间遵循下列指南：

- 彻底定义并文档化设计接口
- 尽可能标准化接口
- 对 HDL 代码参数化

#### A.2.2 将相关组合逻辑放在一起

##### Keeping Related Combinational Logic Together

默认情况下，DC Explorer 不能将逻辑移过层次边界。将相关组合逻辑分到不同块会引入限制逻辑优化的人为障碍。

为获得最佳结果，应用下列策略：

- 将相关组合逻辑及其目标寄存器分组在一起。
- 在处理完整组合路径时，DC Explorer 有灵活性合并逻辑，从而得到更小、更快的设计。将组合逻辑与其 `destination register` 分组还可简化 `timing constraint`，并支持 `sequential optimization`。

- 消除粘合逻辑（glue logic）。

粘合逻辑是连接各块的组合逻辑。将此逻辑移入某一块可通过为 DC Explorer 提供额外灵活性来改善综合结果。消除粘合逻辑也缩短 compile 时间，因为 DC Explorer 需要优化的逻辑级数更少。

例如，假设设计在关键路径上或附近包含三个组合云。图 A.3 给出该设计的不良划分。每个组合云出现在单独块中，因此 DC Explorer 无法充分发挥其组合优化技术。

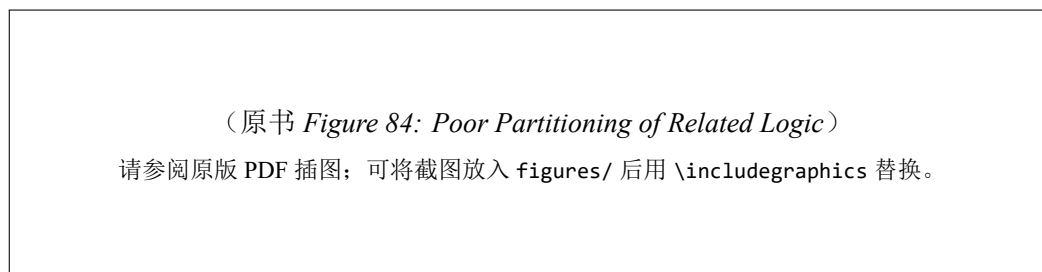


图 A.3: 相关逻辑的不良划分

图 A.4 给出同一设计且无人工边界的情况。在此设计中，DC Explorer 有灵活性在组合云中合并相关功能。

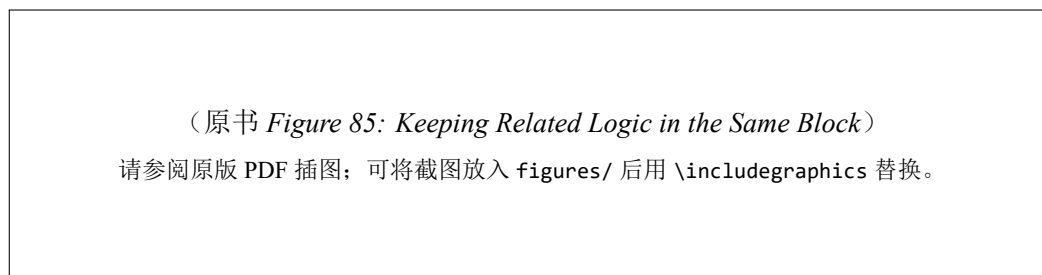


图 A.4: 将相关逻辑保持在同一块中

### A.2.3 对块输出寄存器化

#### Registering Block Outputs

为简化约束定义，确保寄存器驱动块输出，如图 A.5 所示。

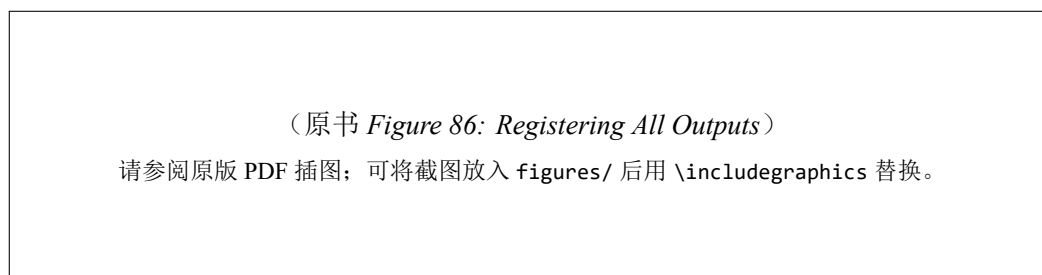


图 A.5: 对所有输出寄存器化

```
set_driving_cell my_flop [all_inputs]
set_input_delay 2 -clock CLK
```

该方法使您能轻松约束每个块，因为：

- 单个块输入上的驱动强度始终等于平均输入驱动的驱动强度
- 来自前一块的输入延时始终等于触发器的路径延时

由于所有输出均已寄存器化时不存在仅组合路径，对设计做时间预算并使用 `set_output_delay` 命令更容易。鉴于每个模块内发生一个时钟周期，各模块的约束简单且相同。

此划分方法可改善仿真性能。所有输出均已寄存器化时，可用仅边沿触发的进程描述模块。敏感列表仅包含时钟，或许还有复位引脚。有限的敏感列表通过使进程在每个时钟周期仅触发一次来加速仿真。

## A.2.4 按设计目标划分

### Partitioning by Design Goal

将具有不同设计目标的逻辑划分到不同块。当设计某些部分比其他部分更面积与时序关键时使用此方法。

为获得最佳综合结果，将非关键速度约束逻辑与关键速度约束逻辑隔离。通过隔离非关键逻辑，可对块施加不同约束，例如最大面积约束。

图 A.6 给出如何分离具有不同设计目标的逻辑。

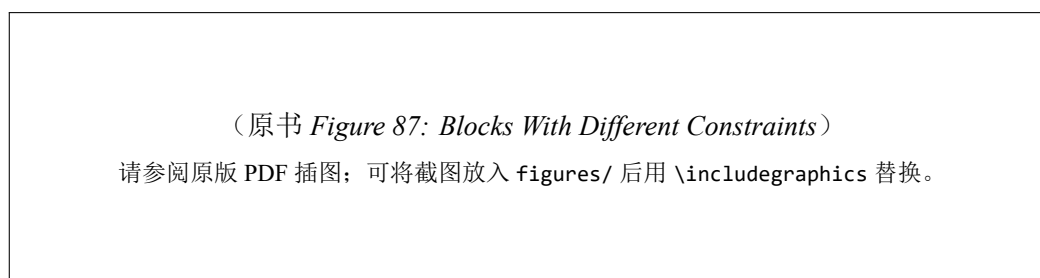


图 A.6: 具有不同约束的块

## A.2.5 按 Compile 技术划分

### Partitioning by Compile Technique

将需要不同 compile 技术的逻辑划分到不同块。当设计同时包含高度结构化逻辑与随机逻辑时使用此方法。

- highly structured logic（如通常包含大型 XOR tree 的 error-detection circuitry）更适合 structuring。
- 随机逻辑更适合 flattening。

## A.2.6 将可共享资源放在一起

### Keeping Sharable Resources Together

DC Explorer 可共享大型 resource（如 adder 或 multiplier），但仅当这些 resource 属于同一 VHDL process 或 Verilog always block 时，才能进行 resource sharing。

例如，若两个独立加法器具有相同目标路径且对其路径有多路复用输出，请将加法器保持在一个 VHDL process 或 Verilog always 块中。若约束允许共享，此方法允许 DC Explorer 共享资源（使用一个加法器而非两个）。图 A.7 给出逻辑示例的可能实现。

（原书 *Figure 88: Keeping Sharable Resources in the Same Process*）  
请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 A.7: 将可共享资源保持在同一进程中

关于资源共享的更多信息，见 HDL Compiler 文档。

### A.2.7 将用户定义资源与其所驱动的逻辑放在一起

#### Keeping User-Defined Resources With the Logic They Drive

用户定义资源是用户定义的函数、过程或宏单元，或用户创建的 DesignWare 组件。DC Explorer 不能自动共享或创建用户定义资源的多个实例。但将这些资源与其所驱动的逻辑放在一起，可在单次实例化无法达到时序目标时，通过手动插入用户定义资源的多次实例化来拆分负载。

图 A.8 说明当信号 `PARITY_ERR` 上的负载过大以致无法满足约束时，通过多次实例化拆分负载。

（原书 *Figure 89: Duplicating User-Defined Resources*）  
请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 A.8: 复制用户定义资源

### A.2.8 隔离特殊功能

#### Isolating Special Functions

将特殊功能（如 I/O pad、时钟生成电路、边界扫描逻辑与异步逻辑）与核心逻辑隔离。图 A.9 给出设计顶层的推荐划分。

（原书 *Figure 90: Recommended Top-Level Partitioning*）  
请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 A.9: 推荐的顶层划分

设计顶层包含 I/O pad 环与中间层次；中间层次包含边界扫描逻辑、时钟生成电路、异步逻辑与核心逻辑的子模块。中间层次的存在是为了允许实例化 I/O pad 的灵活性。隔离时钟生成电路可实现该模块的实例化与仔细仿真。隔离异步逻辑有助于将可测性问题与静态时序分析问题限制在小范围内。

## A.3 面向综合的 HDL 编码

### HDL Coding for Synthesis

HDL 编码是综合的基础，因为它隐含设计的初始结构。编写 HDL 源代码时，始终考虑代码的硬件含义。良好的编码风格可生成更小、更快的设计。本节提供信息以帮助编写高效代码，从而在最短时间内达到设计目标。

主题包括：

- 编写与工艺无关的 HDL
- 使用 HDL 构造
- 编写有效代码

#### A.3.1 编写与工艺无关的 HDL

##### Writing Technology-Independent HDL

使用完全自动综合流程的高层次设计目标是：不实例化任何门或触发器。若达到此目标，您将拥有可读、简洁且可移植的高层次 HDL 代码，可转移到其他厂商或未来工艺。

在某些情况下，HDL Compiler 工具需要编译器指令以提供实现信息，同时仍保持与工艺无关。在 Verilog 中，编译器指令以字符 `//` 或 `/*` 开头。在 VHDL 中，编译器指令以两个连字符 (`--`) 后跟 `pragma` 或 `synopsys` 开头。更多信息见 HDL Compiler 文档。

下列各节讨论保持 HDL 代码与工艺无关的各种方法。

推断组件

**Inferring Components** HDL Compiler 提供推断下列组件的能力：

- 多路选择器 (multiplexer)
- 寄存器
- 三态驱动器
- 多位组件 (multibit components)

这些推断能力在下列各页讨论。更多信息与示例见 HDL Compiler 文档。

推断多路选择器 *Inferring Multiplexers*

HDL Compiler 可从 HDL 代码中的 `case` 语句推断通用多路选择器单元 (MUX\_OP)。若目标工艺库至少包含 2 选 1 多路选择器单元，DC Explorer 将推断的 MUX\_OP 映射到目标工艺库中的多路选择器单元。DC Explorer 在 `compile` 期间根据设计约束确定 MUX\_OP 实现。

使用 `infer_mux` 编译器指令控制多路选择器推断。附着到块时，该指令强制对块中所有 `case` 语句做多路选择器推断；附着到 `case` 语句时，强制对该特定 `case` 语句做多路选择器推断。

### 推断寄存器 *Inferring Registers*

寄存器推断允许您在设计中指定与工艺无关的时序逻辑。寄存器是简单的 1 位存储器件，可以是锁存器或触发器。锁存器是电平敏感存储器件；触发器是边沿触发存储器件。

每当您未在所有条件下指定输出的结果值时（如不完全指定的 `if` 或 `case` 语句），HDL Compiler 推断 D 锁存器。HDL Compiler 也可推断 SR 锁存器与主从锁存器。

每当 Verilog `always` 块或 VHDL `process` 的敏感列表包含边沿表达式（测试信号上升或下降沿）时，HDL Compiler 推断 D 触发器。HDL Compiler 也可推断 JK 触发器与翻转触发器。

### 混合寄存器类型 *Mixing Register Types*

为获得最佳结果，将每个 Verilog `always` 块或 VHDL `process` 限制为单一类型的寄存器推断：锁存器、带异步置位/复位的锁存器、触发器、带异步置位/复位的触发器，或带同步置位/复位的触发器。

在设计中混合上升沿与下降沿触发的触发器时要小心。若模块同时推断上升沿与下降沿触发的触发器，且目标工艺库不含下降沿触发的触发器，DC Explorer 会在下降沿时钟的时钟树中生成反相器。

### 推断无控制信号的寄存器 *Inferring Registers Without Control Signals*

对于推断无控制信号的寄存器，使数据与时钟引脚可从输入端口或通过组合逻辑控制。若门级仿真器不能从输入端口或通过组合逻辑控制数据或时钟引脚，仿真器无法初始化电路，仿真将失败。

### 推断带控制信号的寄存器 *Inferring Registers With Control Signals*

可用异步或同步控制信号初始化或控制触发器状态。

对于推断锁存器上的异步控制信号，使用 `async_set_reset` 编译器指令（VHDL 中为属性）标识异步控制信号。推断触发器时，HDL Compiler 自动标识异步控制信号。

对于推断同步复位，使用 `sync_set_reset` 编译器指令（VHDL 中为属性）标识同步控制。

### 推断三态驱动器 *Inferring Three-State Drivers*

将高阻值（Verilog 中为 `1'bz`，VHDL 中为 `'Z'`）赋给输出引脚，以使 DC Explorer 推断三态门。三态逻辑降低设计可测性并使调试困难。在可能的情况下，用多路选择器替换三态缓冲器。

切勿在条件表达式中使用高阻值。HDL Compiler 始终将与高阻值比较的表达式求值为假，这可能导致门级实现与 RTL 描述行为不同。

关于三态推断的更多信息，见 HDL Compiler 文档。

### 推断多位组件 *Inferring Multibit Components*

多位推断允许您将多路选择器、寄存器与三态驱动器映射到规则结构化逻辑或多位库单元。使用多位组件可带来下列结果：

- 因共享晶体管与优化的晶体管级版图而更小的面积与延时
- sequential gate 中的 clock skew 减小
- sequential banked component 的 clock power 降低

- 改善 datapath 的规则版图

在下列情况下多位组件可能效率不高：

- 作为状态机寄存器
- 在更适合 single-bit design 的小型 based logic 中

在决定映射到多位还是单位组件时，必须权衡多位组件的收益与优化灵活性损失。

将 `infer_multibit` 编译器指令附着到总线信号以推断多位组件。也可在优化后用 `create_multibit` 与 `remove_multibit` 命令在单位与多位实现之间切换。

设计状态机

**Designing State Machines** 可用多种不同格式指定状态机：

- Verilog
- VHDL
- 状态表 (state table)
- PLA

若在 HDL 代码中使用 `state_vector` 与 `enum` 编译器指令，DC Explorer 可从网表提取状态表。在状态表格式中，DC Explorer 不保留 `casex`、`casez` 与 `parallel_case` 信息。DC Explorer 不优化无效输入组合与互斥输入。

图 A.10 给出有限状态机架构。

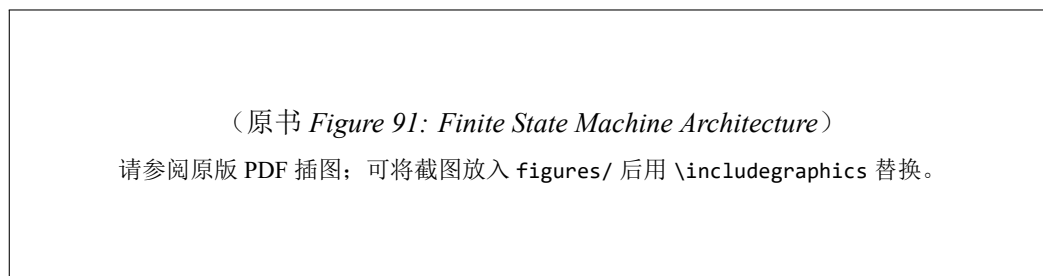


图 A.10: 有限状态机架构

使用提取的状态表具有下列收益：

- 可执行状态最小化
- 可在不同编码风格之间权衡
- 可在不 flattening 设计的情况下使用 don't care 条件
- 自动推导 don't care 状态码

### A.3.2 使用 HDL 构造

#### Using HDL Constructs

下列各节提供关于下列特定 HDL 构造的信息与指南：

- 一般 HDL 构造

- Verilog 宏定义
- VHDL 端口定义

## 一般 HDL 构造

**General HDL Constructs** 本节信息同时适用于 Verilog 与 VHDL。

### 敏感列表 *Sensitivity Lists*

应为每个 Verilog `always` 块或 VHDL `process` 完整指定敏感列表。不完整的敏感列表（如下例所示）可导致 HDL 与门级设计之间的仿真不匹配。

```
always @ (A)
  C <= A | B;
```

代码 A.1: 不完整的敏感列表（Verilog）

```
process (A)
  C <= A or B;
```

代码 A.2: 不完整的敏感列表（VHDL）

### 值赋值 *Value Assignments*

Verilog 与 VHDL 都支持在 RTL 代码中使用立即与延迟值赋值。立即值赋值——由 Verilog 阻塞赋值（`=`）与 VHDL 变量（`:=`）实现——所生成的硬件依赖于赋值顺序。延迟值赋值——由 Verilog 非阻塞赋值（`<=`）与 VHDL 信号（`<=`）实现——所生成的硬件与赋值顺序无关。

为获得正确仿真结果：

- 在 sequential Verilog `always` block 或 VHDL `process` 内使用 delayed（nonblocking）assignment
- 在 combinational Verilog `always` block 或 VHDL `process` 内使用 immediate（blocking）assignment

### if 语句 *if Statements*

当作为连续赋值一部分用在 Verilog `always` 块或 VHDL `process` 中的 if 语句不含 `else` 子句时，DC Explorer 创建锁存器。下列示例给出在综合期间生成锁存器的 if 语句。

```
if ((a == 1) && (b == 1))
  z = 1;
```

代码 A.3: 不正确的 if 语句（Verilog）

```
if (a = '1' and b = '1') then
  z <= '1';
end if;
```

代码 A.4: 不正确的 if 语句（VHDL）

**case 语句** *case Statements*

若 if 语句包含三个以上条件，考虑使用 case 语句以改善设计并行性与代码清晰度。下列示例使用 case 语句实现 3 位译码器。

```
case ({a, b, c})
  3'b000: z = 8'b00000001;
  3'b001: z = 8'b00000010;
  3'b010: z = 8'b00000100;
  3'b011: z = 8'b00001000;
  3'b100: z = 8'b00010000;
  3'b101: z = 8'b00100000;
  3'b110: z = 8'b01000000;
  3'b111: z = 8'b10000000;
  default: z = 8'b00000000;
endcase
```

代码 A.5: 使用 case 语句 (Verilog)

```
case_value := a & b & c;
CASE case_value IS
  WHEN "000" =>
    z <= "00000001";
  WHEN "001" =>
    z <= "00000010";
  WHEN "010" =>
    z <= "00000100";
  WHEN "011" =>
    z <= "00001000";
  WHEN "100" =>
    z <= "00010000";
  WHEN "101" =>
    z <= "00100000";
  WHEN "110" =>
    z <= "01000000";
  WHEN "111" =>
    z <= "10000000";
  WHEN OTHERS =>
    z <= "00000000";
END CASE;
```

代码 A.6: 使用 case 语句 (VHDL)

不完整的 case 语句会导致创建锁存器。VHDL 不支持不完整的 case 语句。在 Verilog 中，可通过 default 子句或 full\_case 编译器指令避免锁存器推断。

尽管 full\_case 指令与 default 子句都防止锁存器推断，它们含义不同。full\_case 指令断言已指定所有有效输入值且无需 default 子句；default 子句为任何未定义输入值指定输出。

为获得最佳结果，使用 default 子句而非 full\_case 指令。若未指定的输入值是 don't care 条件，使用输出值为 x 的 default 子句可生成更小的实现。

若使用 full\_case 指令，每当 case 表达式求值为未指定输入值时，门级仿真可能与 RTL 仿真不匹配。若使用 default 子句，仅当指定了 don't care 条件且 case 表达式求值为未指定输入值时，才

可能发生仿真不匹配。

### 常量定义 *Constant Definitions*

使用 Verilog 的 ``define` 语句或 VHDL 的 `constant` 语句定义全局常量。将全局常量定义保存在单独文件中。使用参数（Verilog）或 `generic`（VHDL）定义局部常量。

示例 A.7 给出包含全局 ``define` 与局部参数的 Verilog 代码片段。示例 A.8 给出包含全局常量与局部 `generic` 的 VHDL 代码片段。

```
// Define global constant in def_macro.v
`define WIDTH 128

// Use global constant in reg128.v
reg regfile[WIDTH-1:0];

// Define and use local constant in module myfile
module myfile (a, b, c);
    parameter WIDTH=128;
    input [WIDTH-1:0] a, b;
    output [WIDTH-1:0] c;
```

代码 A.7: 使用宏与参数（Verilog）

```
-- Define global constant in synthesis_def.vhd
constant WIDTH : INTEGER := 128;

-- Include global constants
library my_lib;
USE my_lib.synthesis_def.all;

-- Use global constant in entity my_design
entity my_design is
    port (a,b : in std_logic_vector(WIDTH-1 downto 0);
          c: out std_logic_vector(WIDTH-1 downto 0));
end my_design;

-- Define and use local constant in entity my_design
entity my_design is
    generic (WIDTH_VAR : INTEGER := 128);
    port (a,b : in std_logic_vector(WIDTH-1 downto 0);
          c: out std_logic_vector(WIDTH-1 downto 0));
end my_design;
```

代码 A.8: 使用全局常量与 Generics（VHDL）

### 使用 Verilog 宏定义

**Using Verilog Macro Definitions** 在 Verilog 中，宏用 ``define` 语句实现。对 ``define` 语句遵循下列指南：

- 仅用 ``define` 语句声明常量。

- 将 ``define` 语句保存在单独文件中。
- 不要使用嵌套的 ``define` 语句。  
读取嵌套超过两层的宏很困难。为使代码可读，不要使用嵌套的 ``define` 语句。
- 不要在模块定义内使用 ``define`。  
在模块定义内使用 ``define` 时，局部宏与全局宏具有相同引用名但不同值。使用参数定义局部常量。

## 使用 VHDL 端口定义

**Using VHDL Port Definitions** 在 VHDL 源代码中定义端口时，遵循下列指南：

- 使用 `STD_LOGIC` 与 `STD_LOGIC_VECTOR` 包。  
使用 `STD_LOGIC` 可避免在综合后设计上需要类型转换函数。
- 不要使用 `buffer` 端口模式。  
将 `port` 声明为 `buffer` 时，该 `port` 必须在整个 `hierarchy` 中均以 `buffer` 模式使用。为简化综合，应将 `port` 声明为 `out`，再定义一个驱动该 `output port` 的内部 `signal`。

## A.3.3 编写有效代码

### Writing Effective Code

本节提供为综合编写高效、可读 HDL 源代码的指南。指南涵盖：

- 标识符
- 表达式
- 函数
- 模块

## 标识符指南

**Guidelines for Identifiers** 好的标识符名称传达信号含义、变量值或模块功能；没有这些信息，硬件描述难以阅读。

遵循下列命名指南以改善 HDL 源代码可读性：

- 确保信号名称传达信号含义或变量值，同时不过于冗长。  
例如，假设有一个表示 `rs1` 浮点操作码的变量。短名称如 `frs1` 不能向读者传达含义；长名称如 `floating_pt_opcode_rs1` 传达含义，但其长度可能使源代码难读。使用如 `fpop_rs1` 的名称可同时满足两个目标。
- 对大小写以及区分名称中各词使用一致的命名风格。  
常用风格包括 C、Pascal 与 Modula。
  - C 风格使用小写名称并用下划线分隔词，例如 `packet_addr`、`data_in`、`first_grant_enable`。
  - Pascal 风格将名称首字母与每个词首字母大写，例如 `PacketAddr`、`DataIn`、`FirstGrantEnable`。
  - Modula 风格对名称首字母用小写，并将后续词首字母大写，例如 `packetAddr`、`dataIn`、`firstGrantEnable`。

选择一种约定并一致应用。

- 避免易混淆字符。  
某些字符（字母与数字）看起来相似且易混淆，例如 O 与 0（零）；1 与 l（一）。
- 避免保留字。
- 名称采用 noun 或 noun 后接 verb 的形式，例如 AddrDecode、DataGrant、PCI\_interrupt。
- 添加后缀以澄清名称含义。  
表 A.2 给出常用后缀及其含义。

表 A.2: 信号名称后缀及其含义

| 后缀    | 含义          |
|-------|-------------|
| _clk  | 时钟信号        |
| _next | 寄存之前的信号     |
| _n    | 低有效信号       |
| _z    | 连接到三态输出的信号  |
| _f    | 使用有效下降沿的寄存器 |
| _xi   | 芯片主输入       |
| _xo   | 芯片主输出       |
| _xod  | 芯片主开漏输出     |
| _xz   | 芯片主三态输出     |
| _xbio | 芯片主双向 I/O   |

表达式指南

**Guidelines for Expressions** 对表达式遵循下列指南：

- 使用括号表示优先级。  
表达式运算符优先级规则令人困惑，因此应使用括号使表达式易于阅读。除非使用 DesignWare 资源，括号对生成逻辑影响很小。难以阅读且无括号的逻辑表达式示例：

```
bus_select = a ^ b & c~^d|b^~e&^f[1:0];
```

- 用函数调用或连续赋值替换重复表达式。  
若特定表达式使用超过两三次，考虑用实现该表达式的函数或连续赋值替换该表达式。

函数指南

**Guidelines for Functions** 对函数遵循下列指南：

- 不要在函数内使用全局引用。  
在过程代码中，函数在被调用时求值。在连续赋值中，当其任一已声明输入改变时求值函数。避免在函数内引用全局名，因为全局值改变时函数可能不会被重新求值。这可能导致 HDL 描述与门级网表之间的仿真不匹配。  
例如，下列 Verilog 函数引用全局名 byte\_sel：

```
function byte_compare;
    input [15:0] vector1, vector2;
    input [7:0] length;

    begin
        if (byte_sel)
            // compare the upper byte
        else
            // compare the lower byte
        ...
    end
endfunction // byte_compare
```

- 注意任务与函数的局部存储是静态的。  
形式参数、输出与局部变量在函数返回后保留其值。每次调用函数时重用局部存储。此存储对调试有用，但存储重用也意味着函数与任务不能递归调用。
- 使用 component implication 时要谨慎。  
可用 map\_to\_module 与 return\_port\_name 编译器指令将函数映射到特定实现。simulation 使用 function 内容；synthesis 则以 gate-level module 替代该 function。使用 component implication 时，RTL model 与 gate-level model 可能不同。因此，在对门级设计运行仿真之前，无法完全验证设计。  
下列功能可能需要 component instantiation 或 functional implication:
  - 用于省电的时钟门控电路
  - 具有潜在冒险的异步逻辑（包括异步逻辑以及在某些状态下有效的异步信号）
  - Datapath 电路（包括大型多路选择器；实例化的宽多路选择器组；存储器元件如 RAM 或 ROM；以及黑盒宏单元）

关于 component implication 的更多信息，见 HDL Compiler 文档。

## 模块指南

### Guidelines for Modules 对模块遵循下列指南：

- 通过端口传递值时避免使用逻辑表达式。  
端口列表可包含表达式，但表达式使调试复杂化。此外，隔离与位域相关的问题很困难，尤其当该位域导致内部端口量与外部端口量不同时。
- 将局部引用定义为 generic（VHDL）或参数（Verilog）。不要将 generic 或参数传入模块。

# 附录 B

## 设计示例

### *Design Example*

优化设计时，可能需要对不同 hierarchy level 和 component 采用不同的 compile strategy。本附录给出使用多种 compile 策略的设计示例。前面各章详细说明如何实现每种 compile 策略。注意：本附录所用设计示例不代表真实应用。

本附录包括以下节：

- 设计描述
- Setup 文件
- 默认约束文件
- Read 脚本
- Compile 脚本

可在 `$SYNOPSIS/doc/syn/guidelines` 访问这些节所述文件，其中 `$SYNOPSIS` 为安装目录路径。

### B.1 设计描述

#### Design Description

该设计示例说明如何使用一组常用 `de_shell` 命令设置 design constraint，以及如何用 script 实现不同的 compile strategy。

该设计使用同步 RTL 与带时钟 D 触发器的组合逻辑。

图 B.1 给出设计示例的框图。顶层设计包含七个模块：`Adder16`、`CascadeMod`、`Comparator`、`Multiply8x8`、`Multiply16x16`、`MuxMod` 与 `PathSegment`。

图 B.2 给出设计示例的层次。

顶层模块及其优化所用的 compile 策略为：

|                |                                                                                                            |
|----------------|------------------------------------------------------------------------------------------------------------|
| <b>Adder16</b> | 使用 registered output 以简化 constraint 设置。由于 endpoint 是 register 的 data pin，无需在 output port 上设置 output delay。 |
|----------------|------------------------------------------------------------------------------------------------------------|

(原书 *Figure 92: Block Diagram for the Design Example*)  
 请参阅原版 PDF 插图；可将截图放入 figures/ 后用 \includegraphics 替换。

图 B.1: 设计示例框图

(原书 *Figure 93: Hierarchy for the Design Example*)  
 请参阅原版 PDF 插图；可将截图放入 figures/ 后用 \includegraphics 替换。

图 B.2: 设计示例层次

|                      |                                                                                                                                                                                            |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CascadeMod</b>    | 使用 hierarchical compile strategy。该设计的 compile script 在 compile 前于顶层 (CascadeMod) 设置 constraint。<br>CascadeMod 设计两次实例化 Adder8 设计。script 对 Comparator module 使用 compile-once-don't-touch 方法。 |
| <b>Comparator</b>    | 是组合块。该设计的 compile 脚本使用虚时钟概念，以说明在设计中使用虚时钟。ChipLevel 设计两次实例化 Comparator。compile 脚本 (针对 CascadeMod) 使用 compile-once-don't-touch 方法解析多重实例。                                                     |
| <b>Multiply8x8</b>   | 说明用于优化设计的基本时序与面积约束。                                                                                                                                                                        |
| <b>Multiply16x16</b> | 在 compile 前 ungroup DesignWare component。Ungroup hierarchical module 可能有助于获得更好的 synthesis result。该 module 的 compile script 在 module 的 primary port 上定义 two-cycle path。                     |
| <b>MuxMod</b>        | 是组合块。该设计的脚本使用虚时钟概念。                                                                                                                                                                        |
| <b>PathSegment</b>   | 在 module 内使用 path segmentation。script 对 module 内的 two-cycle path 使用 set_multicycle_path，并用 group 创建新的 hierarchy level。                                                                     |

示例 B.1 至示例 B.11 给出 ChipLevel 设计的 Verilog 源代码。

```
/* Date: May 11, 1995 */
/* Example Circuit for Baseline Methodology for Synthesis */
/* Design does not show any real-life application but rather
   it is used to illustrate the commands used in the Reference
   Methodology */

module ChipLevel (data16_a, data16_b, data16_c, data16_d, clk, cin, din_a,
                  din_b, sel, rst, start, mux_out, cout1, cout2, s1, s2, op,
                  comp_out1, comp_out2, m32_out, regout);
```

```

input [15:0] data16_a, data16_b, data16_c, data16_d;
input [7:0] din_a, din_b;
input [1:0] sel;
input clk, cin, rst, start;
input s1, s2, op;
output [15:0] mux_out, regout;
output [31:0] m32_out;
output cout1, cout2, comp_out1, comp_out2;

wire [15:0] ad16_sout, ad8_sout, m16_out, cnt;

Adder16 u1 (.ain(data16_a), .bin(data16_b), .cin(cin), .cout(cout1),
            .sout(ad16_sout), .clk(clk));

CascadeMod u2 (.data1(data16_a), .data2(data16_b), .cin(cin), .s(ad8_sout),
               .cout(cout2), .clk(clk), .comp_out(comp_out1), .cnt(cnt),
               .rst(rst), .start(start) );

Comparator u3 (.ain(ad16_sout), .bin(ad8_sout), .cp_out(comp_out2));

Multiply8x8 u4 (.op1(din_a), .op2(din_b), .res(m16_out), .clk(clk));

Multiply16x16 u5 (.op1(data16_a), .op2(data16_b), .res(m32_out), .clk(clk));

MuxMod u6 (.Y_IN(mux_out), .MUX_CNT(sel), .D(ad16_sout), .R(ad8_sout),
           .F(m16_out), .UPC(cnt));

PathSegment u7 (.R1(data16_a), .R2(data16_b), .R3(data16_c), .R4(data16_d),
                .S2(s2), .S1(s1), .OP(op), .REGOUT(regout), .clk(clk));
endmodule

```

代码 B.1: ChipLevel.v

```

module Adder16 (ain, bin, cin, sout, cout, clk);
/* 16-Bit Adder Module */
output [15:0] sout;
output cout;
input [15:0] ain, bin;
input cin, clk;

wire [15:0] sout_tmp, ain, bin;
wire cout_tmp;
reg [15:0] sout, ain_tmp, bin_tmp;
reg cout, cin_tmp;

always @(posedge clk) begin
    cout = cout_tmp;
    sout = sout_tmp;
    ain_tmp = ain;

```

```

    bin_tmp = bin;
    cin_tmp = cin;
end
    assign {cout_tmp,sout_tmp} = ain_tmp + bin_tmp + cin_tmp;
endmodule

```

代码 B.2: Adder16.v

```

module CascadeMod (data1, data2, s, clk, cin, cout, comp_out, cnt, rst, start)
;
input [15:0] data1, data2;
output [15:0] s, cnt;
input clk, cin, rst, start;
output cout, comp_out;
wire co;

Adder8 u10 (.ain(data1[7:0]), .bin(data2[7:0]), .cin(cin), .clk(clk),
.sout(s[7:0]), .cout(co));
Adder8 u11 (.ain(data1[15:8]), .bin(data2[15:8]), .cin(co), .clk(clk),
.sout(s[15:8]), .cout(cout));
Comparator u12 (.ain(s), .bin(cnt), .cp_out(comp_out));

Counter u13 (.count(cnt), .start(start), .clk(clk), .rst(rst));
endmodule

```

代码 B.3: CascadeMod.v

```

module Adder8 (ain, bin, cin, sout, cout, clk);
/* 8-Bit Adder Module */
output [7:0] sout;
output cout;
input [7:0] ain, bin;
input cin, clk;

wire [7:0] sout_tmp, ain, bin;
wire cout_tmp;
reg [7:0] sout, ain_tmp, bin_tmp;
reg cout, cin_tmp;

always @(posedge clk) begin
    cout = cout_tmp;
    sout = sout_tmp;
    ain_tmp = ain;
    bin_tmp = bin;
    cin_tmp = cin;
end
    assign {cout_tmp,sout_tmp} = ain_tmp + bin_tmp + cin_tmp;
endmodule

```

代码 B.4: Adder8.v

```

module Counter (count, start, clk, rst);
/* Counter module */
    input clk;
    input rst;
    input start;
    output [15:0] count;

    wire clk;
    reg [15:0] count_N;
    reg [15:0] count;

    always @ (posedge clk or posedge rst)
        begin : counter_S
            if (rst) begin
                count = 0; // reset logic for the block
            end
            else begin
                count = count_N; // set specified registers of the block
            end
        end

    always @ (count or start)
        begin : counter_C
            count_N = count; // initialize outputs of the block
            if (start) count_N = 1; // user specified logic for the block
            else count_N = count + 1;
        end
    end
endmodule

```

代码 B.5: Counter.v

```

module Comparator (cp_out, ain, bin);
/* Comparator for 2 integer values */
    output cp_out;
    input [15:0] ain, bin;
    assign cp_out = ain < bin;
endmodule

```

代码 B.6: Comparator.v

```

module Multiply8x8 (op1, op2, res, clk);
/* 8-Bit multiplier */
    input [7:0] op1, op2;
    output [15:0] res;
    input clk;

    wire [15:0] res_tmp;
    reg [15:0] res;

    always @(posedge clk) begin

```

```

    res = res_tmp;
end
assign res_tmp = op1 * op2;
endmodule

```

代码 **B.7:** Multiply8x8.v

```

module Multiply16x16 (op1, op2, res, clk);
/* 16-Bit multiplier */
input [15:0] op1, op2;
output [31:0] res;
input clk;

wire [31:0] res_tmp;
reg [31:0] res;

always @(posedge clk) begin
    res = res_tmp;
end
assign res_tmp = op1 * op2;
endmodule

```

代码 **B.8:** Multiply16x16.v

```

`define DATA 2'b00
`define REG 2'b01
`define STACKIN 2'b10
`define UPCOUT 2'b11

```

代码 **B.9:** def\_macro.v

```

module MuxMod (Y_IN, MUX_CNT, D, R, F, UPC);
`include "def_macro.v"
output [15:0] Y_IN;
input [ 1:0] MUX_CNT;
input [15:0] D, F, R, UPC;

reg [15:0] Y_IN;

always @ ( MUX_CNT or D or R or F or UPC ) begin
    case ( MUX_CNT )
        `DATA :
            Y_IN = D ;
        `REG :
            Y_IN = R ;
        `STACKIN :
            Y_IN = F ;
        `UPCOUT :
            Y_IN = UPC;
    endcase
end
end

```

```
endmodule
```

代码 B.10: MuxMod.v

```
module PathSegment (R1, R2, R3, R4, S2, S1, OP, REGOUT, clk);
/* Example for path segmentation */
input [15:0] R1, R2, R3, R4;
input S2, S1, clk;
input OP;
output [15:0] REGOUT;

reg [15:0] ADATA, BDATA;
reg [15:0] REGOUT;
reg MODE;

wire [15:0] product ;

always @(posedge clk)
begin : selector_block
    case(S1)
        1'b0: ADATA <= R1;
        1'b1: ADATA <= R2;
        default: ADATA <= 16'bx;
    endcase
    case(S2)
        1'b0: BDATA <= R3;
        1'b1: BDATA <= R4;
        default: ADATA <= 16'bx;
    endcase
end

/* Only Lower Byte gets multiplied */
// instantiate DW02_mult
DW02_mult #(8,8) U100 (.A(ADATA[7:0]), .B(BDATA[7:0]), .TC(1'b0),
.PRODUCT(product));

always @(posedge clk)
begin : alu_block
    case (OP)
        1'b0 : begin
            REGOUT <= ADATA + BDATA;
        end
        1'b1 : begin
            REGOUT <= product;
        end
        default : REGOUT <= 16'bx;
    endcase
end
```

```
endmodule
```

代码 **B.11**: PathSegment.v

## B.2 Setup 文件

### Setup File

运行设计示例时，将示例 [B.12](#) 中的项目专用 `setup` 文件复制到项目工作目录。该 `setup` file 使用 Tcl subset 编写，可供 `dctcl` command language 使用。

```
# Define the target technology library, symbol library,
# and link libraries
set_app_var target_library lsi_10k.db
set_app_var link_library [concat $target_library ""]
set_app_var search_path [concat $search_path ./src]
set_app_var designer "Your Name"
set company "Synopsys, Inc."
# Define path directories for file locations
set_app_var source_path "./src/"
set_app_var script_path "./scr/"
set_app_var log_path "./log/"
set_app_var ddc_path "./ddc/"
set_app_var db_path "./db/"
set_app_var netlist_path "./netlist/"
```

代码 **B.12**: .synopsys\_dc.setup 文件

更多信息亦见 *Using Tcl With Synopsys Tools* 文档。

#### 参见

Setup 文件

## B.3 默认约束文件

### Default Constraints File

示例 [B.13](#) 所示文件定义设计的默认约束。在后续脚本中，DC Explorer 对每个模块先读取该文件。若某 module 的 script 包含额外 constraint，或包含不同于 default constraint file 的 constraint value，DC Explorer 将采用 module-specific constraint。

```
# Define system clock period
set_app_var clk_period 20

# Create real clock if clock port is found
if {[sizeof_collection [get_ports clk]] > 0} {
    set_app_var clk_name clk
    create_clock -period $clk_period clk
```

```

}

# Create virtual clock if clock port is not found
if {[sizeof_collection [get_ports clk]] == 0} {
    set_app_var clk_name vclk
    create_clock -period $clk_period -name vclk
}

# Apply default drive strengths and typical loads
# for I/O ports
set_load 1.5 [all_outputs]
set_driving_cell -lib_cell IV [all_inputs]

# If real clock, set infinite drive strength
if {[sizeof_collection [get_ports clk]] > 0} {
    set_drive 0 clk
}

# Apply default timing constraints for modules
set_input_delay 1.2 [all_inputs] -clock $clk_name
set_output_delay 1.5 [all_outputs] -clock $clk_name
set_clock_uncertainty -setup 0.45 $clk_name

# Set operating conditions
set_operating_conditions WCCOM

```

代码 B.13: defaults.con

## B.4 Read 脚本

### Read Script

示例 B.14 给出用于读入 ChipLevel 设计的 dctcl 脚本。

read.tcl 脚本将指定 Verilog 文件中的设计信息读入内存。

```

read_file -format verilog ChipLevel.v
read_file -format verilog Adder16.v
read_file -format verilog CascadeMod.v
read_file -format verilog Adder8.v
read_file -format verilog Counter.v
read_file -format verilog Comparator.v
read_file -format verilog Multiply8x8.v
read_file -format verilog Multiply16x16.v
read_file -format verilog MuxMod.v
read_file -format verilog PathSegment.v

```

代码 B.14: read.tcl

## B.5 Compile 脚本

### Compile Scripts

示例 B.15 至示例 B.25 给出用于 compile ChipLevel 设计的 dctcl 脚本。

每个模块的 compile 脚本以该模块命名以便识别。初始 dctcl 脚本文件具有 .tcl 后缀；由 write\_script 命令生成的脚本具有 .wtcl 后缀。

```
# Initial compile with estimated constraints
source "${script_path}initial_compile.tcl"

current_design ChipLevel
write_file -format ddc -hierarchy \
  -output "${ddc_path}ChipLevel_init.ddc"

# Characterize and write_script for all modules
source "${script_path}characterize.tcl"

# Recompile all modules using write_script constraints
remove_design -all
source "${script_path}recompile.tcl"

current_design ChipLevel
write_file -format ddc -hierarchy \
  -output "${ddc_path}ChipLevel_final.ddc"
```

代码 B.15: run.tcl

```
# Initial compile with estimated constraints
source "${script_path}read.tcl"

current_design ChipLevel
source "${script_path}defaults.con"

source "${script_path}adder16.tcl"
source "${script_path}cascademod.tcl"
source "${script_path}comp16.tcl"
source "${script_path}mult8.tcl"
source "${script_path}mult16.tcl"
source "${script_path}muxmod.tcl"
source "${script_path}pathseg.tcl"
```

代码 B.16: initial\_compile.tcl

```
# Script file for constraining Adder16
set_app_var rpt_file "adder16.rpt"
set_app_var design "adder16"

current_design Adder16
source "${script_path}defaults.con"
```

```

# Define design environment
set_load 2.2 sout
set_load 1.5 cout
set_driving_cell -lib_cell FD1 [all_inputs]
set_drive 0 $clk_name

# Define design constraints
set_input_delay 1.35 -clock $clk_name {ain bin}
set_input_delay 3.5 -clock $clk_name cin

compile_exploration

write_file -format ddc -hierarchy -output "${ddc_path}${design}.ddc"

source "${script_path}report.tcl"

```

代码 B.17: adder16.tcl

```

# Script file for constraining CascadeMod
# Constraints are set at this level and then a
# hierarchical compile approach is used

set_app_var rpt_file "cascademod.rpt"
set_app_var design "cascademod"

current_design CascadeMod
source "${script_path}defaults.con"

# Define design environment
set_load 2.5 [all_outputs]
set_driving_cell -lib_cell FD1 [all_inputs]
set_drive 0 $clk_name

# Define design constraints
set_input_delay 1.35 -clock $clk_name {data1 data2}
set_input_delay 3.5 -clock $clk_name cin
set_input_delay 4.5 -clock $clk_name {rst start}
set_output_delay 5.5 -clock $clk_name comp_out

# Use compile-once, dont_touch approach for Comparator
set_dont_touch u12

compile_exploration

write_file -format ddc -hierarchy -output "${ddc_path}${design}.ddc"

source "${script_path}report.tcl"

```

代码 B.18: cascademod.tcl

```

# Script file for constraining Comparator
set_app_var rpt_file "comp16.rpt"
set_app_var design "comp16"

current_design Comparator
source "${script_path}defaults.con"

# Define design environment
set_load 2.5 cp_out
set_driving_cell -lib_cell FD1 [all_inputs]

# Define design constraints
set_input_delay 1.35 -clock $clk_name {ain bin}
set_output_delay 5.1 -clock $clk_name {cp_out}

compile_exploration

write_file -format ddc -hierarchy -output "${ddc_path}${design}.ddc"

source "${script_path}report.tcl"

```

代码 B.19: comp16.tcl

```

# Script file for constraining Multiply8x8
set_app_var rpt_file "mult8.rpt"
set_app_var design "mult8"

current_design Multiply8x8
source "${script_path}defaults.con"

# Define design environment
set_load 2.2 res
set_driving_cell -lib_cell FD1P [all_inputs]
set_drive 0 $clk_name

# Define design constraints
set_input_delay 1.35 -clock $clk_name {op1 op2}

compile_exploration

write_file -format ddc -hierarchy -output "${ddc_path}${design}.ddc"

source "${script_path}report.tcl"

```

代码 B.20: mult8.tcl

```

# Script file for constraining Multiply16x16
set_app_var rpt_file "mult16.rpt"
set_app_var design "mult16"

```

```

current_design Multiply16x16
source "${script_path}defaults.con"

# Define design environment
set_load 2.2 res
set_driving_cell -lib_cell FD1 [all_inputs]
set_drive 0 $clk_name

# Define design constraints
set_input_delay 1.35 -clock $clk_name {op1 op2}

# Define multicycle path for multiplier
set_multicycle_path 2 -from [all_inputs] \
    -to [all_registers -data_pins -edge_triggered]

# Ungroup DesignWare parts
set_app_var designware_cells [get_cells \
    -filter "@is_oper==true"]
if {[sizeof_collection $designware_cells] > 0} {
    set_ungroup $designware_cells true
}

compile_exploration

write_file -format ddc -hierarchy -output "${ddc_path}${design}.ddc"

source "${script_path}report.tcl"
report_timing_requirements -ignored \
    >> "${log_path}${rpt_file}"

```

代码 B.21: mult16.tcl

```

# Script file for constraining MuxMod
set_app_var rpt_file "muxmod.rpt"
set_app_var design "muxmod"

current_design MuxMod
source "${script_path}defaults.con"

# Define design environment
set_load 2.2 Y_IN
set_driving_cell -lib_cell FD1 [all_inputs]

# Define design constraints
set_input_delay 1.35 -clock $clk_name {D R F UPC}
set_input_delay 2.35 -clock $clk_name MUX_CNT
set_output_delay 5.1 -clock $clk_name {Y_IN}

compile_exploration

```

```
write_file -format ddc -hierarchy -output "${ddc_path}${design}.ddc"

source "${script_path}report.tcl"
```

代码 B.22: muxmod.tcl

```
# Script file for constraining path_segment
set_app_var rpt_file "pathseg.rpt"
set_app_var design "pathseg"

current_design PathSegment
source "${script_path}defaults.con"

# Define design environment
set_load 2.5 [all_outputs]
set_driving_cell -lib_cell FD1 [all_inputs]
set_drive 0 $clk_name

# Define design rules
set_max_fanout 6 {S1 S2}

# Define design constraints
set_input_delay 2.2 -clock $clk_name {R1 R2}
set_input_delay 2.2 -clock $clk_name {R3 R4}
set_input_delay 5 -clock $clk_name {S2 S1 OP}

# Perform path segmentation for multiplier
group -design mult -cell mult U100
set_input_delay 10 -clock $clk_name mult/product*
set_output_delay 5 -clock $clk_name mult/product*
set_multicycle_path 2 -to mult/product*

compile_exploration

write_file -format ddc -hierarchy -output "${ddc_path}${design}.ddc"

source "${script_path}report.tcl"
report_timing_requirements -ignored \
    >> "${log_path}${rpt_file}"
```

代码 B.23: pathseg.tcl

```
source "${script_path}read.tcl"

current_design ChipLevel
source "${script_path}defaults.con"

source "${script_path}adder16.wtcl"
compile_exploration
write_file -format ddc -hierarchy -output "${ddc_path}adder16_wtcl.ddc"
set_app_var rpt_file adder16_wtcl.rpt
```

```

source "${script_path}report.tcl"

source "${script_path}cascademod.wtcl"
dont_touch u12
compile_exploration
write_file -format ddc -hierarchy \
    -output "${ddc_path}cascademod_wtcl.ddc"
set_app_var rpt_file cascade_wtcl.rpt
source "${script_path}report.tcl"
source "${script_path}comp16.wtcl"
compile_exploration
write_file -format ddc -hierarchy -output "${ddc_path}comp16_wtcl.ddc"
set_app_var rpt_file comp16_wtcl.rpt
source "${script_path}report.tcl"

source "${script_path}mult8.wtcl"
compile_exploration
write_file -format ddc -hierarchy -output "${ddc_path}mult8_wtcl.ddc"
set_app_var rpt_file mult8_wtcl.rpt
source "${script_path}report.tcl"

source "${script_path}mult16.wtcl"
compile_exploration
write_file -format ddc -hierarchy -output "${ddc_path}mult16_wtcl.ddc"
set_app_var rpt_file mult16_wtcl.rpt
source "${script_path}report.tcl"
report_timing_requirements -ignored \
    >> "${log_path}${rpt_file}"

source "${script_path}muxmod.wtcl"
compile_exploration
write_file -format ddc -hierarchy -output "${ddc_path}muxmod_wtcl.ddc"
set_app_var rpt_file muxmod_wtcl.rpt
source "${script_path}report.tcl"

source "${script_path}pathseg.wtcl"
compile_exploration
write_file -format ddc -hierarchy -output "${ddc_path}pathseg_wtcl.ddc"
set_app_var rpt_file pathseg_wtcl.rpt
source "${script_path}report.tcl"
report_timing_requirements -ignored \
    >> "${log_path}${rpt_file}"

```

代码 B.24: recompile.tcl

```

# This script file creates reports for all modules
set_app_var maxpaths 15

check_design > "${log_path}${rpt_file}"
report_area >> "${log_path}${rpt_file}"

```

```
report_design >> "${log_path}${rpt_file}"
report_cell >> "${log_path}${rpt_file}"
report_reference >> "${log_path}${rpt_file}"
report_port -verbose >> "${log_path}${rpt_file}"
report_net >> "${log_path}${rpt_file}"
report_constraint -all_violators -verbose \
    >> "${log_path}${rpt_file}"
report_timing -path_type end >> "${log_path}${rpt_file}"
report_timing -max_paths $maxpaths \
    >> "${log_path}${rpt_file}"
report_qor >> "${log_path}${rpt_file}"
```

代码 **B.25:** report.tcl

# 附录 C

## 基本命令

### *Basic Commands*

本附录列出综合用基本 `de_shell` 命令，并为每条命令提供简要说明。命令按下列主题分组：

- 定义设计规则的命令
- 定义设计环境的命令
- 设置设计约束的命令
- 分析并解决设计问题的命令

在每一节内，命令按字母顺序列出。

#### C.1 定义设计规则的命令

##### **Commands for Defining Design Rules**

定义设计规则的命令为：

###### **set\_max\_capacitance**

为连接到指定 port 的 net，或为设计中的所有 net，设置 maximum capacitance。

**set\_max\_fanout** 为 output port 设置 expected fanout load value。

###### **set\_max\_transition**

为连接到指定 port 的 net，或为设计中的所有 net，设置 maximum transition time。

###### **set\_min\_capacitance**

为附着到指定端口的网，或设计中的所有网，设置最小电容。

#### C.2 定义设计环境的命令

##### **Commands for Defining Design Environments**

定义设计环境的命令为：

|                                 |                                                                                                                                                                            |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>set_drive</b>                | 设置输入或 inout 端口的驱动值。 <b>set_drive</b> 命令已被 <b>set_driving_cell</b> 命令取代。                                                                                                    |
| <b>set_driving_cell</b>         | 在 input 或 inout port 上设置 attribute，指定驱动这些 port 的 library cell 或 library pin。该命令将 library pin 与 input port 关联，使 delay calculator 能够准确建模 external driver 的 drive capability。 |
| <b>set_fanout_load</b>          | 定义输出端口上的外部扇出负载值。                                                                                                                                                           |
| <b>set_load</b>                 | 定义输入与输出端口以及网上的外部负载值。                                                                                                                                                       |
| <b>set_operating_conditions</b> | 定义当前设计的工作条件。                                                                                                                                                               |

## C.3 设置设计约束的命令

### Commands for Setting Design Constraints

设置设计约束的基本命令为：

|                                                                                             |                                                           |
|---------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| <b>create_clock</b>                                                                         | 在当前设计中创建时钟对象并定义其波形。                                       |
| <b>set_clock_latency, set_clock_uncertainty, set_propagated_clock, set_clock_transition</b> | 在 clock object 或 flip-flop clock pin 上设置 clock attribute。 |
| <b>set_input_delay</b>                                                                      | 相对于时钟信号在引脚或输入端口上设置输入延时。                                   |
| <b>set_max_area</b>                                                                         | 指定当前设计的最大面积。                                              |
| <b>set_output_delay</b>                                                                     | 相对于时钟信号在引脚或输出端口上设置输出延时。                                   |

设置设计约束的高级命令为：

|                            |                                                                                                         |
|----------------------------|---------------------------------------------------------------------------------------------------------|
| <b>group_path</b>          | 对一组 path 或 endpoint 分组，以便进行 cost-function 计算。该命令用于创建 path group、向现有 group 添加 path，或更改现有 group 的 weight。 |
| <b>set_false_path</b>      | 将指定点之间的 path 标记为 false path，并从 timing analysis 中排除所选 path。                                              |
| <b>set_max_delay</b>       | 为当前设计中所选路径指定最大延时目标。                                                                                     |
| <b>set_min_delay</b>       | 为当前设计中所选路径指定最小延时目标。                                                                                     |
| <b>set_multicycle_path</b> | 允许指定 timing path 使用超过一个 clock period 的时间。                                                               |

## C.4 分析并解决设计问题的命令

### Commands for Analyzing and Resolving Design Problems

分析并解决设计问题的命令为：

|                      |            |
|----------------------|------------|
| <b>all_connected</b> | 列出网上的所有扇出。 |
|----------------------|------------|

|                                   |                                                                             |
|-----------------------------------|-----------------------------------------------------------------------------|
| <b>all_registers</b>              | 列出设计中的 sequential element 或 pin。                                            |
| <b>check_design</b>               | 检查当前设计内部表示的一致性，并酌情发出错误与警告消息。                                                |
| <b>check_timing</b>               | 检查施加于 current design 的 timing attribute。                                    |
| <b>get_attribute</b>              | 报告指定属性的值。                                                                   |
| <b>link</b>                       | 定位设计中每个单元的引用。                                                               |
| <b>report_area</b>                | 提供当前设计的面积信息与统计。                                                             |
| <b>report_attribute</b>           | 列出所选 object 的 attribute 及其值。object 可以是 cell、net、pin、port、instance 或 design。 |
| <b>report_cell</b>                | 列出当前设计中的单元及其单元属性。                                                           |
| <b>report_clock</b>               | 显示当前设计上与时钟相关的信息。                                                            |
| <b>report_constraint</b>          | 列出当前设计上的约束及其代价、权重与加权代价。                                                     |
| <b>report_delay_calculation</b>   | 报告延时弧计算的细节。                                                                 |
| <b>report_design</b>              | 显示为当前设计定义的工作条件、线负载模型与模式、时序范围、内部输入与输出，以及禁用的时序弧。                              |
| <b>report_design_mismatch</b>     | 报告为使 design 能够 link 而被绕过的 design mismatch。                                  |
| <b>report_hierarchy</b>           | 列出当前设计的子设计。                                                                 |
| <b>report_missing_constraints</b> | 报告当前设计中缺失的约束。                                                               |
| <b>report_net</b>                 | 若已设置当前实例，则显示该实例所属设计的网信息；否则显示当前设计的网信息。                                       |
| <b>report_path_group</b>          | 列出当前设计中所有时序路径组。                                                             |
| <b>report_port</b>                | 列出当前设计中关于端口的信息。                                                             |
| <b>report_qor</b>                 | 显示当前设计的结果质量信息及其他统计。                                                         |
| <b>report_resources</b>           | 显示关于资源实现的信息。                                                                |
| <b>report_timing</b>              | 列出当前设计的时序信息。                                                                |
| <b>report_timing_requirements</b> | 列出时序路径要求及相关信息。                                                              |
| <b>report_transitive_fanin</b>    | 列出当前实例所选引脚、网或端口的扇入逻辑。                                                       |
| <b>report_transitive_fanout</b>   | 列出当前实例所选引脚、网或端口的扇出逻辑。                                                       |

# 术语表

## Glossary

以下条目按英文字母顺序排列。英文术语保留原文；定义已译为中文，命令名与技术标识符保持英文。

### 标注 (*annotation*)

附着于设计中对象的信息片段，例如附着于网上的电容值；或将此类信息附着到设计中对象的过程。

### 反标 (*back-annotate*)

使用提取及其他后处理信息更新电路设计的过程，这些信息反映设计依赖于实现的特性，例如引脚选择、组件位置或寄生电气特性。反标允许对最终电路做更准确的时序分析。数据由布局后的另一工具生成并传递到综合环境。例如，设计数据库可用实际互连延时更新；这些延时在布局布线之后——确切互连长度已知之后——计算。

### 单元 (*cell*)

见实例 (*instance*)。

### Clock (*clock*)

具有周期行为的定时脉冲源。clock 通过控制数字电路中的 sequential element（如 flip-flop 与 register）来同步数据信号传播。用 `create_clock` 定义 clock。

用 `create_clock` 创建的 clock 默认忽略 clock network 的 delay effect。因此，为进行准确的 timing analysis，需要用 clock latency 与 clock skew 描述 clock network。另见 clock latency 与 clock skew。

### 时钟门控 (*clock gating*)

用逻辑（非反相器或缓冲器）控制时钟信号，以在选定时刻关断时钟信号或修改时钟脉冲特性。

### Clock latency (*clock latency*)

clock signal 从 clock source 传播到设计中特定点所需的时间。clock latency 是 source latency 与 network latency 之和。

source latency 是从实际 clock origin 到设计中 clock definition point 的传播时间。network latency 是从 clock definition point 到第一个 register clock pin 的传播时间。

用 `set_clock_latency` 命令指定时钟延迟。

### Clock skew (*clock skew*)

同一 clock domain 内或不同 clock domain 之间，各 register 处 clock arrival time 的最大差值。本指南在此处用 clock uncertainty 表征该 skew 特性，并使用 `set_clock_uncertainty`

为一个或多个 clock network 指定该值；clock skew 是实际到达时间之差，不能一般性地与包含 jitter、建模裕量等成分的 clock uncertainty 等同。

#### 时钟源 (*clock source*)

将时钟波形施加到设计的引脚或端口。时钟信号到达其所有源传递扇出中的寄存器。时钟可有多源。

对 `create_clock` 使用 `source_object` 选项指定时钟源。

#### 时钟树 (*clock tree*)

时钟源与该源传递扇出中寄存器之间的组合逻辑。时钟树亦称时钟网络，由厂商根据一个时钟域内或时钟域之间寄存器处的物理布局数据综合。

#### Clock uncertainty (*clock uncertainty*)

本指南此处将其用于表征 clock skew；见 clock skew。实际 SDC 分析中，clock uncertainty 是施加于 clock relationship 的 timing margin，不应与实际 clock arrival time 之差混为一谈。

#### 核 (*core*)

用作 ASIC 设计构建块的预设计逻辑块。

#### Critical path (*critical path*)

电路中 delay 最长的 path。电路速度取决于最慢的 register-to-register delay。clock period 不能短于该 delay，否则 signal 无法及时到达下一级 register 并被捕获。

#### 当前设计 (*current design*)

活动设计（正在处理的设计）。多数命令特定于当前设计，即在当前设计上下文中操作。用 `current_design` 命令指定当前设计。

#### 当前实例 (*current instance*)

设计层次中实例专用命令默认操作的实例。用 `current_instance` 命令指定当前实例。

#### Datapath (*datapath*)

用加法器、乘法器、移位器与比较器等算术运算符操纵数据信号的逻辑电路。

#### 设计约束 (*design constraints*)

设计者对设计性能目标的规格，即综合在其下执行的时序与环境限制。DC Explorer 用这些约束——例如低功耗、小面积、高速度或最小成本——指导设计优化以满足面积与时序目标。

设计约束分两类：设计规则约束与设计优化约束。

- 设计规则约束在工艺库中提供。为保证所制造电路正常工作，不得违例。
- 设计优化约束定义时序与面积优化目标。

DC Explorer 按两组约束优化设计综合；但设计规则约束优先级更高。

#### 伪路径 (*false path*)

您不希望 DC Explorer 在时序分析期间考虑的路径。例如两个永不同时使能的多路复用块之间的路径，即不能传播信号的路径。

用 `set_false_path` 命令按路径禁用基于时序的综合。该命令移除指定路径上的时序约束。

**扇入 (*fanin*)** 驱动端点引脚、端口或网（亦称 sink）的引脚。若存在经组合逻辑从该引脚到 sink 的时序路径，则该引脚在 sink 的扇入中。扇入追踪从寄存器时钟引脚或有效起点开始。扇入亦称传递扇入（*transitive fanin*）。

用 `report_transitive_fanin` 命令报告指定 sink 引脚、端口或网的扇入。

**扇出 (*fanout*)** 由源引脚、端口或网驱动的引脚。若存在经组合逻辑从源到该引脚或端口的时序路径，则该引脚在源的扇出中。扇出追踪在寄存器数据引脚或有效端点处停止。扇出亦称传递扇出（*transitive fanout*）或时序扇出。

用 `report_transitive_fanout` 命令报告指定源引脚、端口或网的扇出。

**扇出负载 (*fanout load*)**

表示对总扇出数值贡献的无量纲数。扇出负载不同于负载（电容值）。

DC Explorer 通过将 `fanout_load` 属性与每个输入引脚关联、将 `max_fanout` 属性与单元每个输出（驱动）引脚关联来建模扇出限制，并确保扇出负载之和小于 `max_fanout` 值。

**展平 (*flatten*)** 将设计的组合逻辑路径转换为两级积之和表示。展平期间，DC Explorer 从设计中移除所有中间项及因此所有相关逻辑结构。展平基于约束。

**前向标注 (*forward-annotate*)**

将数据从综合环境传输到设计流程后续所用其他工具。例如，可将标准延时格式（SDF）中的延时与约束数据从综合环境传输以指导布局布线工具。

**生成时钟 (*generated clock*)**

由集成电路内部生成的时钟信号；非直接来自外部源。例如由系统时钟生成的二分频时钟。用 `create_generated_clock` 命令定义生成时钟。

**Hold time (*hold time*)**

clock active edge 之后，data pin 上的 signal 必须保持稳定的时间。hold time 为到达 cell data pin 的 path 建立 minimum-delay requirement。

本指南给出的 hold 检查关系式为：

$$\text{hold} = \text{max clock delay} - \text{min data delay}$$

**理想时钟 (*ideal clock*)**

被视为在时钟网络中传播时无延时的时钟。理想时钟类型是 DC Explorer 的默认。可用 `set_clock_latency` 与 `set_propagated_clock` 命令覆盖默认行为，以获得非零时钟网络延时并指定关于时钟网络延时的信息。

**理想网 (*ideal net*)**

被赋理想时序条件的网——即延迟、转换时间与电容赋值为零。此类网免于时序更新、延时优化与设计规则修复。将打算单独综合的某些高扇出网（如 scan-enable 与复位网）定义为理想网可缩短运行时间。

用 `set_ideal_net` 命令将网指定为理想网。

**Input delay (*input delay*)**

指定从 clock edge 到 signal 到达指定 input port 的 minimum 或 maximum delay 的 constraint。

用 `set_input_delay` 命令相对于指定时钟信号在引脚或输入端口上设置输入延时。

**实例 (*instance*)** 内存中已加载引用（库组件或设计）在电路中的出现；每个实例有唯一名称。设计可包含多个实例；每个实例指向同一引用，但有唯一名称以与其他实例区分。实例亦称单元（cell）。

**叶单元 (*leaf cell*)**

逻辑设计的基本单元。叶单元不能分解为更小的逻辑单元。例如 NAND 门与反相器。

**链接库 (*link library*)**

DC Explorer 用于解析单元引用的工艺库。链接库可包含工艺库与设计文件。链接库还包含已映射网表中单元（库单元以及子设计）的描述。

链接库包括本地链接库（`local_link_library` 属性）与系统链接库（`link_library` 变量）。

**多周期路径 (*multicycle path*)**

数据从起点传播到端点需要超过一个时钟周期的路径。

用 `set_multicycle_path` 命令指定 DC Explorer 应用于确定数据在特定端点何时需要的时钟周期数。

**网表 (*netlist*)** 以 ASCII 或二进制格式描述电路原理图的文件——网表包含设计中电路元件与互连的列表。网表传输是在设计系统或工具之间移动设计信息的最常用方式。

**工作条件 (*operating conditions*)**

设计遇到的工艺、电压与温度范围。DC Explorer 按工艺、电压与温度曲线上的工作点优化设计，并根据工作条件缩放单元与线延时。

默认情况下，工作条件在工艺库的 `operating_conditions` 组中指定。

**优化 (*optimization*)**

逻辑综合过程中 DC Explorer 尝试实现最能满足设计功能、时序与面积要求的工艺库单元组合的步骤。

**Output delay (*output delay*)**

指定从 output port 到捕获该 output 数据的 sequential element 的 minimum 或 maximum delay 的 constraint。该 constraint 确定 signal 必须在 output port 可用的时间，以满足 sequential element 的 setup/hold requirement。

用 `set_output_delay` 命令相对于指定时钟信号在引脚或输出端口上设置输出延时。

**Pad 单元 (*pad cell*)**

位于芯片边界、允许与芯片外集成电路连接或通信的特殊单元。

**路径组 (path group)**

相关路径的组，由 `create_clock` 命令隐式分组或由 `group_path` 命令显式分组。默认情况下，端点由同一时钟打入的路径分配到同一路径组。

**引脚 (pin)**

单元上提供输入与输出连接的部分。引脚可为双向。子设计的端口在父设计中是引脚。

**传播时钟 (propagated clock)**

经时钟网络产生延时的时钟。传播时钟用于确定寄存器时钟引脚处的时钟延迟。由传播时钟打入的寄存器，其边沿时间因从时钟源到寄存器时钟引脚的路径延时而偏斜。

用 `set_propagated_clock` 命令指定时钟延迟经时钟网络传播。

**实时钟 (real clock)**

具有源的时钟，意味着其波形施加到设计中的引脚或端口。通过使用 `create_clock` 命令并包含端口或引脚的源列表来创建实时钟。实时钟可以是理想的或传播的。

**引用 (reference)**

可用作构建更大电路的元素的库组件或设计。引用的结构可以是简单逻辑门或更复杂的设计（RAM 核或 CPU）。设计可包含引用的多次出现；每次出现是一个实例。亦见实例。

**RTL (RTL)**

RTL (register transfer level, 寄存器传输级) 是数字电子电路的寄存器级描述。在数字电路中，寄存器在时钟周期之间存储中间信息；因此 RTL 描述所存储的中间信息、其在设计中的存储位置，以及如何通过设计传输。RTL 在一组寄存器之间的数据流级别建模电路行为。此抽象级别通常几乎不含时序信息，除对一组时钟边沿与特性的引用外。

**Setup time (setup time)**

clock active edge 之前，data pin 上的 signal 必须保持稳定的时间。setup time 为到达 cell data pin 的 path 建立 maximum-delay requirement。

本指南给出的 setup 检查关系式为：

$$\text{setup} = \text{max data delay} - \text{min clock delay}$$

**Slack (slack)**

表示 mapped design 中 path endpoint 处 data arrival time 相对于 required arrival time 的 timing margin。slack 可为正、负或零；具体差值方向取决于所执行的 setup (max-delay) 或 hold (min-delay) 检查，但符号含义一致。

正 slack 表示仍满足 timing constraint；对 setup path，它表示 path delay 在不违例约束的情况下可增加的量。负 slack 表示 timing violation；对 setup path，其绝对值表示为满足约束必须减少的 path delay。hold slack 应按 min-delay 检查解释，不能直接套用 setup path 的“减少 delay”表述。

**结构化 (structuring)**

向设计添加中间变量与逻辑结构，可导致减小设计面积。结构化基于约束。最适合应用于非关键时序路径。

默认情况下，DC Explorer 对设计做 structuring。

**综合 (synthesis)**

从输入 IC 设计生成基于工艺库的优化门级网表的软件过程。综合包括读取 HDL 源代码并基于该描述优化设计。

**符号库 (symbol library)**

包含特定 ASIC 库中所有单元原理图符号的库。DC Explorer 用符号库生成设计原理图。可用 Design Vision 查看设计原理图。

**目标库 (target library)**

DC Explorer 在优化期间映射到的工艺库。目标库包含用于生成网表的单元以及设计系统工作条件的定义。

**工艺库 (technology library)**

综合过程中 DC Explorer 可用的 ASIC 单元库。工艺库可包含每个 ASIC 单元的面积、时序、功耗与功能信息。每个库的工艺特定于特定 ASIC 厂商。

**时序例外 (timing exception)**

对 DC Explorer 假定的默认（单周期）时序行为的例外。为使 DC Explorer 正确分析电路，必须指定设计中不符合默认行为的每条时序路径。时序例外的例子包括伪路径、多周期路径，以及需要与默认计算时间不同的特定最小或最大延时时间的路径。

**时序路径 (timing path)**

规定数据通过设计传播的点到点序列。数据在起点由时钟边沿发射，经组合逻辑元件传播，并在端点由另一时钟边沿捕获。时序路径的起点是输入端口或时序元件的时钟引脚。时序路径的端点是输出端口或时序元件的数据引脚。

**转换延时 (transition delay)**

由驱动引脚改变电压状态所需时间引起的时序延时。

**取消分组 (ungroup)**

移除设计中的层次级别。Ungrouping 将给定层次级别的子设计合并到父单元或设计中。

用 ungroup 命令，或对 compile 使用 auto\_ungroup 选项来 ungroup 设计。

**唯一化 (uniquify)**

解析内存中对同一设计的多个单元引用。

唯一化过程为每个引用原始设计的已实例化单元创建具有唯一设计名的唯一设计副本。

**虚时钟 (virtual clock)**

存在于系统中但不属于该块的时钟。虚时钟不打入当前设计内任何时序器件，也不与引脚或端口关联。用虚时钟作为相对于块外时钟指定输入与输出延时的参考。

用不带关联引脚或端口列表的 create\_clock 命令创建虚时钟。